System Design Interview Kit

Complete Edition — 61 problems with staged walkthroughs

Part I · Foundations and the order of flow

Part II · Questions reported at Amazon, Apple, Meta, Microsoft, Databricks, Google, Netflix (Q1–Q27)

Part III · The classic canon: TinyURL to ChatGPT (Q28–Q61)

Part IV · Master index, top 20, patterns, practice tracker

Prepared for Mal · Senior EM / Director loops · October 2026

Contents

Start here 5

What's in the book 5

Where to start: a 12-week system design track 5

Part I — Foundations 8

A1. How EM / Sr EM / Director design rounds differ from IC rounds 8

A2. The order of flow: what to say and draw, in what order 10

A3. Clarifying Questions Playbook — what to ask and what not to ask 14

A4. Building blocks — internals you must be able to explain 17

A5. Data platform foundations (essential for Databricks, useful everywhere) 28

A6. LLM and ML systems foundations 31

A7. Numbers cheat sheet and estimation 33

A8. The Manager Lens playbook 34

A9. Low-level / object-oriented design 35

A10. Code review and coding round prep 41

Part II — Company question bank (Q1–Q27) 44

Chapter 1 — Amazon 45

Amazon at a glance 45

Q1. Reverse design: "Walk me through a system you own. Where does it break at 10×?" 47

Q2. Design a distributed logging system 57

Q3. Design a system that delivers firmware updates to devices 66

Q4. Design a Dropbox-style file sync service 74

Q5. Design a parking payment system 81

Q6. Design the software behind an Amazon pickup location with lockers 91

Q7. Design a promotion system: "10% cash back on a particular credit card" (and coupons) 98

Q8. Implement Alexa voice search ("Alexa, order paper towels") 104

Chapter 2 — Apple 110

Apple at a glance 110

Q9. Migrate data off a legacy storage system without disrupting customers 112

Q10. Design a new data storage system from scratch — index structure, APIs, EU regionalization (key-value store) 122

Q11. Design a web cache (distributed cache) 131

Q12. Design a load balancer from scratch 138

Q13. Observability platform — and forecasting host spend with a finance partner 145

Chapter 3 — Meta 150

Meta at a glance 150

Q14. Real-time comments on a post with millions of concurrent viewers 151

Q15. Design News Feed (Facebook / Instagram) 159

Q16. A hardware-software system (example: smart glasses capture and sync) 164

Chapter 4 — Microsoft **169**
Microsoft at a glance 169
Q17. Design the notification system for Microsoft Teams 170
Q18. Real-time collaborative document editing (co-authoring) 176

Chapter 5 — Databricks **180**
Databricks at a glance 180
Q19. Design an object store (S3-like) 181
Q20. Design a payment network like Visa/Mastercard 188
Q21. Design a streaming data pipeline with exactly-once results (lakehouse) 195
Q22. Design a multi-tenant job scheduler / compute control plane 201

Chapter 6 — Google (secondary) **206**
Google at a glance 206
Q23. Show 1st/2nd/3rd-degree connections instantly (LinkedIn-style) 207
Q24. Count views/clicks on YouTube Shorts 212
Q25. Monitor ML model performance in real time 217

Chapter 7 — Netflix (secondary) **221**
Netflix at a glance 221
Q26. Distributed rate limiter at millions of requests per second 222
Q27. Design global video distribution 226

Part III — The classic canon (Q28–Q61) **231**

Part III.1 — Core consumer systems **232**
Q28. Design TinyURL (URL shortener) — and Pastebin 232
Q29. Design WhatsApp (1:1 and group chat) — also Messenger, Slack 239
Q30. Design YouTube (upload, transcode, stream) — also TikTok, Spotify 246
Q31. Design Twitter / X (post, home timeline, search, trends) 253
Q32. Design Uber / Lyft (ride matching) — also DoorDash 258

Part III.2 — Location, booking, search and crawling **264**
Q33. Design Yelp / a proximity service ("find restaurants near me") — also Nearby Friends 264
Q34. Design Google Maps (routing and ETA) 268
Q35. Design Ticketmaster (high-demand ticket booking) — also Airbnb / hotel booking 272
Q36. Design search autocomplete (typeahead) 277
Q37. Design a web crawler 281

Part III.3 — Search, ranking and infrastructure building blocks **287**
Q38. Design a web search engine (Google Search) 287
Q39. Design Top-K / trending (heavy hitters) — "top 100 songs/videos/hashtags in the last hour" 291
Q40. Design a distributed message queue (Kafka) 296
Q41. Design a distributed unique ID generator 300
Q42. Design a real-time gaming leaderboard 303

Part III.4 — Finance, real-time media, AI and messaging **306**
Q43. Design a stock exchange (order matching) — also Robinhood (brokerage) 306

Q44. Design Zoom (video conferencing) 313
Q45. Design ChatGPT (an LLM chat service) 316
Q46. Design a recommendation system (Netflix / Amazon / YouTube recommendations) 321
Q47. Design Gmail (email service) 326

Part III.5 — Marketplaces, social and real-time apps 331

Q48. Design a local delivery service (Gopuff / Instacart-style) 331
Q49. Design Instagram (photo upload, feed, stories) 335
Q50. Design Tinder (swipe and match) 339
Q51. Design LeetCode (online judge with contests) 342
Q52. Design Strava (activity tracking) 345
Q53. Design an online auction (eBay-style bidding) 348
Q54. Design a news aggregator (Google News) 351
Q55. Design a price tracking service (CamelCamelCamel) 354
Q56. Design Facebook post search 357
Q57. Design online chess (real-time multiplayer game) 360
Q58. Design a flash sale (limited inventory, huge spike) 363
Q59. Design a donations website (GoFundMe-style, viral spikes) 366
Q60. Design GitHub Actions (CI/CD job runner) 369
Q61. Design an ad click aggregator 372

Part IV — Index and practice plan 376

Top 20: the minimum set to master 376
Problems grouped by pattern 376
Master index: all 61 problems 377
Variants map: "they asked X — practice Y" 379
Company matrix for Part II (Q1–Q27) 380
Practice tracker 381
Sources 382

Start here

This is the complete system design kit in one book: **foundations, the 27 questions reported at your target companies, and the 34 classic questions every interviewer draws from — 61 problems in all**, each with a staged high-level design walkthrough you can rehearse out loud.

What's in the book

Part	What it gives you	When to use it
I. Foundations	How EM design rounds are scored, the order of flow (what to say and draw, in what order), clarifying questions to ask and avoid, building blocks from the inside, data platforms, ML/LLM systems, numbers, the manager lens, low-level design, code review	First two weeks, then as reference
II. Company question bank (Q1–Q27)	Questions reported in 2025–2026 at Amazon, Apple, Meta, Microsoft, Databricks, Google and Netflix, each worked end to end	Before each company's loop, re-read its chapter
III. Classic canon (Q28–Q61)	TinyURL, WhatsApp, YouTube, Uber, Ticketmaster, ChatGPT and the rest	The pattern library behind most questions
IV. Index and practice plan	Master index of all 61, variants map, company matrix, top-20 priority list, tracker, scoring rubric, sources	Plan your weeks; track progress

Where to start: a 12-week system design track

This track fits inside your 6-month plan alongside leadership and coding prep. Budget 4–5 hours a week for system design.

Weeks	Do this	Done when
1–2	Part I: A1 (how you're scored), A2 (order of flow), A3 (clarifying questions). Then A4 building blocks — explain each aloud in 3 minutes	You can explain any building block with one failure mode and one number
3–4	The five pattern-teachers: **Q28 TinyURL → Q29 WhatsApp → Q30 YouTube → Q32 Uber → Q35 Ticketmaster**. Learn A7 numbers	Third attempt of each hits every checkpoint in 45 minutes
5–6	Your highest-value company problems: **Q1 (your own system), Q9 migration, Q2 logging, Q14 live comments, Q10 storage + EU**	Q1 rehearsed with your real numbers

Weeks	Do this	Done when
7–8	**Q19 object store, Q21 streaming pipeline, Q3 OTA, Q5 payments, Q15 feed, Q4 Dropbox, Q17 notifications** + A5 data platform	Can redraw each from memory
9–10	Remaining classics, grouped by pattern (Part IV index): fan-out, geo, booking/contention, search, counting, real-time	One cold attempt each; mark weak patterns
11–12	Mock interviews (2+ per week). Before each real loop, re-read that company's chapter in Part II and its day-before cheat sheet	Mock scores average Strong Hire on the rubric

If time is short, the **top 20 list in Part IV** is the minimum set.

The drill for every problem

1. **Cold attempt (45 min):** read only the prompt; do the whole design out loud on a blank page, recording yourself.
2. **Compare (20 min):** read the walkthrough; mark every gap in red — missed questions, wrong numbers, missing deep dives, no manager lens.
3. **Rehearse (15 min):** say the "**You:**" lines aloud twice, then again from memory in your own words.
4. **Redraw (10 min):** draw the staged diagram from memory in the right order.
5. **Repeat at day 7 and day 30**, cold.

Your engineering background makes you want to go deep immediately. The drill trains the opposite habit: the whole system on the board by minute 20, then depth.

How each walkthrough is laid out

Prompt → clarify (ask / don't ask / assumptions) → requirements → estimates → API and core entities → data model → **high-level design walkthrough** (staged diagrams in drawing order, a script per step, numbered write and read paths, component table, checklist, 60-second summary) → deep dives → scaling → failures → trade-offs → security, operations and cost → manager lens → follow-ups → mock transcript → scoring guide → "say this" lines → your tie-in.

The most-asked problems have every section; less common ones are more compact but always include the staged walkthrough, deep dives, trade-offs and "say this" lines. In the room, walk the high-level design *before* the data model (see A2), even though the book lists the data model first for reference.

Confidence labels (Part II)

- **High:** multiple independent reports from 2025–2026

- **Medium:** one recent first-hand report, or a recent guide citing candidates
- **Low / Likely:** older reports, aggregators, or a strong pattern match to the company's domain

Reported questions are signals, not a syllabus. Part III's classics carry no company label because they appear everywhere.

What "ready" looks like

Skill	Ready when you can…	Where
Scoping	Turn a one-line prompt into requirements, assumptions and out-of-scope in under 5 minutes	A2, A3
Estimation	Produce QPS, storage and bandwidth aloud in 3 minutes and say what they imply	A7
Drawing	Build the design in stages along the request path, narrating, with numbered arrows	A2 + every walkthrough
Building blocks	Explain any block from first principles with a failure mode and a number	A4–A6
Trade-offs	Say "X over Y because Z; the cost is W" for every major choice	Every problem
Manager lens	Weave teams, phasing, migration, risk and cost in at decision points	A8
Low-level depth	Write an LRU cache, rate limiter or log-structured store in 25 minutes	A9
Code review	Find and prioritize the bugs in a page of unfamiliar code	A10

Honest scope

- **Knowledge isn't performance.** Plan at least 10 timed design mocks with people who interrupt you.
- **Your own systems matter most.** Amazon's reverse design, Databricks' technical fit and Apple's real-problem rounds probe what *you* built. Replace every [VERIFY] with real details before you say it in an interview.
- **The coding gap is your biggest risk** for Apple, Meta, Microsoft and Databricks. A9, A10 and the coding track in your schedule address it; only daily practice closes it.

Part I — Foundations

Part I is the toolkit you use in every loop. Parts II and III assume you know it cold. Read A1–A3 first; they change how you *behave* in the room. A4–A10 are reference material you rehearse until you can explain each item out loud without notes.

> **How to use this part.** For each section: read it once, then close the book and explain it aloud in 3 minutes as if to a staff engineer. If you stumble, mark it in your error log. Your goal by Week 8 is to explain every building block in A4 from first principles in under 3 minutes, including one failure mode and one number.

A1. How EM / Sr EM / Director design rounds differ from IC rounds

What is actually being scored

An IC design round mostly scores **technical correctness and depth**. A manager design round scores the same technical bar *minus some depth*, **plus** three things ICs are rarely scored on:

1. **Judgment under ambiguity** — Do you pick a sensible scope, state assumptions, and make defensible calls quickly instead of exploring everything?
2. **Breadth with selective depth** — Can you hold the whole system in view *and* drop into the two or three components where the real risk lives?
3. **Organizational translation** — Can you connect the architecture to how teams would build, own, operate, and evolve it (phasing, ownership boundaries, migration, cost, risk)?

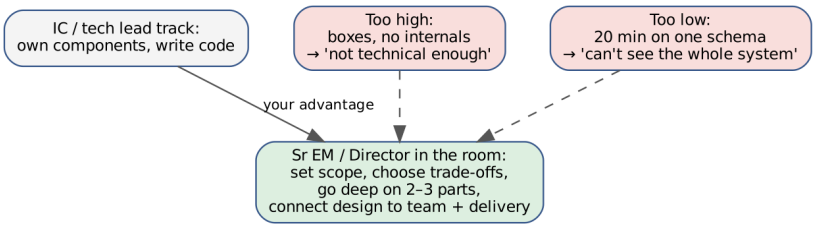


Figure A1.1 — Where an engineering leader should sit in the design round, and the two ways managers fail it.

How the bar differs by company (your targets)

Company	Technical depth expected of EMs	What tips you to Strong Hire	Most common EM failure

Amazon	Medium–high. End-to-end architecture, then let the interviewer pull you into a subcomponent. AWS-shaped thinking is normal.	Full architecture first; operational excellence (alarms, on-call, COE mechanisms) without being asked; Leadership Principles visible in how you make calls.	Diving into one component early and running out of time; thin operational story; saying "we" without "I decided".
Apple	High. Problems often come from the interviewer's real work; second round goes low-level (indexes, APIs, partitioning, regionalization).	Specific, defensible choices; migration and backwards-compatibility realism; privacy/data residency instincts.	Textbook answers with no specifics; weak on migration while two systems run in parallel.
Meta	Medium–high. Product/system design with a strong product-and-metrics flavor.	Clear user-facing requirements, scaling plus *what you would measure*; crisp trade-offs; pace.	Ignoring metrics and product behavior; over-engineering before establishing the core flow.
Microsoft	Medium. Practical cloud design; often paired with leadership discussion; a PM partner may probe trade-offs.	Pragmatic choices, Azure-literate where relevant, clear collaboration and growth-mindset framing.	Generic, unanchored designs; not explaining why a choice fits the customer scenario.
Databricks	Very high. Reported as the hardest round; systems/infra problems; you need a "yes" from every interviewer.	Internals (storage, metadata, consistency, distributed compute); comfort with data-platform concepts; defending your own system in depth.	Staying at box level; not knowing how object storage, logs, or Spark-like systems behave underneath.
Google (secondary)	High. One-line prompts; interviewers push past managed services and ask how you'd build the component.	Setting scope yourself; first-principles explanations of queues, caches, traversals.	Naming products as black boxes; reproducing a memorized architecture.
Netflix (secondary)	Very high and bespoke. Scale, resilience, and operations.	Failure thinking, graceful degradation, chaos-style reasoning.	Generic product clones; ignoring failure modes.

Top reasons managers fail design rounds

1. **Too high-level.** Boxes with labels and no mechanics. If you say "cache," be ready to say what's keyed, the TTL, invalidation, hit rate, and what happens on a miss storm.
2. **Too deep, too early.** Twenty minutes on a schema; no end-to-end flow by minute 20. This is the classic engineer-turned-manager trap and **your** highest risk.
3. **No numbers.** Estimation isn't decoration; it drives choices (one Postgres box vs 200 shards). Do the math out loud.
4. **Unexplained choices.** Every important decision needs "I chose X over Y because Z; the cost is W."
5. **No failure story.** What breaks first at 10×? What happens when a region dies?

6. **Manager lens missing or bolted on.** Five minutes of "and then I'd hire people" at the end doesn't count. Weave it in where decisions have org consequences.
7. **Passive collaboration.** Waiting for the interviewer to drive, or arguing with hints. Treat hints as data.

Your profile: using engineering depth without drowning in it

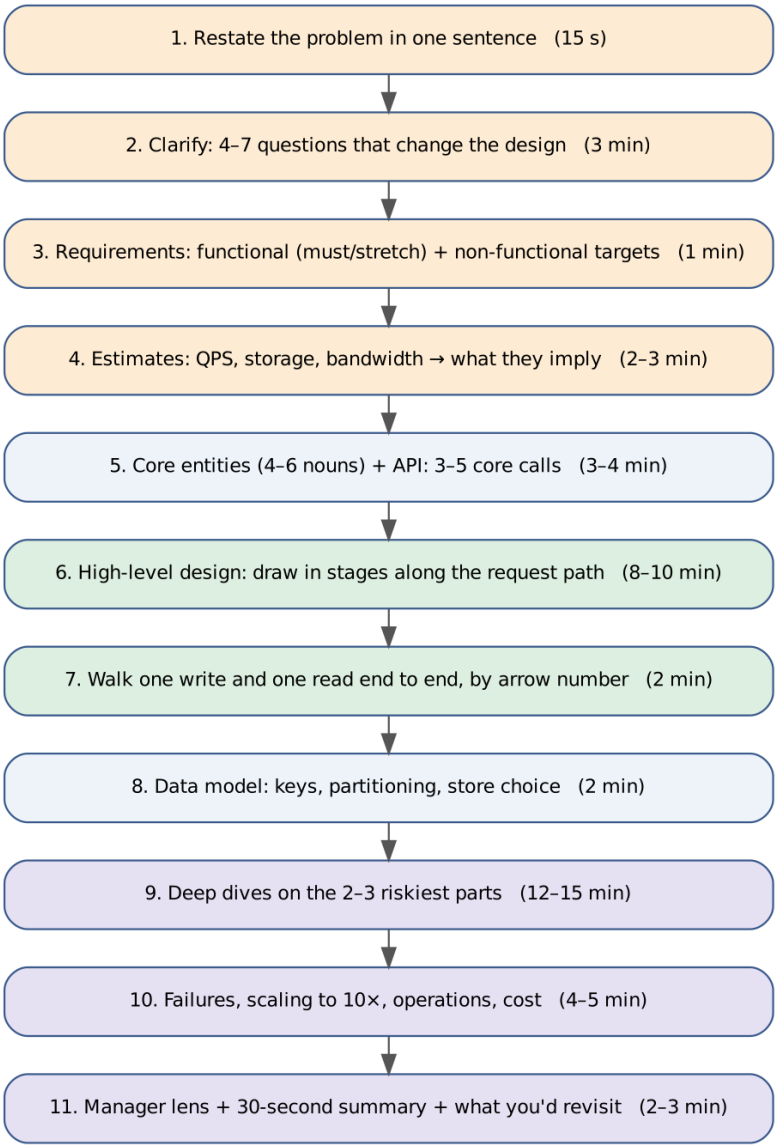
You came up through software engineering, solutions architecture and security architecture. That is a real advantage, because many EM candidates fail by being shallow. It also creates two specific risks: diving in too early, and slipping into "how I'd code it" instead of "how the system and team should work." Use these rules:

- **Depth on invitation or on risk.** Go deep only where (a) the interviewer asks, or (b) the biggest risk in the design lives. Announce it: "The hardest part here is fan-out on write; I'll spend ten minutes there."
- **One layer below the box, by default.** For every component you draw, be ready to go exactly one layer deeper (key, partitioning, consistency, failure). Go two layers only in deep dives.
- **Name your depth explicitly.** "I've built this pattern before at the architecture level — the trap is X." That signals hands-on credibility without a war story.
- **Convert depth into decisions.** After any technical deep dive, close with the leadership consequence: "This is why I'd put the consistency-critical path in one team and keep the read path separate."
- **Security is your secret weapon.** With your AT&T identity background, add one crisp security insight per design (token scoping, tenant isolation, PII flow). Few candidates do; it differentiates you.

A2. The order of flow: what to say and draw, in what order

Interviewers score not just *what* you design but whether they can follow you. A strong candidate narrates in a predictable order, so the interviewer always knows where you are and what's coming. Use this order for every problem in this book until it's automatic.

The speaking order (45 minutes)



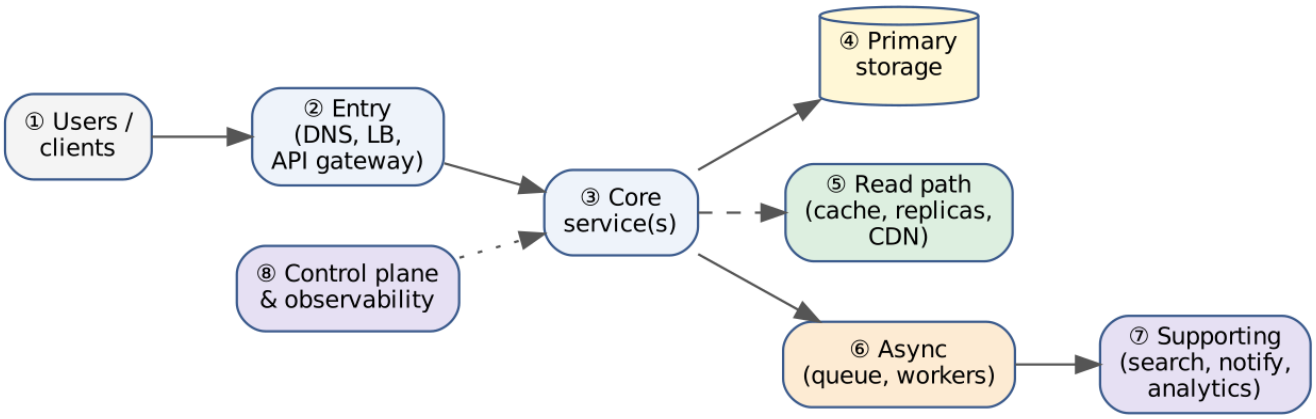
The eleven-step speaking order. Steps 6 and 7 are the high-level design walkthrough — the part most candidates rush.

Step	What you say	Transition phrase to the next step
1. Restate	"So we're building X for Y so that Z."	"Let me ask a few questions before drawing anything."
2. Clarify	Only questions whose answers change the design; state assumptions for the rest	"Let me write down what we're building."
3. Requirements	3–5 must-haves, stretch items, 3–4 non-functional targets with numbers	"Quick math to size this."

Step	What you say	Transition phrase to the next step
4. Estimates	QPS avg/peak, storage/year, bandwidth — and what each number implies	"Here's the API the clients will use."
5. Core entities + API	List the 4–6 main nouns (~2 min), then 3–5 calls with key fields; idempotency where it matters; identity from auth, never from the request body	"Let me draw the system, starting from the user."
6. High-level design	Draw in stages along the request path (below)	"Let me walk one request through it."
7. Walk a request	Number the arrows; narrate one write and one read	"Now the data model, because it drives partitioning."
8. Data model	Entities, keys, partitioning, store choice and why	"The hardest part here is X — I'd like to go deep there."
9. Deep dives	2–3 riskiest components; alternatives and trade-offs	"Let me stress-test this."
10. Failures & scale	What breaks first at 10×, failure modes, SLOs, cost	"If I were building this with a team…"
11. Manager lens & close	Teams, phasing, risks, decide vs delegate; 30-second summary	"Happy to go deeper anywhere you'd like."

The drawing order for the high-level design

Always draw **left to right along the path of a request**, adding one layer at a time. This is the same order used in every walkthrough's staged diagrams.



Draw ①–④ first (the simplest working system), then add ⑤–⑧ only as the requirements demand them.

1. **Users / clients** — who sends the first request.
2. **Entry** — DNS, load balancer, API gateway (auth, rate limits).
3. **Core service(s)** — the business logic, stateless.

4. **Primary storage** — the source of truth. *Stop here and say: "This is the simplest version that works."*
5. **Read path** — caches, replicas, CDN — added because the estimates say reads dominate.
6. **Async path** — queues and workers for anything slow, spiky, or fan-out heavy.
7. **Supporting systems** — search, notifications, analytics, ML.
8. **Control plane and observability** — config, rollout, metrics, alerting.

Why this order works: the interviewer sees a working system within three minutes, every box you add after that has a stated reason ("the estimates show 100:1 reads, so…"), and you never run out of time with half a diagram.

Narration rules while drawing

- **Talk while you draw.** Never go silent for more than ~20 seconds. Say what the box is and why it exists as you add it.
- **Number the arrows.** Write 1, 2, 3 on the write path and a, b, c on the read path. Then step 7 is easy: "Request comes in at 1…".
- **One reason per box.** "Queue here because extraction takes minutes and traffic spikes at quarter end." If you can't give a reason, the box doesn't belong yet.
- **Name the store and why** in one sentence when you draw it; save the detail for the data model step.
- **Mark what you're deferring.** "I'll come back to caching after the data model." It shows control of time.
- **Check in after the diagram:** "That's the end-to-end design. Shall I go deeper on the write path or on storage?"

Common ordering mistakes

Mistake	Why it hurts	Instead
Drawing a cache, Kafka and CDN in the first minute	Looks pattern-matched; no reasons	Start with the simplest working system, add with reasons
Starting with the database schema	No end-to-end view by minute 20	Schema comes after the request walk (step 8)
Drawing everything, then explaining	Interviewer can't follow; time runs out	Narrate box by box
Never walking a request	Interviewer can't verify the design works	Always do step 7 with numbered arrows
Deep-diving before the diagram is complete	Classic engineer-turned-manager trap	Finish ①–⑧ at a simplified level first

Two habits from widely used frameworks

Hello Interview teaches **Requirements → Core entities → API → (Data flow) → High-level design → Deep dives**, the same backbone as above. Two habits worth keeping:

- **Core entities before the API.** Two minutes listing the main nouns (User, Ride, Driver, Trip…) makes the API and diagram much faster.
- **Build the design to satisfy the API, then deepen.** Draw just enough to make each endpoint work, then use deep dives to meet the non-functional requirements. The staged diagrams in this book follow exactly that: Step 1 makes it work, later steps make it scale.

For EM loops, keep two extra moves: estimates that drive decisions, and the manager lens at the end.

Clock discipline

Check the clock at minutes 10, 20 and 35. If the high-level diagram isn't complete at minute 20, say: "*I'm going to leave this part simplified and move on so we have time for the hard parts.*" Interviewers reward that self-awareness; running out of time with half a system is a common No Hire reason.

Recovery moves

- **Interviewer adds a constraint mid-way** ("now it must work offline"): restate it, name what it changes, redesign only the affected part.
- **You realize you made a mistake:** say so immediately — "Actually, that won't work under concurrent writes; let me fix it." Self-correction scores well; defending a bad choice scores badly.
- **You don't know an internal:** reason from first principles and be honest — "I haven't built this exact thing, but the mechanism has to be roughly X because of Y."
- **Interviewer goes silent:** check in — "Is this the level of detail you'd like, or should I go deeper on storage?"

A3. Clarifying Questions Playbook — what to ask and what not to ask

Clarifying questions are scored. Good ones show you know which facts change the design; bad ones show you're waiting to be told what to build. The rule: **ask only questions whose answers change your design, and for everything else state an assumption.** Aim for 4–7 questions in 3–5 minutes.

Questions TO ASK — grouped, with why they matter

Group	Question	Why it matters / what it changes

Users & use cases	"Who are the primary users, and what are the two or three things they must be able to do?"	Defines the core flows you design first; everything else is secondary.
	"Is this consumer-facing, internal, or a platform other teams build on?"	Platform → API contracts, multi-tenancy, quotas. Consumer → latency, abuse, scale spikes.
	"What's the single most important experience to get right?"	Tells you where to spend deep-dive time; interviewers often hint here.
Scale	"Roughly how many daily active users / requests / devices?"	Decides single-node vs sharded; drives every number.
	"Are there spikes — launches, events, time-of-day peaks?"	Peak-to-average ratio sizes capacity, queues and autoscaling.
	"How fast is it growing — design for today or 10×?"	Sets how much you over-provision and where you leave seams for later.
Read/write pattern	"Is it read-heavy or write-heavy? Roughly what ratio?"	Read-heavy → caching, replicas, precompute. Write-heavy → log-structured storage, partitioning, async.
	"How big are the objects — bytes, KBs, or GBs?"	Large objects → blob storage + metadata DB + chunking + CDN.
Latency	"What latency do users expect for the key action — p99 under 200 ms, or is a few seconds fine?"	Synchronous vs async; precompute vs compute-on-read; edge vs region.
Consistency	"If two users look at the same thing, must they see the same value immediately, or is a few seconds of lag OK?"	Strong vs eventual consistency; choice of datastore and replication.
	"Can we ever lose or double-count an event? (Money, inventory, counts)"	Exactly-once semantics, idempotency keys, transactions vs approximate counters.
Availability	"What's the availability target, and what should happen if a dependency is down — degrade or fail?"	Multi-AZ vs multi-region; graceful degradation design.
Geography / residency	"Global users? Any data residency requirements, e.g., EU data staying in the EU?"	Regional partitioning, replication topology, compliance boundaries. (Apple asks this.)
Existing systems	"Greenfield, or replacing / integrating with something that exists?"	Migration, dual-write, backwards compatibility become central. (Apple, Amazon.)
Security & privacy	"Is there sensitive data — PII, payments, health — and who may see what?"	Encryption, access control, tenant isolation, audit logs.

Cost	"Is cost a primary constraint, or optimize for latency and reliability first?"	Storage tiering, precompute vs on-demand, spot capacity. (Apple finance round; Databricks.)
Interviewer focus	"Is there a part you'd especially like me to go deep on?"	Free signal about the rubric. Ask after scoping, not first.

Questions NOT TO ASK — and why they hurt

Don't ask	Why it hurts	Say instead
"What database should I use?"	That's the decision you're being scored on.	"Given heavy key-based reads, I'll use a wide-column store; here's why…"
"Should I use Kafka?"	Same — and it reads as tool-first thinking.	"I need a durable, replayable log partitioned by user — a Kafka-style log fits."
"Is this enough?" / "Is this right?"	Signals low confidence; puts the interviewer in the role of grader mid-answer.	"That covers the core flow. Shall I go deeper on X or move to scaling?"
"What are the requirements?" (open-ended)	Hands scoping back; at Google this alone costs points.	Propose requirements and ask to confirm.
A 12th clarifying question at minute 10	Analysis paralysis; you're out of time for design.	State the remaining assumptions in one sentence and start.
"What language/framework?"	Irrelevant to architecture; signals IC framing.	Skip it.
Implementation minutiae before the high-level design ("What's the exact JSON field name?")	Wrong altitude; burns time.	Leave for API section.
Questions that don't change the design ("Will there be a logo?")	Noise; makes you look unfocused.	Only ask if the answer changes a box or arrow.
"How does your team do it today?" (in a generic round)	Can feel like fishing for the answer.	Fine in Apple-style "real problem" rounds; otherwise state assumptions.

When the interviewer is vague: state assumptions

Some interviewers (often at Google, sometimes Netflix/Databricks) deliberately answer "you decide." Don't stall. Use this pattern:

> *"I'll assume about 50 million daily users, a 100:1 read-to-write ratio, p99 under 200 ms for reads, and eventual consistency of a few seconds being fine for this feed. I'll design for 10× headroom. If any of those are off, tell me and I'll adjust."*

Write the assumptions in a corner of the board. Refer back to them when you make choices ("because we assumed eventual consistency is OK…").

Recovering from a weak question

If you realize you asked something irrelevant, don't apologize at length — pivot: "Actually that won't change the design much; let me focus on what does." Then ask the high-value one.

Company-specific clarification norms

- **Amazon:** Structured, often less back-and-forth. Clarify quickly, then lay out the full architecture before deep dives. Ask about the customer ("Who's the customer and what do they care about?") — it signals Customer Obsession.
- **Apple:** Problems may come from the interviewer's real work. Ask about existing systems, migration constraints, privacy and regions. These questions are welcome and expected.
- **Meta:** Ask about product behavior and success metrics ("What does success look like — engagement, latency, freshness?").
- **Microsoft:** Ask about the customer scenario and enterprise constraints (tenancy, compliance, identity).
- **Databricks:** Ask about data volume, consistency of metadata, multi-tenancy and cloud-provider constraints.
- **Google:** Minimal answers by design. Set scope yourself quickly and confidently.

A4. Building blocks — internals you must be able to explain

For every block: **how it works**, **when to use it**, **limits and failure modes**, **numbers to quote**, and **how you'd build it from scratch** if an interviewer (Google, Databricks, Apple) pushes past the product name. AWS / Azure equivalents are listed so you can speak Amazon's and Microsoft's language, but always lead with the mechanism.

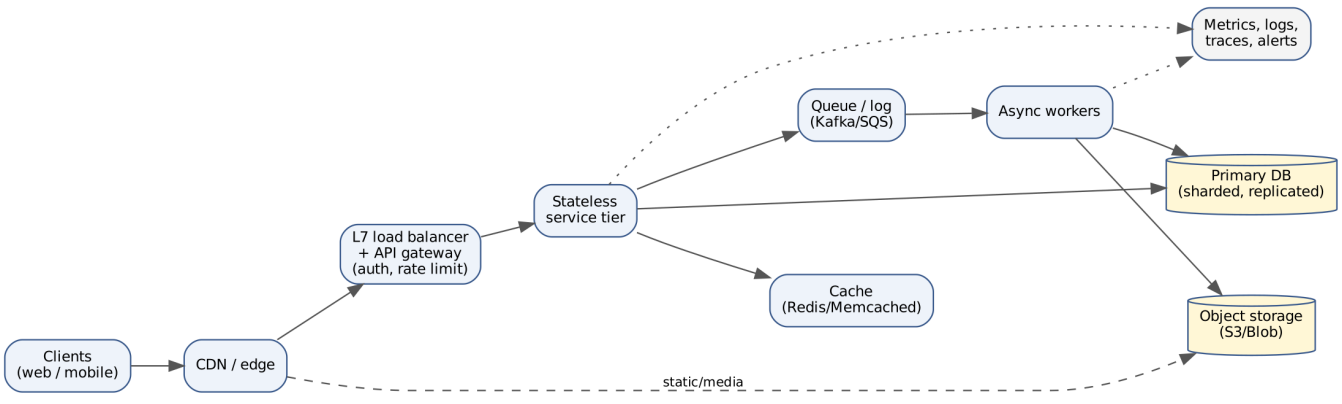


Figure A4.1 — The baseline web-scale architecture. Almost every design starts here; your job is to say which parts this problem stresses and change those.

1. Load balancing (L4 vs L7)

How it works	**L4** balances TCP/UDP connections by IP/port; fast, protocol-agnostic, can't see URLs. **L7** terminates HTTP/TLS and routes on path, header, cookie; enables auth, rate limiting, canary routing. Algorithms: round robin, least connections, weighted, consistent hashing (sticky by key), power-of-two-choices (pick 2 random, choose less loaded — near-optimal and cheap).
Failure modes	LB itself as SPOF (run active-active pairs, anycast/DNS in front); unhealthy backends kept in rotation (tune health checks); connection draining missed during deploys; hot backends from sticky sessions.
Numbers	A modern L7 proxy (Envoy/NGINX) handles tens of thousands of RPS per core-cluster; L4 handles millions of packets/sec.
Build from scratch	(Apple asked this.) Components: listener; backend registry with health checker (active probes + passive outlier detection); selection algorithm; connection pool; config hot-reload; metrics. Discuss: how health state propagates, how to drain, how to avoid thundering herd when a backend returns (slow-start weights).
AWS / Azure	NLB (L4), ALB (L7) / Azure Load Balancer (L4), Application Gateway, Front Door (global L7).

2. CDN and edge caching

How it works	Points of presence cache content near users. Pull model (fetch on miss from origin) or push (pre-populate). Cache key = URL + selected headers. Origin shield = mid-tier cache that collapses misses so origin sees one request per object per region.
Use when	Static assets, media, video segments, public API responses with short TTLs, large downloads (firmware/OTA updates).
Failure modes	Cache stampede on popular-object expiry; serving stale private data (cache-key mistakes leaking per-user content — a security incident); purge latency.
Numbers	Edge hit ratio target 90–99% for static media; edge RTT tens of ms vs 100–200 ms cross-continent.
AWS / Azure	CloudFront / Azure Front Door, Azure CDN.

3. Caching patterns, invalidation and stampede protection

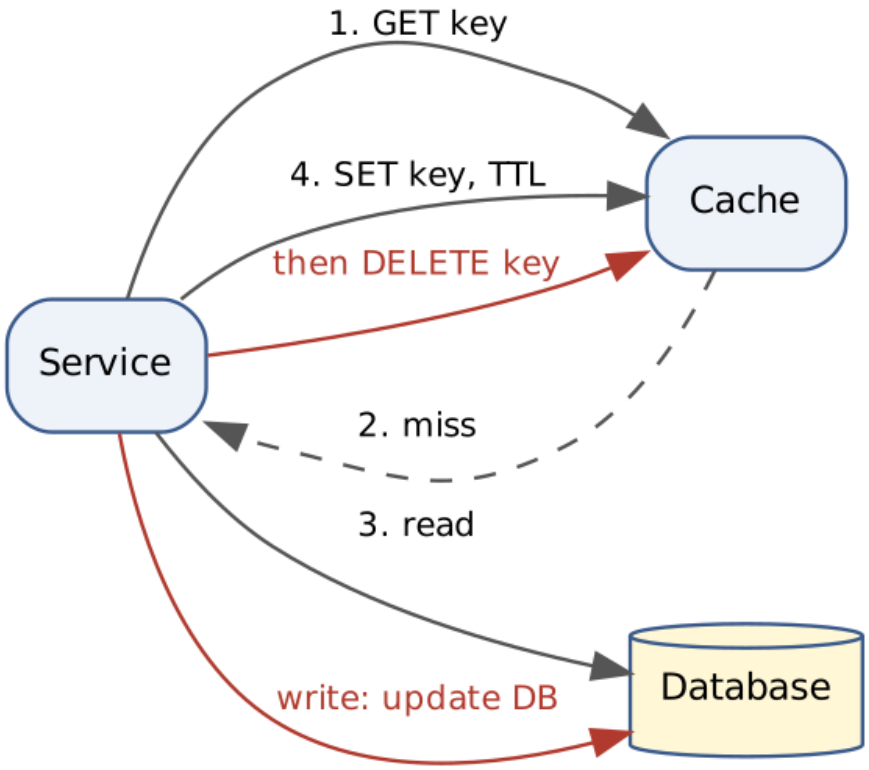


Figure A4.2 — Cache-aside: read through the cache, fill on miss; on write, update the database then delete the key.

Pattern	How	Pros	Cons / when not
Cache-aside (lazy)	App reads cache → miss → DB → set cache. Writes update DB then **delete** key.	Simple; cache holds only hot data; survives cache loss.	First read slow; race: stale value written back after delete (mitigate with short TTL, versioned keys, or delayed double-delete).
Read-through	Cache library loads from DB on miss.	Cleaner app code.	Same staleness issues; couples cache to data source.
Write-through	Write to cache and DB synchronously.	Cache always fresh.	Write latency; caches cold data.
Write-back (behind)	Write to cache; flush to DB async.	Very fast writes; absorbs bursts.	Data loss on cache failure — avoid for money/inventory.
Precompute / materialized views	Compute results ahead (feeds, counters, degree-of-connection).	Fast predictable reads.	Storage cost; staleness; fan-out cost on writes.

Why "delete, not update" on write: two concurrent writers updating the cache can leave the older value last; deleting forces the next reader to load the current DB value.

Stampede (thundering herd) protection: (1) *request coalescing / single-flight* — one loader per key, others wait; (2) *probabilistic early expiry* — refresh slightly before TTL with probability rising toward expiry; (3) *stale-while-revalidate* — serve stale while one worker refreshes; (4) *TTL jitter* so keys don't expire together.

Numbers: Redis ~100k+ ops/sec per core, sub-ms latency in-region. Cache size = hot working set (often 20% of data serving 80% of reads). Target hit rate 90%+; every 1% of misses at 1M RPS is 10k DB reads/sec.

Eviction: LRU (recency), LFU (frequency — better for stable popularity), TTL. Build-from-scratch LRU = hashmap + doubly linked list (see A9).

4. Consistent hashing

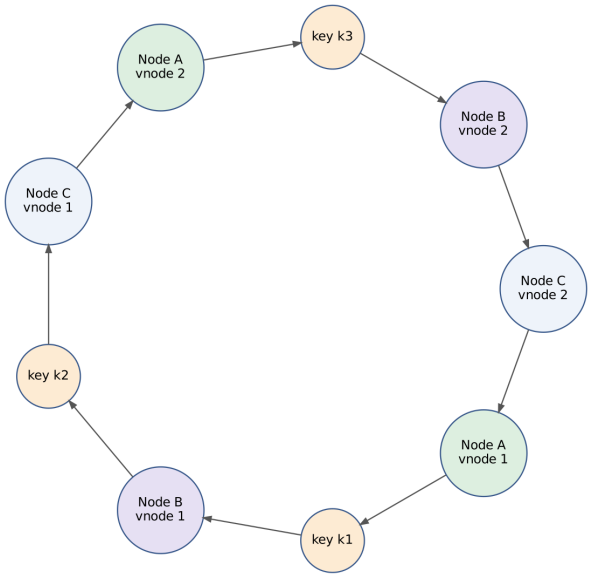


Figure A4.3 — Keys map clockwise to the next virtual node. Each physical node owns many vnodes, so adding a node moves only ~1/N of keys.

Why: with `hash(key) mod N`, changing N remaps almost every key (cache wipeout, mass data movement). Consistent hashing remaps only ~1/N. **Virtual nodes** (100–256 per server) smooth load and let you weight bigger machines. **Alternatives:** rendezvous (highest-random-weight) hashing — simpler, no ring; jump consistent hash — fast, but only for numbered buckets. **Hot keys** still break it: one celebrity key overloads one node → replicate hot keys, add random suffix and fan reads, or cache in front.

5. Sharding and rebalancing

Strategy	How	Good for	Risk
Hash	hash(key) → shard	Even distribution; point lookups	Range scans across all shards

Range	Key ranges per shard (split as they grow)	Range scans, time series	Hot spots on monotonically increasing keys (timestamps) — prefix with a hash/ bucket
Directory / lookup	Service maps key → shard	Flexible moves, tenant placement	Directory is critical path; cache it
Geo / tenant	Shard by region or customer	Residency, isolation, noisy-neighbor control	Uneven tenant sizes; big tenants need sub-sharding

Choosing the shard key is the decision: it must match the dominant access pattern and avoid cross-shard transactions on the hot path. **Rebalancing:** many small fixed partitions (e.g., 4,096) assigned to nodes so you move whole partitions; or dynamic split/merge (Bigtable/HBase/ Spanner style). **Resharding live:** dual-write → backfill → verify → cut reads → cut writes → clean up. This is the same playbook as Apple's legacy-storage migration question.

6. Replication and quorums

- **Leader–follower:** writes to leader, async/sync to followers. Simple; replication lag causes stale reads (read-your-writes violations). Fix: read your own writes from leader for a window, or track a log position per session.
- **Multi-leader:** writes in multiple regions; needs conflict resolution (last-writer-wins with clocks — loses data; CRDTs; app-level merge).
- **Leaderless (Dynamo-style):** write to W of N replicas, read from R; if **R + W > N** reads overlap the latest write (e.g., N=3, W=2, R=2). Repair via read-repair, hinted handoff, Merkle-tree anti-entropy. Amazon's Dynamo paper is worth knowing by name for Amazon loops.

7. Consistency models and CAP / PACELC

Model	Guarantee	Example need
Linearizable (strong)	Every read sees the latest committed write, as if one copy	Balances, inventory decrement, leader election, uniqueness
Sequential / causal	Operations respect causality (reply after the post)	Comments, chat
Read-your-writes	A user sees their own updates	Profile edits, settings
Eventual	Replicas converge if writes stop	Like counts, feeds, view counts

CAP: during a network partition, choose consistency or availability. **PACELC** is the more useful version: if **P**artition, choose **A** or **C**; **E**lse, choose **L**atency or **C**onsistency. Say it this way in interviews: "Even without failures, strong consistency costs a cross-replica round trip on every write; I'll pay that only for the money path."

8. Choosing a datastore

Type	Examples	Choose when	Avoid when
Relational (SQL)	Postgres, MySQL, Aurora, Azure SQL	Transactions, joins, constraints, moderate scale; most systems start here	Write volume beyond a sharded cluster's comfort; schemaless blobs
NewSQL / distributed SQL	Spanner, CockroachDB, Aurora DSQL, Cosmos DB (strong mode)	Global strong consistency + SQL + horizontal scale	Cost/latency sensitive simple KV workloads
Key-value	DynamoDB, Redis, Cosmos DB	Known access patterns, single-key lookups, massive scale	Ad-hoc queries, multi-entity transactions
Wide-column	Cassandra, Bigtable, HBase, ScyllaDB	Huge write throughput, time-ordered data per key (messages, events, metrics)	Cross-partition queries, strong multi-row transactions
Document	MongoDB, Cosmos DB, DocumentDB	Nested, evolving entity shapes	Heavy relational joins
Time-series	Prometheus, InfluxDB, Timestream, M3	Metrics, telemetry, downsampling, retention tiers	General entities
Graph	Neo4j, Neptune	Multi-hop traversals on modest graphs	Planet-scale social graphs (use sharded adjacency lists + precompute; Google expects you to write the BFS)
Search	Elasticsearch/ OpenSearch	Full-text, faceting, log search	Primary source of truth
Object storage	S3, Azure Blob, GCS	Blobs, media, data lake files; 11 nines durability designs	Low-latency small random updates

9. Storage engines: B-trees vs LSM trees

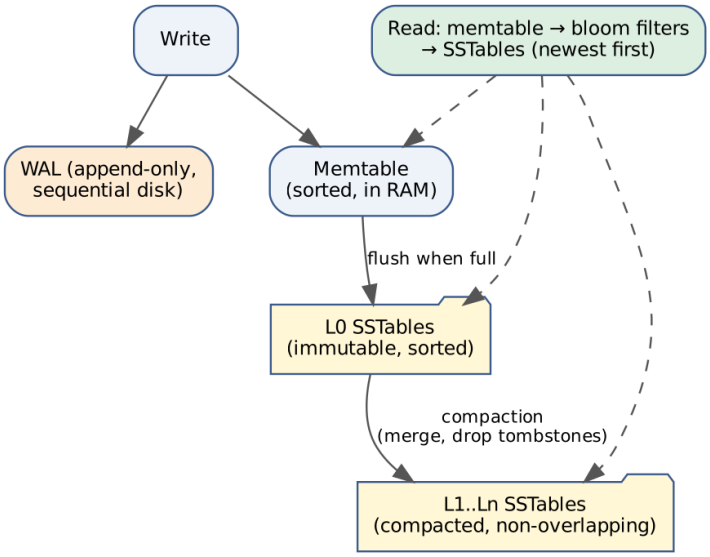


Figure A4.4 — LSM write path: WAL for durability, memtable in RAM, flush to immutable SSTables, compaction in the background.

	B-tree (Postgres, MySQL InnoDB)	**LSM tree (Cassandra, RocksDB, Bigtable)**
Writes	In-place page updates; random I/O	Sequential appends; very high write throughput
Reads	Predictable, one tree walk	May check several SSTables; bloom filters reduce cost
Amplification	Write amplification from page rewrites	Write amplification from compaction; space amplification before compaction
Pick when	Read-heavy, transactional	Write-heavy, time-ordered, append-mostly

WAL (write-ahead log): every change is appended to a log and fsynced before acknowledging; on crash, replay the log. The same idea powers database replication, Kafka, and Delta Lake's transaction log. If you can explain WAL well, you can explain half of distributed storage.

10. Inverted indexes and search

Map each term → posting list of document IDs (+ positions, frequencies). Query = intersect/union posting lists, score with BM25 (term frequency × inverse document frequency, length-normalized). Index built in immutable segments merged in the background (LSM-like). Sharding: by document (each shard has a full index for its docs; query scatters to all) — the common choice; by term (fewer shards per query, but hot terms). Near-real-time: new segments become searchable after refresh (~1s).

11. Queues vs logs, and delivery semantics

	Queue (SQS, RabbitMQ, Service Bus)	Log (Kafka, Kinesis, Event Hubs)
Model	Message consumed and deleted; competing consumers	Append-only, retained; consumers track offsets; replayable
Ordering	Usually none (FIFO variants limited)	Ordered within a partition
Use for	Task distribution, work queues, retries with DLQ	Event streams, fan-out to many consumers, rebuilding state, CDC

Delivery semantics: at-most-once (may lose), at-least-once (may duplicate — the practical default), exactly-once (achieved as *effectively* once: at-least-once delivery + idempotent processing or transactional offset commits). **Say it plainly:** "End-to-end exactly-once is really at-least-once plus idempotency."

Build a queue from scratch (Google-style): partitioned append-only segment files per topic; producer appends with offset; index file maps offset → byte position; consumers keep committed offsets; replication via leader + followers with an in-sync replica set; ack after replication to a quorum; retention by time/size; consumer groups assign partitions to consumers.

12. Idempotency

Client sends an **idempotency key** (UUID) with each mutating request. Server stores `(key → result)` with a TTL in the same transaction as the side effect; repeats return the stored result. Required for payments, orders, any retried write. Consumers dedupe by `event_id` or use natural idempotency (set status = SHIPPED, not "increment").

13. Distributed transactions: sagas and the outbox pattern

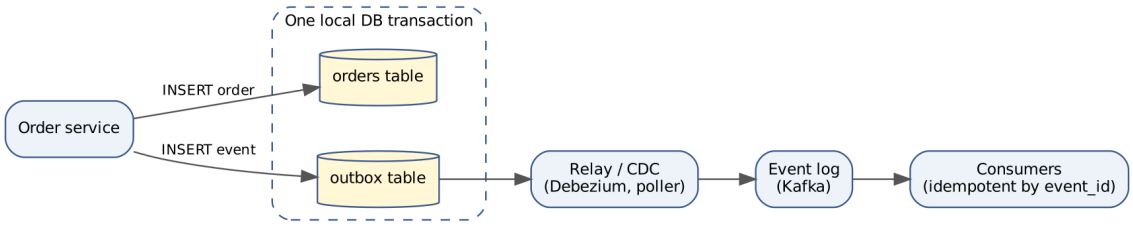


Figure A4.5 — Transactional outbox: the business write and the event are committed atomically in one local transaction; a relay publishes the event afterward.

Problem it solves: "write DB then publish to Kafka" can fail between the two (dual-write problem). Outbox makes it atomic locally; consumers must be idempotent (relay may publish twice).

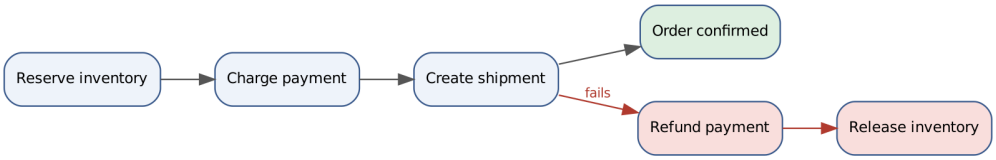


Figure A4.6 — Saga: a sequence of local transactions with compensating actions on failure.

Sagas replace two-phase commit across services. Orchestrated (a coordinator drives steps — easier to reason about and observe) vs choreographed (services react to events — looser coupling, harder to follow). 2PC is acceptable inside one database/cluster; avoid it across services (blocking coordinator, availability cost).

14. Consensus (Raft / Paxos basics)

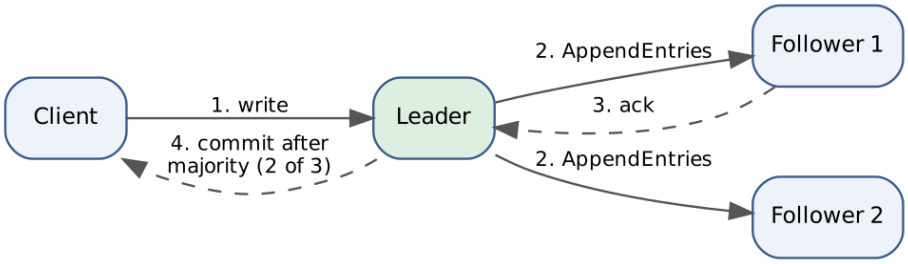


Figure A4.7 — Raft replication: the leader commits an entry once a majority has persisted it.

Used for leader election, configuration, locks, metadata (etcd, ZooKeeper, Chubby, Spanner's Paxos groups). Key points: majority quorum (2 of 3, 3 of 5) tolerates $\lfloor (N-1)/2 \rfloor$ failures; terms/epochs prevent split brain; leader leases for fast reads; keep consensus groups small and data small — use them for *metadata*, not bulk data. **Fencing tokens:** a lock holder that paused (GC) may wake up after losing the lock; storage must reject writes with stale tokens.

15. Rate limiting

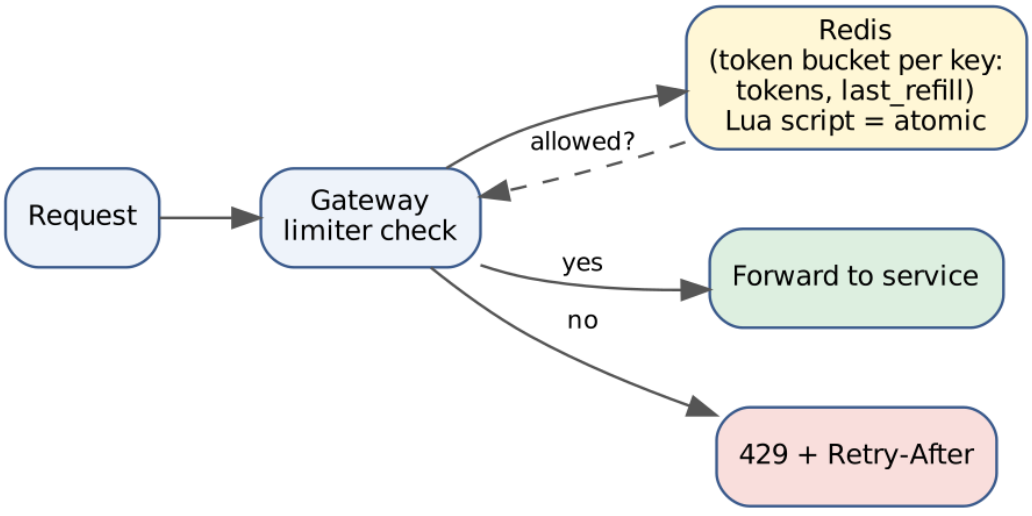


Figure A4.8 — Distributed rate limiter: atomic token-bucket check in Redis via a Lua script.

Algorithm	Behavior	Trade-off
Token bucket	Refill r tokens/sec up to burst b; each request takes one	Allows bursts; the default choice
Leaky bucket	Queue drained at fixed rate	Smooths output; adds latency
Fixed window counter	Count per minute	Simple; 2× burst at window edges
Sliding window log	Store timestamps	Exact; memory heavy
Sliding window counter	Weighted current + previous window	Good accuracy, cheap

At millions of RPS (Netflix-style): local in-memory buckets per instance with periodic sync to a central store (approximate but fast); shard counters by key; fail open vs fail closed is a business decision — say which and why.

16. Unique ID generation

UUIDv4: no coordination, 128-bit, poor index locality. **UUIDv7 / ULID:** time-ordered, good index locality — modern default. **Snowflake:** 64-bit = timestamp (41 bits) + worker ID (10) + sequence (12) → ~4k IDs/ms/worker, sortable; watch clock skew (refuse to issue if the clock goes backward). DB sequences: simple, a bottleneck at scale (allocate ranges/blocks per server).

17. Geo-indexing

Geohash encodes lat/lng into a string prefix; nearby points share prefixes; search the cell and its 8 neighbors. **Quadtree:** recursively split dense areas; good for uneven density. **Google S2 / Uber H3:** sphere-aware hierarchical cells; H3 hexagons have uniform neighbor distances. Use for ride-sharing, delivery, nearby search, regionalization.

18. Real-time delivery: WebSockets, SSE, long polling

WebSockets: bidirectional persistent connections (chat, live comments). Stateful gateways — need a connection registry (user → gateway), heartbeats, reconnect with resume token. **SSE:** server → client stream over HTTP; simpler, auto-reconnect; good for live comments/notifications display. **Long polling:** fallback. **Numbers:** a tuned gateway host holds ~100k–1M idle connections; plan reconnect storms after deploys (jittered backoff, gradual drain).

19. Blob storage and chunking

Split files into chunks (4–8 MB, or content-defined chunking for dedup); store chunks in object storage keyed by content hash; metadata DB holds file → ordered chunk list + versions. Gives resumable uploads, dedup, delta sync (Dropbox-style — reported at Amazon). Upload via pre-signed URLs directly to storage so bytes bypass your service tier.

20. Stream processing

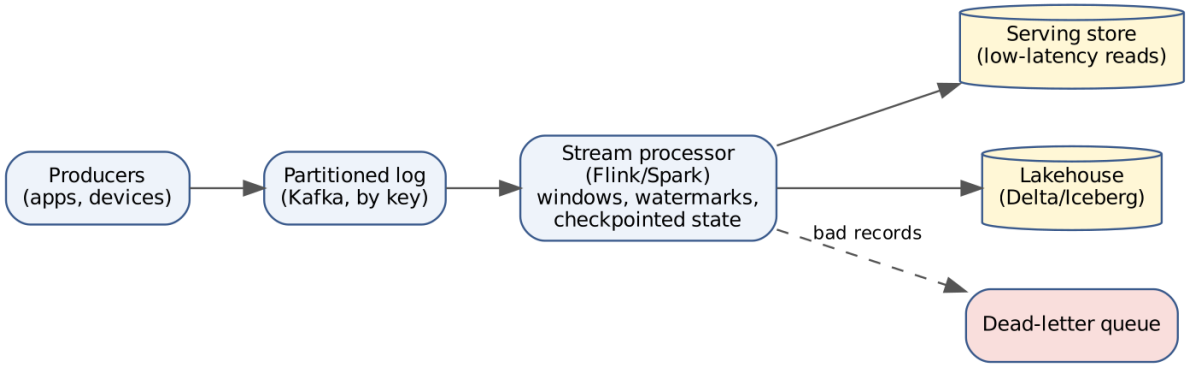


Figure A4.9 — Streaming pipeline with checkpointed state, two sinks and a dead-letter queue.

Event time vs processing time; windows (tumbling, sliding, session); **watermarks** declare "no more events older than T expected," letting windows close; late data handled by allowed lateness or side outputs. **State** (counts, joins) checkpointed to durable storage for recovery; **exactly-once** by checkpoint + transactional/idempotent sinks. Use for click counters (Google: YouTube Shorts clicks), logs/metrics pipelines, fraud detection, ML monitoring.

Batch vs streaming: Lambda architecture = batch for correctness + stream for speed (two codebases); Kappa = everything is a stream, reprocess by replaying the log. Modern lakehouse stacks (Delta + Structured Streaming) blur this: the same table is both a streaming sink and a batch source.

21. Service discovery, circuit breakers, retries

- **Service discovery:** registry (Consul, etcd, Kubernetes DNS/endpoints) + client-side or proxy-side (service mesh) load balancing.
- **Timeouts** on every call, set from the downstream p99 plus margin; total budget propagated (deadline propagation).
- **Retries** only for idempotent operations, with exponential backoff + full jitter, capped, and a **retry budget** (e.g., ≤10% extra load) to avoid retry storms.
- **Circuit breaker:** closed → open after error threshold (fail fast) → half-open probes → closed.
- **Bulkheads:** separate pools per dependency so one slow dependency can't exhaust all threads.
- **Load shedding:** reject low-priority work early under overload (queue-length or latency based); prioritize critical traffic.

22. Multi-region and disaster recovery

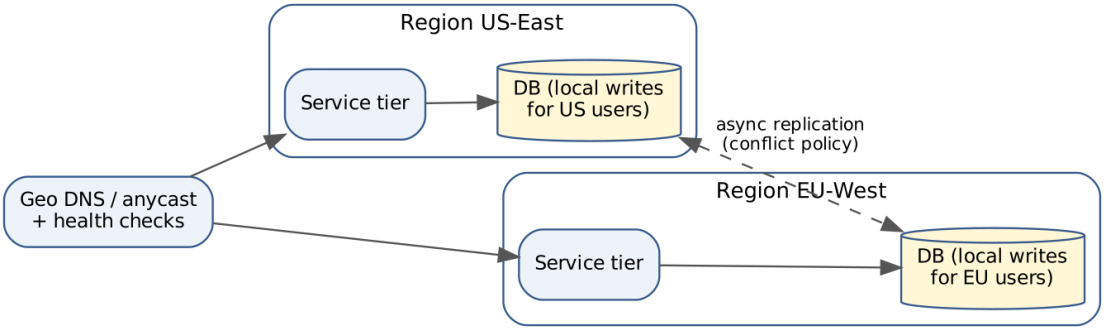


Figure A4.10 — Active-active with users homed to a region; async cross-region replication.

Topology	RPO / RTO	Cost & complexity	When
Backup & restore	Hours / hours	Low	Internal tools
Pilot light / warm standby	Minutes / tens of minutes	Medium	Most business systems
Active-passive (hot)	Seconds–minutes / minutes	High	Customer-facing critical
Active-active	~0 for homed data / seconds	Highest; conflict handling	Global consumer products; residency (EU users homed in EU)

Cell-based architecture (Amazon talks about this publicly): split the service into independent cells, each serving a subset of customers, to limit blast radius. Great "say this" line for Amazon: "I'd make the cell the unit of deployment and failure."

23. Observability

Four golden signals: latency, traffic, errors, saturation. **RED** (rate, errors, duration) for services, **USE** (utilization, saturation, errors) for resources. Define **SLIs** (e.g., % of requests < 300 ms), **SLOs** (99.9% over 28 days), **error budgets** (what you can spend on releases). Page on SLO burn rate (fast and slow burn), not on CPU. Traces with propagated context for cross-service latency. Structured logs with request IDs. This section is what Amazon's Operational Excellence round probes.

A5. Data platform foundations (essential for Databricks, useful everywhere)

Databricks interviewers expect you to reason about distributed compute and storage internals. Amazon (logging, analytics), Google (logs/metrics pipeline) and Meta (metrics) also lean on this material.

Distributed compute: the Spark execution model

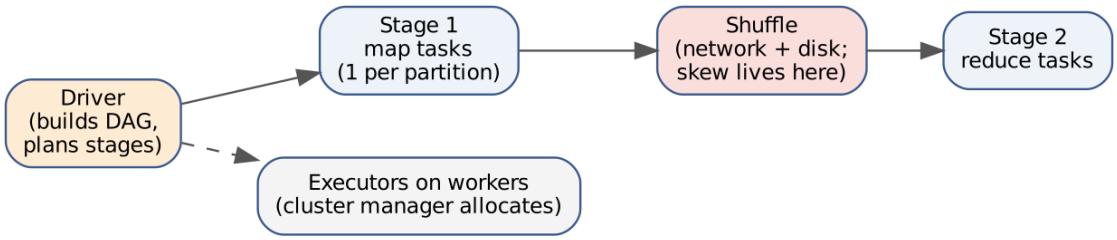


Figure A5.1 — The driver builds a DAG of stages; shuffles split stages and are where skew and cost concentrate.

- **Driver** plans the job: logical plan → optimized plan (Catalyst) → physical plan → DAG of **stages** separated by **shuffles**; each stage runs one **task per partition** on executors.
- **Narrow** transformations (map, filter) stay within a partition; **wide** ones (groupBy, join) shuffle data across the network — the expensive part.
- **Skew:** one hot key → one giant task → the whole stage waits. Fixes: salting the key, broadcast joins for small tables, adaptive query execution (AQE) splitting skewed partitions.
- **Partition sizing:** aim for ~100–200 MB per partition; too many tiny tasks → scheduling overhead; too few → poor parallelism and memory spills.
- **Fault tolerance:** lineage lets lost partitions be recomputed; shuffle files persist so retries don't redo everything.

Columnar formats

Parquet stores data column by column in row groups with min/max statistics per column chunk, dictionary and run-length encoding, and compression. Readers skip columns they don't need (*projection pushdown*) and row groups whose stats exclude the filter (*predicate pushdown*). Result: analytics scans read 5–20× less data than row formats.

Table formats: ACID on object storage (Delta Lake, Iceberg)

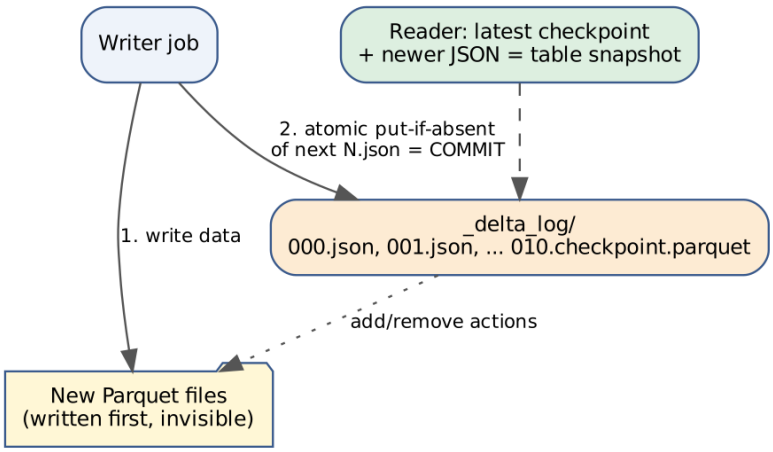


Figure A5.2 — Delta Lake commit protocol: write data files first, then atomically create the next numbered log entry.

- Object stores have no transactions or rename atomicity across many files. A table format adds a **transaction log** that defines which files make up each table version.
- **Delta:** `_delta_log/` holds ordered JSON commits (add/remove file actions, schema, stats) plus periodic Parquet checkpoints. A commit succeeds if the writer creates the next version file first (**put-if-absent**, or a coordinating service where the store lacks it). Conflicts → optimistic concurrency: re-read, check whether the concurrent commit touched the same files, retry or fail.
- **Iceberg:** metadata tree (metadata file → manifest lists → manifests → data files); atomic swap of a pointer in a catalog.
- Features you should name: time travel (read version N), schema evolution, MERGE/upserts, compaction (OPTIMIZE) of small files, data skipping via stats, Z-ordering / liquid clustering for multi-column locality.

Lakehouse vs warehouse vs lake

	Data lake	Warehouse	Lakehouse
Storage	Open files on object storage	Proprietary managed storage	Open files + table format on object storage
Transactions	None	Full	ACID via the log
Workloads	ML, raw data, cheap	BI/SQL, fast	BI + ML + streaming on one copy
Weakness	"Data swamp," no consistency	Cost, lock-in, data copies	Small-file management, metadata scale

Object-store internals (Databricks "design an object store")

- **Split data plane and metadata plane.** Metadata service: bucket/key → object version → list of chunk locations; must support listing by prefix (range-partitioned, sorted keys) and strong read-after-write consistency.
- **Data plane:** storage nodes hold chunks; durability via replication (3×) for hot/small data or **erasure coding** (e.g., 10 data + 4 parity: survives 4 losses at 1.4× overhead) for large/cold data.
- **Writes:** client streams chunks → storage nodes acknowledge → metadata commit makes the object visible atomically. Multipart upload for large objects.
- **Background:** scrubbing for bit rot (checksums), repair when nodes die, garbage collection of orphan chunks, rebalancing.
- **Hot spots:** sequential key names concentrate on one metadata partition → hash-prefix or auto-split partitions.

Query engines and optimizers

Parse → logical plan → rule-based rewrites (predicate pushdown, column pruning, constant folding) → cost-based optimization using statistics (join order, join strategy: broadcast hash vs shuffle hash vs sort-merge) → physical plan → vectorized execution (process column batches with SIMD; Databricks Photon is a vectorized engine). Adaptive execution re-plans at runtime using actual stage statistics.

Streaming correctness

Exactly-once in Structured Streaming / Flink: source offsets + state checkpointed together; sink is idempotent or transactional (Delta writes keyed by batch ID are idempotent). Watermarks bound state size for aggregations and stream-stream joins.

Governance and lineage

Central catalog (e.g., Unity Catalog, Glue, Purview) defines a namespace (catalog.schema.table), permissions (grants on tables, columns, rows), audit logs, and lineage (which jobs produced which tables). Row/column-level security and attribute-based policies. For your insurance background: PII tagging, policyholder data masking, retention rules — a credible tie-in.

Multi-tenant platforms: control plane vs data plane

- **Control plane** (vendor-run): user management, notebooks, job scheduling, cluster lifecycle API, metadata.
- **Data plane** (often in the customer's cloud account): compute clusters reading/writing customer storage. Serverless variants run data plane in vendor accounts with strong isolation.
- Design concerns: tenant isolation (network, credentials, noisy neighbors), quotas, per-tenant encryption keys, regional deployment for residency, blast-radius limits (cells), upgrade orchestration across thousands of workspaces.
- **Cost per workload:** autoscaling on queue depth/backlog, spot/preemptible instances with graceful decommissioning, auto-termination of idle clusters, caching hot data on local SSD.

A6. LLM and ML systems foundations

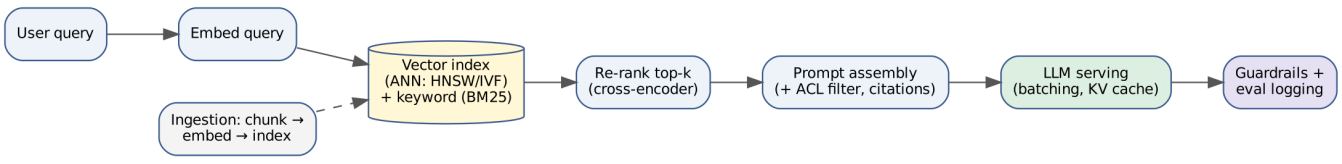


Figure A6.1 — Retrieval-augmented generation (RAG) serving path, with offline ingestion feeding the index.

Model serving

- **Latency anatomy:** time to first token (prefill, compute-bound, grows with prompt length) + time per output token (decode, memory-bandwidth-bound).
- **Throughput levers:** continuous batching (add/remove sequences mid-batch), KV-cache management (paged attention), quantization (8-bit/4-bit), speculative decoding, prompt/prefix caching for shared system prompts.
- **Capacity planning:** tokens/sec per GPU at target latency × peak concurrent requests × average output length → GPU count; add headroom for spikes; route by model size; fall back to smaller model under load.
- **Routing:** small/cheap model first, escalate to bigger model when needed (cascades); separate interactive and batch pools.

RAG specifics

- **Ingestion:** parse → chunk (semantic or ~300–800 tokens with overlap) → embed → index with metadata (source, ACLs, timestamps). Re-embed on model change (versioned indexes).
- **Retrieval:** hybrid (vector ANN + keyword BM25), then re-rank; filter by user permissions *before* the model sees content (security!).
- **ANN indexes:** HNSW (graph, high recall, memory heavy), IVF/PQ (clustered + compressed, cheaper memory).
- **Grounding:** require citations; refuse when retrieval confidence is low.

Evaluation, monitoring, guardrails

- **Offline evals:** golden datasets, LLM-as-judge with human calibration, task metrics (accuracy, faithfulness, retrieval recall@k).
- **Online:** user feedback, A/B tests, latency/cost dashboards, refusal rates.
- **Model monitoring** (Google reported "monitor ML model performance in real time"): input feature drift (PSI, KL divergence), prediction distribution shift, delayed ground-truth labels joined back for accuracy, segment-level slicing, alert thresholds, shadow and canary deployments, automatic rollback.
- **Guardrails:** input filters (prompt injection, PII), output filters (policy, toxicity), tool-use permissions, rate limits per user, audit logs.
- **Feature store:** offline store (training, point-in-time correct joins) + online store (low-latency serving) built from the same definitions to avoid training/serving skew.
- **Cost per request:** (prompt tokens × input price) + (output tokens × output price) + retrieval cost; reduce with caching, shorter prompts, smaller models, batching.

Your tie-in: intelligent document processing in insurance is a real applied-ML system — classification, extraction, confidence thresholds, human-in-the-loop, feedback for retraining. Use it.

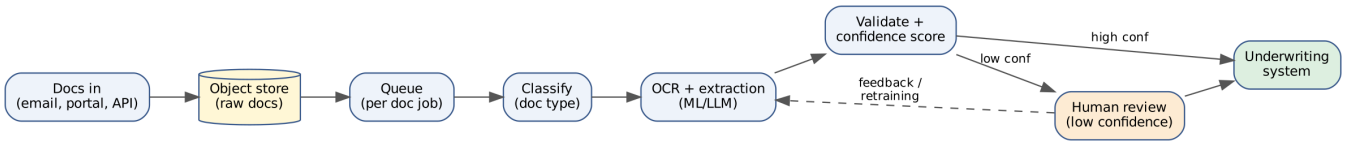


Figure A6.2 — Template for describing your document-processing system: confidence-based routing to human review with a feedback loop. Fill in your real components and numbers [VERIFY].

A7. Numbers cheat sheet and estimation

Latency numbers (orders of magnitude)

Operation	Approx.
L1 cache / main memory reference	~1 ns / ~100 ns
Read 1 MB sequentially from memory	~10–50 µs
SSD random read	~100 µs
Read 1 MB sequentially from SSD	~0.5–1 ms
Round trip within a data center / AZ	~0.5 ms
Cross-AZ round trip	~1–2 ms
HDD seek	~5–10 ms
Cross-continent round trip (US–EU)	~70–150 ms

Throughput rules of thumb

Component	Rough capacity per node
Stateless web/app server	1k–10k RPS (depends on work per request)
Redis	~100k+ ops/sec
Postgres/MySQL (tuned, simple queries)	~5k–20k QPS reads; fewer writes; scale reads with replicas
Kafka broker	Hundreds of MB/s; partitions ~10 MB/s each comfortably
Cassandra node	~10k–50k writes/sec
WebSocket gateway	~100k–1M idle connections

Handy conversions

- 1 day ≈ 86,400 s ≈ **10^5 s** for estimation. 1M requests/day ≈ **12 RPS**. 100M/day ≈ 1,200 RPS.
- Peak ≈ **2–3× average** for consumer traffic (higher for events/launches).
- 1 KB × 1B rows = 1 TB. 1 TB/day ≈ 365 TB/yr; ×3 replication ≈ 1.1 PB.

- Bandwidth: RPS × payload. 10k RPS × 100 KB = 1 GB/s.
- Availability: 99.9% = ~43 min/month down; 99.99% = ~4.3 min/month; 99.999% = ~26 s/month.

Worked estimations

1. Real-time comments (Meta). 2B DAU; 5% comment/day, avg 2 comments → 200M comments/day ≈ 2.3k writes/s avg, ~7k peak; a single viral live post may get 50k comments/min (~800/s) to 1M+ concurrent viewers. Reads dominate: 1M viewers × 1 update batch/2 s = 500k pushes/s for one hot post → you need fan-out via a pub/sub tier and batching, not per-comment pushes. Storage: 200M × 500 B ≈ 100 GB/day.

2. Distributed logging (Amazon). 50k hosts × 1 KB/line × 100 lines/s = 5 GB/s ≈ 430 TB/day raw. Compression ~10× → 43 TB/day. Hot index 7 days (~300 TB), warm/cold to object storage for 90 days–7 years. Ingest via partitioned log (~500 partitions at 10 MB/s each).

3. Firmware/OTA updates (Amazon). 100M devices; 500 MB image; staged over 7 days → 50 PB of transfer total if all download full images → must use CDN + delta updates (maybe 50 MB) → 5 PB; ~8 GB/s average, more at wave peaks. Control plane: device check-ins every 6 h ≈ 4.6k RPS.

4. Click counter for short videos (Google). 50B views/day ≈ 580k events/s avg, ~1.5M peak. Aggregate in stream processor (per video per minute), write ~10M active videos × 1 row/min ≈ 170k writes/s to the serving store — or batch further. Approximate counts fine for display; exact for monetization path.

5. Object store metadata (Databricks). 100B objects × 300 B metadata ≈ 30 TB metadata → must be a sharded, range-partitioned store; 1M metadata ops/s at peak → hundreds of partitions.

A8. The Manager Lens playbook

Weave these in at decision points — not as a closing speech. Each is one or two sentences in the room.

Lens	What to say (example)	When to say it
Team topology	"I'd split ownership along the write path and read path: ingestion team, serving team, and a platform team for the shared log. That boundary matches the API between them."	After the high-level diagram
Conway's law	"I'd avoid splitting this service across two teams; the coupling would show up as coordination cost."	When drawing service boundaries
Build vs buy	"For the queue I'd use a managed service; it's undifferentiated. The ranking logic is where we differentiate, so we build that."	When choosing infrastructure
Phasing	"MVP in one quarter: single region, polling instead of push. GA adds push and multi-AZ. Scale phase adds multi-region."	End of high-level design

Migration	"We can't big-bang this. Dual-write, backfill, shadow-read compare, then cut over by cohort with a kill switch."	Any existing-system problem
Risk	"Top risk is hot partitions on viral content; I'd run a load test with synthetic skew in week three."	Deep dive close
Cost	"Storage dominates cost; tiering to cold storage after 30 days cuts it ~70%."	Estimation or ops section
Ops readiness	"Before launch: SLOs agreed, runbooks, on-call trained, load test at 2× peak, game day."	Ops section
Decide vs delegate	"I'd own the consistency decision and the API contract; schema details and library choices I'd delegate to the tech lead."	When asked about your role
Success metrics	"I'd measure p99 latency, delivery success rate, and cost per million events — and a business metric like comment engagement."	Summary

Leading a design review where two senior engineers disagree

Interviewers (Amazon especially) ask this. A strong answer:

1. **Restate both options and the decision criteria** before discussing preferences (latency, cost, operability, reversibility, team skills).
2. **Separate one-way doors from two-way doors.** Reversible decisions: decide fast, let the owner choose. Irreversible ones (data model, public API): slow down, prototype or spike.
3. **Ask for data:** a time-boxed spike or benchmark on the contested point.
4. **Decide and document:** write a short decision record (context, options, decision, consequences). Name a decider.
5. **Disagree and commit:** the engineer whose option lost gets credit for raising the risks and often owns the mitigation.

A9. Low-level / object-oriented design

Apple's second design round, Microsoft's coding-adjacent rounds and Databricks' "systems" round can go down to classes, interfaces and concurrency. For a manager, the bar is clean decomposition, correct concurrency, and clear API contracts — not clever tricks.

Principles to name out loud

- **Single responsibility** and small interfaces; depend on abstractions (pass a `Clock` or `Storage` interface so components are testable).
- **Immutability** for value objects; mutable state behind one owner.
- **Patterns worth naming:** Strategy (pluggable eviction or pricing), Observer (events), Factory, Decorator (add caching/metrics around a client), State (order lifecycle).

- **Concurrency primitives:** mutex/lock, read-write lock, condition variables, atomic counters, compare-and-swap. Rule: hold locks briefly; never call external services while holding a lock; acquire multiple locks in a global order to avoid deadlock.
- **API contracts:** pre/postconditions, error types, idempotency, thread-safety documented.

Example 1 — LRU cache (thread-safe)

O(1) get/put via a hashmap pointing into a doubly linked list ordered by recency.

```
import threading

class _Node:
    __slots__ = ("key", "val", "prev", "next")
    def __init__(self, key=None, val=None):
        self.key, self.val, self.prev, self.next = key, val, None, None

class LRUCache:
    def __init__(self, capacity: int):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.cap = capacity
        self.map = {}
        self.head, self.tail = _Node(), _Node()   # sentinels
        self.head.next, self.tail.prev = self.tail, self.head
        self.lock = threading.Lock()

    def _remove(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _add_front(self, n):
        n.next, n.prev = self.head.next, self.head
        self.head.next.prev = n
        self.head.next = n

    def get(self, key):
        with self.lock:
            n = self.map.get(key)
            if n is None:
                return None
            self._remove(n); self._add_front(n)
            return n.val

    def put(self, key, val):
        with self.lock:
            n = self.map.get(key)
            if n:
                n.val = val
                self._remove(n); self._add_front(n)
                return
            if len(self.map) == self.cap:
                lru = self.tail.prev
                self._remove(lru)
                del self.map[lru.key]
            n = _Node(key, val)
            self.map[key] = n
            self._add_front(n)
```

Follow-ups to expect: TTLs (store expiry, check on get, lazy + periodic sweep); LFU instead; sharding the lock (N segments by key hash) to reduce contention; making it distributed (consistent hashing across nodes, see A4).

Example 2 — Token-bucket rate limiter

```
import time, threading

class TokenBucket:
    def __init__(self, rate_per_sec: float, burst: int, clock=time.monotonic):
        self.rate, self.capacity, self.clock = rate_per_sec, burst, clock
        self.tokens, self.last = float(burst), clock()
        self.lock = threading.Lock()

    def allow(self, cost: int = 1) -> bool:
        with self.lock:
            now = self.clock()
            self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
            self.last = now
            if self.tokens >= cost:
                self.tokens -= cost
                return True
            return False

class RateLimiter:
    """One bucket per client key."""
    def __init__(self, rate, burst):
        self.rate, self.burst = rate, burst
        self.buckets, self.lock = {}, threading.Lock()

    def allow(self, key) -> bool:
        with self.lock:
            b = self.buckets.get(key)
            if b is None:
                b = self.buckets[key] = TokenBucket(self.rate, self.burst)
        return b.allow()
```

Points to make: injected clock for testability; `monotonic` not wall time (immune to clock changes); unbounded `buckets` map needs eviction (LRU from Example 1) for memory safety; distributed version moves state to Redis with an atomic script (Figure A4.8).

Example 3 — Bounded thread-safe blocking queue

```
import threading
from collections import deque

class BoundedQueue:
    def __init__(self, capacity: int):
        self.cap, self.items = capacity, deque()
        self.lock = threading.Lock()
        self.not_full = threading.Condition(self.lock)
        self.not_empty = threading.Condition(self.lock)

    def put(self, item, timeout=None) -> bool:
        with self.not_full:
            if not self.not_full.wait_for(lambda: len(self.items) < self.cap, timeout):
                return False                      # backpressure: caller decides
            self.items.append(item)
            self.not_empty.notify()
            return True

    def get(self, timeout=None):
        with self.not_empty:
            if not self.not_empty.wait_for(lambda: self.items, timeout):
                raise TimeoutError
            item = self.items.popleft()
            self.not_full.notify()
            return item
```

Points to make: conditions are re-checked in a loop (`wait_for`) because of spurious wakeups; bounded capacity = backpressure; this is the in-process version of the queues in A4 §11.

Example 4 — Booking / reservation system (no double booking)

```
from dataclasses import dataclass
from enum import Enum
import threading, uuid

class Status(Enum):
    HELD = "held"; CONFIRMED = "confirmed"; CANCELLED = "cancelled"

@dataclass
class Reservation:
    id: str; resource_id: str; slot: int; user_id: str; status: Status

class BookingService:
    def __init__(self, repo, payments, clock):
        self.repo, self.payments, self.clock = repo, payments, clock
        self._locks = {}                      # per (resource, slot) in-process locks
        self._guard = threading.Lock()

    def _lock_for(self, k):
        with self._guard:
            return self._locks.setdefault(k, threading.Lock())

    def hold(self, resource_id, slot, user_id) -> Reservation:
        with self._lock_for((resource_id, slot)):
            if self.repo.active_reservation(resource_id, slot):
                raise ValueError("slot taken")
            r = Reservation(str(uuid.uuid4()), resource_id, slot, user_id, Status.HELD)
            self.repo.save(r, hold_expires_at=self.clock() + 600)
            return r

    def confirm(self, reservation_id, idempotency_key):
        r = self.repo.get(reservation_id)
        if r.status is Status.CONFIRMED:
            return r                          # idempotent
        self.payments.charge(r.user_id, idempotency_key=idempotency_key)
        r.status = Status.CONFIRMED
        self.repo.save(r)
        return r
```

Say this: "In-process locks only work on one node. In production the real guarantee is a **unique constraint** on (resource_id, slot) for active reservations in the database, or a conditional write (DynamoDB condition expression). The lock is an optimization, the constraint is the correctness." Also: hold expiry job; State pattern for status transitions; idempotent confirm and payment.

Example 5 — Minimal key-value storage engine (append-only log + in-memory index)

```
import os, json

class LogKV:
    """Bitcask-style: append-only data file, in-memory key -> offset index."""
    def __init__(self, path):
        self.path, self.index = path, {}
        open(path, "a").close()
        self._rebuild()
        self.f = open(path, "a+b")

    def _rebuild(self):
        with open(self.path, "rb") as f:
            off = 0
            for line in f:
                rec = json.loads(line)
                if rec.get("del"):
                    self.index.pop(rec["k"], None)
                else:
                    self.index[rec["k"]] = off
                off += len(line)

    def put(self, k, v):
        line = (json.dumps({"k": k, "v": v}) + "\n").encode()
        self.f.seek(0, os.SEEK_END)
        off = self.f.tell()
        self.f.write(line); self.f.flush(); os.fsync(self.f.fileno())   # durable before ack
        self.index[k] = off

    def get(self, k):
        off = self.index.get(k)
        if off is None:
            return None
        self.f.seek(off)
        return json.loads(self.f.readline())["v"]

    def delete(self, k):
        line = (json.dumps({"k": k, "del": True}) + "\n").encode()
        self.f.seek(0, os.SEEK_END)
        self.f.write(line); self.f.flush(); os.fsync(self.f.fileno())
        self.index.pop(k, None)
```

Follow-ups (Databricks/Apple love these): compaction (rewrite live keys to a new file, swap atomically); crash in the middle of a write (torn last line — validate with a checksum per record and truncate); index too big for memory (sparse index + sorted segments → you've reinvented an LSM tree, A4 §9); concurrency (single writer thread, many readers); fsync batching (group commit) for throughput.

A10. Code review and coding round prep

How to read unfamiliar code in an interview

1. **Get the contract first:** what is this function supposed to do? Inputs, outputs, side effects. Ask if unclear.
2. **Trace one happy path** with a concrete input, out loud.
3. **Walk the checklist below** section by section, noting issues by severity.
4. **Prioritize:** correctness and security bugs first, then performance, then readability. Say which are blocking.
5. **Propose fixes**, not just complaints, and mention the test you'd add for each bug.

Bug-hunting checklist

Area	Look for
Correctness	Off-by-one, wrong comparison operator, integer division, mutation of inputs, wrong default values, swapped arguments
Edge cases	Empty input, single element, duplicates, None/null, very large values, unicode, negative numbers
Error handling	Swallowed exceptions, resources not closed, partial failure leaving inconsistent state
Concurrency	Shared mutable state without locks, check-then-act races, deadlock order, non-atomic read-modify-write
Complexity	Nested loops on large inputs, repeated lookups in lists, string concatenation in loops, N+1 queries
Security	Injection (SQL/command), secrets in code/logs, missing authz checks, unvalidated input, unsafe deserialization
Reliability	Missing timeouts, retries on non-idempotent calls, unbounded caches/queues
Readability	Names, function length, magic numbers, missing tests

Practice snippets (find the issues, then check the answers)

Snippet 1 — pagination helper

```
def paginate(items, page, page_size):
    start = page * page_size
    end = start + page_size - 1
    return items[start:end]
```

> **Answer:** Off-by-one: Python slices are end-exclusive, so `- 1` drops the last item of every page. Also unclear whether `page` is 0- or 1-based (document it); no validation for negative page or page_size ≤ 0.

Fix: `end = start + page_size`; validate inputs; add tests for the first page, last partial page, and an empty list.

Snippet 2 — account transfer

```
def transfer(db, from_id, to_id, amount):
    bal = db.query(f"SELECT balance FROM accounts WHERE id = {from_id}")
    if bal >= amount:
        db.execute(f"UPDATE accounts SET balance = balance - {amount} WHERE id = {from_id}")
        db.execute(f"UPDATE accounts SET balance = balance + {amount} WHERE id = {to_id}")
```

Answer: (1) **SQL injection** via f-strings — use parameterized queries. (2) **Race condition**: check-then-act; two concurrent transfers can both pass the check and overdraw. Use a transaction with `SELECT … FOR UPDATE` or a conditional update `WHERE id=? AND balance >= ?` and check rows affected. (3) **No atomicity**: a crash between the two updates loses money — wrap in one transaction. (4) No validation that `amount > 0` or that accounts differ. (5) Floating-point money if amount is a float — use integer cents or Decimal. (6) No idempotency key for retries. (7) Silent no-op on insufficient funds — return or raise an error.

Snippet 3 — caching user profiles

```
_cache = {}

def get_profile(user_id):
    if user_id in _cache:
        return _cache[user_id]
    profile = http_get(f"https://profiles/api/{user_id}")
    _cache[user_id] = profile
    return profile
```

Answer: Unbounded cache → memory leak (use LRU with max size). No TTL → stale profiles forever. Errors are cached too if `http_get` returns an error payload. No timeout on the HTTP call. Concurrent misses for the same user cause a stampede (single-flight). Module-level global hurts testability (inject the cache and client). Possibly caching PII without considering access rules.

Snippet 4 — counter shared across threads

```
class Stats:
    def __init__(self):
        self.count = 0
        self.lock = threading.Lock()

    def incr(self):
        self.count += 1

    def snapshot_and_reset(self):
        with self.lock:
            c = self.count
            self.count = 0
        return c
```

> **Answer:** `incr` doesn't take the lock; `+=` is a read-modify-write and is not atomic, so increments can be lost, and an increment can race with the reset. Fix: wrap `incr` in the same lock (or use per-thread counters summed on snapshot for high throughput).

Snippet 5 — find duplicates

```
def find_duplicates(nums):
    dups = []
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j] and nums[i] not in dups:
                dups.append(nums[i])
    return dups
```

> **Answer:** Correct but O(n²) comparisons plus O(k) `not in dups` list checks. With 1M items this never finishes. Fix: one pass with a `seen` set and a `dups` set → O(n). Mention whether output order matters (preserve first-seen order if needed) and test with empty list, all-unique, all-same.

Systems-programming questions under time pressure

Databricks candidates have reported a systems-programming question that felt too large for an hour. Strategy:

1. Spend 3–5 minutes agreeing on a **reduced scope**: "In an hour I'll build the core path — single process, in-memory index, durable log. I'll describe compaction and concurrency rather than code them. Does that work?"
2. Write the **interfaces first** (class and method signatures), then fill the core method.
3. Narrate what you're skipping and how you'd do it.
4. Leave five minutes to walk through a test case and failure scenarios.

Working with AI coding assistants (when permitted)

Meta loops have been reported to include AI-assisted coding and Apple has allowed AI tools in a CoderPad round; Amazon prohibits them. Always ask the rules first. When allowed:

- Use it for boilerplate and syntax recall, not for the core algorithm you're being evaluated on.
- **Read every line it produces aloud** and correct it; interviewers are watching your judgment of generated code — this is effectively a code-review round.
- Write the tests yourself; ask the assistant to explain or refactor, then verify.
- Say when output is wrong: "This misses the empty-input case; I'll fix that."

Part II — Company question bank (Q1–Q27)

These are the questions reported at your target companies in 2025–2026, organized by company with primary targets first: **Amazon, Apple, Meta, Microsoft, Databricks**, then Google and Netflix. Each chapter opens with a one-page profile — loop structure, how design is scored, Strong Hire signals, pitfalls — and a question bank with confidence labels and sources.

How to use this part: work through the top-20 problems from Part IV during your main prep. Then, in the week before each company's loop, re-read that company's chapter: the profile, its walkthroughs, and the "say this" lines.

Problems asked at several companies appear once, under the company where they're most reported. The company matrix in Part IV shows where else each one comes up.

Chapter 1 — Amazon

Amazon at a glance

Loop	Recruiter → 60-min hiring manager screen (Leadership Principles + system design) → written narrative (~2 pages, two prompts to choose from) → ~5 one-hour rounds: System Design, People Management, Operational Excellence, Technical Program/Product Management; one is run by the Bar Raiser
Coding	Rare for SDMs (reported in under 6% of SDM loops); some L6 loops add it — confirm with your recruiter
How design is scored	End-to-end architecture first, then depth where the interviewer pulls you. Failure thinking unprompted. Leadership Principles are scored inside the design round too
Strong Hire signals	Numbers drive decisions; operational excellence volunteered (alarms, on-call, correction-of-error mechanisms); "I decided X because..."; cost awareness (Frugality); customer framing (Customer Obsession)
Pitfalls	Diving into one component early; "we" without "I"; thin operations story; designs that don't hold at scale
AWS literacy	Expected. Name the AWS service *and* explain the mechanism underneath
AI tools	Prohibited during interviews

Amazon question bank (reported, 2025–2026 unless noted)

#	Question	Confidence	Source
Q1	Walk through a system your team owned; where does it break at 10×? (reverse design)	High	Aced SDM guide 2026; Exponent Amazon SD guide 2026
Q2	Design a distributed logging system	Medium	Aced SDM guide 2026
Q3	Design a system that delivers firmware updates to devices	Medium	Aced SDM guide 2026
Q4	Design a Dropbox clone	Medium	Glassdoor, senior SWE, Jul 2026
Q5	Design a parking payment system	Medium	IGotAnOffer: repeated across roles on Glassdoor (2026 update)
Q6	Software behind an Amazon pickup location with lockers	Low	IGotAnOffer SDM list (older)

#	Question	Confidence	Source
Q7	Promotion giving 10% cash back on a credit card / coupon system	Low–Medium	IGotAnOffer SDM list; Exponent (LLD, L4)
Q8	Implement Alexa voice search	Medium	Aced SDM guide 2026
—	ATM system; shopping cart under 10× spikes; rate limiter; distributed cache; URL shortener; social network	Low	Glassdoor / aggregators — covered by patterns in other chapters

> **Amazon-specific habit:** in every walkthrough below, notice three moves you must make unprompted: (1) the full architecture on the board before any deep dive; (2) an operations section with specific alarms and a correction-of-error mechanism; (3) one sentence tying a decision to a Leadership Principle in plain words, without naming the principle ("we'd rather a customer wait 2 seconds than get double-charged").

Q1. Reverse design: "Walk me through a system you own. Where does it break at 10×?"

> Amazon (High) · also Databricks "technical fit", Apple, Meta project retro · Sources: Aced Amazon SDM guide (2026); Exponent Amazon system design guide (2026)

This is the most important problem in the book for you, because it's about *your* system. The walkthrough uses your intelligent document processing (IDP) system as the example. Replace every [VERIFY] with your real details before you rehearse.

1. The prompt

"Pick a system your team owns. Draw it. Tell me where it breaks if traffic grows 10×, and what you'd change." Variants: "What would you redesign if you started over?", "What was the hardest technical decision on that system?", "Walk me through your most severe outage on it."

2. Clarify

This one flips the usual flow: *you* hold the facts. Your clarifying move is to agree scope.

Ask:

- "Should I go end to end or focus on one subsystem?" — tells you how deep they want to go.
- "Is 10× in volume, in tenants, or in latency expectations?" — each breaks different parts.

Don't:

- Don't pick a system you only oversaw from a distance. You will be asked about partition keys and alarms.
- Don't pick a system where you can't share numbers. Approximate is fine ("on the order of 50k documents a day"); vague isn't.

3–6. Requirements, estimates, API and data model (state them for *your* system)

Prepare this as a one-minute fact sheet you can recite:

Item	Your system (example shape — fill in real values)
Purpose	Ingest submissions (applications, loss runs, schedules) and extract structured fields for underwriting [VERIFY]
Users	Underwriters, brokers via portal/email, downstream rating systems [VERIFY]
Volume today	e.g., 20k documents/day, peak 3× at quarter end; avg 12 pages [VERIFY]

Item	Your system (example shape — fill in real values)
Latency target	e.g., extracted fields within 15 minutes for 95% of submissions [VERIFY]
Accuracy target	e.g., ≥95% field accuracy on auto-processed docs; low-confidence docs routed to humans [VERIFY]
Core APIs	`POST /submissions` (upload), `GET /submissions/{id}/extraction`, webhook to underwriting [VERIFY]
Data model	Submission (id, broker, line of business, status), Document (id, type, pages, storage URI), Extraction (field, value, confidence, model version, reviewer) [VERIFY]
Stack	e.g., S3, SQS, Lambda/containers, OCR + ML/LLM extraction, Postgres, workflow engine [VERIFY]

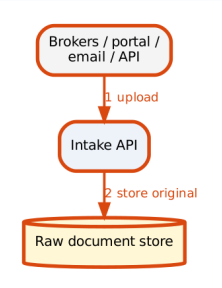
Estimate the 10× point out loud: 20k docs/day × 12 pages = 240k pages/day ≈ 3 pages/s average, ~10/s at peak. At 10× that's ~100 pages/s at peak. If OCR + extraction takes ~2 s per page on one worker, you need ~200 concurrent workers at 10× peak, versus ~20 today. That's what tells you where it breaks.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

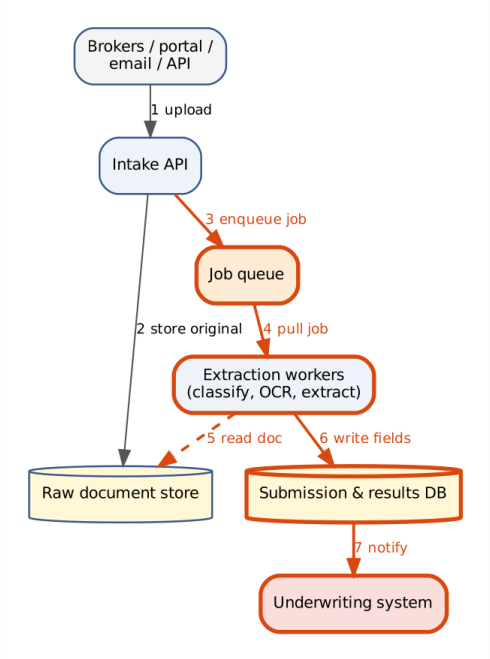
Step 1 · Intake and durable storage (1 min)



> **You:** “Documents arrive from brokers through the portal, email and an API. The intake API validates and gives each submission an ID, and the first thing we do is store the original in object storage. Store first, process later.”

Why this step now: Start with what the system receives and the one thing it must never do — lose a document.

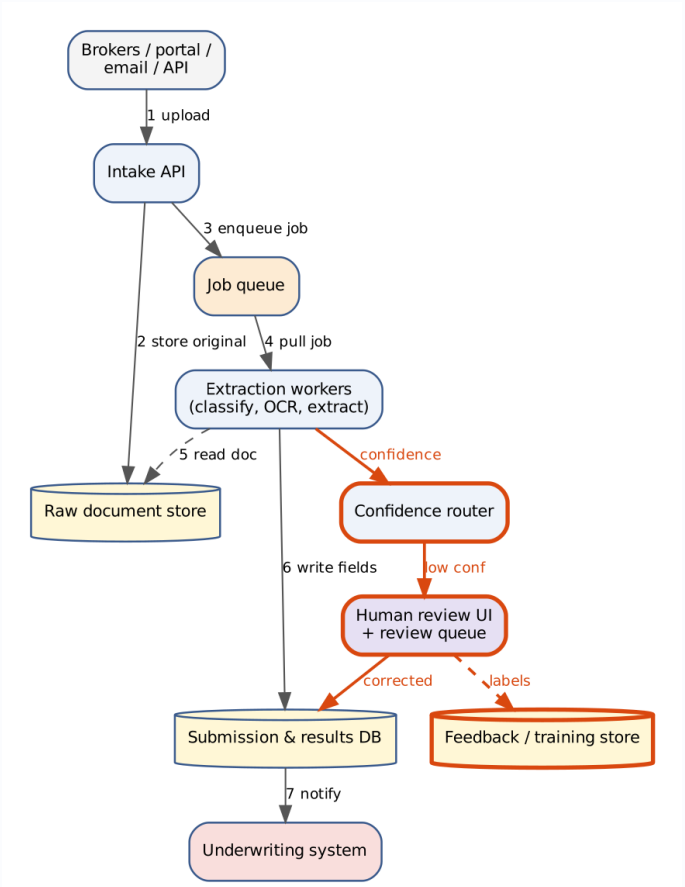
Step 2 · Async processing pipeline (2 min)



> **You:** “Intake enqueues one job per document. Workers pull jobs, classify the document type, run OCR and extraction, and write fields with confidence scores and the model version to the results database. Underwriting gets notified when a submission is ready.”

Why this step now: This is the core value path; draw it before any optimization.

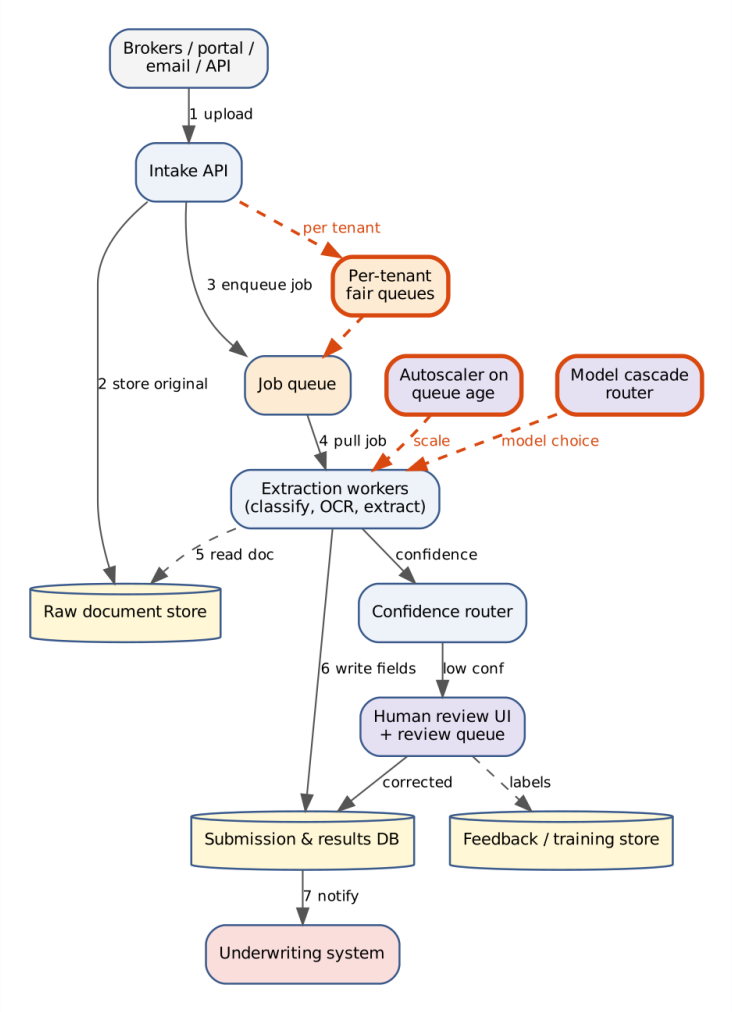
Step 3 · Human-in-the-loop and feedback (1–2 min)



> **You:** “A confidence router decides: high confidence goes straight through, low confidence goes to a review queue. Reviewers' corrections update the record and become training data, which is how the auto-processing rate improves.”

Why this step now: Shows you understand the real system is people plus models.

Step 4 · What changes at 10× (2–3 min)



> **You:** “At ten times the volume, three things change: workers autoscale on queue age rather than CPU, a model cascade sends easy documents to a cheaper model to control cost, and per-tenant fair queues stop one broker's bulk upload from starving everyone else.”

Why this step now: The question is about 10× — only now do you add the scaling pieces, each tied to a specific break.

Narrate one request end to end (numbers match the arrows)

Happy path (high-confidence document)

1. Broker uploads via portal or API (1).
2. Intake API stores the original in object storage before anything else (2).
3. Intake enqueues a job with the submission and document IDs (3).
4. A worker pulls the job (4), reads the file (5), classifies, OCRs and extracts.
5. Worker writes fields, confidence and model version to the results DB (6).
6. Underwriting is notified that the submission is ready (7).

Low-confidence path

1. Router sees confidence below threshold for a document type.
2. Document lands in the review queue; a reviewer corrects fields in the UI.
3. Corrected fields go to the DB; the correction is saved as a training label.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Brokers / portal / email / API	Submit documents	Real entry points; volume spikes at quarter end	Portal, email ingestion, REST
Intake API	Authenticate, validate, assign submission ID	Single front door; idempotent uploads	API Gateway + service
Raw document store	Durable copy of every original file	Store first, process later: nothing is lost if processing fails	S3 (versioned, encrypted)
Job queue	One job per document	Decouples intake from slow extraction; absorbs spikes	SQS / Kafka
Extraction workers (classify, OCR, extract)	Classify doc type, OCR, extract fields with confidence	The core work; scales horizontally	Containers / Lambda + OCR + ML/LLM
Submission & results DB	Submission status, extracted fields, model version	Source of truth for downstream and audit	Postgres / Aurora
Underwriting system	Consumes structured data	The customer of this pipeline	Internal system via events/webhook
Confidence router	Send low-confidence docs to humans	Accuracy vs reviewer cost is a tunable policy	Rules in worker / workflow step
Human review UI + review queue	Reviewers correct fields	Keeps accuracy high where models are unsure	Internal web app + queue
Feedback / training store	Corrections become labeled data	Raises auto-processing rate over time	S3 + labeling dataset
Autoscaler on queue age	Scale workers by oldest job age	At 10x, CPU-based scaling lags; queue age tracks SLA	KEDA / custom metric
Model cascade router	Cheap model first, expensive model for hard docs	Cost is the first thing to break at 10x	Routing policy per doc type
Per-tenant fair queues	Isolate big brokers	One broker's 50k-doc dump shouldn't stall everyone	Weighted fair queuing

Before you leave the high-level design, check you have said:

☐ How the 10× number was derived (math)
☐ The order things break: cost → reviewers → DB → hot tenants
☐ What I personally decided vs what the team built

60-second summary to say once the diagram is complete:

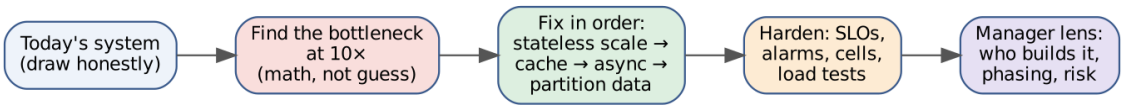


Figure Q1.2 — The five-step structure for any reverse-design answer.

> **You:** "I'll use our document intake and extraction platform. It turns broker submissions into structured data for underwriters. Today it handles about [20k] documents a day with a [15-minute] target. Let me draw it end to end, then I'll do the 10× math and show you the three places it breaks, in the order they'd break."
>
> "Documents arrive by portal, email and API. We store the raw file in object storage first — that's our durable record — then enqueue a job. Workers classify the document type, run OCR and extraction, score confidence, and either push to the underwriting system or route to a human review queue. Reviewers' corrections flow back as training data."

8. Deep dives — where it breaks at 10×, in order

Break 1: extraction throughput and cost (first to break). At 10× you need ~10× worker capacity. Autoscaling on queue depth handles volume, but cost scales linearly and any ML/LLM API has rate limits.

- *Fix:* autoscale on queue age, not CPU; batch pages per model call; route easy document types to a cheaper model and only hard ones to the expensive one (cascade); cache results by document hash for resubmissions.
- *Say:* "The first thing to break isn't a server, it's the extraction budget. I'd track cost per document as a first-class metric."

Break 2: the human review queue. If 15% of documents go to humans today, 10× volume means 10× reviewers. That's an org problem, not a server problem.

- *Fix:* raise auto-processing rates with better models and active learning; prioritize the queue by business value (submission premium, SLA); measure reviewer throughput.
- *Say:* "At 10×, my binding constraint becomes people, so the design has to change the ratio, not just add workers."

Break 3: the metadata database. Status updates per page, polling clients and dashboards on one Postgres primary. At 10× writes and polling, the primary saturates.

- *Fix:* move job-state updates to per-document granularity; replace polling with webhooks/events; read replicas for dashboards; partition by submission date if needed.

Break 4: hot tenants and quarter-end spikes. One large broker dumps 50k documents; everyone else waits.

- *Fix:* per-tenant fair queues or weighted fair scheduling; quotas; a separate priority lane for SLA-bound submissions.

9. Scaling evolution

Stage	Design
Today (1×)	Single queue, autoscaled workers, one Postgres, one region [VERIFY]
3×	Queue-age autoscaling, model cascade, read replicas, per-tenant fairness
10×	Partitioned job state, event-driven notifications, cost-per-doc budget controls, cell-based deployment by line of business so one bad batch can't stall all lines

10. Failures and degradation

- Extraction model API down → documents stay safely queued (raw file already durable); alert on queue age; fall back to a secondary model or human queue for SLA-bound items.
- Poison document (crashes the parser) → retry with backoff, then dead-letter after N attempts; alarm on DLQ depth.
- Bad model release → shadow-evaluate new model versions on a sample; canary by document type; roll back on accuracy drop.

11. Trade-offs to volunteer

Decision	Options	My choice	Why
Sync vs async processing	API waits vs queue + webhook	Async	Extraction takes seconds to minutes; queues absorb spikes
One model vs cascade	Best model always vs cheap-first	Cascade	Cost scales with volume; most docs are easy
Shared pipeline vs cells	One pipeline vs per-line-of-business cells	Cells at 10×	Blast radius; one broker's bad batch shouldn't stall all lines

12. Security, operations and cost

- **Security:** PII and policyholder data → encryption at rest with KMS keys, least-privilege roles per stage, redaction in logs, audit trail of who viewed a document. Mention your security architecture background here.
- **Operations:** SLIs = % of submissions extracted within 15 min; auto-processing rate; field accuracy (sampled audit); DLQ depth; queue age. Page on SLO burn rate and queue age; ticket on accuracy drift.
- **Cost:** cost per document by model tier; monthly forecast tied to submission volume.

13. Manager lens

> **You:** "Organizationally, I'd have the platform team own ingestion and the workflow engine, an ML team own models and evaluation, and a small operations team own the review tooling. The decision I own personally is the auto-processing threshold, because it's a business risk trade-off between accuracy and reviewer cost — I'd set it with the underwriting leaders, not alone."

14. Follow-ups with model answers

1. **"What would you redesign from scratch?"** — "I'd have made extraction results versioned by model version from day one. When we upgraded models we couldn't easily compare or re-run historical documents. [VERIFY — pick your real regret.]"
2. **"What was your hardest technical decision?"** — Pick one with real stakes (build vs buy the OCR/LLM layer; sync vs async; where to put the confidence threshold). Give the options, the data you used, what you chose and what happened.
3. **"How do you know extraction quality is good in production?"** — "Sampled human audits per document type each week, accuracy tracked by model version, and alerts when field-level confidence distributions shift."
4. **"What happens at quarter-end when volume triples?"** — Queue-age autoscaling, pre-warmed capacity scheduled ahead of known peaks, and priority lanes for SLA submissions.
5. **"Tell me about an outage on this system."** — Use the four-part incident story: detection (which alarm, how fast), mitigation (what you did in the first hour), root cause (the real one, not the symptom), mechanism (what you built so it can't recur).
6. **"What did you personally do vs your team?"** — "I set the architecture direction for async processing and the confidence-threshold model, pushed for cost-per-document as a metric, and made the build-vs-buy call on OCR. My tech lead designed the workflow engine; my engineers built the extraction services."

15. Mock transcript (condensed)

> **Interviewer:** Pick a system and tell me where it breaks at 10×.
>
> **You:** Document intake and extraction for underwriting. ~20k docs/day today. Let me draw it end to end, then do the math. *(draws pipeline)* 240k pages/day, ~10 pages/s peak, ~2 s per page means ~20 concurrent workers today, ~200 at 10×.

> **Interviewer:** So you just add workers?
>
> **You:** Workers scale with queue-age autoscaling. The first real break is cost and model rate limits — I'd add a model cascade and cost-per-doc budgets. The second break is human review: 10× docs means 10× reviewers unless we raise the auto-processing rate, so that's a model-quality investment.
>
> **Interviewer:** What would page you at 2 a.m.?
>
> **You:** Queue age over 10 minutes for SLA-bound submissions, and DLQ depth spiking. Accuracy drift creates a ticket, not a page.
>
> **Interviewer:** What did you personally decide here?
>
> **You:** The async architecture, the confidence-threshold policy with underwriting leadership, and buying OCR rather than building it.

16. Scoring guide

Rating	What it sounds like
Strong Hire	Real numbers; identifies non-obvious breaks (cost, humans, hot tenants) in order; concrete fixes; operations and personal decisions crisp; candid about what they'd redo
Hire	Solid diagram and reasonable scaling fixes; some numbers; ops present but generic
Lean No Hire	"We'd add more servers"; vague ownership; can't answer partition or alarm questions
No Hire	Can't draw their own system; blames others; no numbers

17. Mistakes and "say this"

Mistakes: choosing a system you didn't really own; no math; listing technologies instead of decisions; "we" throughout.

Say this:

- "The first thing to break isn't a server — it's the budget."
- "At 10×, my constraint becomes people, so the design has to change the ratio."
- "Here's the decision I owned, and here's the one I'd make differently."
- "I'd make the line of business the unit of failure."

18. Your tie-in

This *is* the tie-in. Prepare three versions: IDP (above), the underwriting platform, and one from your engineering/architect years (for example, a Marriott.com or AT&T identity component). [VERIFY all details.]

Q2. Design a distributed logging system

> Amazon (Medium) · Google variant: "logs and metrics pipeline, processed in under a minute" · Source: Aced Amazon SDM guide (2026); Aced Google SD guide (2026)

1. The prompt

"Design a system that collects logs from all our services and lets engineers search them." Variants: add real-time alerting; add metrics; "process within one minute"; multi-tenant (many teams, quotas).

2. Clarify

Ask (and why):

- "How many hosts/services, and roughly how much log volume per day?" — sizes everything.
- "What's the search freshness requirement — seconds or minutes?" — drives indexing pipeline design.
- "What retention: how long searchable, how long archived?" — hot/warm/cold tiers dominate cost.
- "Who queries — humans debugging, or also automated alerting?" — adds a streaming path.
- "Any compliance needs — PII in logs, audit retention?" — redaction and immutability.
- "Is losing some debug logs acceptable during overload?" — lets you shed load gracefully.

Don't ask: "Should I use Elasticsearch?" · "What log format do they use?" (assume structured JSON with some unstructured; normalize).

State: "I'll assume 50k hosts, ~5 GB/s raw at peak, search freshness under a minute, 7 days hot, 90 days warm, 1+ year cold for compliance, and that dropping DEBUG under overload is acceptable but ERROR and audit logs are not."

3. Requirements

- **Functional (must):** ingest from all hosts; search by time range + service + level + free text; tail live logs; retention tiers.
- **Functional (stretch):** alert rules on patterns; log-to-metric extraction; trace ID correlation.
- **Non-functional:** ingest durability for ERROR/audit (no loss); freshness p95 < 60 s; search p95 < 3 s for last-24h queries; 99.9% ingest availability; per-team quotas.

4. Estimates

- 50k hosts × 100 lines/s × 1 KB = **5 GB/s** raw at peak ≈ 430 TB/day if sustained (use ~200 TB/day average).
- Compression ~10× → ~20 TB/day stored. 7-day hot ≈ 140 TB on SSD (+1 replica = 280 TB). 90-day warm in object storage ≈ 1.8 PB.
- Ingest log: at ~10 MB/s per partition, 5 GB/s needs ~500 partitions.
- Search: 200 engineers, ~1 query/min each at peak = a few queries/s, but each may scan TBs → the challenge is scan efficiency, not QPS.

5. API

```
POST /v1/logs:batch        {source, host, entries:[{ts, level, service, msg, attrs, trace_id}]}
                           → 202 Accepted (agent retries with backoff on 429/503)
GET  /v1/search?q=...&service=...&level=ERROR&from=...&to=...&cursor=...
                           → {hits:[...], next_cursor, partial:boolean}
WS   /v1/tail?service=...  → live stream
POST /v1/alerts            {query, threshold, window, notify}
```

6. Data model

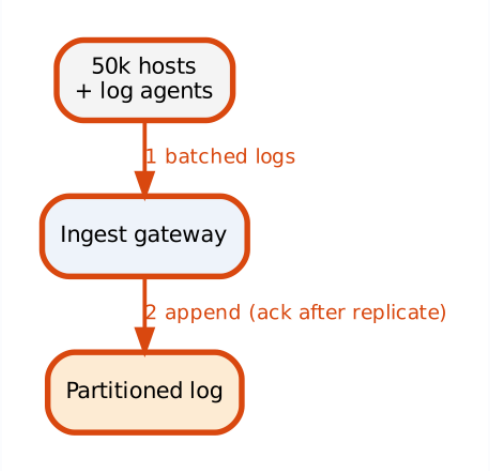
- **Log event:** `(tenant, service, ts, host, level, trace_id, msg, attrs{})`.
- **Hot index segments:** partitioned by `(tenant, time bucket = 1 hour)`, sharded by hash of service within the bucket. Inverted index on tokens of `msg` + keyword fields; columnar storage for `ts, level, service` to filter fast.
- **Cold:** Parquet files in object storage partitioned by `tenant/date/hour/service`, with per-file min/max timestamps for pruning.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

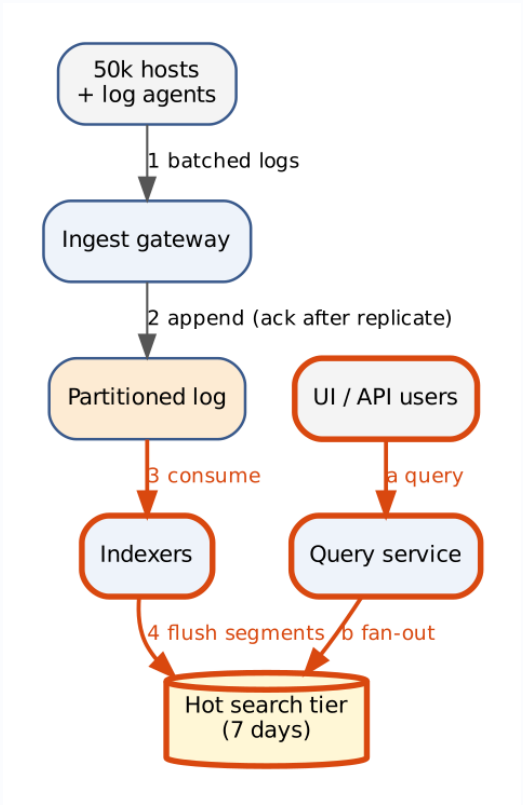
Step 1 · Ingest path (2 min)



> **You:** “Every host runs an agent that tails log files, batches, compresses and buffers on local disk. It sends to a stateless ingest gateway that authenticates, enforces per-team quotas, and returns 429 to slow agents down. The gateway appends to a partitioned durable log and only acknowledges after the batch is replicated.”

Why this step now: Follow the data from where it's born; establish durability first.

Step 2 · Indexing and search (2 min)

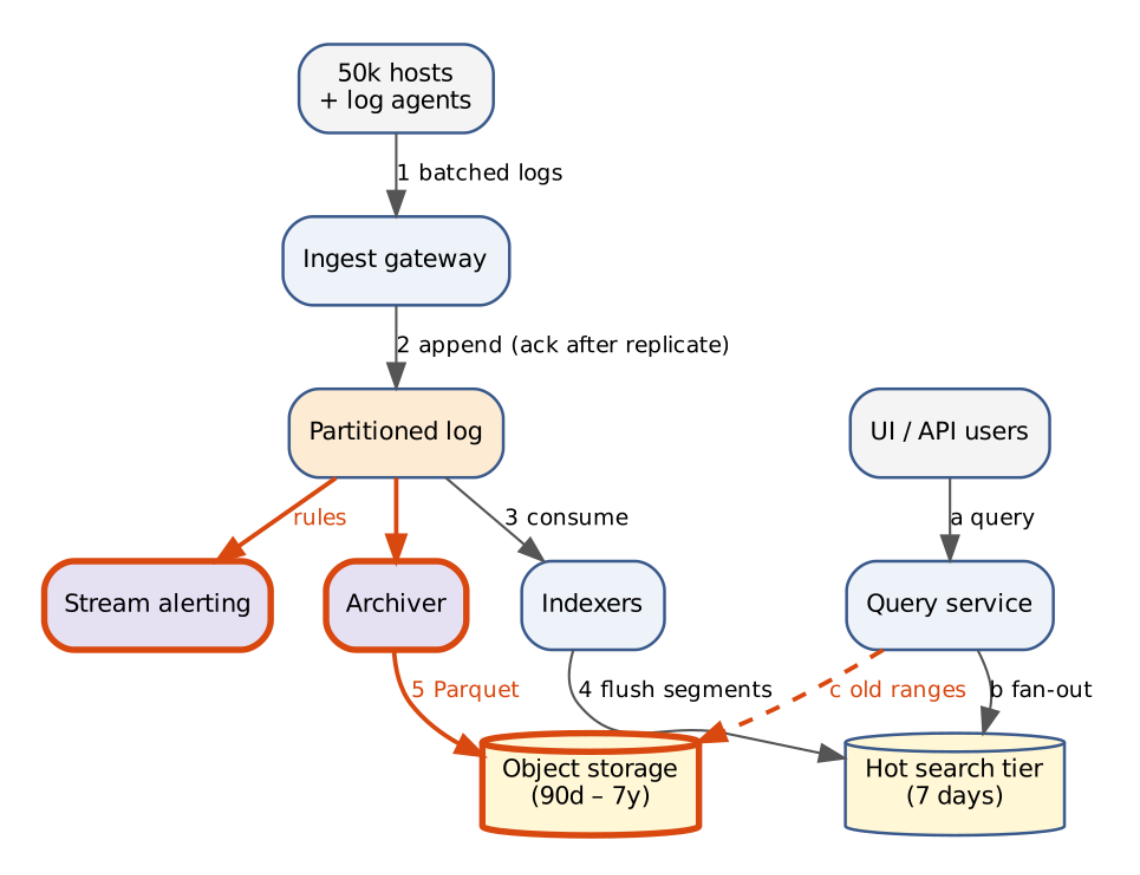


> **You:** “Indexers consume the log, parse and enrich each line, and build index segments in a hot search tier that holds seven days. Users query through a query service that picks the

> segments for the time range and service, fans out, merges results by timestamp and paginates."

Why this step now: Now the main use case — search — works end to end.

Step 3 · Retention and alerting (1–2 min)



> **You:** "In parallel, an archiver writes compressed Parquet to object storage partitioned by tenant, date and hour for long retention, and the query service can scan it for old ranges. A stream processor on the same log evaluates alert rules so alerts fire in seconds."

Why this step now: Add the cost and secondary use cases only after the core path is clear.

Narrate one request end to end (numbers match the arrows)

Write path (a log line)

1. Agent batches and compresses lines, sends to the gateway (1).
2. Gateway checks quota, appends to the log, acks after replication (2).
3. Indexer consumes (3), parses, flushes a segment to the hot tier every 10–30 s (4).
4. Archiver writes the same data as Parquet to object storage (5).

Read path (an engineer searches)

1. User submits query with time range and filters (a).
2. Query service selects segments by time and tenant, fans out to shards (b).

3. For ranges older than 7 days, it scans Parquet in object storage (c).
4. Results merged by timestamp, returned with a cursor; partial results flagged.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
50k hosts + log agents	Tail files, batch, compress, buffer to disk	Buffer at the source so network blips lose nothing	Fluent Bit / CloudWatch agent / Vector
Ingest gateway	Auth, quotas, backpressure (429)	One place to enforce per-team limits	Stateless service behind LB
Partitioned log	Durable buffer, replayable	Shock absorber between ingest and indexing	Kafka / Kinesis
Indexers	Parse, enrich, build index segments	Turns raw lines into searchable segments	Consumer group workers
Hot search tier (7 days)	Inverted index + columns for recent logs	Fast search where most queries land	OpenSearch / custom segments on SSD
Query service	Fan out by time+service, merge, paginate	Hides sharding from users; partial results	Stateless service
UI / API users	Search, tail, dashboards	The people debugging	Web UI, CLI
Archiver	Write compressed Parquet by tenant/date/hour	Cheap long retention	Consumer job
Object storage (90d – 7y)	Cold logs, compliance retention	Storage is the dominant cost	S3 + lifecycle to Glacier
Stream alerting	Evaluate alert rules on the stream	Alerts in seconds, not after indexing	Flink / Kinesis Analytics

Before you leave the high-level design, check you have said:

☐ Volume math → storage tiering
☐ Loss policy: audit/ERROR durable, DEBUG best-effort
☐ Per-team quotas
☐ Freshness measured with canary log lines

60-second summary to say once the diagram is complete:

> **You:** "Let me draw the whole pipeline first. An agent on every host tails log files, batches and compresses, and buffers on local disk so a network blip doesn't lose data. It sends to a stateless ingest gateway that authenticates, enforces per-team quotas and returns 429 when a team exceeds its budget."
>
> "The gateway writes to a partitioned durable log — Kinesis or Kafka — partitioned by service and host. That log is the decoupling point: indexing can fall behind without losing data. Indexers consume, parse, enrich (add team, region), and build index segments for the hot search tier,

> which holds seven days. In parallel an archiver writes compressed Parquet to object storage for long-term retention. A stream processor on the same log evaluates alert rules."
>
> "Queries go to a query service that fans out to the index shards covering the time range, merges results and paginates. For older ranges it scans Parquet in object storage, which is slower but cheap."

8. Deep dives

Deep dive 1: no loss for important logs, graceful loss for the rest.

- Agent: disk-backed buffer (e.g., 1 GB), at-least-once send with retries; dedupe via `(host, file, offset)` sequence IDs.
- Gateway acks only after the log has persisted the batch (replicated to 2+ brokers).
- Under overload: priority by level. Gateway sheds DEBUG/INFO first (tells agents to sample), never ERROR/audit.
- *Say:* "I'd make the loss policy explicit: audit and ERROR are durable, DEBUG is best-effort. That's a product decision I'd confirm with service owners."

Deep dive 2: the index and search.

- Time-partitioned segments: most queries are "last N hours," so segment by hour and prune by time first, then by service, then search the inverted index.
- Segment lifecycle: write to small in-memory segments, flush every ~10–30 s (this is your freshness), merge into bigger segments in the background (LSM-like).
- Query fan-out: query service picks segments by time and tenant, scatters, gathers top results by timestamp, returns a cursor. Timeouts per shard; return partial results flagged `partial: true` rather than failing.
- Alternative: index only metadata fields and brute-force scan compressed message text (Loki-style) — far cheaper storage, slower free-text search. Mention as a cost lever.

Deep dive 3: multi-tenancy and quotas.

- Per-team ingest quotas in GB/day with burst; enforced at the gateway with token buckets (A9 Example 2).
- Noisy neighbor: one team's log storm → their partitions lag; isolate big tenants on dedicated partitions.
- Chargeback: report GB ingested and stored per team — drives teams to log less.

9. Scaling evolution

Stage	Design
Small (one team)	Agents → managed search cluster directly
Target	Log in the middle, hot index 7 days, Parquet archive, query fan-out

Stage	Design
10×	Cells per region/tenant group; metadata-only indexing for low-value logs; tiered sampling; separate alerting pipeline

10. Failures

- Indexers fall behind → log retains data (e.g., 3 days); alert on consumer lag; search shows "data delayed" banner. Scale indexers by partition.
- Hot tier node loss → replicas; rebuild from the log or from archive.
- Region outage → agents buffer locally; cross-region failover for the ingest endpoint for audit logs.
- Query of death (search across 90 days, all services) → query cost estimation, limits, async "export" job instead.

11. Trade-offs

Decision	Options	My choice	Why
Direct-to-index vs log in the middle	Agents write to search cluster vs Kafka/Kinesis first	Log in middle	Absorbs spikes, replay, multiple consumers
Full-text index vs metadata-only	Index all tokens vs labels only	Full-text for hot 7 days, metadata-only for cold	Search speed where it matters, cost elsewhere
Push vs pull agents	Agents push vs collectors scrape	Push	Simpler across networks; buffers at source
Exact vs sampled DEBUG	Keep all vs sample	Sample under quota pressure	Frugality without losing signal

12. Security, operations and cost

- **Security:** PII redaction at the agent or gateway (regex + structured-field allowlists); tenant isolation on query; immutable audit logs (object lock); access logs on the logging system itself.
- **Operations:** SLIs — ingest success rate, end-to-end freshness (synthetic canary log line every 10 s per region, measure time to searchable), search p95 latency, consumer lag. Page on freshness SLO burn and ingest error rate.
- **Cost:** storage dominates. Levers: compression, 7-day hot window, cold Parquet, sampling DEBUG, per-team chargeback. Say a number: "Moving from 30-day hot to 7-day hot cuts SSD cost by roughly 75%."

13. Manager lens

> **You:** "I'd run this as three teams: ingest (agents, gateway, log), index and search, and the query/UX + alerting team. The shared contract is the log schema, which I'd version and govern

> centrally. Phase 1 is ingest and search for one region in a quarter; phase 2 adds alerting and archive; phase 3 adds cells and cost controls. The top risk is a log storm from one bad deploy taking out ingest for everyone — that's why quotas ship in phase 1, not later."

14. Follow-ups

1. **"How do you guarantee ordering?"** — Only per host/stream (partition by host); global ordering isn't needed. Sort by timestamp at query time; clock skew handled by tolerating a small window.
2. **"How do you make logs searchable in under a minute?"** — Segment flush interval of 10–30 s plus indexer lag alarms; measure with canary log lines.
3. **"A team logs 10× more after a bad deploy. What happens?"** — Their quota trips at the gateway; agents sample DEBUG; ERROR still flows; team is alerted; other teams unaffected.
4. **"How would you add tracing correlation?"** — Require `trace_id` in structured logs; index it as a keyword; link from traces UI.
5. **"Why not just use CloudWatch / a vendor?"** — "At small scale I would. At 5 GB/s, vendor cost is often the deciding factor, and we need custom quotas and retention — I'd do a cost model before deciding." (Frugality, build vs buy.)
6. **"How do you reprocess logs after a parsing bug?"** — Replay from the log (within retention) or re-index from the Parquet archive.

15. Mock transcript

> **Interviewer:** Design a logging system for our services.
>
> **You:** A few questions first: how many hosts, freshness target, retention, and is losing DEBUG under overload OK? … I'll assume 50k hosts, 5 GB/s peak, under-a-minute freshness, 7 days hot and a year archived. That's about 20 TB/day compressed, so storage tiering will matter most for cost. *(draws full pipeline)*
>
> **Interviewer:** What if the indexers fall behind?
>
> **You:** Nothing is lost — the log keeps three days. Search shows a delay banner, we alarm on consumer lag, and indexers scale by partition. That's exactly why I put a log in the middle.
>
> **Interviewer:** How do you stop one team from hurting everyone?
>
> **You:** Per-team token buckets at the gateway, priority by level so ERROR always flows, and dedicated partitions for the biggest tenants.

16. Scoring guide

Rating	Signals
Strong Hire	Volume math → tiering; log in the middle with clear reasoning; explicit loss policy; quotas; freshness measured with canaries; cost levers with numbers
Hire	Correct pipeline and storage; some ops; fewer numbers

Rating	Signals
Lean No Hire	Agents writing straight to Elasticsearch; no retention/cost story; no overload plan
No Hire	Can't get logs from hosts to search coherently

17. Mistakes and "say this"

Mistakes: ignoring cost (storage *is* the problem); no backpressure; one giant index; no multi-tenancy.

Say this: "The log is my shock absorber." · "Audit is durable, DEBUG is best-effort — I'd make that explicit." · "I'd measure freshness with a canary log line, not trust indexer metrics."

18. Your tie-in

Your platform almost certainly had a logging/observability stack. Mention one real decision (retention, vendor cost, PII redaction in logs for policyholder data). [VERIFY]

Q3. Design a system that delivers firmware updates to devices

> Amazon (Medium) · also Meta hardware-software, Apple · Source: Aced Amazon SDM guide (2026)

1. The prompt

"Design over-the-air (OTA) firmware updates for millions of devices (Echo, Kindle, Ring)." Variants: staged rollout with auto-rollback; devices on poor networks; security of the update chain.

2. Clarify

Ask:

- "How many devices, how many models, and how big is an image?" — bandwidth and CDN sizing.
- "How quickly must a release reach the fleet — days or hours? Any emergency security patches?" — wave design.
- "What are devices' connectivity patterns — always online, intermittent, battery-constrained?" — pull vs push, resumable downloads.
- "What happens if an update fails mid-install — do devices have A/B partitions?" — brick risk.
- "Who can initiate a rollout, and what approvals are required?" — security and governance.

Don't ask: "Should devices use MQTT?" (decide it) · "What chip do they use?" (irrelevant).

State: "100M devices, ~30 models, 500 MB full images, deltas ~10% size, devices check in every 6 hours, A/B partitions so a failed install rolls back locally. Normal rollouts over a week; security patches within 48 hours."

3. Requirements

- **Must:** publish signed images per model/version; target cohorts; staged waves; devices download and verify; report status; automatic pause/rollback on health regression.
- **Stretch:** delta updates; scheduled install windows (night); per-customer opt-outs (enterprise fleets).
- **Non-functional:** never brick a device; cryptographic integrity end to end; check-in API 99.99%; bandwidth cost minimized.

4. Estimates

- Check-ins: 100M / 6 h ≈ **4.6k RPS** average; allow 3× for synchronized wakeups → ~15k RPS. Add jitter so devices don't sync.

- Bandwidth for a release: full image 100M × 500 MB = 50 PB — too much. With deltas (~50 MB): 5 PB over 7 days ≈ **8 GB/s** average. CDN is mandatory.
- Telemetry: 100M devices × few events per update ≈ 300M events per rollout.

5. API

```
POST /v1/devices/{id}/checkin   {model, current_version, hw_rev, battery, network}
     → {update: {version, delta_from, url (signed, short TTL), sha256, signature,
install_window}} | {none, next_checkin_s}
POST /v1/devices/{id}/status    {version, phase: downloaded|verified|installed|failed|
rolled_back, error_code}
POST /v1/releases               {model, version, artifact_uri, notes}            (internal,
requires approval)
POST /v1/rollouts               {release, cohorts, waves:[{pct, min_hours, health_gates}]}
POST /v1/rollouts/{id}:pause | :resume | :rollback
```

6. Data model

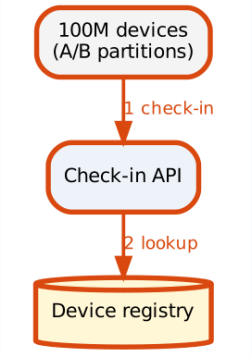
- `Release(model, version, full_uri, delta_uris{from_version→uri}, sha256, signature, status)`
- `Rollout(id, release, cohort_filter, waves[], current_wave, state, gates)`
- `Device(id, model, hw_rev, version, cohort_bucket = hash(id) % 10000, last_checkin, region)` in a key-value store keyed by device_id.
- `DeviceUpdateStatus(device_id, rollout_id, phase, ts, error)` — time series to the telemetry pipeline, aggregated per rollout/wave.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

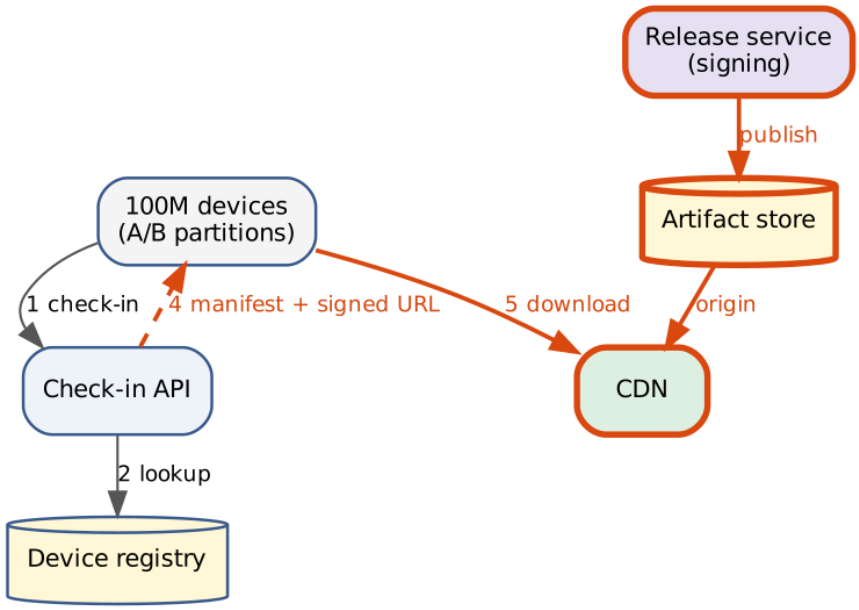
Step 1 · Devices ask, server decides (1 min)



> **You:** “Devices poll a stateless check-in API every few hours with jitter, sending their model and current version. The API looks the device up in a registry keyed by device ID.”

Why this step now: Start with the device and the decision it needs answered.

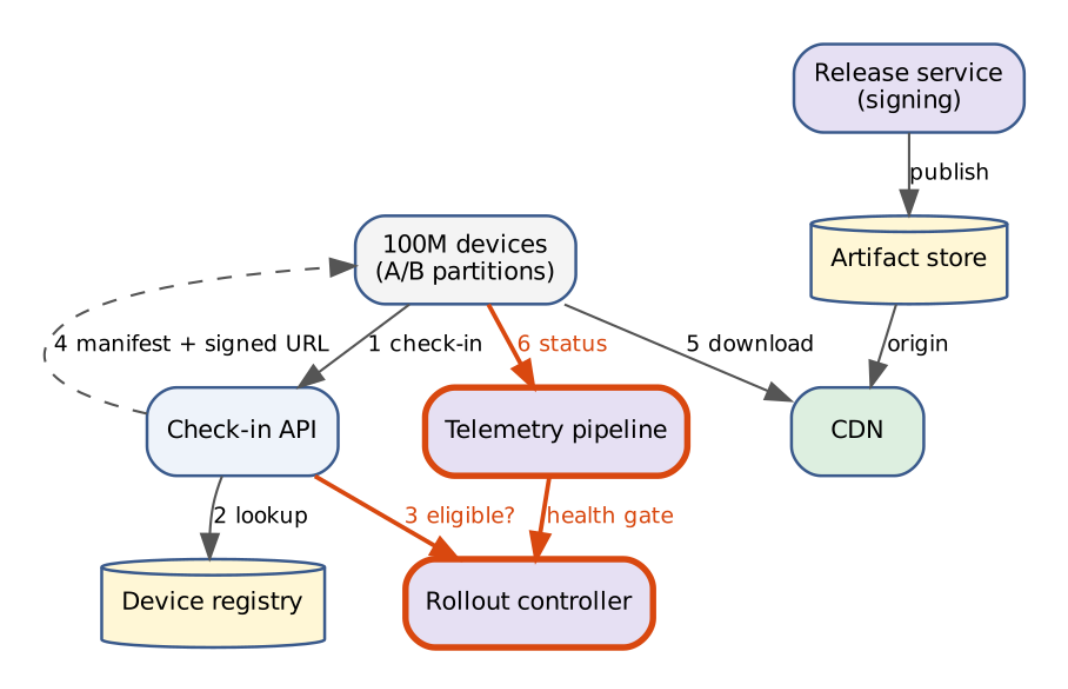
Step 2 · Releases and bytes (2 min)



> **You:** “Engineers publish a release through a signing service; signed full images and deltas go to an artifact store behind a CDN. If an update applies, check-in returns a manifest with a short-lived signed CDN URL, the hash and the signature. Bytes always come from the CDN, never from our services.”

Why this step now: Separate decisions (our services) from bytes (CDN) early — that's the key scale insight.

Step 3 · Safe rollout loop (2 min)



> **You:** "The rollout controller decides eligibility: is this device's cohort bucket inside the current wave? Devices report status and the telemetry pipeline computes install success and crash rate per wave. The controller advances, holds, or auto-pauses based on those health gates."

Why this step now: The control loop is what makes it safe; add it once the basic flow works.

Narrate one request end to end (numbers match the arrows)

Update a device

1. Device checks in with model and version (1); API reads the registry (2).
2. API asks the rollout controller whether the device's bucket is in the current wave (3).
3. API returns manifest: version, signed CDN URL, sha256, signature (4).
4. Device downloads resumably from the CDN (5), verifies signature, installs to the inactive A/B slot, reboots.
5. Device reports status (6); telemetry feeds the controller's health gates.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
100M devices (A/B partitions)	Check in, download, verify, install, report	The fleet; must never brick	Device updater agent
Check-in API	Answer 'is there an update for me?'	Stateless decision point; devices poll with jitter	Stateless service, mTLS
Device registry	Model, version, cohort bucket, last check-in	Targeting needs device facts	DynamoDB keyed by device_id

CDN	Serve image and delta bytes	50 PB naive transfer — bytes must not hit our services	CloudFront / multi-CDN
Artifact store	Signed full images + deltas	Origin for CDN	S3
Release service (signing)	Publish signed releases	Integrity of the update chain	HSM-backed signing
Rollout controller	Waves, eligibility, health gates, auto-pause	Safe rollout is a control loop	Service + state DB
Telemetry pipeline	Install results, crash rates per wave	Feeds the health gates	Kinesis + stream aggregation

Before you leave the high-level design, check you have said:

☐ Bandwidth math → CDN + deltas
☐ Waves with automatic health gates
☐ A/B partitions for local rollback
☐ Signing chain and anti-rollback

60-second summary to say once the diagram is complete:

> **You:** "Two planes. The control plane is where engineers publish a signed release and define a rollout — which cohorts, which waves, and what health gates. The data plane is devices checking in and downloading bytes."
>
> "Devices poll a stateless check-in API every few hours with jitter. The API looks up the device in the registry and asks the rollout controller whether this device is eligible: is its model in the release, is its cohort bucket inside the current wave percentage? If yes, it returns a manifest with a short-lived signed CDN URL, the hash and the signature. The bytes come from the CDN, never from our services."
>
> "The device downloads resumably, verifies the signature with a key baked into the bootloader, installs to the inactive A/B partition, reboots, and reports status. Telemetry — install success and post-update crash rates — feeds the rollout controller, which advances waves only when health gates pass and auto-pauses if they fail."

8. Deep dives

Deep dive 1: staged rollout with automatic health gates.

- Cohort bucket = `hash(device_id) % 10000`, stable across releases so the same canary devices don't always go first — or deliberately rotate (salt by release).
- Waves: 0.1% → 1% → 10% → 50% → 100%, each with minimum soak time and gates: install success ≥ 99.5%, crash rate not worse than baseline + X%, no spike in support contacts.
- Controller loop (every few minutes): compute metrics for the current wave vs control group; advance, hold, or pause; pause notifies the owner.

```
def evaluate_wave(rollout, metrics, baseline):
    w = rollout.current_wave
    if metrics.install_success < w.gates.min_success:     return "PAUSE"
    if metrics.crash_rate > baseline.crash_rate * (1 + w.gates.max_crash_increase):
        return "PAUSE"
    if metrics.devices_reporting < w.gates.min_sample:   return "HOLD"   # not enough data
    if hours_since(w.started_at) < w.min_hours:          return "HOLD"
    return "ADVANCE"
```

Deep dive 2: never brick a device.

- A/B partitions: install to inactive slot; bootloader tries new slot; if boot health check fails N times, revert to old slot automatically — no network needed.
- Battery and storage checks before install; install windows at night.
- Resumable download with HTTP range requests; verify hash before install.

Deep dive 3: security of the update chain (your differentiator).

- Images signed in an offline/HSM-backed signing service; device verifies signature against a public key in secure boot.
- Anti-rollback: device refuses versions below a monotonic security counter (prevents downgrade attacks to vulnerable versions).
- Signed URLs with short TTL; mutual TLS for device check-in with per-device certificates.
- Two-person approval for releases; audit log.
- *Say:* "Compromise of the update pipeline is the worst-case attack — it's remote code execution on 100 million devices. I'd treat signing keys like the crown jewels."

9. Scaling evolution

Stage	Design
Small	Devices download from object storage directly; manual waves
Target	CDN, deltas, automated health gates, cohort buckets
10× / multi-product	Rollout service as a platform for all device teams; per-region cells; peer-assisted distribution on local networks (devices share updates)

10. Failures

- Bad release → health gate pauses at 0.1–1%; rollback = publish previous version as a new rollout (anti-rollback counter permitting) or device-local revert if boot fails.
- CDN regional outage → multi-CDN with steering; devices retry with backoff.
- Thundering herd after a power outage → jittered check-in; `next_checkin_s` returned by server lets you spread load.
- Check-in API down → devices simply check later; no harm (fail safe).

11. Trade-offs

Decision	Options	Choice	Why
Push vs pull	Server pushes via MQTT vs devices poll	Poll with jitter, push nudge for urgent patches	Polling is simpler and fails safe; push for security emergencies
Full vs delta images	Full only vs per-version deltas	Deltas for top N source versions, full as fallback	10× bandwidth savings; long tail gets full image
Cohort assignment	Random each release vs stable	Stable buckets, salted per release	Reproducible, avoids same devices always first

12. Security, operations, cost

- **Security:** covered in deep dive 3.
- **Operations:** per-rollout dashboard (wave, success, failures by error code, model, region); page on gate failures for security patches; weekly review of failed-install error codes.
- **Cost:** CDN egress dominates — deltas, compression, regional caching, off-peak scheduling.

13. Manager lens

> **You:** "This is a platform used by many device teams, so I'd build it as a self-service rollout service with guardrails: teams define releases and waves; the platform enforces signing, gates and approvals. One team owns the control plane, one owns the device-side updater library with firmware teams. The decision I'd own is the default safety policy — minimum wave sizes and gates — because it's the thing teams will want to bypass under deadline pressure."

14. Follow-ups

1. **"Urgent security patch must reach 90% of devices in 24 hours. What changes?"** — Compressed waves (1% for 1 h, then 10%, then 100%), push nudges to online devices, pre-position on CDN, keep the health gates but shorten soak.
2. **"Devices offline for months?"** — Return a path to the latest version: chain of deltas or full image; enforce minimum supported version on reconnect.
3. **"How do you test before 0.1%?"** — Internal dogfood fleet, lab device farm across hardware revisions, staged by region.
4. **"How do you handle enterprise customers who want control?"** — Per-customer policies (defer N days, maintenance windows), exposed in an admin API.
5. **"How do you detect a bad update that doesn't crash but breaks a feature?"** — Feature-level health metrics (wake-word success rate, Wi-Fi reconnects) compared to a holdback control group.

15. Mock transcript

> **Interviewer:** Design firmware updates for 100 million devices.
>
> **You:** Device count, image size, rollout speed, connectivity, and do devices have A/B partitions? … I'll assume 500 MB images, deltas around 50 MB, weekly rollouts and 48-hour security patches. Full images to 100M devices would be 50 PB — so CDN plus deltas is non-negotiable. *(draws control plane, check-in, CDN, telemetry loop)*
>
> **Interviewer:** What if a release has a bug that bricks devices?
>
> **You:** Three layers: the 0.1% wave catches it with health gates and auto-pauses; A/B partitions mean the bootloader reverts locally after failed boots; and we never expand without soak time. The worst case is a few thousand devices that revert themselves.
>
> **Interviewer:** What's your biggest security concern?
>
> **You:** The signing key. Compromise means code execution on the whole fleet, so signing lives in an HSM with two-person approval, and devices enforce anti-rollback counters.

16. Scoring guide

Rating	Signals
Strong Hire	Bandwidth math → CDN + deltas; automated wave gates with metrics; A/B rollback; signing chain and anti-rollback; thundering-herd jitter; platform/manager framing
Hire	CDN, waves, rollback; security mentioned but shallow
Lean No Hire	Push all devices at once; no health gates; no signing
No Hire	Serving bytes from app servers; no rollback story

17. Mistakes and "say this"

Mistakes: treating it as a file download problem; no fleet-health feedback loop; ignoring security.

Say this: "The CDN serves bytes; my services only make decisions." · "The rollout controller is a control loop, not a schedule." · "Worst case is remote code execution on the fleet, so signing keys are the crown jewels."

18. Your tie-in

Your AT&T mobile identity security architecture work is directly relevant to device identity, certificates and signing. One sentence: "From my security architecture work on mobile identity, the device-identity piece is where these systems usually get compromised." [VERIFY]

Q4. Design a Dropbox-style file sync service

> Amazon (Medium — reported in a 2026 senior loop) · common at Meta, Microsoft (OneDrive), Google (Drive) · Source: Glassdoor Amazon senior SWE review (Jul 2026)

1. The prompt

"Design Dropbox: users store files and they sync across devices." Variants: sharing and permissions; large files; offline edits and conflicts; version history.

2. Clarify

Ask:

- "How many users and how much storage per user?" — sizes storage.
- "Typical file sizes — documents or large media?" — chunking strategy.
- "Must edits from one device appear on others in seconds or minutes?" — notification design.
- "Do we need shared folders and collaborative editing, or just sync?" — sharing adds permissions and fan-out; co-editing is a different problem (see Microsoft Q18).
- "Offline edits allowed? How do we resolve conflicts?" — conflict policy.

Don't ask: "Which cloud provider?" · "Do we need a web UI?" (assume yes, it's just another client).

State: "500M users, 100M daily active, avg 10 GB each, typical files under 10 MB but some multi-GB; sync visible within seconds; shared folders; offline edits with conflict copies."

3. Requirements

- **Must:** upload/download; sync changes across a user's devices; version history; shared folders; resumable large uploads.
- **Stretch:** deduplication across users; LAN sync; selective sync.
- **Non-functional:** durability 11 nines for content; metadata strongly consistent per namespace; sync latency < 5 s for small changes; bandwidth-efficient.

4. Estimates

- Storage: 500M × 10 GB = **5 EB** logical; dedup and compression reduce physical significantly (assume 30–50%).
- Writes: 100M DAU × 10 file changes/day = 1B changes/day ≈ **12k changes/s**, peak ~35k/s.
- Upload bandwidth: if average change uploads ~1 MB of new chunks → 12 GB/s average.

- Metadata: 500M users × 10k files × 500 B ≈ 2.5 PB of metadata — needs sharding by namespace.

5. API

```
POST /v1/files:commit     {namespace_id, path, parent_rev, chunk_hashes:[...], size, mtime}
                         → {missing_chunks:[...]} | {rev}         (client uploads only
missing chunks)
PUT  <presigned URL>      (chunk bytes, direct to blob store)
GET  /v1/changes?namespace_id=...&cursor=...  → {entries:[...], cursor, has_more}
GET  /v1/changes:longpoll?cursor=...          → {changes: bool}    (held up to ~60 s)
GET  /v1/files/{id}/revisions
POST /v1/shares            {path, grantee, role}
```

6. Data model

- `Namespace(id, owner, type: user|shared)` — the unit of sharding and consistency.
- `FileEntry(namespace_id, path, file_id, latest_rev, is_deleted)` — primary key `(namespace_id, path)`.
- `Revision(file_id, rev, chunk_list[], size, author_device, ts)`.
- `Journal(namespace_id, seq, op, path, rev)` — append-only per namespace; clients sync by `seq` cursor.
- `Chunk(hash → blob location, refcount)` — content-addressed.

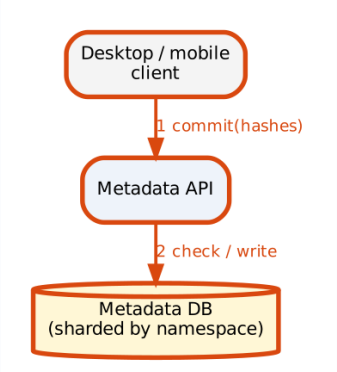
Metadata in a sharded relational store (MySQL/Postgres shards by namespace_id) — we need transactions within a namespace.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

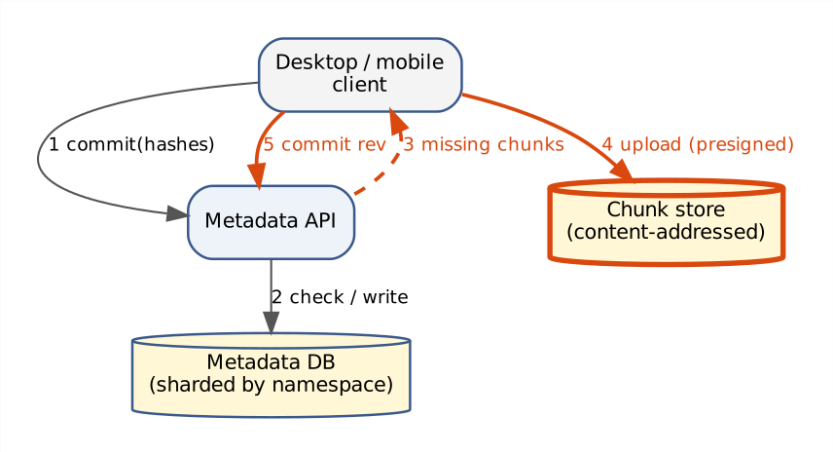
Step 1 · Metadata path (1 min)



> **You:** “The client watches the folder and splits changed files into chunks with a SHA-256 hash each. It sends the metadata API the list of hashes for the new revision. Metadata lives in a database sharded by namespace — a user's folder or a shared folder.”

Why this step now: Establish the most important split: metadata is small and transactional.

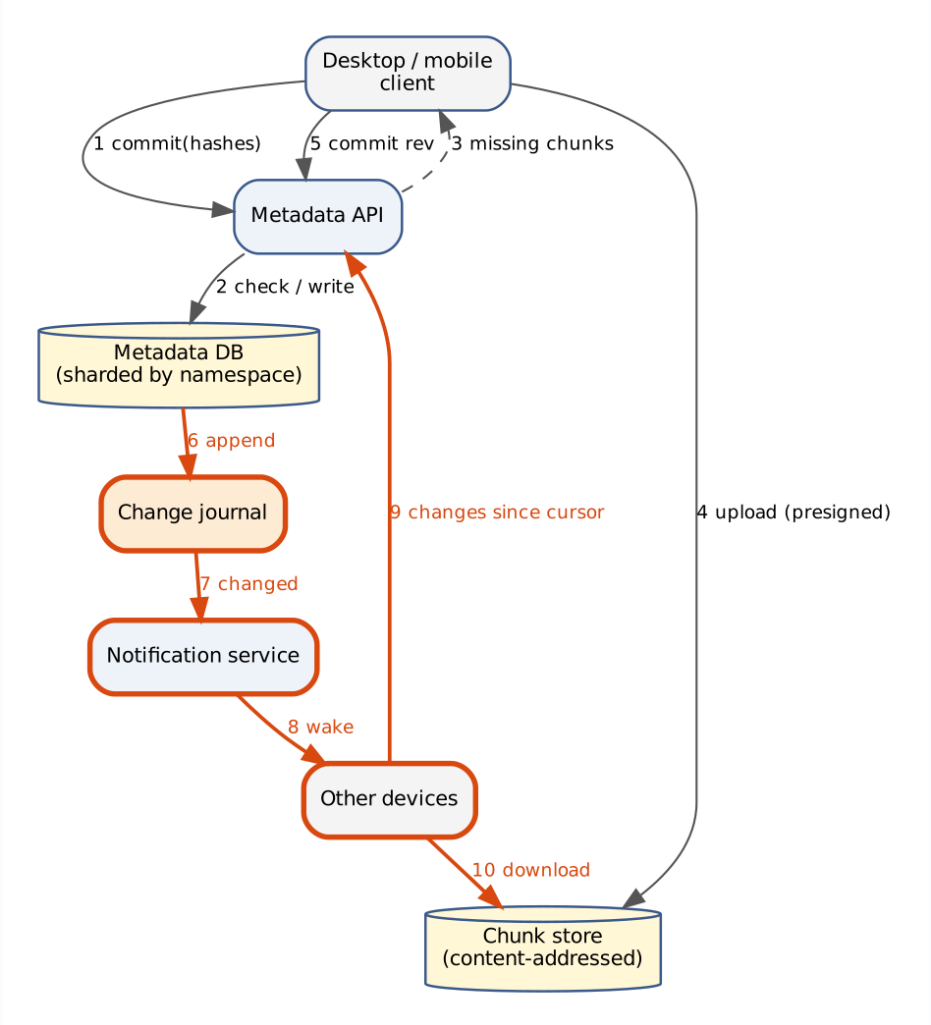
Step 2 · Bytes path (1–2 min)



> **You:** “The API replies with which chunks it doesn't have. The client uploads only those, directly to a content-addressed chunk store via pre-signed URLs, then commits the revision — conditional on the parent revision so concurrent edits are detected.”

Why this step now: Bytes bypass your services; dedup falls out of content addressing.

Step 3 · Sync to other devices (1–2 min)



> **You:** “Each commit appends to the namespace's change journal. Other devices hold a long-poll with a notification service; when the journal advances, they wake, ask for changes since their cursor, and download only the chunks they need.”

Why this step now: Only now add the real-time part.

Narrate one request end to end (numbers match the arrows)

Upload a changed file

1. Client chunks and hashes the file, sends hashes to the metadata API (1).
2. API checks which chunks exist (2) and returns the missing list (3).
3. Client uploads missing chunks straight to the chunk store (4).
4. Client commits the revision with parent_rev (5); conflict → conflicted copy.

Another device syncs

1. Commit appends to the namespace journal (6); notification service sees it (7).
2. Waiting devices are woken (8), request changes since their cursor (9).

3. They download only missing chunks (10) and apply locally.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Desktop / mobile client	Watch folder, chunk + hash files, local DB	Most of the sync logic lives on the client	Native sync engine
Metadata API	Commit revisions, list changes	Small, consistent, transactional operations	Stateless service
Metadata DB (sharded by namespace)	Files, revisions, chunk lists, journal	Ordering and transactions per namespace	Sharded MySQL / Postgres
Chunk store (content-addressed)	Chunks keyed by SHA-256	Dedup and resumable uploads	S3 / custom blob store
Change journal	Ordered changes per namespace	Devices sync by cursor	Journal table + event stream
Notification service	Hold long-polls, wake clients	Seconds-level sync without polling the DB	Long-poll / WebSocket fleet
Other devices	Pull changes, download chunks	Where the sync shows up	Same client

Before you leave the high-level design, check you have said:

☐ Bytes vs metadata split
☐ Upload-only-missing-chunks
☐ Namespace as unit of consistency
☐ Conflict policy

60-second summary to say once the diagram is complete:

You: "The core idea is separating bytes from metadata. The client watches the folder, splits changed files into ~4 MB chunks and hashes each. It sends the metadata service the list of hashes; the service replies with which chunks it doesn't already have. The client uploads only those, directly to blob storage via pre-signed URLs, then commits the new revision."

"Each commit appends an entry to the namespace's journal with a sequence number. Other devices hold a long-poll on 'changes since my cursor'; when the journal advances, they wake up, fetch the change list, and download the chunks they need."

8. Deep dives

Deep dive 1: chunking and dedup.

- Fixed-size chunks are simple but an insert near the start shifts every chunk. **Content-defined chunking** (rolling hash boundaries, e.g., Rabin fingerprint) keeps most chunks identical after small edits → much less upload.
- Dedup by SHA-256 hash across all users saves storage; privacy concern: hash-existence probing can reveal whether someone has a file — mitigate by per-user proof-of-possession or not exposing dedup to the client across users.

Deep dive 2: consistency and conflicts.

- Commit is conditional on `parent_rev` (optimistic concurrency): if another device committed first, the server rejects; client creates a "conflicted copy" with device name — the Dropbox approach. Simple, never loses data.
- Journal per namespace gives a total order of changes within that namespace — simple sync with a cursor.

Deep dive 3: notifications at scale.

- 100M devices holding long-polls → notification servers hold connections (≈100k–1M per host), subscribe to namespace journal updates through a pub/sub bus. Notification carries only "something changed," not data — clients fetch from the changes API.

9. Scaling evolution

Stage	Design
Small	Upload whole files to S3; metadata in one Postgres; clients poll
Target	Chunking + dedup, sharded metadata by namespace, journal cursors, long-poll notification tier
10×	Own blob storage tier with erasure coding (cost), cold tiering for old versions, LAN sync

10. Failures

- Upload interrupted → chunk-level resume; commit only after all chunks present.
- Metadata shard down → that namespace is read-only; others unaffected.
- Notification tier down → clients fall back to periodic polling.
- Corrupted chunk → hash verification on read; repair from replicas/parity.

11. Trade-offs

Decision	Options	Choice	Why
Conflict policy	Last-writer-wins vs conflicted copy vs merge	Conflicted copy	Never loses user data; merging is file-type specific

Decision	Options	Choice	Why
Chunking	Fixed vs content-defined	Content-defined	Small edits upload small deltas
Metadata store	NoSQL vs sharded SQL	Sharded SQL by namespace	Transactions and ordering within a namespace
Notification	WebSockets vs long-poll	Long-poll	Simpler through proxies; data fetched via API anyway

12. Security, operations, cost

- **Security:** encryption at rest per chunk, TLS, share-link tokens with expiry, permission checks on every metadata op, audit logs for enterprise.
- **Operations:** SLIs — commit success rate, sync latency (time from commit to other device seeing it), upload success; alert on journal lag.
- **Cost:** storage is everything — dedup, compression, erasure coding over 3× replication (1.4× vs 3×), cold tier for old revisions.

13. Manager lens

> **You:** "Three teams: client sync engine, metadata and sync service, storage. The client team is the riskiest because bugs there lose user data on millions of machines — so client releases get the same staged rollout we'd use for firmware. The decision I own is the conflict policy, because it's a user-trust decision."

14. Follow-ups

1. **"How do you sync a 50 GB file?"** — Chunked multipart upload, parallel streams, resumable; commit when complete; show progress.
2. **"Shared folder with 10,000 members — what changes?"** — Shared namespace with its own journal; members subscribe to it; permission checks cached; fan-out is to notification subscribers only.
3. **"How do you restore a deleted folder from last month?"** — Journal + revisions are immutable; restore = new commit pointing to old revisions.
4. **"How would you add search?"** — Index file names/metadata from the journal stream; content extraction async for supported types.
5. **"How do you keep storage cost down?"** — Dedup, erasure coding, lifecycle to cold storage, limit version history by plan.

15. Mock transcript

> **Interviewer:** Design Dropbox.
>
> **You:** Users, storage per user, file sizes, sync latency, sharing, offline edits? … 500M users, 10 GB each — that's 5 exabytes logical, so storage efficiency is the game. Let me draw it: client

chunks and hashes, metadata service tells it which chunks are missing, bytes go straight to blob storage, commit appends to a per-namespace journal, other devices long-poll the journal.

Interviewer: Two devices edit the same file offline.

You: The commit is conditional on the parent revision. Second commit fails; the client keeps both as a conflicted copy. We never silently lose data.

Interviewer: Why content-defined chunking?

You: With fixed chunks, inserting a byte at the start changes every chunk hash. Rolling-hash boundaries keep unchanged regions identical, so we upload only what changed.

16. Scoring guide

Rating	Signals
Strong Hire	Bytes vs metadata split; chunk + hash + upload-missing-only; journal + cursor sync; explicit conflict policy; storage cost levers with math
Hire	Chunking and metadata DB; sync by polling; conflicts mentioned
Lean No Hire	Whole-file uploads through app servers; no conflict strategy
No Hire	No separation of storage and metadata; can't explain sync

17. Mistakes and "say this"

Mistakes: routing bytes through your servers; ignoring conflicts; single metadata DB.

Say this: "Bytes go direct to storage; my servers only move metadata." · "The namespace is my unit of consistency and sharding." · "I'd rather create a conflicted copy than lose a user's work."

18. Your tie-in

Document storage and versioning for submissions and policy documents is a close analog. [VERIFY]

Q5. Design a parking payment system

Amazon (Medium — repeated across roles on Glassdoor) · payment patterns also at Databricks (Q20), Apple, Microsoft · Source: IGotAnOffer Amazon system design guide (2026 update)

1. The prompt

"Design a payment system for parking: drivers start a session, pay, and enforcement can check whether a car has paid." Variants: pay-by-plate with cameras; dynamic pricing; garages with gates; multiple cities.

2. Clarify

Ask:

- "Street parking, garages with gates, or both?" — gates need low-latency, offline-capable decisions.
- "How do drivers pay — app, kiosk, plate recognition?" — entry points and identity.
- "Prepay a fixed duration, or pay on exit by actual time?" — session model.
- "How many cities/lots and sessions per day?" — scale.
- "Who are the other users — enforcement officers, city operators?" — extra read APIs and reporting.
- "What payment methods — cards, wallets, accounts?" — PSP integration and PCI scope.

Don't ask: "Should I use Stripe?" · "What database?"

State: "Both street and garage; app, kiosk and plate cameras; 2M sessions/day across 500 cities; pay-on-exit for garages, prepay with extensions on street; cards and wallets through a payment processor; we must never double-charge."

3. Requirements

- **Must:** start/extend/end session; price calculation by zone rules; authorize and capture payment; enforcement lookup by plate; receipts; refunds.
- **Stretch:** dynamic pricing; reservations; fleet accounts.
- **Non-functional:** exactly-once charging (no double charge, no lost revenue); enforcement lookup p99 < 200 ms; gate decisions < 1 s and working during network loss; PCI DSS compliance; auditability.

4. Estimates

- 2M sessions/day ≈ 23/s average; ~150/s at morning peaks. Payments similar order. **This is not a throughput problem — it's a correctness and reliability problem.** Say that out loud.
- Enforcement lookups: 5k officers × 1 scan/30 s ≈ 170 RPS.
- Storage: 2M × 2 KB × 365 ≈ 1.5 TB/year — fits comfortably in a relational DB.

5. API

```
POST /v1/sessions            {zone_id, plate, payment_method_id, duration_min?}
Idempotency-Key: <uuid>
     → {session_id, ends_at, quoted_price}
POST /v1/sessions/{id}:extend {minutes}  Idempotency-Key
POST /v1/sessions/{id}:end   → {final_price, receipt_id}
GET  /v1/enforcement/plates/{plate}?zone_id=...  → {paid: bool, ends_at}
POST /v1/refunds             {session_id, amount, reason}  Idempotency-Key
```

6. Data model

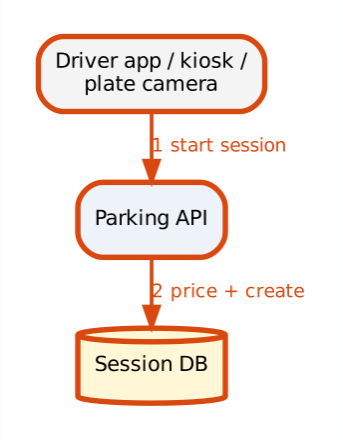
- `Zone(id, city, rules_version, timezone)`, `RateRule(zone_id, version, schedule, price_per_min, max_minutes)` — versioned so a session's price is reproducible.
- `Session(id, zone_id, plate, user_id, start, end, state, rules_version, quoted_price)`.
- `Payment(id, session_id, idempotency_key UNIQUE, amount, currency, state: CREATED→AUTHORIZED→CAPTURED|FAILED|REFUNDED, psp_ref)`.
- `LedgerEntry(id, payment_id, account, debit, credit, ts)` — double-entry, append-only.
- Enforcement view: `active_by_plate(plate, zone) → ends_at` in a cache, updated from session events.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

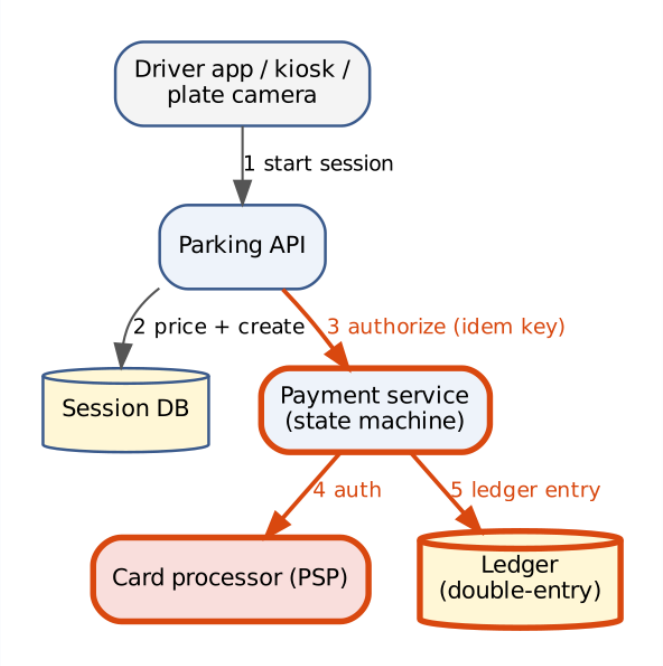
Step 1 · Parking domain (1 min)



> **You:** “Drivers start a session from the app, a kiosk or a plate camera. The parking API prices it using the zone's rule version and creates the session in a relational database — volume is only about 25 per second, so correctness, not scale, drives the design.”

Why this step now: Say the 'correctness, not scale' insight as early as possible.

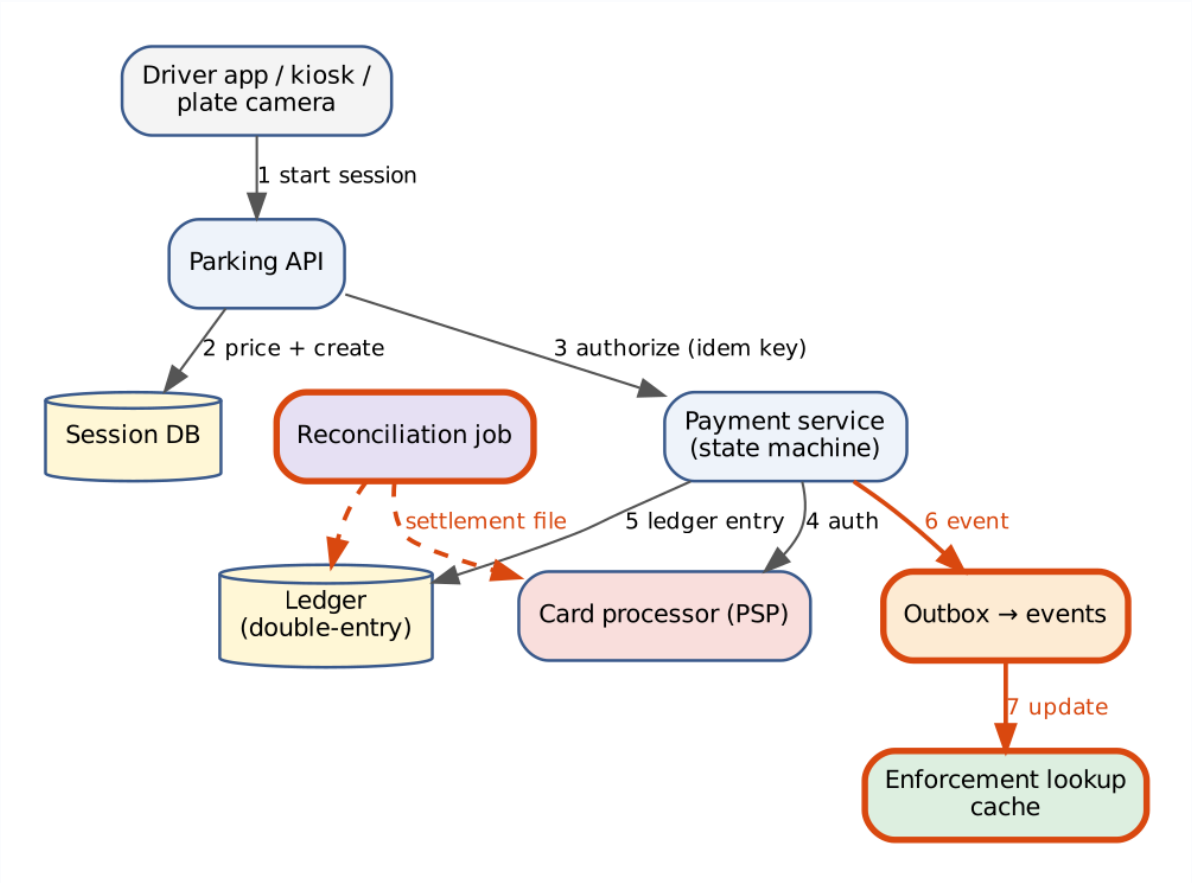
Step 2 · Money path (2 min)



> **You:** “The parking API asks the payment service to authorize, with an idempotency key. The payment service is a state machine: it records the payment, calls the processor with the same key, and writes double-entry ledger entries. Cards are tokenized by the processor, so we never store card numbers.”

Why this step now: Isolate money into its own service before anything else touches it.

Step 3 · Events, enforcement, reconciliation (1–2 min)



> **You:** “Every state change writes an event to an outbox in the same transaction; a relay publishes it. Enforcement reads a cache of paid plates updated from those events. A daily reconciliation job compares our ledger with the processor's settlement file and flags mismatches for finance.”

Why this step now: Secondary consumers and correctness checks come last.

Narrate one request end to end (numbers match the arrows)

Start and pay for a session

1. Driver starts a session (1); parking API prices it by rule version and creates it (2).
2. Parking API calls payment service to authorize with an idempotency key (3).
3. Payment service calls the processor with the same key (4); on timeout state = UNKNOWN.
4. Ledger entries written (5); event written to outbox in the same transaction (6).
5. Relay publishes; enforcement cache updated for the plate (7).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Driver app / kiosk / plate camera	Start, extend, end sessions	Multiple entry points, one session model	Mobile app, kiosk, ALPR

Parking API	Sessions and pricing by zone rule version	Owns the parking domain	Service
Session DB	Sessions, zones, versioned rate rules	Low volume, needs transactions	Postgres
Payment service (state machine)	Authorize, capture, refund with idempotency	Money correctness lives in one place	Service + Postgres
Card processor (PSP)	Charges cards, tokenizes PANs	Keeps us out of most PCI scope	Stripe / Adyen / bank PSP
Ledger (double-entry)	Append-only record of every money movement	Explain every cent	Postgres append-only tables
Outbox → events	Publish payment/session events reliably	No dual-write bugs	Outbox table + relay to Kafka
Enforcement lookup cache	plate+zone → paid until	Officers need <200 ms checks	Redis
Reconciliation job	Compare ledger vs PSP settlement files	Catches unknown states and PSP errors	Daily batch

Before you leave the high-level design, check you have said:

☐ 25 TPS → correctness problem
☐ Idempotency end to end incl. timeouts
☐ Ledger + reconciliation
☐ Tokenization / PCI

60-second summary to say once the diagram is complete:

> **You:** "Three parts. The parking side owns zones, pricing rules and sessions. The payment service owns money movement as a state machine with idempotency keys and a double-entry ledger. And an event stream feeds enforcement lookups, receipts and reporting."
>
> "When a driver starts a session, the parking API prices it using the zone's rule version, creates the session, and asks the payment service to authorize the card. On end — or at expiry for prepaid — we capture the final amount. Every state change writes to the ledger and to an outbox in the same transaction, and a relay publishes events. Enforcement reads a cache of active sessions by plate, updated from those events."

8. Deep dives

Deep dive 1: never double-charge — idempotency and the payment state machine.

- Client generates an idempotency key per user intent; payment service stores `(key → result)` with a unique constraint inside the same transaction that creates the payment record.

- Call the processor with *our* idempotency key too, so retries after a timeout don't create a second charge.
- Timeouts are the dangerous case: if the processor call times out, the state is UNKNOWN — don't retry blindly; query the processor by our reference, or let reconciliation resolve it.

```
def authorize(session_id, amount, idem_key):
    with db.transaction():
        existing = db.payments.get_by_key(idem_key)
        if existing:
            return existing                          # replay returns same result
        p = db.payments.insert(session_id=session_id, amount=amount,
                               idem_key=idem_key, state="CREATED")
    try:
        r = psp.authorize(amount, reference=p.id, idempotency_key=idem_key, timeout=3)
        new_state = "AUTHORIZED" if r.approved else "FAILED"
    except Timeout:
        new_state = "UNKNOWN"                        # resolved by status query /
reconciliation
    with db.transaction():
        db.payments.update_state(p.id, new_state)
        db.ledger.append_for(p, new_state)           # double-entry if money moved
        db.outbox.insert(event="payment."+new_state.lower(), payment_id=p.id)
    return db.payments.get(p.id)
```

Deep dive 2: the ledger and reconciliation.

- Double-entry: every movement debits one account and credits another; the sum of all entries is zero. Append-only — corrections are new entries, never updates.
- Daily reconciliation compares our ledger with the processor's settlement file; mismatches go to an exceptions queue for finance ops. This is how you catch UNKNOWN states and processor errors.
- *Say:* "The ledger is the source of truth for money; everything else is a view."

Deep dive 3: gates that keep working offline.

- Garage gate controllers cache recent entries and valid passes locally; on exit with no connectivity, they record the exit, open the gate (or charge a capped fee) and sync later. Business decision: revenue risk vs blocking cars. Say you'd decide it with the operations lead.

9. Scaling evolution

Stage	Design
Single city	Monolith + Postgres + one processor
Target (500 cities)	Separate payment service, ledger, outbox, enforcement cache, rules versioning
10× / international	Regional deployments for data residency, multiple processors with routing/failover, multi-currency ledger

10. Failures

- Processor down → queue captures for later (authorization holds); for new sessions, allow "pay later" with account on file, or second processor.
- Duplicate client requests → idempotency.
- Cache/enforcement view stale → read-through to the session DB on misses; officers see "last updated" time.
- Ledger/processor mismatch → reconciliation exceptions queue; never auto-"fix" money without review.

11. Trade-offs

Decision	Options	Choice	Why
Datastore	NoSQL vs relational	Relational	Low volume, transactions and constraints matter most
Auth-then-capture vs immediate charge		Auth at start, capture at end	Final price unknown until exit
Processor timeouts	Retry vs mark UNKNOWN	UNKNOWN + status query + reconciliation	Blind retry risks double-charge
Gate offline	Block vs allow-and-sync	Allow with cap, sync later	Customer experience; bounded revenue risk

12. Security, operations, cost

- **Security:** never store card numbers — tokenize via the processor to stay out of most PCI scope; encrypt plates (PII in some jurisdictions); least-privilege for refunds with approval thresholds; audit every refund.
- **Operations:** SLIs — authorization success rate, capture success, enforcement lookup latency, reconciliation mismatch count. Page on auth success drop (could be a processor outage); alert finance on mismatches.
- **Cost:** processor fees dominate; batch small captures where rules allow; route by cost across processors.

13. Manager lens

> **You:** "I'd separate a payments team from the parking product team early, because payments has different risk, compliance and on-call needs. Phase 1: one city, one processor, ledger from day one — retrofitting a ledger is painful. Phase 2: enforcement and reconciliation. Phase 3: multi-processor and international. The decision I'd keep: money correctness beats feature speed, so the payments team has veto on anything touching the ledger."

14. Follow-ups

1. **"The processor charged the card but we timed out. What happens?"** — Payment is UNKNOWN; we query the processor by our reference; if charged, mark AUTHORIZED; reconciliation catches anything missed. Never retry without the idempotency key.
2. **"How do you change prices without affecting active sessions?"** — Versioned rules; session records the rule version it started under.
3. **"How do you handle refunds safely?"** — Refund is a new payment-like record with its own idempotency key, approval threshold, ledger entries reversing the original.
4. **"How do enforcement officers check a plate quickly?"** — Cache keyed by plate+zone with ends_at; updated from session events; read-through fallback.
5. **"Multiple currencies?"** — Store amounts as integer minor units with currency code; ledger per currency; FX at settlement.

15. Mock transcript

> **Interviewer:** Design a parking payment system.
>
> **You:** Street or garage, how drivers pay, prepay or pay-on-exit, scale? … 2M sessions a day is only ~25 per second, so this is a correctness problem, not a scale problem. My priorities are never double-charging, always being able to explain every cent, and keeping gates working offline.
>
> **Interviewer:** How do you prevent double charges?
>
> **You:** Idempotency keys stored with a unique constraint in the same transaction as the payment row, passed through to the processor. And on a timeout I don't retry — I mark the payment unknown and query the processor.
>
> **Interviewer:** Why a ledger?
>
> **You:** Because finance will ask me to explain every cent. Double-entry, append-only, reconciled daily against the processor's files.

16. Scoring guide

Rating	Signals
Strong Hire	Calls out correctness over scale; idempotency end to end incl. timeouts; state machine; ledger + reconciliation; PCI tokenization; offline gates trade-off
Hire	Idempotency and transactions; basic payment flow
Lean No Hire	Heavy scale design for 25 TPS; no timeout handling
No Hire	Stores card numbers; retries charges blindly

17. Mistakes and "say this"

Mistakes: over-engineering for scale; ignoring the timeout case; updating balances in place.

Say this: "At 25 transactions a second, this is a correctness problem, not a scale problem." · "A timeout is an unknown, not a failure." · "The ledger is the source of truth for money; everything else is a view."

18. Your tie-in

Insurance has premium billing, payments and reconciliation with carriers/brokers. If your org touched billing or premium flows, use it. [VERIFY]

Q6. Design the software behind an Amazon pickup location with lockers

> Amazon (Low — older SDM report; strong domain fit) · Source: IGotAnOffer Amazon SDM question list

1. The prompt

"Design Amazon Locker: customers choose a locker at checkout, a driver deposits the package, the customer picks it up with a code." Variants: package doesn't fit; returns via locker; locker offline.

2. Clarify

Ask: "How many locker sites and slots?" · "Locker sizes and how we match packages?" · "How long do we hold packages?" · "Does the locker have reliable connectivity?" · "Returns too, or just deliveries?"

Don't ask: "What hardware do the lockers run?" (assume an embedded controller with a small local DB and intermittent connectivity).

State: "50k locker sites, ~60 slots each in 3 sizes, 3-day pickup window, intermittent connectivity, deliveries and returns."

3. Requirements

- **Must:** show available lockers at checkout; reserve a slot for the delivery date; driver deposit; customer pickup by code; expiry and return-to-sender.
- **Non-functional:** never assign the same slot twice; pickup works offline at the locker; codes secure against guessing.

4. Estimates

- 50k × 60 = 3M slots. If each turns over every ~2 days → 1.5M deliveries/day ≈ 17/s; checkout availability queries much higher (~5k RPS at peak, cacheable per region).

5. API

```
GET  /v1/lockers?lat=..&lng=..&package_dims=..&date=..  → [{site_id, distance,
available_sizes}]
POST /v1/reservations {site_id, size, order_id, window}  Idempotency-Key → {reservation_id}
POST /v1/sites/{id}/deposits {reservation_id, slot_id, driver_id}       (from driver app /
locker)
POST /v1/sites/{id}/pickups  {code}                                       (at locker;
offline-capable)
```

6. Data model

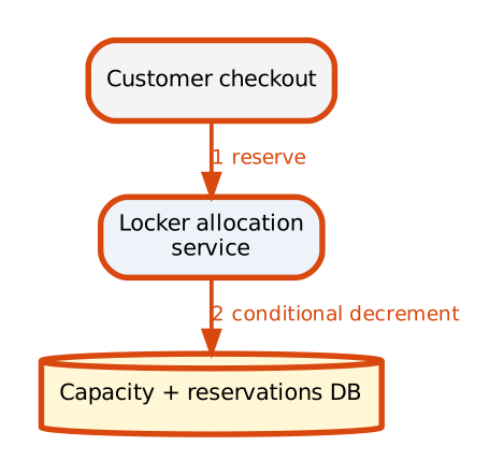
- `Site(id, geo, timezone, status)`, `Slot(site_id, slot_id, size, state: FREE|RESERVED|OCCUPIED|OUT_OF_SERVICE, reservation_id)`.
- `Reservation(id, order_id, site_id, size, window_start, window_end, state, code_hash)`.
- Capacity per site/date/size (counter) for fast availability: `capacity(site, date, size) = slots - reserved`.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

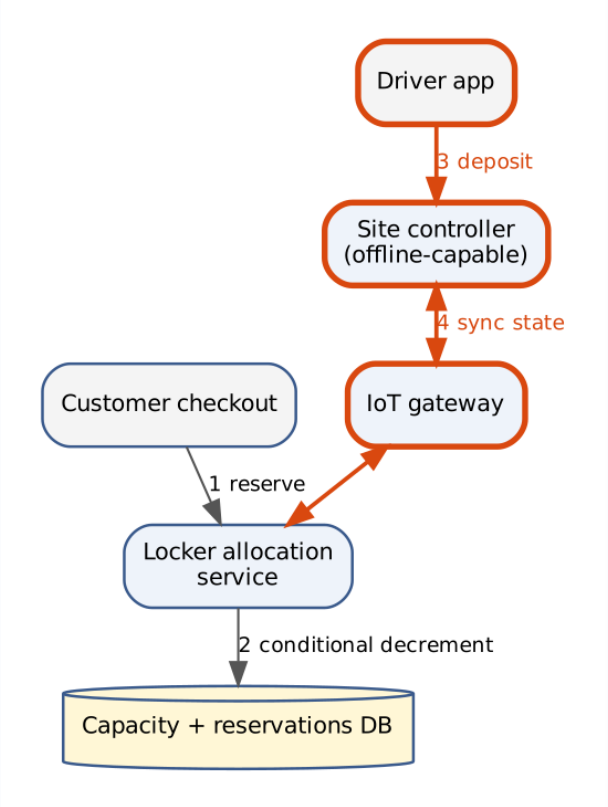
Step 1 · Reserve capacity at checkout (1 min)



> **You:** "At checkout the customer picks a site. The allocation service reserves capacity — a conditional decrement of the counter for that site, delivery date and size — not a physical slot, because the delivery date and final box size can change."

Why this step now: Lead with the key insight: capacity early, physical slot late.

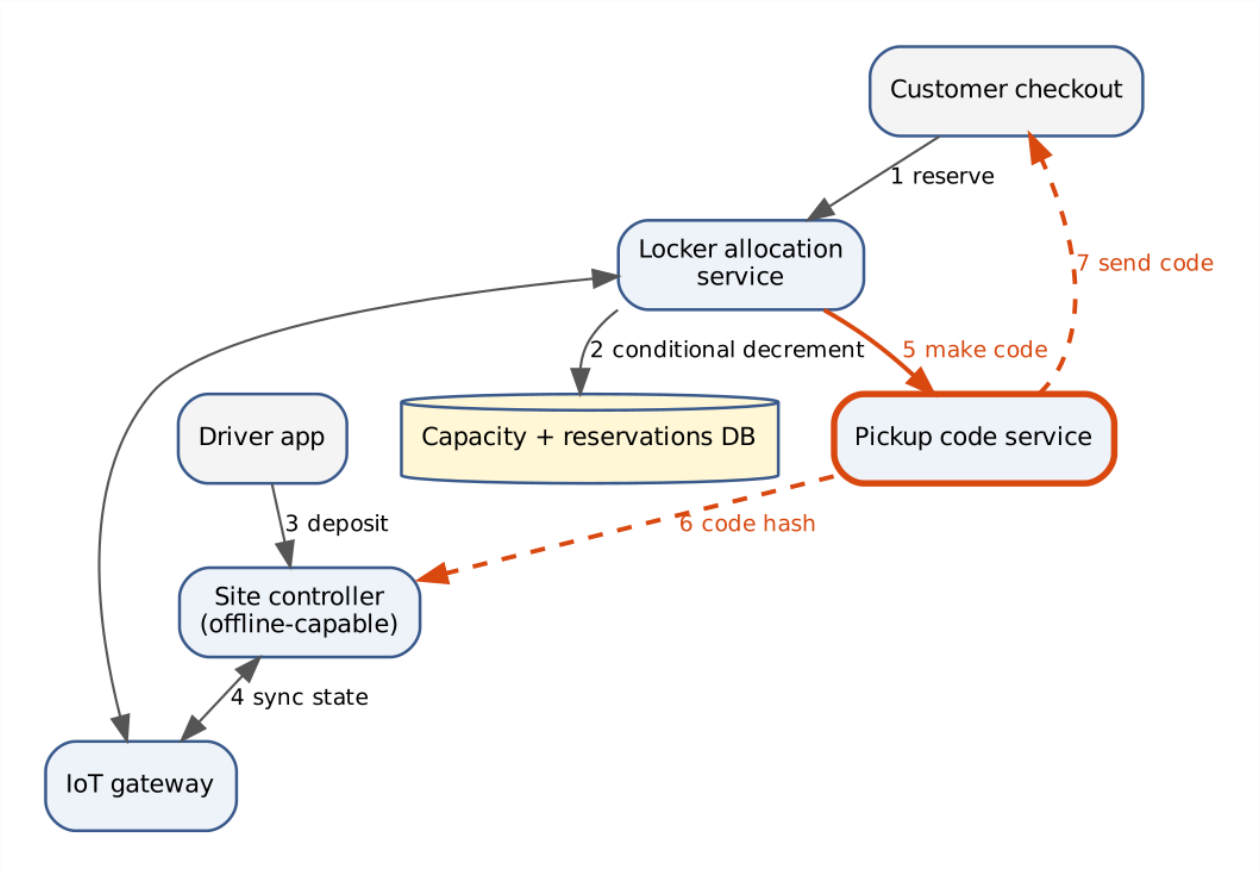
Step 2 · Deposit at the site (1–2 min)



> **You:** “When the driver arrives, the site controller picks a free slot of the right size and opens it. The controller is the single writer for its own slots and keeps a local database, so it works offline and syncs state through an IoT gateway.”

Why this step now: Physical-world constraints (offline) shape the design.

Step 3 · Pickup (1 min)



> **You:** "A code service generates a random one-time code, sends the customer the code and the site only its hash. Pickup is verified locally at the locker, rate-limited per keypad."

Why this step now: Security and offline pickup finish the flow.

Narrate one request end to end (numbers match the arrows)

Delivery and pickup

1. Checkout reserves capacity (1) via a conditional counter decrement (2).
2. Driver deposits (3); controller assigns a slot and syncs state (4).
3. Code service creates a code (5), sends hash to the site (6) and code to the customer (7).
4. Customer enters the code at the locker; verified locally, door opens.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Customer checkout	Pick a locker site	Where demand enters	Retail app
Locker allocation service	Reserve capacity by site/ date/size	Prevents overbooking	Service
Capacity + reservations DB	Counters and reservations	Conditional decrements	DynamoDB / Postgres

Driver app	Deposit packages	Physical workflow	Mobile app
IoT gateway	Secure device messaging	Sites are intermittently connected	AWS IoT Core / MQTT
Site controller (offline-capable)	Assign physical slot, open doors, verify codes	Single writer for its slots; works offline	Embedded controller + local DB
Pickup code service	Generate codes, store hashes	Secure offline pickup	Service

Before you leave the high-level design, check you have said:

☐ Capacity vs physical slot
☐ Single writer per site
☐ Offline pickup with hashed codes

60-second summary to say once the diagram is complete:

> **You:** "The key insight is that we reserve *capacity* at checkout and assign a *physical slot* at deposit time. At checkout we don't know exactly when the package arrives or its final box size, so we decrement a capacity counter for that site, date and size. When the driver arrives, the locker controller picks a free slot of the right size and opens it."
>
> "Each site has a controller with a local database: it knows its slots and the pickup codes for packages inside. It syncs with the cloud through an IoT gateway — MQTT with per-device certificates. Pickup works offline because the code check is local."

8. Deep dives

Deep dive 1: no double-booking. Capacity decrement is a conditional write (`UPDATE capacity SET reserved = reserved + 1 WHERE site=? AND date=? AND size=? AND reserved < slots`). Physical slot assignment happens at the site controller, which is the single writer for its own slots — no distributed race.

Deep dive 2: offline pickup security. Cloud generates a random 8+ digit code per reservation, stores only a hash, sends the hash (and slot mapping on deposit) to the site. Locker verifies locally. Rate-limit attempts per keypad (lock out after N failures). Codes expire. The QR option encodes a signed token the locker can verify with a public key.

Deep dive 3: package doesn't fit. Driver app offers the next size up if free; otherwise reroute to the nearest site with capacity and notify the customer. Track mis-sizing rate per seller as a metric (it's a product-quality signal).

9–11. Scaling, failures, trade-offs

- **Failures:** site offline → deposits/pickups recorded locally, synced later; cloud treats site state as eventually consistent and holds a safety buffer of capacity. Controller hardware failure → site marked out of service; reroute reservations.
- **Trade-off:** reserve physical slot at checkout (simple, wastes capacity because delivery dates slip) vs capacity counters + assign at deposit (better utilization) → **choose the latter**.
- **Scale:** availability queries cached per region with short TTL; reservation writes partitioned by site.

12. Security, operations, cost

- Device certificates, signed commands to open doors, tamper alarms, video at sites.
- SLIs: deposit success rate, pickup success, site online %; alert on sites offline > 30 min or repeated door faults.
- Cost: minimize truck reroutes (expensive) via accurate capacity.

13. Manager lens

> **You:** "Two teams with different release cadences: cloud services ship daily; locker firmware ships monthly through staged OTA. The contract between them — the sync protocol — is the thing I'd guard most carefully, versioned and backward compatible."

14. Follow-ups

1. **"Delivery slips a day — what happens to the reservation?"** — Capacity counter moves to the new date; if full, offer alternatives.
2. **"Customer doesn't pick up?"** — After 3 days, slot flagged for return; driver collects; refund flow triggered.
3. **"How do returns work?"** — Customer gets a code to open an empty slot; locker reports occupancy; driver picks up.
4. **"How do you know a door is actually closed with a package inside?"** — Door sensors + weight/IR sensor; mismatch triggers a support case.

15–17. Mock, scoring, say this

> **Interviewer:** Design the software for Amazon lockers.
>
> **You:** I'll reserve capacity at checkout and assign the physical slot at deposit, because delivery timing is uncertain. The site controller is the single writer for its slots and works offline, so pickup never depends on the network.

Strong Hire: capacity-vs-slot insight; offline-first site; secure code design; reroute flow. **No Hire:** central DB assigns slots at checkout, pickup requires network.

Say this: "Reserve capacity early, assign the physical slot late." · "The site is the single writer for its own slots."

Q7. Design a promotion system: "10% cash back on a particular credit card" (and coupons)

> Amazon (Low–Medium) · Exponent reports a coupon system as an Amazon low-level design (L4) question · Source: IGotAnOffer Amazon SDM list; Exponent Amazon SD guide (2026)

1. The prompt

"Design a system that gives 10% cash back when customers pay with a specific co-branded card, capped at $50/month." Variants: coupon codes; budget-limited campaigns; stacking rules.

2. Clarify

Ask: "Applied at checkout as a discount, or credited after settlement?" (changes fraud and returns handling) · "Budget limits per campaign and per customer?" · "What happens on returns?" · "Stacking with other promotions?" · "Scale — orders per second at peak (Prime Day)?"

State: "Cash back credited after the card transaction settles, 10% capped at $50/customer/month, campaign budget $10M, reversed on returns, stacks with coupons but not with other card offers; peak 50k orders/s on Prime Day."

3. Requirements

- **Must:** define campaigns (rules, eligibility, budget, dates); show estimated reward at checkout; compute and credit after settlement; enforce caps exactly; reverse on refund.
- **Non-functional:** checkout evaluation p99 < 20 ms (in the critical path); never exceed budget by more than a tiny tolerance; auditable.

4. Estimates

- Checkout evaluations: 50k/s peak × (say) 3 promotion checks = 150k evaluations/s → rules must be evaluated **in memory**, not by DB queries.
- Redemptions: only for eligible card payments, maybe 5% of orders → 2.5k/s peak — counters must handle hot keys (the campaign budget counter).

5. API

```
POST /v1/promotions:evaluate {customer_id, cart, payment_method}  → {applicable:[{promo_id, est_reward}]}
POST /v1/campaigns {rules, budget, per_customer_cap, start, end}   (admin)
Events consumed: order.placed, payment.settled, order.refunded
```

6. Data model

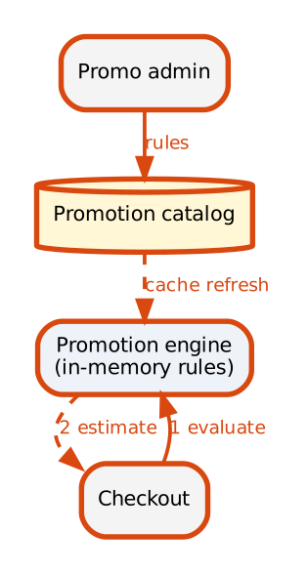
- `Campaign(id, rule_expr, budget_total, budget_used, per_customer_cap, period, state)`.
- `CustomerUsage(customer_id, campaign_id, period, amount_used)` — conditional increments.
- `Redemption(id, order_id, campaign_id, amount, state: PENDING|CREDITED|REVERSED)` with unique `(order_id, campaign_id)` for idempotency.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

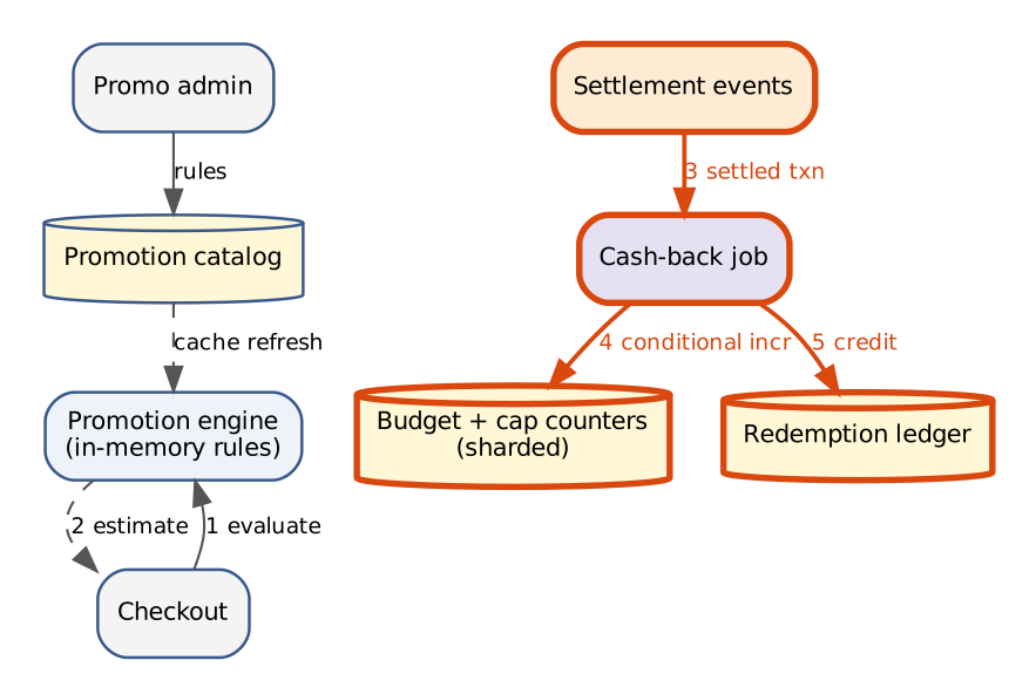
Step 1 · Fast path at checkout (1 min)



> **You:** “Marketing defines campaigns in a catalog. The promotion engine caches compiled rules in memory, so checkout can ask 'what applies?' and get an estimate in a few milliseconds. Nothing is committed here.”

Why this step now: Separate the fast path from the money path immediately.

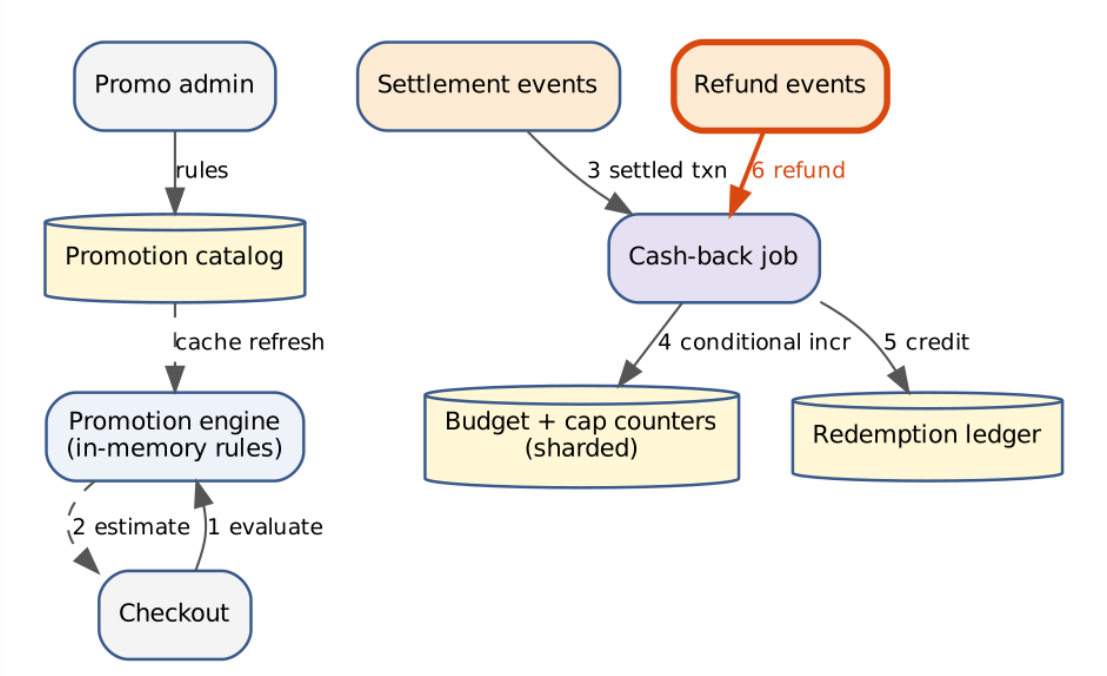
Step 2 · Money path after settlement (2 min)



> **You:** "When the card transaction settles, a cash-back job consumes the event, re-checks eligibility, and does conditional increments on the customer's monthly cap and the campaign budget. If both succeed, it writes a redemption and credits the customer."

Why this step now: Exactness lives here.

Step 3 · Reversals (30 s)



> **You:** “Refund events trigger reversal entries and restore cap usage.”

Why this step now: Close the money loop.

Narrate one request end to end (numbers match the arrows)

Earn cash back

1. Checkout asks the engine (1) and shows an estimate (2).
2. Settlement event arrives (3); job increments caps and budget conditionally (4).
3. Redemption written and customer credited (5).
4. On refund (6), proportional reversal and cap restored.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Promo admin	Define rules, budgets, dates	Marketing owns campaigns	Admin UI
Promotion catalog	Versioned campaign rules	Small dataset, cached everywhere	Postgres
Checkout	Asks for applicable promotions	Fast path	Retail checkout
Promotion engine (in-memory rules)	Evaluate eligibility, estimate reward	150k evals/s need in-memory rules	Service with cached compiled rules
Settlement events	Card transaction settled	Credit only real, settled spend	Kafka
Cash-back job	Re-check eligibility, enforce caps, credit	Money path, exact	Consumer workers
Budget + cap counters (sharded)	Conditional increments	Exact caps; hot key sharded	DynamoDB / Postgres
Redemption ledger	Credits and reversals	Auditable money	Postgres
Refund events	Returns trigger reversals	Cap usage restored	Kafka

Before you leave the high-level design, check you have said:

☐ Estimate vs commit split
☐ Exact caps under concurrency
☐ Hot budget counter sharded

60-second summary to say once the diagram is complete:

> **You:** "I'd split the fast path from the money path. At checkout, the promotion engine evaluates rules in memory — campaigns are a small dataset, cached in each instance and refreshed on change — and shows an *estimated* reward. Nothing is committed yet."

> "When the card transaction settles, a cash-back job consumes the settlement event, re-evaluates eligibility, and does conditional increments on the customer's monthly usage and the campaign budget. If both succeed, it writes a redemption and credits the customer. Refunds generate reversal entries."

8. Deep dives

Deep dive 1: exact caps under concurrency. `UPDATE customer_usage SET used = used + :amt WHERE customer=? AND campaign=? AND period=? AND used + :amt <= cap` — check rows affected. If the cap is partially left, credit the remainder (`min(10%, cap - used)`).

Deep dive 2: the hot budget counter. One campaign budget row at 2.5k increments/s is a hot key. Options: (a) split the budget into N sub-counters (shards), each with budget/N, increment a random shard, rebalance remaining budget periodically; (b) allocate budget in blocks to each worker (lease 1,000 dollars at a time). Accept tiny overspend tolerance (< 0.01%) and say so. This is a classic Amazon-scale insight.

Deep dive 3: rule engine. Rules as a small DSL compiled to predicates (`card_bin IN [...] AND category NOT IN [gift_cards] AND order_total >= 25`). Versioned; tested in a sandbox against historical orders before launch ("what would this have cost last week?").

9–11. Failures and trade-offs

- Settlement events duplicated → unique (order_id, campaign_id).
- Engine cache stale after campaign end → campaigns carry end time; evaluation checks it locally.
- Trade-off: credit at checkout (instant gratification, fraud and return risk) vs after settlement (safer) → after settlement for cash back; at checkout for coupons with stricter limits.

12–13. Security, ops, cost, manager lens

- Fraud: account farming for promotions → velocity checks, device fingerprinting, per-payment-instrument caps.
- Ops: dashboard of budget burn rate vs plan; alert when burn rate predicts exhaustion before campaign end.
- Manager lens: "Marketing owns campaign definitions; engineering owns guardrails. I'd require a cost simulation on historical data before any campaign over $1M launches — that's a mechanism, not a meeting."

14. Follow-ups

1. **"Budget runs out mid-day — what do customers see?"** — Checkout estimates disappear once remaining budget < threshold; edge cases credited if eligible at order time per policy.
2. **"Customer returns half the order?"** — Proportional reversal; cap usage restored.

3. **"How do you test a new promotion safely?"** — Shadow-evaluate on live traffic without crediting; compare predicted cost.

15–17. Mock, scoring, say this

> **You:** "Checkout gets an estimate from in-memory rules; money is committed only after settlement with conditional increments. The budget counter is a hot key, so I'd shard it and accept a tiny overspend tolerance."

Strong Hire: fast path vs money path split; exact caps; hot counter sharding; reversals; simulation mechanism. **No Hire:** DB query per rule per checkout; no concurrency control on caps.

Say this: "Estimate on the fast path, commit on the money path." · "A single budget row is a hot key — I'd shard it and own the tolerance explicitly."

Q8. Implement Alexa voice search ("Alexa, order paper towels")

> Amazon (Medium) · Source: Aced Amazon SDM guide (2026)

1. The prompt

"Design voice search for shopping on Alexa." Variants: reorder previous purchases; disambiguation by voice; multi-language.

2. Clarify

Ask: "End-to-end from audio, or start from text after speech recognition?" (scopes ML vs search) · "Is the goal search results or completing a purchase?" · "Latency budget for a voice response?" · "Do we personalize with purchase history?" · "Languages/locales?"

State: "I'll design from audio to response, focused on the search/ranking path; ASR and NLU are existing services I'll treat as components with SLAs. End-to-end response under ~1.5 s. Personalized. Purchase requires confirmation."

3. Requirements

- **Must:** understand intent (search, reorder, add to cart); return 1–2 spoken results (not 20 — voice is a narrow channel); confirm before purchase; use history ("the usual").
- **Non-functional:** p95 response < 1.5 s end to end; high precision for top result; safe purchasing (no accidental orders).

4. Estimates

- Say 50M shopping voice queries/day ≈ 600/s avg, ~2k/s peak. Latency budget split: ASR ~300 ms, NLU ~100 ms, retrieval + ranking ~200 ms, response + TTS ~300 ms, network ~300 ms.

5. API (internal)

```
POST /v1/voice/shopping:query {user_id, device_id, locale, transcript, nlu:{intent, slots}}
  → {speech_text, candidates:[{asin, title_short, price}], action: confirm|clarify|none}
POST /v1/voice/shopping:confirm {session_id, asin, quantity}
```

6. Data model

- Product index: ASIN, title, short spoken title, brand, category, attributes, availability, price, popularity.

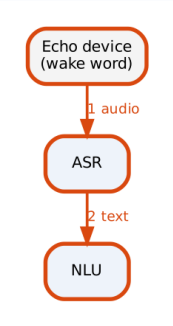
- User context: purchase history (frequently reordered items), Prime status, default address, preferences.
- Session: dialog state for multi-turn clarification.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

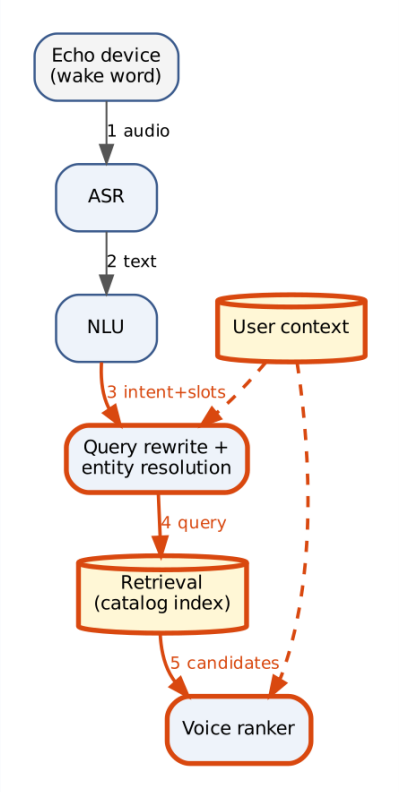
Step 1 · Understand the request (1 min)



> **You:** “Wake word detection runs on the device. Audio streams to speech recognition, then NLU extracts the intent — buy — and slots like product and quantity. I'll treat ASR and NLU as existing services with latency budgets.”

Why this step now: Scope the ML pieces as components; the shopping problem starts after NLU.

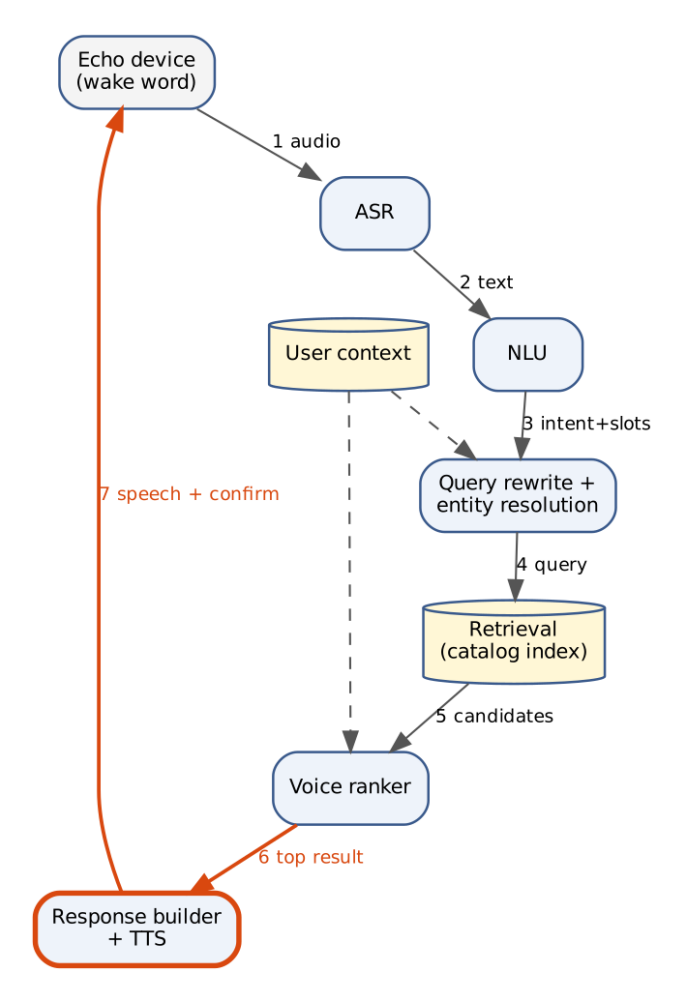
Step 2 · Find the right product (2 min)



> **You:** “A query rewrite step resolves things like 'my usual coffee' from purchase history and corrects ASR errors. Retrieval pulls candidates from the catalog index, and a voice-specific ranker picks one or two, heavily personalized.”

Why this step now: This is the core of the question.

Step 3 · Respond safely (1 min)



> **You:** “The response builder says one short result and asks for confirmation before buying; text-to-speech plays it on the device.”

Why this step now: Purchasing by voice must be safe.

Narrate one request end to end (numbers match the arrows)

'Alexa, order paper towels'

1. Device streams audio (1); ASR returns n-best text (2).
2. NLU extracts intent=buy, product=paper towels (3).
3. Rewrite resolves against history; retrieval gets candidates (4, 5).
4. Ranker picks the top item (6); device says it and asks to confirm (7).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Echo device (wake word)	Detect wake word, stream audio	On-device for privacy and latency	Device firmware

ASR	Speech → text (n-best)	Existing service with SLA	Speech model
NLU	Intent + slots	Turns text into a shopping action	NLU model
Query rewrite + entity resolution	Resolve 'the usual', fix ASR errors	Voice queries are noisy and personal	Service + history lookup
Retrieval (catalog index)	Candidates by BM25 + vectors	Find plausible products fast	Search cluster
Voice ranker	Pick 1–2 results	On voice only position 1 matters	Learning-to-rank model
User context	Purchase history, Prime, address	Personalization is the strongest signal	Feature store / KV
Response builder + TTS	Short spoken answer, confirmation	Safe purchasing	TTS service

Before you leave the high-level design, check you have said:

☐ Latency budget per stage (~1.5 s total)
☐ Voice = one-result interface
☐ Confirmation and purchase safety

60-second summary to say once the diagram is complete:

> **You:** "The pipeline is wake word on the device, then speech-to-text, then intent and slot extraction — 'buy', product 'paper towels', quantity. The shopping part starts with a query rewrite that resolves 'the usual' or 'my coffee' from purchase history and fixes ASR errors. Retrieval pulls candidates from the catalog index, a ranker optimized for voice picks one or two, and the response builder says something short and asks for confirmation."
>
> "Voice changes the ranking problem: with a screen, top-10 relevance is fine. On voice, only position one matters, so precision at 1 and personalization dominate."

8. Deep dives

Deep dive 1: query understanding with ASR noise. ASR returns an n-best list ("paper towels" / "paper tower"). Score candidates against catalog vocabulary and user history; phonetic matching; prefer interpretations that match past purchases. Low confidence → ask a clarifying question rather than guess.

Deep dive 2: voice-friendly ranking. Features: past purchases of this item/brand (strongest), Prime eligibility, availability, rating, price; penalize items with long, unpronounceable titles; use a shortened spoken title. Offline learning-to-rank trained on voice outcomes (confirmed purchases vs "no"), online A/B tests measuring confirmation rate and returns.

Deep dive 3: safe purchasing. Always confirm; voice PIN option; kids/household profiles; purchase limits; easy cancellation window. Track accidental-order rate as a guardrail metric.

9–11. Scaling, failures, trade-offs

- Latency: cache popular queries' candidate sets; precompute per-user "reorder" lists; run retrieval and personalization lookups in parallel.
- Failure: ranking service slow → fall back to "your usual" item or popularity-ranked results; never block the whole response.
- Trade-off: one result (simple, risky) vs three options read aloud (safer, tedious) → one strong result with confirmation, offer "or something else?" when confidence is moderate.

12–13. Security, ops, manager lens

- Privacy: voice recordings and transcripts are sensitive — retention controls, user deletion, minimal logging.
- Metrics: confirmation rate, abandonment, accidental orders, p95 latency per stage.
- Manager lens: "Four teams touch this — ASR, NLU, search, and the voice experience team. I'd own the end-to-end latency budget and the success metric (confirmed purchases without returns), and split the budget per team so each knows its target."

14. Follow-ups

1. **"User says 'order batteries' — which ones?"** — If history has a clear winner, propose it; else ask one clarifying question (size); never read a list of ten.
2. **"How do you evaluate a new ranker?"** — Offline replay on logged sessions, then A/B on confirmation rate and returns.
3. **"New locale?"** — Locale-specific ASR/NLU models, catalog availability per marketplace, localized spoken titles.

15–17. Mock, scoring, say this

> **You:** "On voice, only position one matters. So I'd optimize precision at one with personalization, keep a 1.5-second budget split across stages, and always confirm before buying."

Strong Hire: latency budget per stage; voice-specific ranking insight; ASR noise handling; safety guardrails; cross-team ownership. **No Hire:** treats it as text search returning a list.

Say this: "Voice is a one-result interface." · "I'd own the end-to-end latency budget and give each team its slice."

Chapter 2 — Apple

Apple at a glance

Loop	Hiring manager screen → system design with an engineer (a real problem the team is working on) → final day of four back-to-back rounds: people management, code review/ coding, cross-org partnership (e.g., finance), second system design (low-level)
Coding	Yes — CoderPad, easy–medium (e.g., validate a Sudoku grid), with optimization follow-ups; AI tools have been allowed in at least one recent loop — ask
How design is scored	Requirements discipline, specific defensible choices, operational realism (migration, backwards compatibility, partial failure), low-level depth (indexes, API surface, partitioning), regionalization
Strong Hire signals	You engage with the interviewer's real problem; you go from boxes to index structures and API contracts; privacy and data residency instincts; migration without customer impact
Pitfalls	Textbook patterns with no specifics; not engaging with the team's domain; a generic "Why Apple?"
Culture	Teams set their own formats; EMs expected to be hands-on technically

Apple question bank (reported, 2026)

#	Question	Confidence	Source
Q9	Migrate data off a legacy storage system without disrupting customers or losing data; run two systems in parallel; backwards compatibility	Medium–High (multiple related prompts in one report set)	Aced Apple EM guide (2026)
Q10	Design a new data storage system from scratch — index structure, APIs, regionalization for EU users; design a key-value store	Medium	Aced Apple EM guide (2026)
Q11	Design a web cache	Medium	Aced Apple EM guide (2026)
Q12	Design a load balancer from scratch	Medium	Aced Apple EM guide (2026)
Q13	Observability platform + forecasting host spend with a finance partner	Medium (cross-org round)	Exponent Apple EM experience (2026)
—	Validate a 3×3 Sudoku grid (coding)	Medium	Aced Apple EM guide (2026)

Apple-specific habit: the first design round is often the interviewer's actual problem. Ask about the existing system, its constraints and what's been tried. Treat it as a working session with a colleague. Bring privacy and regionalization into every design without being asked.

Q9. Migrate data off a legacy storage system without disrupting customers

> Apple (Medium–High) · also Amazon (reverse design), Microsoft, Databricks · Source: Aced Apple EM guide (2026)

1. The prompt

"We have a legacy storage system that's hitting scalability and partitioning limits. We're building a new in-house store. How do you migrate without disrupting the customer experience or losing data?" Follow-ups reported: "How do you run both systems in parallel?" and "How do you make the new store backwards compatible with existing data layers?"

2. Clarify

Ask (this is the round where questions matter most):

- "What's the data — size, number of records, and access pattern (key lookups, scans, writes per second)?" — sets backfill duration and cutover design.
- "What are the legacy system's specific limits — partition hot spots, max size, latency?" — tells you what the new design must fix, not just copy.
- "Who are the clients — one service, or many teams calling it directly?" — many direct clients means you need a facade first.
- "Can we tolerate any downtime or read-only windows?" — usually no, but ask.
- "Is the schema changing, or just the storage engine?" — schema change adds transformation and dual-schema reads.
- "What does 'losing data' mean here — any write lost, or eventual repair acceptable?" — defines verification rigor.
- "Is there a deadline — e.g., legacy hardware end of life?" — drives risk appetite.

Don't ask: "Should we just do a big-bang cutover?" (you know the answer) · "What tool do you use for CDC?" (decide).

State: "I'll assume 2 PB, ~5 billion records, 200k reads/s and 20k writes/s at peak, dozens of client services, no downtime allowed, schema evolving slightly, and zero tolerance for lost writes."

3. Requirements

- **Must:** zero customer-visible downtime; no lost or corrupted writes; ability to roll back at every step; backwards compatibility for existing clients during transition; verifiable correctness.
- **Non-functional:** migration must not degrade legacy performance (throttle backfill); latency at cutover equal or better; progress observable.

4. Estimates

- Backfill: 2 PB at 2 GB/s sustained throttled throughput ≈ 1M seconds ≈ **12 days**. Plan for 3–4 weeks with throttling during peaks and retries.
- Dual-write overhead: 20k writes/s × 2 → new store must already handle production write load before cutover.
- Shadow reads at 1% sampling = 2k comparisons/s → comparator must be lightweight and async.

5. API (the facade)

```
interface Store {
  get(key, opts{consistency}) -> Record | NotFound
  put(key, record, opts{if_version}) -> version
  delete(key, opts{if_version})
  scan(prefix, cursor, limit) -> page
}
// Routing config (per key range / client / cohort):
//   mode ∈ {LEGACY_ONLY, DUAL_WRITE_LEGACY_READ, DUAL_WRITE_SHADOW_READ,
NEW_READ_LEGACY_FALLBACK, NEW_ONLY}
```

6. Data model

- **Migration state per key range:** `(range_id, mode, backfill_watermark, verified_until, last_mismatch_rate)`.
- **Records carry a version** (monotonic, e.g., legacy update timestamp + sequence) so the newest write always wins regardless of which path wrote it — this is what makes backfill and dual-write safe together.
- **Schema adapter:** new schema is a superset; adapter maps legacy ↔ new (field renames, defaults).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

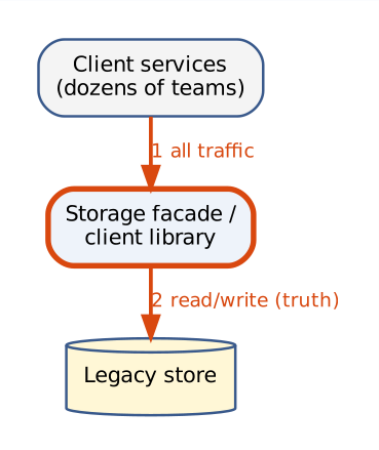
Step 1 · Today's state (30 s)



> **You:** “Today dozens of services read and write the legacy store directly. That's the real problem: with many direct callers, we can't control a migration.”

Why this step now: Draw the current system honestly — Apple's interviewer probably owns it.

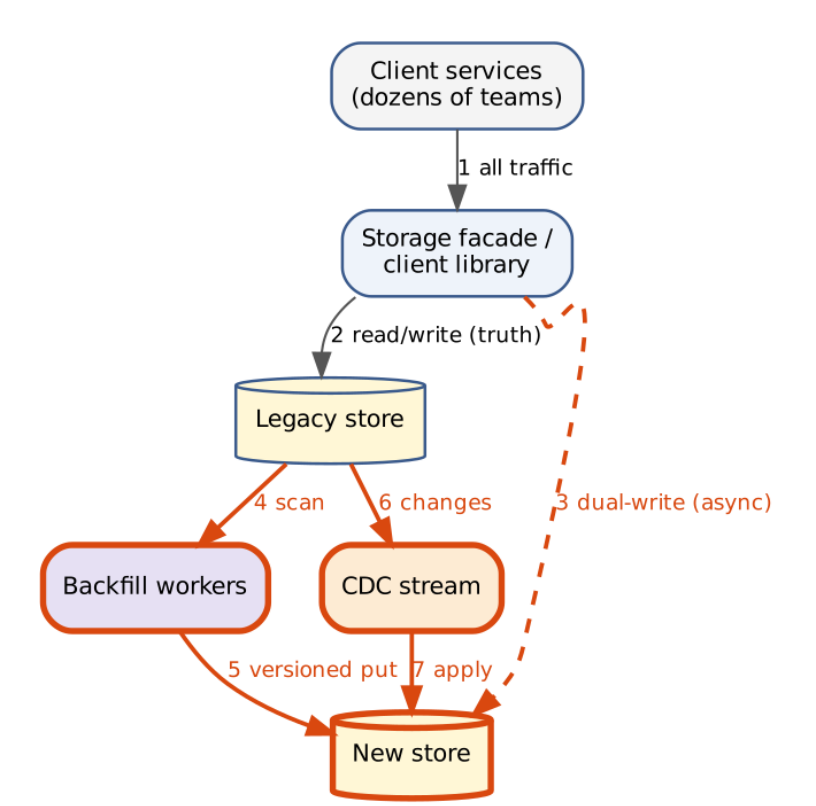
Step 2 · Phase 0: put a facade in front (1–2 min)



> **You:** “Step zero is a facade or client library that every read and write goes through, with routing controlled by flags per key range and cohort. Until all traffic goes through one place, nothing else is safe.”

Why this step now: This is the decision that separates strong answers.

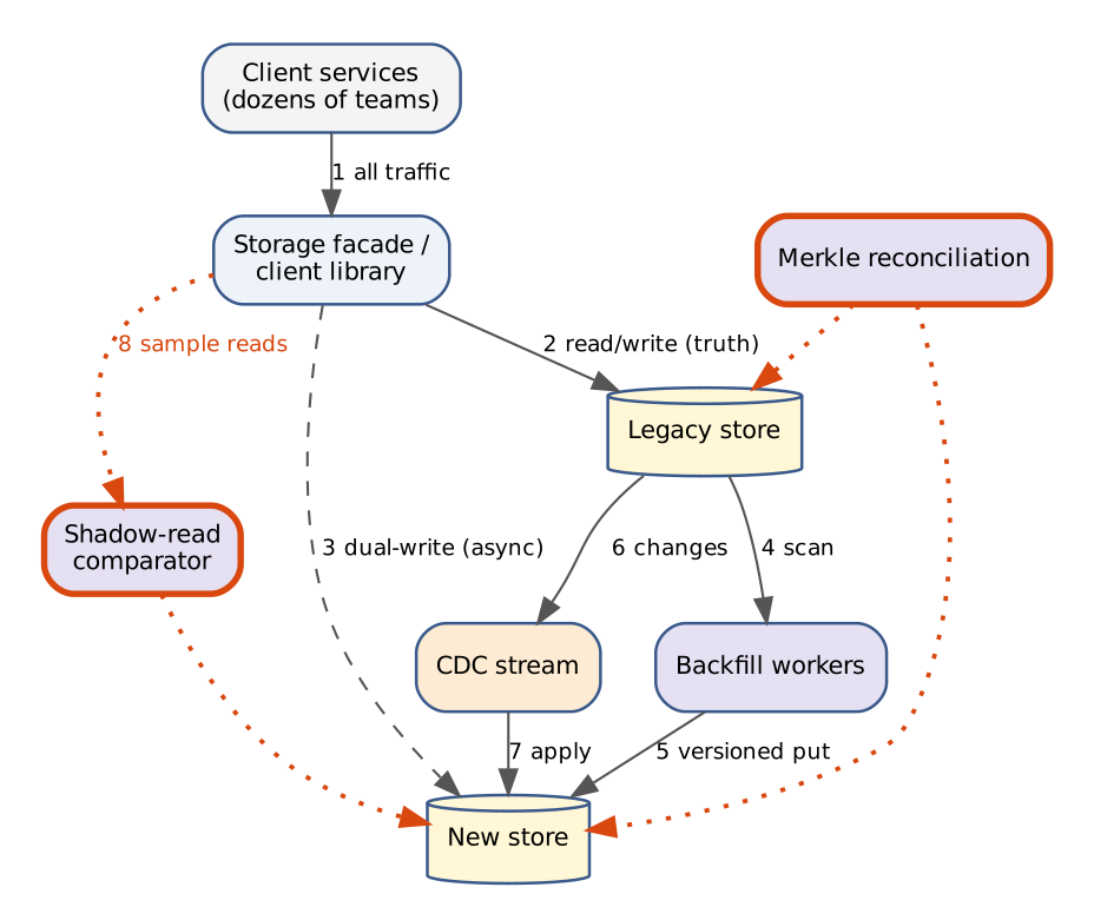
Step 3 · Phases 1–2: dual-write, backfill, CDC (2 min)



> **You:** “The facade dual-writes to the new store asynchronously, with legacy still the source of truth. Backfill workers copy history range by range, throttled, and a CDC stream catches writes during the copy. Every write to the new store is conditional on a record version, so an old backfill write can never overwrite a newer live write.”

Why this step now: The data-movement mechanics.

Step 4 · Phases 3–5: verify, cut over, flip (2 min)



> **You:** "A comparator samples reads from both stores and tracks mismatches per range; Merkle reconciliation covers cold data. When a range has zero unexplained mismatches for a week, we serve its reads from the new store with fallback, then flip writes, keeping legacy writes for a rollback window."

Why this step now: Verification and reversible cutover end the story.

Narrate one request end to end (numbers match the arrows)

A write during migration

1. Client calls the facade (1); facade writes legacy, the source of truth (2).
2. Facade asynchronously writes the new store with the record's version (3); failures go to a repair queue.

Moving history

1. Backfill scans legacy by range (4) and writes conditional-on-version to the new store (5).
2. CDC captures every legacy change (6) and applies it with the same version check (7).

Proving it

1. Comparator samples reads from both stores (8); mismatches classified and fixed.

2. Merkle reconciliation compares checksums per key range for cold data.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Client services (dozens of teams)	Read and write records	Many direct callers is the core risk	Existing services
Legacy store	Source of truth today	Hitting scale/partition limits	Existing DB
Storage facade / client library	Route reads/writes per key range & cohort via flags	Can't control a migration without one choke point	Library + routing config service
New store	Target system	Fixes legacy limits	In-house store
Backfill workers	Copy history range by range, throttled, resumable	Bring existing data across	Batch workers
CDC stream	Catch writes made during backfill or by bypassing clients	Closes the gap	Change data capture → Kafka
Shadow-read comparator	Compare legacy vs new responses async	Proof before cutover	Async verifier + metrics
Merkle reconciliation	Checksum key ranges in both stores	Covers cold data shadow reads miss	Batch job

Before you leave the high-level design, check you have said:

☐ Facade first
☐ Version-ordered conditional writes + tombstones
☐ Exit criteria per phase
☐ Rollback window after write flip

60-second summary to say once the diagram is complete:

You: "I'd never do a big-bang migration on a system like this. The plan has six phases, and each one is reversible by flipping a flag."

"**Phase 0 — facade.** First I put a storage facade, or a client library, between every client and the legacy store. Until every read and write goes through one place, we can't control the migration. With dozens of clients this alone might be a quarter of work, and it's where I'd put my strongest engineers."

"**Phase 1 — dual-write.** The facade writes to legacy as the source of truth and asynchronously to the new store. Failures writing to the new store don't fail the request; they go to a repair queue."

"**Phase 2 — backfill.** Throttled, resumable workers copy historical data range by range, while a change-data-capture stream from legacy closes the gap for writes that happen during the copy. Versioned records mean a stale backfill write can never overwrite a newer dual-write."

> "**Phase 3 — shadow reads.** The facade reads from legacy, serves that answer, and also reads from the new store in the background and compares. We track mismatch rate per key range and investigate every class of mismatch."
>
> "**Phase 4 — cutover reads, range by range.** When a range has had zero unexplained mismatches for, say, a week, we serve reads from the new store, with fallback to legacy on not-found. We start with internal users, then 1%, 10%, 100%."
>
> "**Phase 5 — flip the source of truth.** Writes go to the new store first; we keep writing to legacy for a rollback window. **Phase 6** — decommission after the window, once nothing reads from legacy."

8. Deep dives

Deep dive 1: ordering between backfill, CDC and dual-writes. The classic bug: backfill reads a record at version 5, a live update writes version 6 to both stores, then backfill writes version 5 into the new store — overwriting newer data. Fix: conditional writes on version.

```
def apply_to_new_store(key, record, version):
    # Only write if newer than what's there: compare-and-set on version.
    while True:
        current = new_store.get_version(key)          # None if absent
        if current is not None and current >= version:
            return "SKIPPED_STALE"
        if new_store.put_if_version(key, record, version, expected=current):
            return "APPLIED"
        # lost a race with another writer; loop and re-check
```

Deletes need tombstones with versions too, or a backfill can resurrect a deleted record.

Deep dive 2: verification — how do you *know* nothing was lost?

- Shadow reads catch read-path differences on live traffic.
- Bulk reconciliation per key range: compute checksums/Merkle trees over sorted key ranges in both stores; drill into mismatched sub-ranges. Cheap to run repeatedly.
- Count-based checks by range and by day as a coarse sanity check.
- Every mismatch class gets a root cause before cutover (adapter bug, timezone, encoding, ordering).

Deep dive 3: running two systems in parallel and backwards compatibility.

- Facade hides both stores; clients see one API.
- New schema is additive; adapter fills defaults for legacy-shaped reads; old clients keep working because the facade serves the old shape until they upgrade.
- Avoid dual-*read* in the hot path except fallback-on-miss; dual reads double latency risk.
- Capacity: the new store must take full production write load from phase 1 — load-test it before dual-write starts.

9. Scaling evolution / phasing timeline

Phase	Duration (example)	Exit criteria
0 Facade	4–10 weeks	100% of traffic through facade; no latency regression
1 Dual-write	2 weeks	New-store write success ≥ 99.99%; repair queue drains
2 Backfill + CDC	3–4 weeks	All ranges backfilled; CDC lag < 1 min
3 Shadow read	2–4 weeks	Mismatch rate 0 unexplained for 7 days per range
4 Read cutover	2–4 weeks	100% reads from new; p99 latency ≤ legacy
5 Write flip + rollback window	4 weeks	No rollbacks needed
6 Decommission	2 weeks	Zero legacy traffic; data archived

10. Failures

- New store outage during phases 1–3 → no customer impact (legacy is source of truth); repair queue catches up.
- After phase 5, new store outage → that's why we keep writing to legacy during the rollback window; flip back.
- Backfill overloads legacy → adaptive throttling based on legacy latency; pause during peak hours.
- Adapter bug corrupts records → caught by shadow reads before cutover; re-backfill affected ranges.

11. Trade-offs

Decision	Options	Choice	Why
Big bang vs incremental		Incremental by key range + cohort	Reversible at each step; limits blast radius
Sync vs async dual-write	Fail request if new write fails vs async + repair	Async + repair in early phases	Don't let the new system hurt customers before it's trusted
CDC vs dual-write only		Both	Dual-write misses clients not yet on the facade and writes during backfill windows
Verification	Shadow reads only vs + Merkle reconciliation	Both	Shadow covers hot keys; reconciliation covers cold data

12. Security, operations, cost

- **Security/privacy:** migration copies everything — encryption in transit and at rest in the new store, same or stricter access controls, and *data residency preserved* (EU data must stay in EU regions during migration).
- **Operations:** migration dashboard per range: mode, backfill %, CDC lag, mismatch rate, error rates; runbook per phase with explicit rollback steps; a single migration owner on call.
- **Cost:** running both systems for months is expensive — say so, and plan decommission dates with finance.

13. Manager lens

> **You:** "The technical plan is the easier half. The harder half is dozens of client teams. I'd publish the migration plan with dates and exit criteria, give teams a client library so adopting the facade is a dependency bump, and track adoption weekly. I'd own the go/no-go for each cutover phase personally, with written criteria agreed in advance, so the decision isn't made under deadline pressure."

14. Follow-ups

1. **"A client writes directly to legacy, bypassing the facade. What happens?"** — CDC catches the write, so the new store stays correct; we detect bypassing clients from legacy access logs and chase them.
2. **"Shadow reads show 0.01% mismatches. Do you cut over?"** — No. Every mismatch class needs an explanation. Benign classes (e.g., timestamps formatted differently) get normalized in the comparator; real bugs get fixed and re-verified.
3. **"How do you handle deletes?"** — Versioned tombstones in both stores; backfill respects them; garbage-collect tombstones after the window.
4. **"How long do you keep legacy after cutover?"** — A defined rollback window (e.g., 4 weeks) with dual-writes, then read-only, then archive.
5. **"How would you test the new store before dual-write?"** — Replay a sample of production traffic (captured requests) against it; load test at 2× peak.
6. **"What if the schema changes in an incompatible way?"** — Expand–contract: add new fields first, migrate readers, then remove old fields.

15. Mock transcript

> **Interviewer:** We need to move off our legacy store. How would you do it?
>
> **You:** Can I ask about the data and the clients first? Size, access patterns, how many services call it directly, and whether the schema changes? … With dozens of direct clients, step zero is a facade, otherwise we can't control routing.
>
> **Interviewer:** How do you keep both stores consistent?

> **You:** Dual-write through the facade with legacy as source of truth, backfill plus CDC for history, and versioned conditional writes so an old backfill write can never overwrite a newer live write.
>
> **Interviewer:** How do you know you're done?
>
> **You:** Shadow reads with zero unexplained mismatches per range for a week, plus Merkle-tree reconciliation for cold data. Then range-by-range read cutover with fallback, and only then flip the source of truth — keeping legacy writes for a rollback window.

16. Scoring guide

Rating	Signals
Strong Hire	Facade first; phased, reversible plan with exit criteria; version-ordered conditional writes; tombstones; shadow reads + reconciliation; client-adoption plan; residency preserved
Hire	Dual-write + backfill + cutover; some verification
Lean No Hire	Backfill then switch; no plan for writes during the copy
No Hire	Downtime window or big-bang cutover

17. Mistakes and "say this"

Mistakes: forgetting writes during backfill; no version ordering; no rollback path; ignoring client teams.

Say this: "Every phase is a flag flip, so every phase is reversible." · "An old backfill write must never overwrite a newer live write — so every write is conditional on version." · "Zero *unexplained* mismatches, not zero mismatches."

18. Your tie-in

Your cloud modernization almost certainly included data migrations from on-prem or legacy policy systems. This is your strongest Apple story. [VERIFY: the system, the volume, how you ran dual systems, one thing that went wrong.]

Q10. Design a new data storage system from scratch — index structure, APIs, EU regionalization (key-value store)

> Apple (Medium) · also Databricks, Google, Amazon (DynamoDB-style) · Source: Aced Apple EM guide (2026)

1. The prompt

"Design a new storage system from scratch. How would you structure the index and expose APIs? How would you regionalize it to support European users?" Variant: "Design a key-value store."

2. Clarify

Ask:

- "What's stored — small records (KBs) or blobs?" — KV store vs object store.
- "Access patterns: point lookups by key, range scans, secondary attributes?" — index design.
- "Consistency: read-your-writes, strong, or eventual?" — replication protocol.
- "Scale: keys, total size, QPS?"
- "What do EU users need — data stored only in the EU, or just low latency?" — residency vs performance are different requirements.
- "Multi-tenant (many internal teams) or one service?"

State: "Small records under 10 KB; point gets/puts plus range scans within a partition key; one secondary index; strong consistency per key; 10 billion keys, 100 TB, 500k reads/s and 100k writes/s; EU users' data must stay in the EU; multiple internal teams as tenants."

3. Requirements

- **Must:** `get/put/delete` by key with conditional writes; range scan within a partition; secondary index on one attribute; per-key strong consistency; data residency by region; multi-tenant quotas.
- **Non-functional:** p99 get < 10 ms in-region; 99.99% availability; durable (replicated across 3 availability zones).

4. Estimates

- 100 TB / partition size of ~10 GB → **~10k partitions**. ×3 replicas = 30k partition replicas. At ~200 replicas per node with 2 TB each → ~150 nodes per region set.
- 500k reads/s / 10k partitions = 50 reads/s per partition on average — fine; hot keys are the risk.

5. API

```
PUT    /v1/tables/{t}/items/{pk}/{sk}   body, If-Match: <version>        → 200 {version} |
412
GET    /v1/tables/{t}/items/{pk}/{sk}?consistency=strong|eventual        → {item, version}
DELETE /v1/tables/{t}/items/{pk}/{sk}   If-Match
GET    /v1/tables/{t}/items/{pk}?sk_from=..&sk_to=..&limit=..&cursor=..  → {items,
next_cursor}
GET    /v1/tables/{t}/indexes/{idx}?value=..&cursor=..                    → {items,
next_cursor}   (eventually consistent)
PUT    /v1/tables/{t}  {home_region_policy: "user_region", indexes:[...], quotas}
```

Design notes to say: composite key (partition key + sort key) gives both point lookups and ordered range scans; versions enable optimistic concurrency; cursors are opaque and encode the last key; secondary index reads are explicitly eventually consistent.

6. Data model and index structure

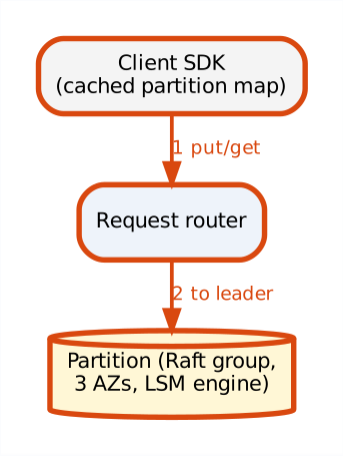
- **Primary index:** data partitioned by hash(partition key) into ranges of hash space; within a partition, items sorted by sort key in an LSM tree (A4 §9) — fast writes, ordered scans.
- **Partition map:** `hash range → replica set (leader, followers), region` stored in a small metadata service (Raft group), cached by routers and SDKs.
- **Secondary index:** a separate internal table keyed by `(index_value, primary_key)`, updated asynchronously from each partition's replication log. Global secondary indexes are eventually consistent; local secondary indexes (same partition key) can be updated synchronously.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

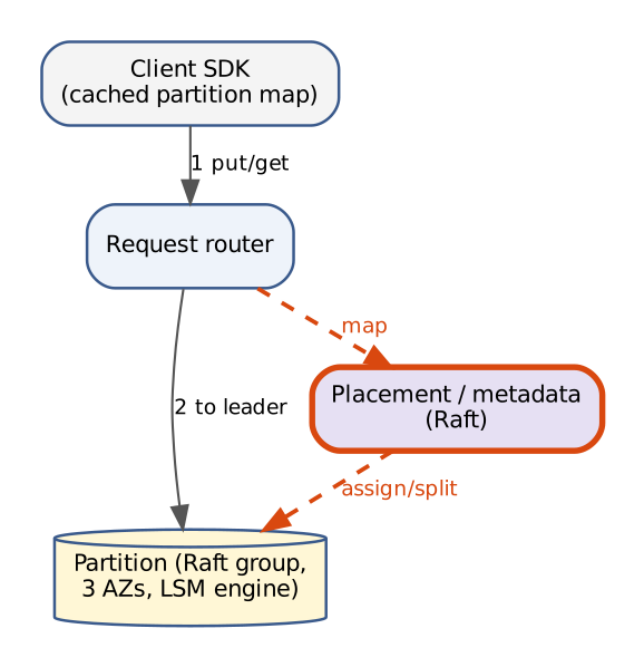
Step 1 · Data path in one region (1–2 min)



> **You:** “Clients use an SDK with a cached partition map. A router sends each request to the leader of the partition that owns the key. Each partition is a Raft group with three replicas across availability zones, running an LSM storage engine sorted by sort key.”

Why this step now: Show how a key finds its data first.

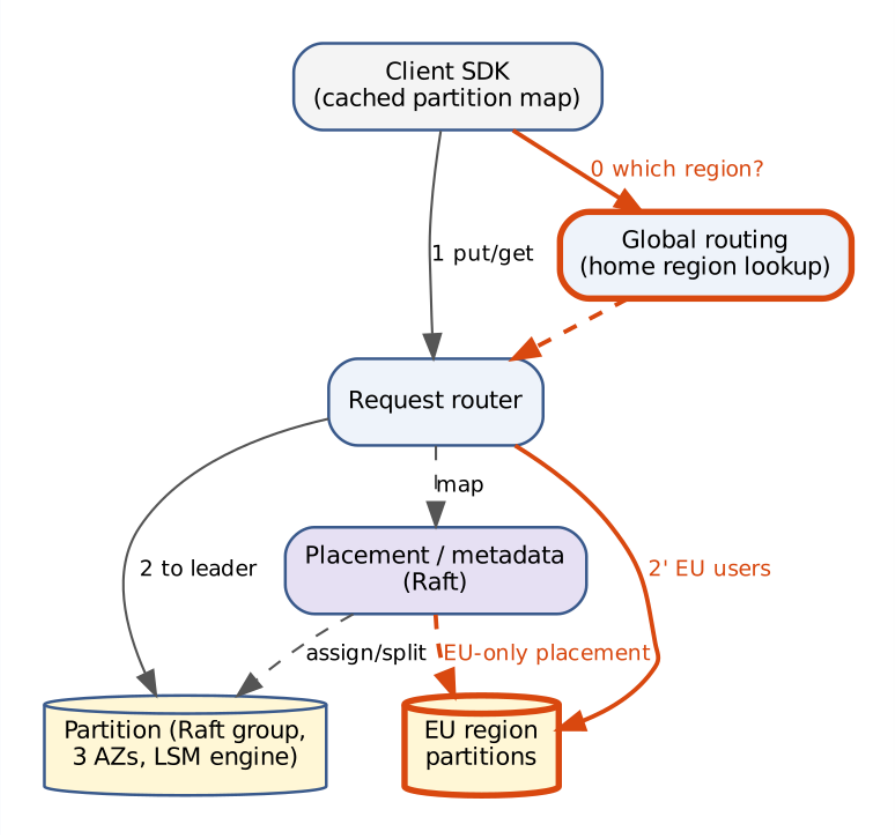
Step 2 · Placement and elasticity (1 min)



> **You:** “A small placement service, itself a Raft group, owns the partition map: it splits partitions that get too big or hot and moves replicas. Routers and SDKs cache the map and refresh on 'wrong partition' errors.”

Why this step now: Control plane separate from data plane.

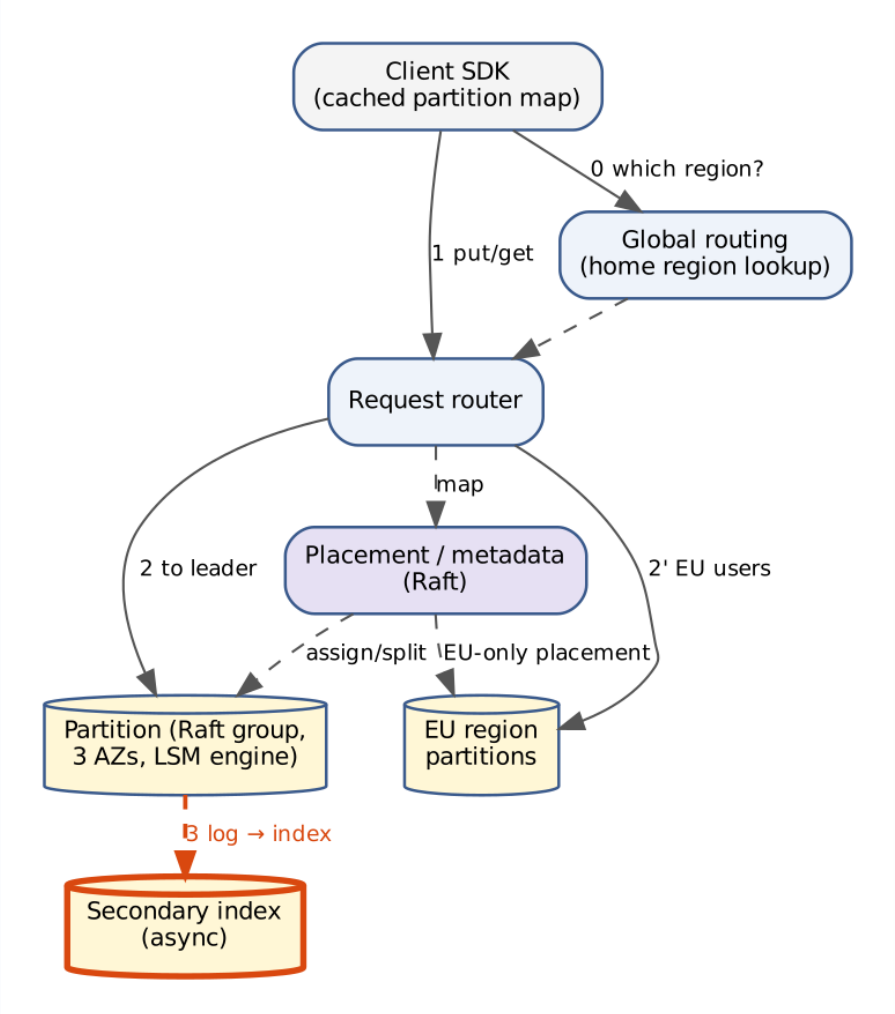
Step 3 · Regionalization for EU (1–2 min)



> **You:** “Each user has a home region. A global routing step looks it up and sends EU users to the EU deployment. The placement service only places EU partitions on EU nodes, so residency is enforced by placement, not by convention.”

Why this step now: The Apple follow-up — answer it with structure, not a sentence.

Step 4 · Secondary index (30 s)



> **You:** “Secondary indexes are separate internal tables fed asynchronously from each partition's replication log; the API labels those reads as eventually consistent.”

Why this step now: Index structure question answered explicitly.

Narrate one request end to end (numbers match the arrows)

Strongly consistent write

1. SDK resolves the user's home region (0) and partition, sends to router (1).
2. Router forwards to the partition leader (2 / 2′ for EU); leader appends to Raft log, replicates to a majority, applies to the LSM memtable, acks.
3. Replication log feeds the async secondary index (3).

Strongly consistent read

1. Router sends to the leader, which serves under a leader lease (no consensus round).
2. Eventually consistent reads may go to any replica.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Client SDK (cached partition map)	Route requests to the right partition	Avoids a lookup per request	Client library
Request router	Forward to partition leader, retry on map change	Thin clients and auth	Stateless service
Partition (Raft group, 3 AZs, LSM engine)	Store a key range; replicate writes	Per-key strong consistency	Raft + RocksDB-like engine
Placement / metadata (Raft)	Partition map, splits, moves, residency rules	Elasticity and residency enforced centrally	Small Raft cluster (etcd-like)
EU region partitions	EU users' data only on EU nodes	Residency is a placement constraint	Same engine, EU-only placement
Global routing (home region lookup)	Route a user to their home region	Users can call from anywhere	Edge routing + account directory
Secondary index (async)	Index on an attribute	Query by non-key attribute	Internal table fed by replication log

Before you leave the high-level design, check you have said:

- ☐ Residency vs latency distinction
- ☐ Composite key: partition key + sort key
- ☐ Raft per partition + leases
- ☐ Async GSI contract

60-second summary to say once the diagram is complete:

> **You:** "Clients use an SDK that caches the partition map. A request goes to a router, which sends it to the leader replica of the partition owning that key. Each partition is a Raft group of three replicas across availability zones; the leader appends the write to its log, replicates to a majority, applies it to the local LSM storage engine, and acknowledges. That gives per-key strong consistency."
>
> "Strongly consistent reads go to the leader, using a leader lease so we don't need a round of consensus per read. Eventually consistent reads can go to any replica — cheaper and lower latency."
>
> "A placement service holds the partition map and decides splits and moves when a partition gets too big or too hot."

8. Deep dives

Deep dive 1: regionalization for EU users (the Apple follow-up).

- Separate **residency** (legal: EU personal data stored and processed in the EU) from **latency** (EU users served from nearby).
- Each tenant's data gets a **home region** chosen by policy (e.g., user's account region). The partition key includes or maps to the home region, so EU users' partitions are placed only on EU nodes. Replicas stay within the EU (across EU availability zones or two EU regions for DR).
- A global routing layer (or the SDK) looks up the user's home region and routes there. Non-EU services needing EU data call an EU-hosted API — the data doesn't leave.
- Metadata that's truly global (e.g., account → home region mapping) contains no personal data, or is minimized.
- Moving a user between regions (they relocate) = a controlled migration job (Q9 pattern) with audit.
- *Say:* "Residency is a placement constraint, not a cache setting. I'd enforce it in the placement service so no misconfiguration can put an EU partition on a US node."

Deep dive 2: partition splits and hot keys.

- Split when a partition exceeds size (10 GB) or sustained load; new partition map version propagated; routers retry on "wrong partition" errors with the new map.
- Hot keys: per-key rate limits, an optional read cache for hot items, write sharding by suffix for counters (application-level).

Deep dive 3: the write path inside a node.

- Raft log entry → apply to memtable → WAL already covered by the Raft log → flush to SSTables → compaction. Bloom filters per SSTable for fast negative lookups. Tombstones for deletes, purged in compaction after replication confirms.

9. Scaling evolution

Stage	Design
Small	Single-region, replicated SQL sharded by key
Target	Raft-replicated partitions, LSM engine, placement service, async GSIs, per-region residency
10×	Automatic splitting/merging, tiering cold data to object storage, per-tenant cells

10. Failures

- Leader node fails → Raft elects a new leader in seconds; clients retry; in-flight writes either committed (majority) or not acknowledged.
- AZ fails → partitions keep a majority in the other two AZs.

- Placement service down → data path keeps working with cached maps; no splits until restored.
- Region down → for EU-only data, failover only to another EU region; never to US.

11. Trade-offs

Decision	Options	Choice	Why
Replication	Leaderless quorum (Dynamo) vs Raft per partition	Raft	Per-key strong consistency and conditional writes are requirements
Engine	B-tree vs LSM	LSM	Write-heavy, range scans within partition, compaction acceptable
Secondary indexes	Sync vs async	Async (global), sync (local)	Sync global indexes require distributed transactions
Residency	Encrypt everywhere vs place data in-region	Placement in-region	Legal requirement is location, not just encryption

12. Security, operations, cost

- Per-tenant encryption keys; access policies per table; audit logs per region; deletion requests (right to erasure) propagate to indexes, backups (with documented retention) and caches.
- SLIs: p99 latency per operation and region, availability, replication lag, compaction backlog; page on availability and latency burn.
- Cost: storage efficiency (compression, compaction tuning), right-sizing replica count per tier.

13. Manager lens

> **You:** "Storage engine, replication/placement, and the API/SDK are three distinct teams with different skills. The decision I'd own is the consistency contract we promise to tenants — once teams depend on it we can't weaken it. For EU residency I'd involve privacy and legal early and turn their requirements into placement rules tested in CI."

14. Follow-ups

1. **"How do you do a range scan across partitions?"** — Range scans are within a partition key by design; cross-partition queries go through secondary indexes or an analytics export.
2. **"How do strongly consistent reads avoid a consensus round?"** — Leader leases with bounded clock drift; if the lease is uncertain, fall back to a read index (confirm leadership with a heartbeat to a majority).
3. **"A user deletes their account. What happens?"** — Delete in primary, async in indexes, crypto-shred per-user keys for backups, confirm completion to the requester.

4. **"How would you add transactions across keys?"** — Within one partition: easy (single Raft group). Across partitions: two-phase commit over Raft groups with a transaction coordinator (Spanner-style) — expensive; offer it only if needed.

15. Mock transcript

> **Interviewer:** Design a new storage system. How would you regionalize it for Europe?
>
> **You:** First, is Europe a latency requirement or a residency requirement? … If it's residency, it's a placement constraint: each user has a home region, their partitions only live on EU nodes, replicas stay in the EU, and the placement service enforces it so a misconfiguration can't violate it.
>
> **Interviewer:** How is the index structured?
>
> **You:** Composite key — partition key hashed to place data, sort key ordered inside the partition in an LSM tree — so point lookups and range scans within a partition are both fast. Secondary indexes are separate internal tables updated from the replication log, eventually consistent, and the API says so.

16. Scoring guide

Rating	Signals
Strong Hire	Residency vs latency distinction; placement-enforced residency; composite key design; Raft per partition with leases; async GSI with explicit contract; split/hot-key handling
Hire	Sharding, replication, basic regional routing
Lean No Hire	"Replicate everywhere" for EU; no index design
No Hire	Can't explain how data is located or replicated

17. "Say this"

"Residency is a placement constraint, not a cache setting." · "The API should tell callers which reads are eventually consistent." · "The consistency contract is a one-way door."

18. Your tie-in

Insurance data has strict regulatory and state-level requirements; if you handled data residency, retention, or PII controls, use it here. [VERIFY]

Q11. Design a web cache (distributed cache)

> Apple (Medium) · Amazon (distributed cache, Low), Netflix (CDN caching) · Source: Aced Apple EM guide (2026)

1. The prompt

"Design a web cache" — interpret with the interviewer: an HTTP caching proxy in front of origin servers, or a distributed in-memory cache (Memcached/Redis-like) used by services. Ask which; the walkthrough covers the distributed service cache and notes HTTP specifics.

2. Clarify

Ask: "HTTP reverse-proxy cache or an application-level key-value cache?" · "Read QPS, object sizes, working-set size?" · "How stale can data be?" · "How are updates signaled — TTL only, or invalidation events?" · "Personalized content?" (cache-key safety)

State: "Application-level distributed cache for read-heavy services: 2M reads/s, objects ~2 KB, 500 GB hot working set, staleness up to a few seconds acceptable, invalidation on writes."

3. Requirements

- **Must:** get/set/delete with TTL; scale horizontally; survive node loss without stampeding the database; invalidation on source changes.
- **Non-functional:** p99 < 2 ms; hit rate > 95%; no cross-user data leakage.

4. Estimates

- 500 GB hot set / 64 GB usable per node ≈ 8 nodes; with headroom and replicas → ~20 nodes.
- 2M reads/s / 20 nodes = 100k ops/s per node — at the edge of a single Redis instance per node; use multiple shards per host or client-side L1 caching.
- At 95% hit rate, DB sees 100k reads/s — make sure it can; at 90% it's 200k/s. Say that 5 points of hit rate halve or double DB load.

5. API

```
get(key) -> value | miss
get_multi(keys[]) -> map
set(key, value, ttl, if_version?)
delete(key)
lease_get(key) -> value | (miss, lease_token)     // stampede control
set_with_lease(key, value, lease_token)
```

6. Data model

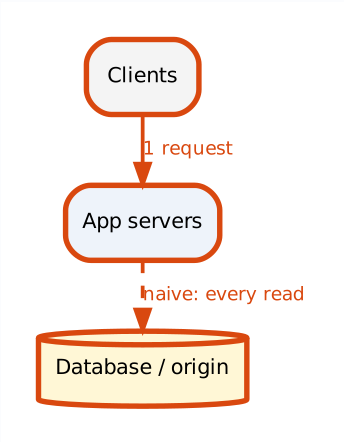
- In-memory hash table per shard; slab allocation for memory efficiency; LRU or LFU per slab class; TTL per item.
- Key naming: `service:entity:id:version` — versioned keys make invalidation cheap (bump the version).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

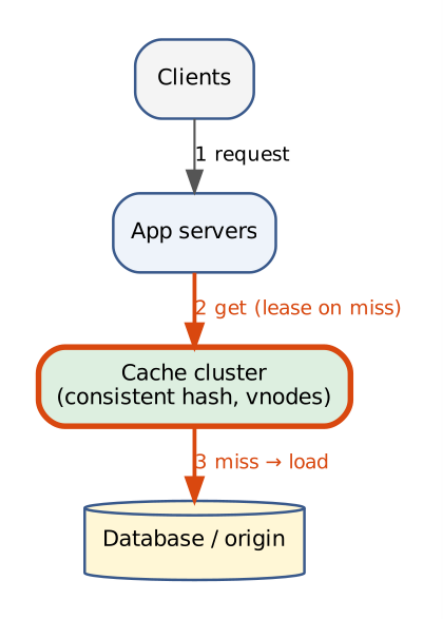
Step 1 · The problem (30 s)



> **You:** “Without a cache, every read hits the database — at two million reads a second that's not possible. The cache's job is to protect the database.”

Why this step now: Frame what the cache is for.

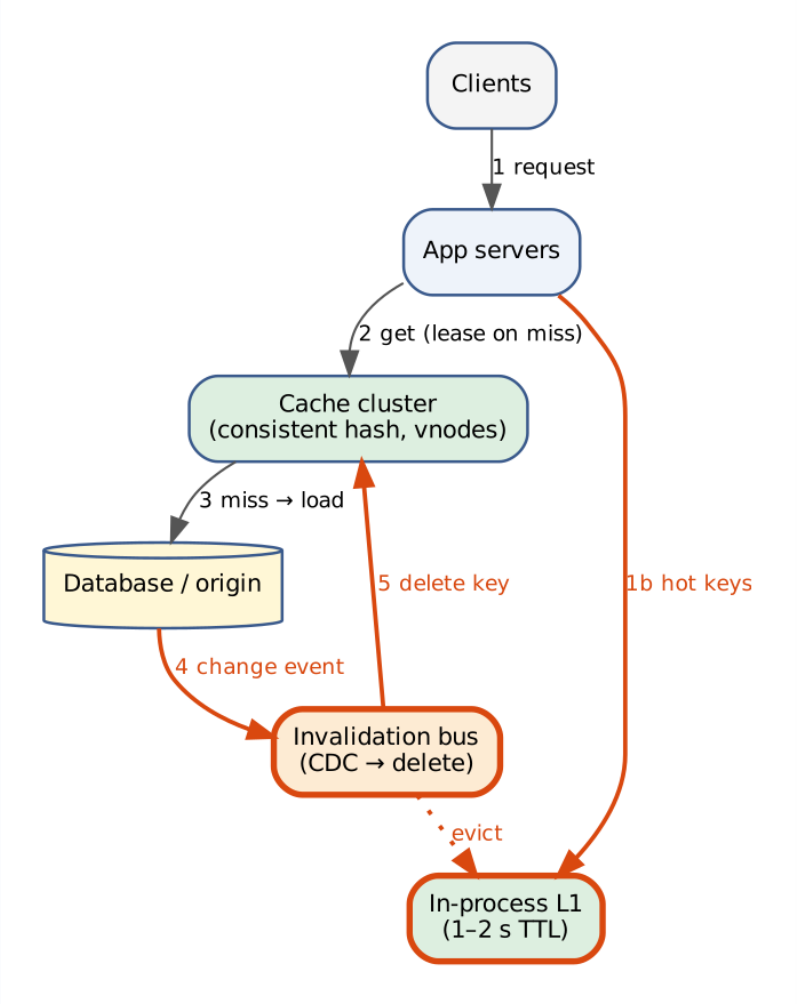
Step 2 · Distributed cache tier (1–2 min)



> **You:** “App servers hash keys onto a consistent-hash ring with virtual nodes to pick a cache node. On a miss, the cache gives one caller a lease to load from the database; others wait or get slightly stale data, which prevents stampedes.”

Why this step now: Core mechanism and stampede control.

Step 3 · Hot keys and freshness (1 min)



> **You:** "A tiny in-process L1 cache with a one-to-two-second TTL absorbs celebrity keys. Database change events drive an invalidation bus that deletes affected keys; TTLs are the safety net."

Why this step now: Edge cases interviewers probe.

Narrate one request end to end (numbers match the arrows)

Read

1. Request hits app server (1); L1 checked for hot keys (1b).
2. L2 cache get (2); on miss, lease holder loads from DB (3) and sets with lease token.

Write

1. App writes the DB; change event emitted (4).
2. Invalidation bus deletes the key in L2 (5) and evicts L1; next read reloads.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech

Clients	Requests		—
App servers	Business logic; read-through the cache	Where cache lookups happen	Service fleet
Database / origin	Source of truth	What the cache protects	Postgres / service
Cache cluster (consistent hash, vnodes)	Hot data in memory	2M reads/s can't hit the DB	Memcached / Redis cluster
In-process L1 (1–2 s TTL)	Absorb celebrity keys	One hot key shouldn't melt one cache node	Local LRU
Invalidation bus (CDC → delete)	Delete keys when data changes	Freshness with TTL as backstop	CDC → Kafka → deleters

Before you leave the high-level design, check you have said:

☐ Hit-rate math vs DB load
☐ Delete-on-write and the stale-set race
☐ Leases for stampedes
☐ Cache key includes tenant/user scope

60-second summary to say once the diagram is complete:

> **You:** "Clients hash keys onto a consistent-hash ring with virtual nodes to pick a cache node, so adding or losing a node moves only about one Nth of keys. Each app server also keeps a tiny in-process L1 cache with a TTL of a second or two for the hottest keys — that's what absorbs celebrity keys."
>
> "Writes go to the database; a change-data-capture stream publishes invalidations that delete the affected keys from L2 and broadcast to L1s. TTLs are the safety net if an invalidation is lost."

8. Deep dives

Deep dive 1: stampede control with leases (Facebook's memcache paper approach). On a miss, the cache hands out a lease token to one client; other clients get "wait/retry" or a slightly stale value. Only the lease holder can set the key. This prevents thousands of simultaneous DB loads when a hot key expires, and prevents stale sets racing with deletes.

Deep dive 2: node failure without a DB meltdown. Losing a node means ~1/N of keys miss at once. Options: replicate hot keys; a "gutter" pool that temporarily serves keys for failed nodes with short TTLs; warm the replacement node gradually.

Deep dive 3: consistency. The race: reader misses, reads DB (old), writer updates DB and deletes cache, reader sets old value → stale until TTL. Fixes: leases (set rejected if a delete happened after lease issue), versioned keys, short TTLs on volatile data.

HTTP cache notes (if the interviewer meant that): respect `Cache-Control`, `ETag`/`If-None-Match` revalidation, `Vary` headers in the cache key, never cache responses with `Set-Cookie` or `private`, request collapsing for concurrent misses.

9–11. Failures and trade-offs

Decision	Options	Choice	Why
Invalidation	TTL only vs event-driven deletes	Events + TTL backstop	Freshness with safety net
Update vs delete on write		Delete	Avoids concurrent-update ordering bugs
Replication	None vs replicate hot keys	Replicate hot keys only	Memory cost vs stampede risk
L1 in-process	Yes/no	Yes, ~1–2 s TTL	Absorbs hot keys, tiny staleness

12–13. Security, ops, manager lens

- Cache keys must include tenant/user scope for personalized data — the most common cache security bug is serving one user's data to another.
- SLIs: hit rate per key prefix, p99 latency, evictions, memory, DB load. Alert on hit-rate drops (they predict DB overload).
- Manager lens: "I'd offer caching as a platform with a client library that bakes in leases, key versioning and metrics, so product teams don't each reinvent invalidation bugs."

14. Follow-ups

1. **"A key gets 1M reads/s."** — L1 in-process cache, replicate the key across nodes with suffixes, or push to the CDN if public.
2. **"How do you add capacity without a hit-rate cliff?"** — Consistent hashing + vnodes; add nodes gradually; warm from replicas.
3. **"How do you know what to cache?"** — Measure miss cost × frequency; cache expensive, frequently read, rarely changing data first.

15–17. Mock, scoring, say this

> **You:** "Five points of hit rate is the difference between a healthy database and an outage, so I treat hit rate as a production SLO and design for stampedes explicitly with leases."

Strong Hire: consistent hashing, two tiers, lease-based stampede control, invalidation race explained, cache-key security. **No Hire:** "put Redis in front" with no invalidation or failure story.

Say this: "Delete on write, don't update." · "The cache's job is to protect the database, so I design for the day the cache fails."

Q12. Design a load balancer from scratch

> Apple (Medium) · Google-style "build the component yourself" · Source: Aced Apple EM guide (2026)

1. The prompt

"Design a load balancer from scratch." Interviewers want internals: how traffic is distributed, health checking, config changes, and how the LB itself avoids being a single point of failure.

2. Clarify

Ask: "L4 (TCP) or L7 (HTTP)?" · "Traffic volume and connection counts?" · "Internal service-to-service or internet-facing?" · "Sticky sessions needed?" · "Do backends have different capacities?"

State: "Internet-facing HTTP service, 1M RPS peak, TLS, mixed backend sizes, no sticky sessions except for a few legacy routes, zero-downtime config changes."

3. Requirements

- **Must:** distribute requests across healthy backends; health checking; TLS termination; routing rules by host/path; weights; connection draining; no single point of failure.
- **Non-functional:** added latency < 1 ms p99; handles backend failure within seconds; config changes without dropping connections.

4. Estimates

- 1M RPS / ~50k RPS per L7 proxy instance (with TLS, keep-alive) → ~20 proxies + 50% headroom → 30.
- L4 tier in front distributes TCP connections to the L7 proxies; a few L4 nodes handle millions of packets/s.

5. Interfaces

```
Data plane: listener(port, tls_cert) → route_table → cluster → endpoint picker
Control plane API:
  PUT /clusters/{name}   {endpoints:[{addr, weight, zone}], health_check:{path, interval, thresholds}, lb_policy}
  PUT /routes            [{match:{host, path_prefix}, cluster, timeout, retries}]
  GET /stats             per-cluster RPS, errors, latency, endpoint health
```

6. Data model

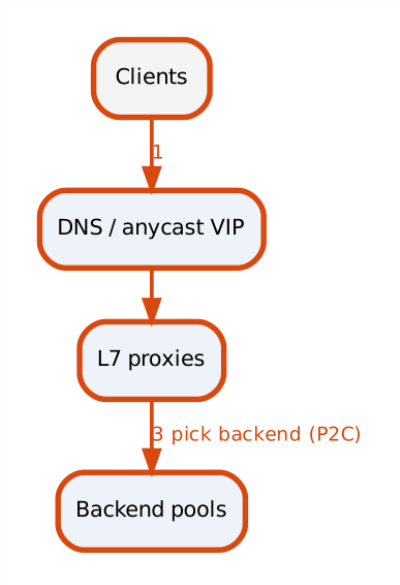
- Route table (immutable snapshot, versioned); clusters with endpoint lists; per-endpoint health state and outlier stats; per-endpoint active request count (for least-request).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

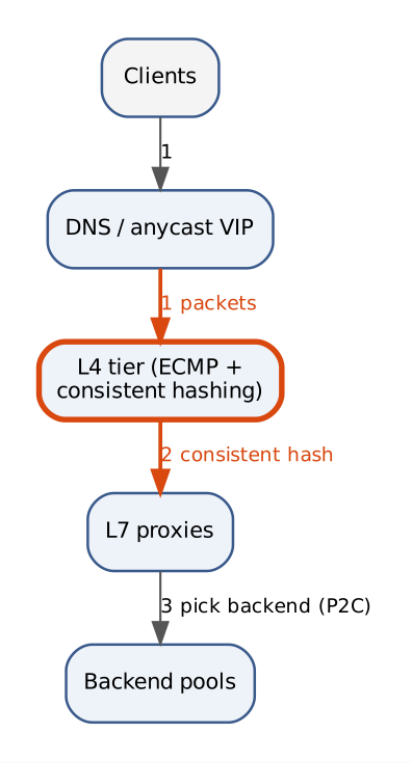
Step 1 · Simplest working shape (30 s)



> **You:** “Clients resolve a virtual IP; L7 proxies terminate TLS, match routes, and choose a backend.”

Why this step now: Start simple, then remove single points of failure.

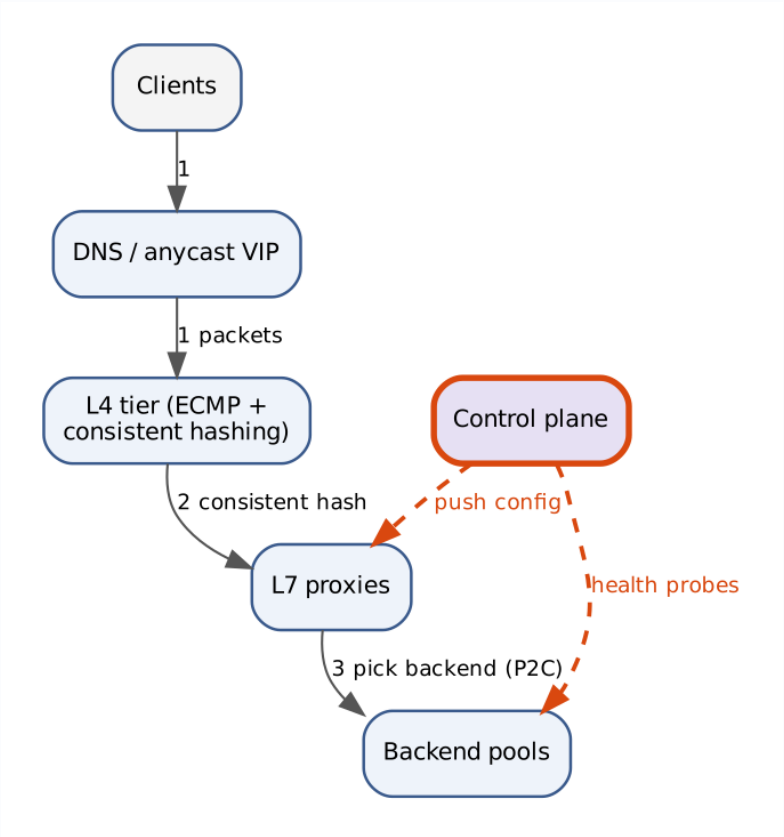
Step 2 · Two-tier: L4 in front of L7 (1–2 min)



> **You:** “In front, an L4 tier: routers spread packets over L4 nodes with ECMP, and each L4 node consistently hashes the connection to an L7 proxy, so adding or losing nodes breaks very few connections.”

Why this step now: Shows you know how real load balancers scale.

Step 3 · Control plane (1 min)



> **You:** “A separate control plane pushes versioned config snapshots and health information. Proxies keep serving the last good config if it's down.”

Why this step now: Separation of data and control planes.

Narrate one request end to end (numbers match the arrows)

A request

1. Client resolves the VIP; packets reach an L4 node via ECMP (1).
2. L4 hashes the 5-tuple to an L7 proxy (2).
3. L7 terminates TLS, matches route, picks a backend with power-of-two-choices (3), applies timeout/retry budget.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Clients	Connect to a VIP		—
DNS / anycast VIP	One address, many entry points	Global entry	Route 53 / anycast BGP
L7 proxies	TLS, routing, backend selection, retries	Request-aware balancing	Envoy / NGINX / custom

Backend pools	Serve requests	—	Service fleets
L4 tier (ECMP + consistent hashing)	Spread connections across L7 proxies	Removes the L7 tier as a SPOF; survives node changes	Maglev-style
Control plane	Config snapshots, health, weights	Data plane must not depend on it per request	xDS-style config service

Before you leave the high-level design, check you have said:

☐ L4 vs L7 roles
☐ P2C selection
☐ Active + passive health with panic threshold
☐ Safe config rollout

60-second summary to say once the diagram is complete:

> **You:** "Two layers. The L4 tier advertises a virtual IP via anycast or BGP; routers spread packets across L4 nodes with ECMP, and each L4 node uses consistent hashing on the connection 5-tuple so a connection always lands on the same L7 proxy even if an L4 node is added or removed. The L7 proxies terminate TLS, parse HTTP, match routes, and pick a backend."
>
> "A separate control plane holds configuration and pushes versioned snapshots to proxies. Proxies keep serving the last good config if the control plane is down — the data plane never depends on the control plane at request time."

8. Deep dives

Deep dive 1: backend selection. Round robin ignores load; least-connections needs global state; **power of two choices** — pick two random healthy endpoints, choose the one with fewer active requests — gets near-optimal balance with only local state. Weighted variants for mixed instance sizes. Consistent hashing for the few sticky routes.

```
import random
def pick(endpoints):
    healthy = [e for e in endpoints if e.healthy]
    if not healthy: raise NoHealthyUpstream
    if len(healthy) == 1: return healthy[0]
    a, b = random.sample(healthy, 2)
    return a if a.active / a.weight <= b.active / b.weight else b
```

Deep dive 2: health checking. Active checks (HTTP GET /health every 2 s; 3 failures → unhealthy; 2 successes → healthy) plus passive outlier detection (eject endpoints with 5 consecutive 5xx or high latency for 30 s). Panic threshold: if more than ~50% of endpoints look unhealthy, the health signal is probably wrong — balance across all rather than overload the few "healthy" ones. Slow-start: newly healthy endpoints ramp weight over a minute.

Deep dive 3: zero-downtime changes. Config snapshots swapped atomically; existing connections finish on old config. Proxy upgrades via connection draining: stop accepting new connections, finish in-flight requests, exit. Hot restart passes listening sockets to the new process.

9–11. Failures and trade-offs

- L7 proxy dies → L4 consistent hashing remaps only its connections; clients retry idempotent requests.
- Bad config pushed → validation, staged rollout of config (canary proxies first), automatic rollback on error-rate spike.
- Retry storms → retry budgets per cluster (e.g., ≤ 20% extra), only for idempotent methods.

Decision	Choice	Why
Algorithm	Power of two choices (weighted)	Near-optimal with local state
Health	Active + passive + panic threshold	Fast detection without false mass ejection
Control plane coupling	Push snapshots; data plane independent	Control plane outage doesn't take down traffic

12–13. Security, ops, manager lens

- TLS certificate rotation automated; DDoS protection at the L4/edge (SYN cookies, rate limits); request size limits; header sanitization.
- SLIs: added latency, 5xx generated by the LB vs by backends, ejection events, config push success.
- Manager lens: "The load balancer is a tier-0 dependency, so config changes go through the same staged rollout as code. I'd never let one global config push reach every proxy at once — that's how one typo becomes a global outage."

14. Follow-ups

1. **"How does the L4 tier avoid breaking connections when nodes change?"** — Consistent hashing (Maglev-style) plus connection tracking tables.
2. **"Backend is slow but not failing."** — Latency-based outlier detection; least-request balancing naturally sends it less traffic.
3. **"Global load balancing across regions?"** — GeoDNS/anycast to nearest healthy region; health-aware failover; capacity-aware spillover.

15–17. Mock, scoring, say this

> **You:** "The data plane must never depend on the control plane at request time. Proxies keep serving the last good config."

Strong Hire: L4/L7 layering, power-of-two choices, active+passive health with panic threshold, safe config rollout. **No Hire:** one box doing round robin with no failure story.

Say this: "Power of two choices gives near-optimal balance with only local state." · "If half the fleet looks unhealthy, the health check is probably what's broken."

Q13. Observability platform — and forecasting host spend with a finance partner

> Apple (Medium — cross-org round with a cost-management partner) · Source: Exponent Apple EM interview experience (2026)

This combines a design question with Apple's cross-org partnership round, where a recent candidate was asked how they'd forecast host spend for an observability team and work with finance on cost.

1. The prompt

"Design an internal observability platform (metrics, logs, traces) for thousands of services — and explain how you'd work with finance to manage its host cost."

2. Clarify

Ask: "How many services/hosts, and what's growing fastest — metrics, logs, or traces?" · "Who pays — central budget or chargeback to teams?" · "What's the cost trend and what's finance worried about — growth rate, unpredictability?" · "Retention requirements?"

State: "5,000 services, 200k hosts, spend growing 40% a year against 15% traffic growth — finance wants predictability and a plan to bend the curve."

3–6. Requirements, estimates, data model (summary)

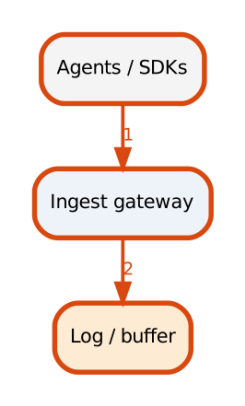
- **Must:** ingest metrics/logs/traces; query and dashboards; alerting; per-team usage attribution; cost forecast.
- **Estimates:** metrics: 200k hosts × 1,000 series = 200M active series; at 1 sample/10 s ≈ 20M samples/s. Cardinality (unique label combinations) is the cost driver, not hosts.
- **Data model for cost:** `usage(team, signal_type, unit, quantity, day)` × `unit_cost(signal_type, tier)` → cost per team per day.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

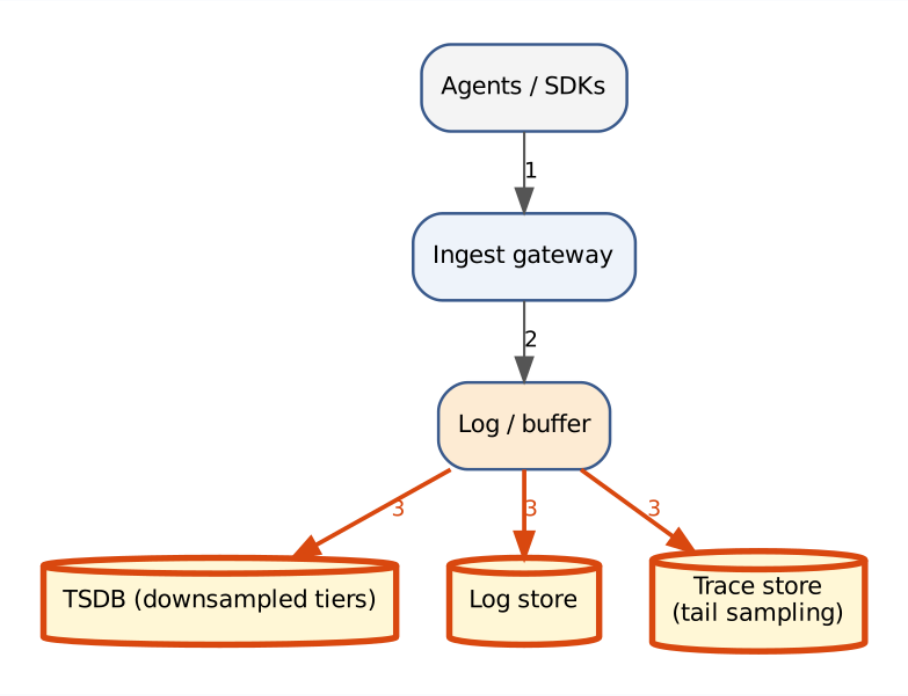
Step 1 · Ingest with cost control built in (1 min)



> **You:** “Agents send through an ingest gateway that enforces per-team quotas, sampling and cardinality limits, then into a buffer log.”

Why this step now: Cost control starts at ingest.

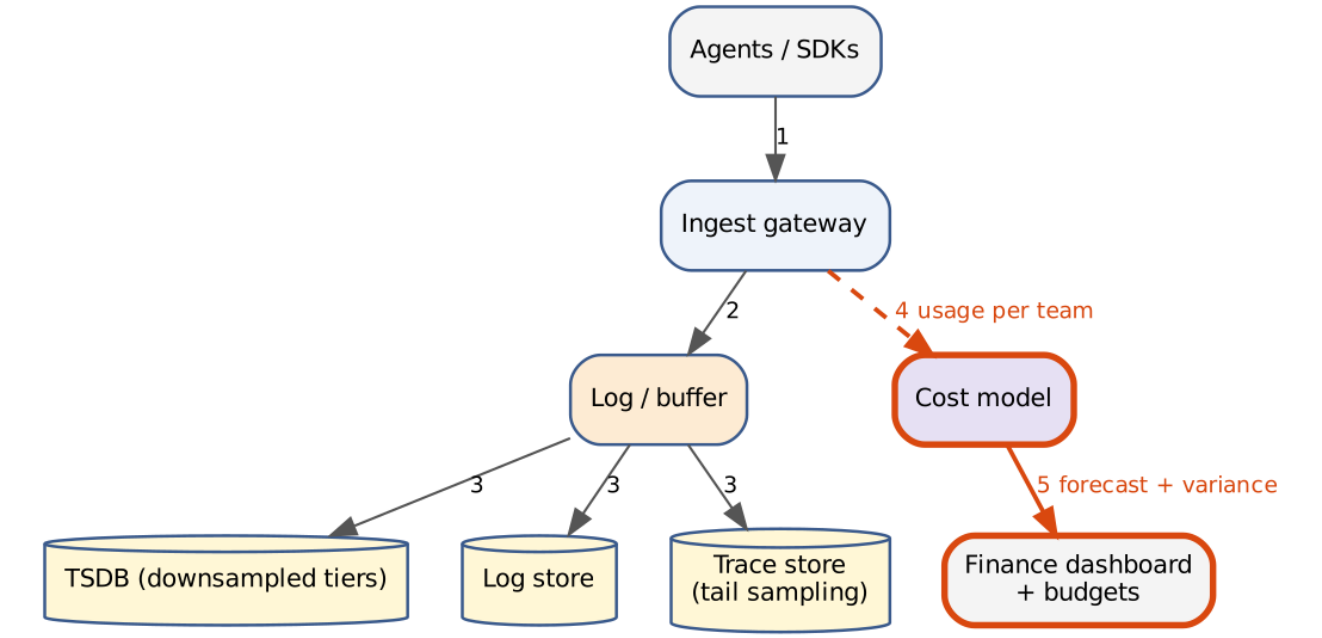
Step 2 · Purpose-built stores (1 min)



> **You:** “Metrics go to a time-series database with downsampled tiers, logs to the log store, traces to a trace store with tail sampling.”

Why this step now: Each signal has a different cost driver.

Step 3 · Cost attribution and forecast (1–2 min)



> **You:** "The gateway emits usage per team; a cost model multiplies by unit costs and forecasts by driver. Finance gets showback, forecast versus actual, and the efficiency roadmap."

Why this step now: This is what the cross-org round is really about.

Narrate one request end to end (numbers match the arrows)

From signal to dollars

1. Agent sends signals (1); gateway applies quotas/sampling and records usage (2, 4).
2. Signals land in their stores (3).
3. Cost model computes per-team cost and driver-based forecast for finance (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Agents / SDKs	Emit metrics, logs, traces	—	OpenTelemetry
Ingest gateway	Quotas, sampling, cardinality limits	Cost control point	Service
Log / buffer	Decouple ingest from stores	Spikes	Kafka
TSDB (downsampled tiers)	Metrics	Purpose-built storage	Prometheus/M3/ Mimir
Log store	Logs	See Q2	OpenSearch + S3
Trace store (tail sampling)	Traces	Keep errors/slow, sample rest	Tempo / Jaeger

Cost model	Usage × unit cost per team; forecast by driver	Finance needs predictability	Batch job + warehouse
Finance dashboard + budgets	Showback, forecast vs actual	Shared language with finance	BI dashboards

Before you leave the high-level design, check you have said:

☐ Cardinality as cost driver
☐ Unit economics
☐ Driver-based forecast + variance cadence

60-second summary to say once the diagram is complete:

> **You:** "Architecturally: agents and SDKs send to an ingest gateway, which enforces per-team quotas and sampling, then into a log; metrics go to a time-series database with downsampled tiers, logs to a log store (see the logging design), and traces to a trace store with tail-based sampling — keep all error and slow traces, sample the rest. The piece that matters for this conversation is the gateway emitting usage per team, which feeds a cost model."

8. Deep dives

Deep dive 1: what actually drives cost. Metrics cost is driven by cardinality (a label like `user_id` explodes series counts); logs by volume and retention; traces by sampling rate. Levers: cardinality limits and linting at ingest, downsampling (10 s → 1 min → 1 h tiers), retention by tier, tail sampling.

Deep dive 2: forecasting host spend (the finance conversation).

1. **Unit economics first:** cost per million samples ingested, per GB of logs stored per day, per million spans. Break host spend into these units.
2. **Drivers:** usage = f(services, hosts, traffic, team behavior). Forecast each driver from engineering roadmaps (new services launching, traffic plans) rather than extrapolating total spend.
3. **Model:** `forecast = Σ_signal (driver_forecast × usage_per_driver × unit_cost)`, with scenarios (base, high-growth, with-efficiency-projects).
4. **Variance tracking:** monthly forecast vs actual, explained by driver; share with finance proactively.
5. **Bend the curve:** efficiency roadmap with dollar estimates (e.g., cardinality limits −20%, log retention change −15%), chargeback or showback so teams see their costs.

> **You (to the finance partner):** "Our spend is growing 40% while traffic grows 15%, and the gap is mostly metric cardinality and log retention. I'll give you a driver-based forecast you can tie to the product roadmap, a monthly variance report, and three efficiency projects with

> estimated savings and dates. If we don't get to the target by Q3, here's the trade-off we'd bring to you: shorter retention for non-critical logs."

Deep dive 3: chargeback vs showback. Showback (visibility) first to build trust and fix obvious waste; chargeback later once attribution is accurate. Engineering and finance agree on the allocation method up front.

9–13. Failures, trade-offs, manager lens

- Observability must be more available than what it observes: separate failure domains; alerting path independent of the query path.
- Trade-off: dropping data vs paying — make it explicit per tier with service owners.
- Manager lens: "I'd hold a monthly cost review with finance and the top ten spending teams. My job is translating engineering choices into dollars and dollars back into engineering priorities."

14. Follow-ups (cross-org round style)

1. **"Finance says cut 20% next quarter. What do you do?"** — Show the efficiency options with savings, risk and effort; propose a mix; flag what we'd lose (e.g., 30-day → 14-day log retention for tier-3 services); agree before acting.
2. **"How do you handle a forecast miss?"** — Explain by driver (e.g., unplanned service launch), update the model, and add an early-warning signal (new high-cardinality metrics detected at ingest).
3. **"How do you get teams to care?"** — Showback dashboards, budgets per org, recognition for teams that reduce cost, platform defaults that are cheap by design.

15–17. Mock, scoring, say this

Strong Hire: speaks finance's language (unit costs, drivers, variance), ties cost to architecture levers, proposes a cadence and mechanisms. **Lean No Hire:** "we'll optimize later," no forecast method.

Say this: "I forecast drivers, not totals." · "Cardinality, not hosts, is what we're really paying for." · "Showback first to build trust, chargeback once attribution is accurate."

18. Your tie-in

Cloud modernization almost certainly involved cloud cost management with finance. Use a real number and a real lever. [VERIFY]

Chapter 3 — Meta

Meta at a glance

Loop	Screening: one behavioral + one system/product design. Loop: coding (some reports mention AI-assisted coding), product or system design, project retro, people management
How design is scored	Product-minded scaling: clear user flows, scale math, and **what you'd measure**. Interviewers have reported focusing on "scaling + metrics"
Strong Hire signals	User experience defined first; real-time and fan-out handled with numbers; metrics and experimentation; crisp trade-offs at pace
Pitfalls	Over-engineering before the core flow works; no metrics; ignoring hot/viral content

Meta question bank

#	Question	Confidence	Source
Q14	Real-time comment system for a Facebook post with millions of concurrent users (focus: scaling + metrics)	High (two separate reports)	Glassdoor Meta EM reviews
Q15	Design Facebook News Feed / Instagram	Medium–High	Glassdoor Meta EM reviews
Q16	A hardware-software system design	Medium (vague report)	Glassdoor Meta EM review
—	Generic industry-standard system design ×2 + LeetCode-style coding	Medium	Glassdoor Meta EM review

Q14. Real-time comments on a post with millions of concurrent viewers

> Meta (High) · also Microsoft (Teams/live events), Amazon (live shopping), Netflix (live) · Source: Glassdoor Meta EM reviews (two reports)

1. The prompt

"Design a real-time comment system under a Facebook post that may have millions of concurrent active users." One report says the interviewer focused on **scaling and metrics**.

2. Clarify

Ask:

- "Is this a normal post or a live video/event post? What's the most concurrent viewers we design for?" — the viral case drives everything.
- "Must viewers see every comment, or is a representative stream acceptable at very high volume?" — this is *the* key question; at 10k comments/s nobody can read them all.
- "How fresh — within a second or two?"
- "Do we need ranking (top comments) as well as a live stream?"
- "Reactions too, or just comments?"
- "What metrics define success — engagement, latency, delivery rate?" — Meta wants you to ask this.

Don't ask: "WebSockets or polling?" (decide) · "What's the comment length limit?" (assume).

State: "Up to 5M concurrent viewers on a viral live post, up to 10k comments/s on that post at peak, delivery to viewers within ~2 seconds, and at that volume viewers see a sampled/ranked live stream rather than every comment; full comment list is browsable with pagination."

3. Requirements

- **Must:** post a comment (persisted, visible to author immediately); live stream of new comments to viewers; paginated history; moderation (spam, abuse) before broad distribution.
- **Stretch:** top-comment ranking; reactions; replies.
- **Non-functional:** comment write durable and p99 < 300 ms; live delivery p95 < 2 s; system must survive a single post with millions of viewers (hot key); graceful degradation.

4. Estimates

- **Writes (global):** 2B DAU × 5% × 2 comments = 200M/day ≈ 2.3k/s average, ~7k/s peak. Easy.

- **Hot post:** 10k comments/s on one post — a single partition hot key on writes.
- **Fan-out:** naive = 10k comments/s × 5M viewers = 50B deliveries/s. Impossible → **batching and sampling**: push one update batch per viewer every 1–2 s with at most ~20 comments → 5M viewers / 2 s = 2.5M messages/s for the hot post. Still big, but spread across many gateways.
- **Connections:** 5M concurrent for one post; global maybe 100M+ live connections → ~200–1,000 gateway hosts at 100k–500k connections each.

5. API

```
POST /v1/posts/{post_id}/comments   {text, client_comment_id}  → {comment_id, created_at}
GET  /v1/posts/{post_id}/comments?order=recent|top&cursor=...  → {comments[], next_cursor}
WS   /v1/live  → client sends {subscribe: post_id}; server pushes {post_id, comments:[...],
total_count}
```

6. Data model

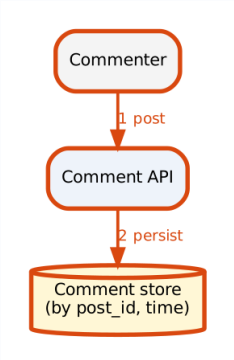
- `Comment(post_id, comment_id [time-ordered ID], author_id, text, created_at, status: visible|hidden|pending)` — partitioned by `post_id`, clustered by `comment_id` (wide-column store: Cassandra/TAO-like).
- Hot post: split a post's comments across N sub-partitions (`post_id#bucket`) when its write rate crosses a threshold; reads merge buckets.
- `Subscriptions`: in-memory on gateways (`post_id → local connections`) and in the pub/sub tier (`post_id → set of gateways`).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

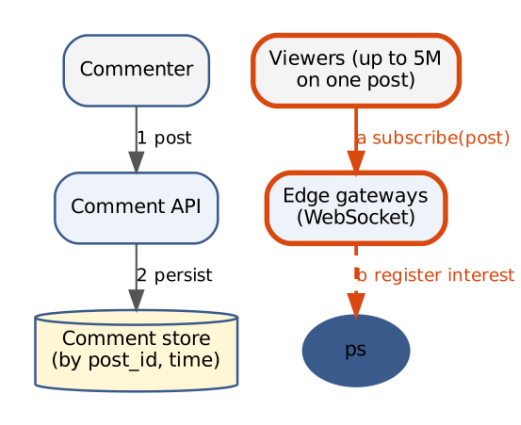
Step 1 · Write path (1 min)



> **You:** “A comment goes to the comment API, which rate-limits, runs fast integrity checks, and persists it in a store partitioned by post. The author sees it immediately from local state.”

Why this step now: The easy half first.

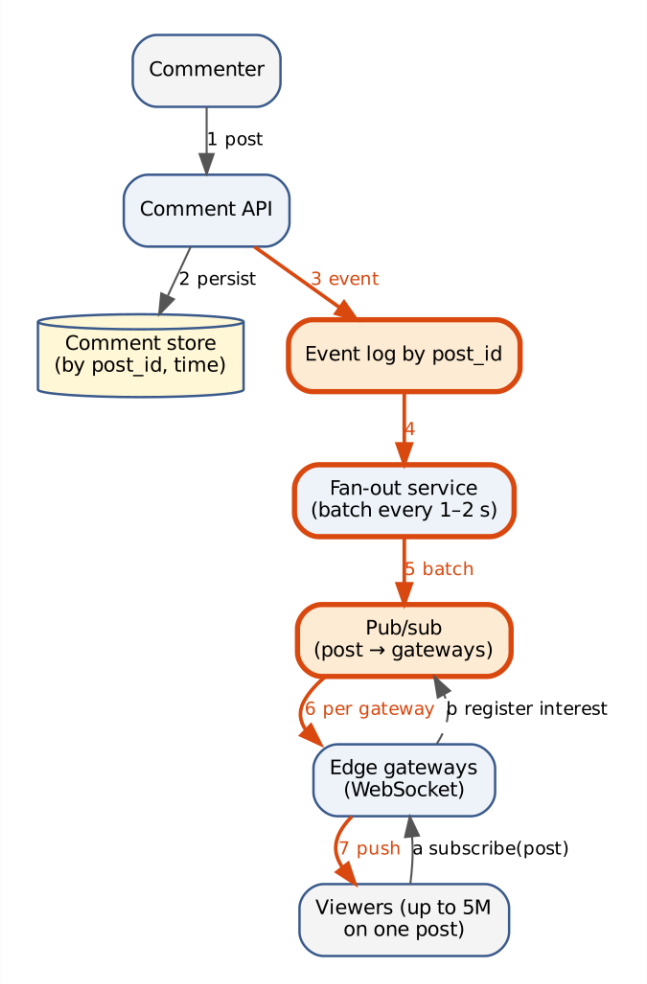
Step 2 · Viewers connect (1 min)



> **You:** "Viewers hold WebSockets to edge gateways and subscribe to the post. Each gateway registers with the pub/sub tier that it has viewers for that post."

Why this step now: Set up the delivery topology before fan-out.

Step 3 · Fan-out (2 min)



> **You:** “The API also publishes an event to a log partitioned by post. A fan-out service builds a batch per post every one to two seconds — every comment for quiet posts, a ranked sample for viral ones — and publishes it once per gateway. Each gateway writes to its local connections from memory.”

Why this step now: Do the fan-out math out loud here.

Narrate one request end to end (numbers match the arrows)

Comment to viewers

1. Comment posted (1) and persisted (2); event appended to log (3).
2. Fan-out consumes (4), batches per post, publishes once (5).
3. Pub/sub sends one message per subscribed gateway (6).
4. Each gateway pushes to its local viewers (7).

Viewer joins

1. Viewer subscribes on its gateway (a); gateway registers interest (b).
2. Initial comments fetched from the store; live updates follow.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Commenter	Post a comment	—	App
Comment API	Rate limit, integrity checks, persist	Write path	Service
Comment store (by post_id, time)	Durable comments, pagination	History and source of truth	Wide-column store
Viewers (up to 5M on one post)	Watch the live stream	The fan-out problem	App
Edge gateways (WebSocket)	Hold connections; local subscriptions	Multiplication happens here, in memory	WS gateway fleet
Event log by post_id	Comment events	Decouple write from delivery	Kafka
Fan-out service (batch every 1–2 s)	Build per-post batches; sample hot posts	Cuts messages by orders of magnitude	Stream workers
Pub/sub (post → gateways)	Route batches only to gateways with viewers	One message per gateway, not per viewer	Pub/sub tier

Before you leave the high-level design, check you have said:

☐ Fan-out math (50B naive → batching)
☐ Two-level fan-out
☐ Hot post on write (bucketing)

☐ Metrics + canary viewers

60-second summary to say once the diagram is complete:

> **You:** "Let me separate the write path from the delivery path. On write: the comment API rate-limits, runs fast integrity checks, writes to the comment store partitioned by post, returns success, and publishes an event to a log partitioned by post_id. The author sees their comment immediately from the local client state — no need to wait for fan-out."
>
> "On delivery: viewers hold a WebSocket to an edge gateway and subscribe to the post. Each gateway tells the pub/sub tier 'I have viewers for post X.' A fan-out service consumes the log and, for each post, builds a batch every second or two — for normal posts every comment, for hot posts a ranked sample. It publishes the batch once per gateway, not once per viewer. Each gateway then writes it to its local connections. So the expensive multiplication happens at the edge, in memory, spread across hundreds of machines."

8. Deep dives

Deep dive 1: the fan-out math and the two-level fan-out.

- Level 1: fan-out service → gateways subscribed to the post (hundreds to thousands of messages per batch).
- Level 2: gateway → its local connections (e.g., 10k viewers of that post on that gateway → 10k socket writes from memory).
- Batching every 1–2 s cuts message count by orders of magnitude and smooths load. For quiet posts, push immediately.

Deep dive 2: hot post on the write side. One post at 10k writes/s overloads one partition. Detect hotness (write rate per post), switch to bucketed partitions `post_id#0..15`, write to a random bucket; readers merge by timestamp. Counter for total comments uses sharded counters too.

Deep dive 3: what viewers see when volume is huge (product + scale). Above ~20 comments/s, show a *sample*: prefer comments from friends, high-quality signals, verified accounts, diversity; always include the viewer's own comments and replies to them. This is a product decision; propose it and validate with experiments.

Deep dive 4: metrics (what the interviewer focused on).

Metric	Why	Target
Comment post success rate, p99 latency	Write health	≥ 99.95%, < 300 ms
End-to-end delivery latency (comment created → rendered on viewers)	Real-time promise	p95 < 2 s
Delivery ratio (comments delivered / expected per viewer)	Detect silent drops	≥ 99% for normal posts

Metric	Why	Target
Connections per gateway, reconnect rate	Capacity and stability	Alert on reconnect spikes
Fan-out lag per post	Hot post backlog	< 2 s
Engagement: comments per viewer, time watching	Product health	A/B tested
Integrity: spam rate, moderation latency	Safety	Track by region/ language

Measure delivery latency with **synthetic canary viewers** on each gateway cluster and with client-side timestamps sampled back.

9. Scaling evolution

Stage	Design
Small	Clients poll `GET comments?since=` every few seconds
Target	WebSockets + pub/sub + per-post batching; sharded comment store
Viral 10×	Hot-post bucketing, sampled live stream, regional fan-out trees (fan-out service in each region), degrade to longer batch intervals

10. Failures and degradation

- Gateway dies → clients reconnect with jittered backoff to another gateway, resubscribe with last-seen comment_id to fill the gap from storage.
- Fan-out lag on a hot post → increase batch interval (2 s → 5 s), reduce sample size; never block writes.
- Comment store slow → accept writes into the log first (durable), persist asynchronously, show to author immediately; degrade history pagination.
- Deploy causes reconnect storm → drain gateways gradually, server-suggested reconnect delays.

11. Trade-offs

Decision	Options	Choice	Why
Push vs pull	Poll vs WebSocket/SSE	Push with poll fallback	Seconds-level freshness at scale
Per-comment vs batched delivery		Batched (1–2 s)	Reduces messages by orders of magnitude
Every comment vs sampled		Sampled above a threshold	Humans can't read 10k/s; protects the system

Decision	Options	Choice	Why
Fan-out location	Central per-viewer vs gateway-local	Gateway-local	Multiplication happens in memory at the edge

12. Security, operations, cost

- **Integrity:** rate limits per user and per post; spam classifiers inline (fast) and async (deep); hide-then-review for borderline content; ability to freeze comments on a post.
- **Ops:** SLOs on delivery latency and write success; runbook for "viral event" (pre-scale gateways for scheduled live events).
- **Cost:** gateway fleet and bandwidth dominate; batching and compression (permessage-deflate) reduce bandwidth.

13. Manager lens

> **You:** "Three teams: the comments service (write path, storage), real-time infrastructure (gateways, pub/sub, shared with Messenger and live features), and integrity. I'd make real-time delivery a platform shared across products, since building it per feature is how you end up with five fragile systems. The product decision — sampling comments on viral posts — I'd drive with the product manager and validate with an experiment, measuring engagement and complaints."

14. Follow-ups

1. **"How does a viewer who reconnects not miss comments?"** — Resubscribe with last-seen comment_id; gateway fetches the gap from the store (bounded, e.g., last 50).
2. **"How do you show the total comment count?"** — Sharded counter, approximate in real time, exact asynchronously.
3. **"How would you rank 'top comments'?"** — Score = engagement (reactions, replies) with time decay + author affinity + quality; recompute periodically; cache top-N per post.
4. **"How do you test this before a big live event?"** — Load test with synthetic viewers at 2× expected; pre-warm gateways; game day.
5. **"Global audience?"** — Regional fan-out services subscribe to the post's log; cross-region traffic is one stream per region, not per viewer.

15. Mock transcript

> **Interviewer:** Design real-time comments for a post with millions of concurrent viewers.
>
> **You:** The key question: at 10k comments a second, do viewers need every comment? … I'll assume a ranked sample above a threshold. Quick math: naive fan-out is 10k × 5M = 50 billion deliveries a second, so I'll batch per post every 1–2 seconds and fan out in two levels — to gateways, then from gateway memory to sockets.
>
> **Interviewer:** What would you measure?

> **You:** End-to-end delivery latency with canary viewers, delivery ratio to catch silent drops, write success and latency, fan-out lag per post, reconnect rate — and engagement per viewer as the product metric, A/B tested when we change sampling.

16. Scoring guide

Rating	Signals
Strong Hire	Fan-out math; two-level fan-out with batching; hot post handling on write and read; sampling as a product decision; strong metrics list with measurement method
Hire	WebSockets + pub/sub; sharded storage; some metrics
Lean No Hire	Per-viewer pushes per comment; no hot-key thinking
No Hire	Polling the database per viewer

17. "Say this"

"The multiplication should happen at the edge, in memory." · "Nobody can read ten thousand comments a second, so sampling is a product decision, not just a scaling trick." · "I'd measure delivery with canary viewers, not trust server-side counters."

18. Your tie-in

If your org built any real-time notifications for underwriters or brokers (status updates on submissions), reference the pattern. [VERIFY]

Q15. Design News Feed (Facebook / Instagram)

> Meta (Medium–High) · common everywhere ("design Twitter") · Source: Glassdoor Meta EM reviews

1. The prompt

"Design News Feed." Instagram variant: photo/video posts, follow graph (asymmetric), Explore.

2. Clarify

Ask: "How many users, average friends/follows, and the max (celebrities)?" · "Ranked or chronological?" · "Media types?" · "Feed freshness — how quickly should a new post appear?" · "What's the success metric — time spent, meaningful interactions?"

State: "2B users, 1B DAU, avg 300 friends, some accounts with 100M followers; ranked feed; posts visible to friends within ~a minute; media stored separately."

3. Requirements

- **Must:** publish posts; fetch a ranked feed with pagination; media upload; like/comment counts.
- **Non-functional:** feed load p99 < 500 ms; high availability (stale feed is better than an error); eventual consistency fine.

4. Estimates

- Feed reads: 1B DAU × 10 loads/day = 10B/day ≈ **115k reads/s**, peak ~350k/s.
- Posts: 1B × 0.5 = 500M/day ≈ 6k/s.
- Fan-out on write: 6k posts/s × 300 friends = **1.8M feed inserts/s** — feasible with sharded caches; but a 100M-follower post = 100M inserts → don't push celebrities.
- Feed cache: 1B users × 500 post IDs × 8 B = 4 TB — fits in a distributed cache.

5. API

```
POST /v1/posts {text, media_ids[], audience}       → {post_id}
GET  /v1/feed?cursor=...&limit=20                   → {items:[{post, author, counts,
reason}], next_cursor}
POST /v1/media:upload  → presigned URL
```

6. Data model

- `Post(post_id, author_id, created_at, text, media_refs, audience)` — sharded by author_id.
- `Graph`: adjacency lists `friends(user_id) → [ids]` (TAO-like graph cache).
- `FeedCache(user_id) → list of (post_id, author_id, ts)` capped at ~500.

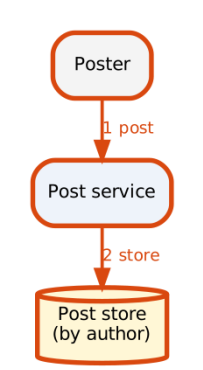
- Media: object storage + CDN.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

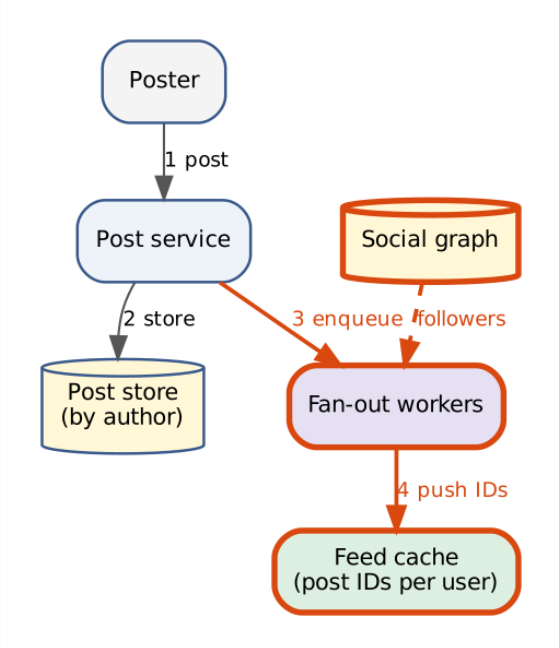
Step 1 · Publishing (30 s)



> **You:** “A post goes to the post service and is stored, sharded by author.”

Why this step now: Start simple.

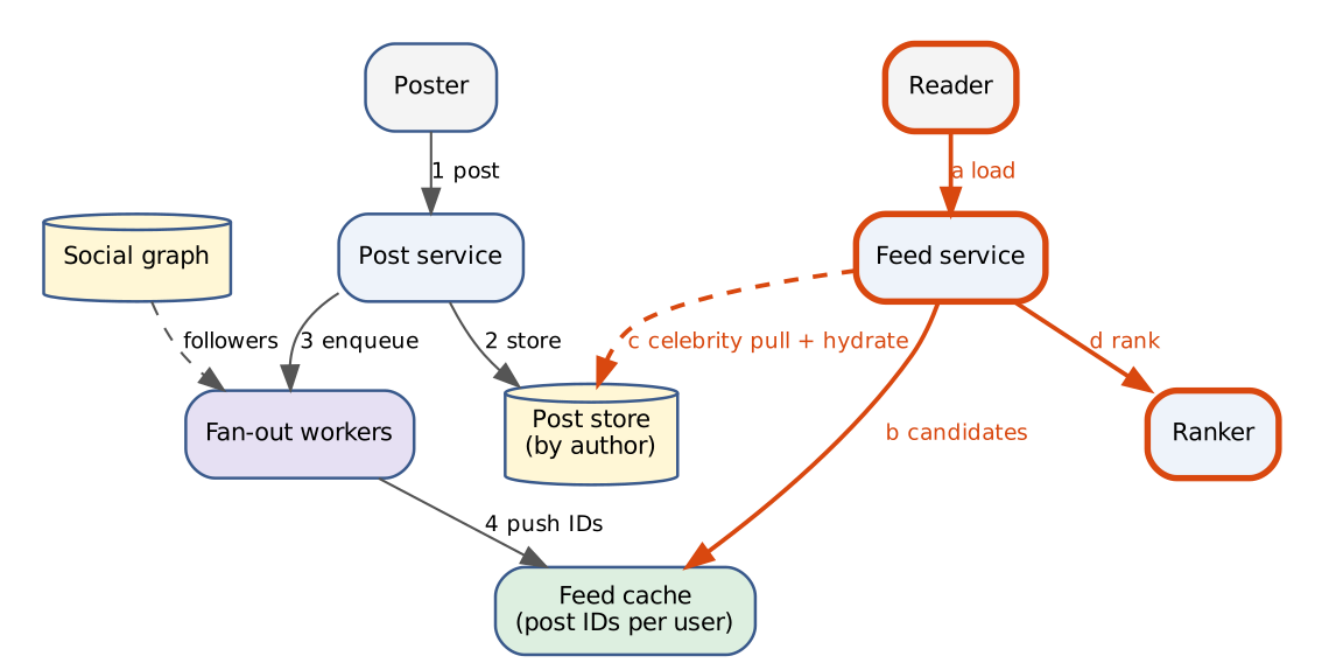
Step 2 · Fan-out on write (1–2 min)



> **You:** “Fan-out workers look up the author's followers and push the post ID into each follower's feed cache — for normal authors only. Accounts with huge followings are not pushed.”

Why this step now: State the hybrid decision here with the math.

Step 3 · Reading the feed (1–2 min)



> **You:** "The feed service reads your candidate IDs, pulls recent posts from celebrities you follow, hydrates posts and counts, and sends candidates to the ranker, which returns the top 20."

Why this step now: Read path completes the design.

Narrate one request end to end (numbers match the arrows)

Post

1. Post stored (1, 2); fan-out enqueued (3); IDs pushed to followers' caches (4).

Load feed

1. Reader requests feed (a); candidates from cache (b); celebrity posts pulled and all hydrated (c); ranked (d) and returned.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Poster	Create posts	—	App
Post service	Store posts	Source of truth	Service
Post store (by author)	Posts	—	Sharded store

Fan-out workers	Push post IDs to followers' feeds	Cheap reads for normal authors	Queue consumers
Social graph	Friend/follower lists	Who receives what	TAO-like graph cache
Feed cache (post IDs per user)	Precomputed candidates	Fast feed loads	Redis / memcache
Reader	Load feed	—	App
Feed service	Merge pushed IDs + pulled celebrity posts, hydrate	Hybrid fan-out	Service
Ranker	Score and order candidates	Ranked feed	ML ranking service

Before you leave the high-level design, check you have said:

☐ Fan-out math and celebrity threshold
☐ IDs only, hydrate on read
☐ Fallback to recency

60-second summary to say once the diagram is complete:

> **You:** "The core trade-off is fan-out on write versus on read. Pushing every post into every friend's feed makes reads cheap, but a celebrity with 100 million followers turns one post into 100 million writes. Pulling at read time is cheap to write but expensive to read. So I'd use a hybrid: push for normal users, pull for high-follower accounts."
>
> "When you load your feed, the feed service reads your precomputed list of candidate post IDs, merges in recent posts from the celebrities you follow, hydrates the posts and counts in parallel from caches, and sends ~500 candidates to the ranker, which returns the top 20."

8. Deep dives

Deep dive 1: ranking pipeline. Candidate generation (~500–1,000) → lightweight first-pass model → heavy model on top ~100 (predict probabilities of like, comment, share, hide) → value score = weighted sum → diversity rules (not five posts from one author) → integrity demotion. Features from a feature store; models trained offline on engagement logs; A/B tested.

Deep dive 2: freshness and consistency. New post appears in your own feed immediately (read-your-writes from the author's client). Friends see it after async fan-out (seconds to a minute). Deleted posts: delete from post store; feed entries are filtered at hydration time (don't chase every feed cache).

Deep dive 3: availability. If ranking is slow, return chronological from cache. If the feed cache misses (inactive user), rebuild on read from friends' recent posts (pull) and repopulate. Don't precompute feeds for inactive users — saves huge storage.

9–11. Scaling, failures, trade-offs

Decision	Choice	Why
Fan-out	Hybrid push/pull by follower count threshold (~10k+)	Bounds write amplification
Feed store	Cache of IDs only, hydrate on read	Small, cheap, always-fresh content
Inactive users	No precompute; build on read	Most users are not daily
Ranking failure	Fallback to recency	Stale feed beats an error

12–13. Ops, cost, manager lens

- SLIs: feed load latency, empty-feed rate, ranking timeout rate, fan-out lag. Product metrics via experiments.
- Manager lens: "Feed infra, ranking/ML, and integrity are separate teams. The shared contract is the candidate and feature interfaces. I'd guard the experimentation platform — every ranking change ships behind an A/B test with guardrail metrics like hides and reports."

14. Follow-ups

1. **"Instagram vs Facebook — what changes?"** — Asymmetric follows (more celebrity skew → more pull), media-heavy (CDN, transcoding), Explore uses interest-based candidates, not just the graph.
2. **"How do you paginate a ranked feed?"** — Snapshot candidate set per session with a cursor; don't re-rank between pages or users see duplicates.
3. **"How do you handle a user who follows 5,000 people?"** — Cap candidate retrieval per source; rely on ranking.

15–17. Mock, scoring, say this

> **You:** "Fan-out on write for normal users, on read for celebrities — that bounds write amplification. Feeds store IDs only and hydrate on read, and if ranking times out, I serve recency. A stale feed beats an error."

Strong Hire: hybrid fan-out with math, ranking stages, inactive-user insight, graceful degradation, experiments. **No Hire:** query all friends' posts from the DB on every load at 350k RPS.

Q16. A hardware-software system (example: smart glasses capture and sync)

> Meta (Medium — the report says only "a design question for a hardware-software system") · Practice for Meta Reality Labs, Amazon devices, Apple · This specific scenario is a representative example, not a reported prompt.

1. The prompt (representative)

"Design the system that lets smart glasses capture photos and videos and sync them to the user's phone and the cloud." Other possible framings: fitness tracker telemetry; smart home device control; VR headset updates.

2. Clarify — what's different with hardware

Ask: "What are the device constraints — battery, storage, connectivity (Bluetooth to phone, Wi-Fi)?" · "Is the phone always nearby?" · "Privacy expectations — on-device processing, indicators when recording?" · "Fleet size and firmware update model?" · "Which features need cloud vs can run on device?"

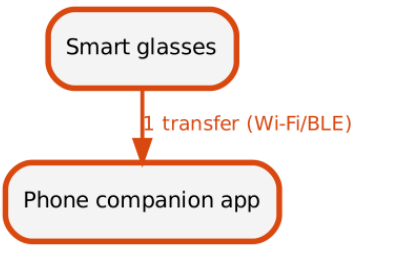
State: "Glasses with limited battery and 32 GB storage, BLE always, Wi-Fi sometimes; phone usually nearby; media encrypted; capture LED required; 10M devices; OTA updates via the phone."

3. Key design points (walkthrough)

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

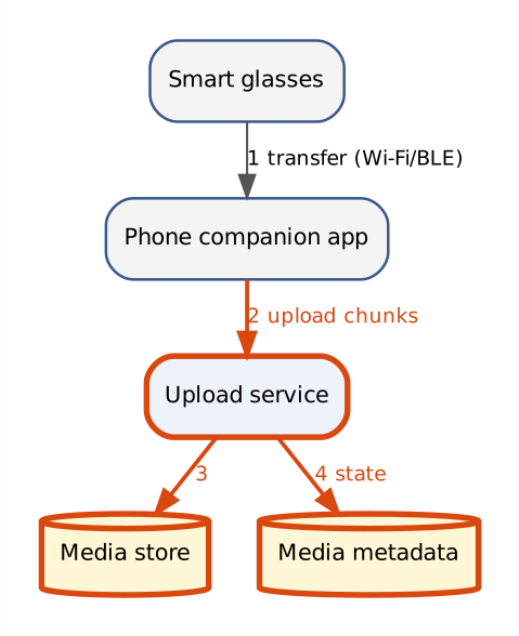
Step 1 · Device to phone (1 min)



> **You:** “Glasses capture, encode with the hardware encoder, and queue items locally, encrypted. The phone app pulls them — Bluetooth for small items and thumbnails, Wi-Fi for bulk — ideally while charging.”

Why this step now: Battery is the product; design device-first.

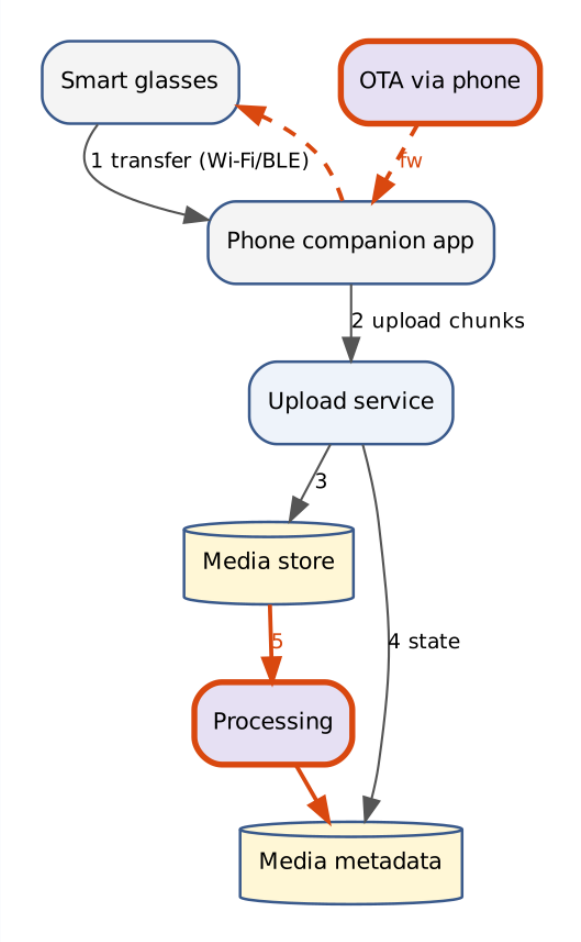
Step 2 · Phone to cloud (1 min)



> **You:** “The phone uploads resumable chunks to an upload service; media goes to storage, and a metadata record tracks each item's state from captured to confirmed.”

Why this step now: Reliable sync over bad networks.

Step 3 · Processing and updates (1 min)



> **You:** “Cloud processing transcodes and generates thumbnails, with AI features only on opt-in. Firmware updates flow back through the phone using the staged rollout pattern.”

Why this step now: Close the loop.

Narrate one request end to end (numbers match the arrows)

Photo to cloud

1. Glasses transfer to phone (1).
2. Phone uploads chunks (2) to storage (3); metadata updated (4).
3. Processing runs (5); device deletes local copy only after confirmation.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Smart glasses	Capture, encode, local encrypted queue	Battery and storage constrained	Device firmware
Phone companion app	Pull media over Wi-Fi/BLE, show user	Relay with better connectivity	Mobile app

Upload service	Resumable chunked uploads	Unreliable networks	Service + presigned URLs
Media store	Encrypted media	—	Object storage
Media metadata	Item state, device, timestamps	Sync state machine	KV / SQL
Processing	Transcode, thumbnails, opt-in AI	Heavy work off device	Workers / GPU
OTA via phone	Firmware updates	See Q3	Rollout service

Before you leave the high-level design, check you have said:

☐ Battery/connectivity constraints
☐ Stable item IDs + idempotent upload
☐ Privacy: capture LED, consent

60-second summary to say once the diagram is complete:

> **You:** "Hardware changes the design in three ways: the device is constrained, connectivity is intermittent, and updates are slow and risky. So I design device-first and offline-first."

- **On device:** capture → encode efficiently (hardware encoder) → store in a local queue with metadata → encrypt with a device key. Prioritize battery: transfer large files only over Wi-Fi or when charging; BLE for small control messages and thumbnails.
- **Phone as relay:** companion app pulls media from the glasses (Wi-Fi direct for bulk), shows it to the user, uploads to cloud with resumable chunked uploads (Q4 pattern) when on Wi-Fi/charging per user settings.
- **Cloud:** upload service, media store, metadata DB, processing (transcode, thumbnails, optional AI features with user consent).
- **Sync protocol:** each media item has a stable ID generated on device; idempotent uploads; state machine `captured → transferred_to_phone → uploaded → confirmed`; device deletes local copy only after confirmation (and only if user chose that).
- **Firmware:** OTA via phone (Q3 pattern): staged waves, A/B partitions, signed images.
- **Privacy:** capture LED enforced in firmware (can't be disabled by software), on-device processing where possible, user controls for cloud backup, encryption in transit and at rest.

4. Estimates

- 10M devices × 20 photos + 2 min video/day ≈ 20 × 3 MB + 120 MB ≈ 180 MB/device/day → 1.8 PB/day if all uploaded; realistically 30% opt into cloud backup → ~540 TB/day. Storage cost drives tiering and compression decisions.

5. Trade-offs to discuss

Decision	Choice	Why
Direct-to-cloud from glasses vs via phone	Via phone	Battery and radio constraints; phone has better connectivity
On-device vs cloud AI	On-device for latency/privacy-sensitive features; cloud for heavy models with consent	Battery vs capability vs privacy
Transfer timing	Defer bulk to Wi-Fi/charging	Battery life is the product

6. Metrics and manager lens

- Metrics: capture-to-phone latency, transfer success rate, battery drain per transfer, sync backlog per device, crash rates by firmware version.
- Manager lens: "Hardware, firmware, mobile app and cloud teams ship on very different cadences — hardware is fixed for years, firmware monthly, cloud daily. I'd design protocol versioning so the cloud supports old firmware for years, and I'd run a cross-team integration test lab with real devices."

7. "Say this"

"On hardware, battery is the product, so the system defers bulk work to Wi-Fi and charging." · "The device deletes local media only after the cloud confirms." · "Hardware lives for years, so the cloud must speak every protocol version still in the field."

Chapter 4 — Microsoft

Microsoft at a glance

Loop	Hiring manager interview → 4–5 loop rounds (system design, possibly a lighter coding round, leadership/behavioral, often a PM partner) → "As Appropriate" (AA) interview with a senior leader if feedback is positive, who makes the final call
How design is scored	Practical cloud design, customer scenario clarity, collaboration, growth mindset; Azure literacy helps
Pitfalls	Generic designs not tied to the customer scenario; weak collaboration signals with the PM partner
Data caveat	Recent Microsoft EM-specific design reports are thin. Q17 is reported (for SWE); Q18 is a strong domain pattern. Practice both for patterns

Microsoft question bank

#	Question	Confidence	Source
Q17	Design the notification system for Microsoft Teams	Medium (SWE guide, 2026)	CleverPrep Microsoft guide
Q18	Design real-time collaborative editing (Word/Loop co-authoring)	Low / Likely	Domain pattern
—	Also practice from other chapters: Q4 file sync (OneDrive), Q2 logging, Q14 live comments (Teams live events), Q10 storage with residency (EU Data Boundary)	—	—

Q17. Design the notification system for Microsoft Teams

> Microsoft (Medium) · generic notification design also at Meta, Amazon · Source: CleverPrep Microsoft SWE guide (2026)

1. The prompt

"Design notifications for Teams: mentions, messages, meeting reminders, app alerts — across desktop, mobile and email."

2. Clarify

Ask: "Which notification types and which channels?" · "Scale — users, events per day?" · "Enterprise requirements — tenant policies, compliance, quiet hours?" · "Presence-aware? (don't push to phone if active on desktop)" · "Delivery guarantees — must every mention be delivered?"

State: "300M monthly users, 100M daily; ~5B notification-worthy events/day; channels: in-app activity feed, desktop toast, mobile push, email digest; presence-aware routing; tenant admin policies; mentions must be delivered at least once; others best-effort with batching."

3. Requirements

- **Must:** generate notifications from events; respect user preferences, do-not-disturb and tenant policies; route by presence; dedupe across devices; activity feed with read state synced across devices.
- **Non-functional:** mention delivery p95 < 3 s; no duplicate push storms; 99.95% availability; per-tenant data residency.

4. Estimates

- 5B events/day ≈ 58k/s average, ~200k/s peak (Monday 9 a.m. per time zone).
- After filtering/aggregation, maybe 1.5B notifications delivered/day ≈ 17k/s avg.
- Activity feed storage: 100M users × 200 items × 500 B = 10 TB.

5. API

```
Internal: publish(event{type, tenant, actor, targets[], thread_id, priority})
GET  /v1/activity?cursor=...                → feed items with read state
POST /v1/activity/{id}:read
PUT  /v1/preferences  {type → channels, quiet_hours, mute_threads[]}
Device registration: POST /v1/devices {platform: apns|fcm|wns, token}
```

6. Data model

- `NotificationPrefs(user_id, type, channels, quiet_hours, muted_threads)`.
- `ActivityItem(user_id, item_id, type, thread_id, actor, ts, read)` — partitioned by user.
- `DeviceToken(user_id, device_id, platform, token, last_seen)`.
- `Presence(user_id → active_device, status)` in a fast in-memory store.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

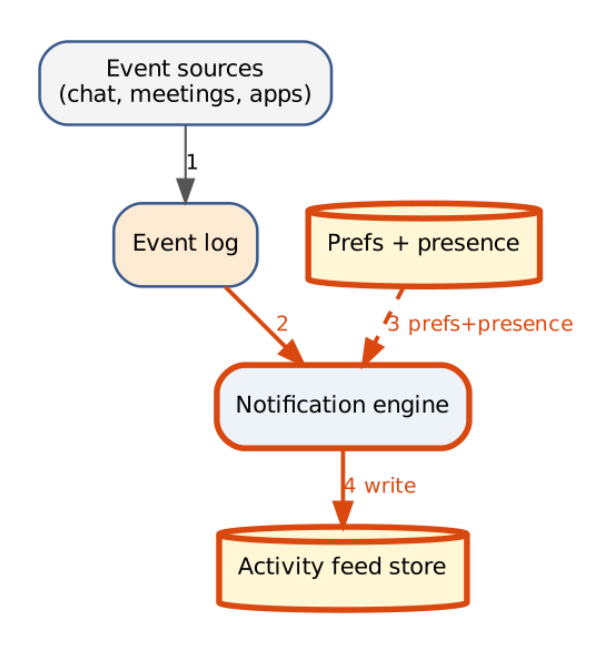
Step 1 · Events in (30 s)



> **You:** “Chat, meetings and apps publish typed events to a log; they never call push providers directly.”

Why this step now: Platform boundary first.

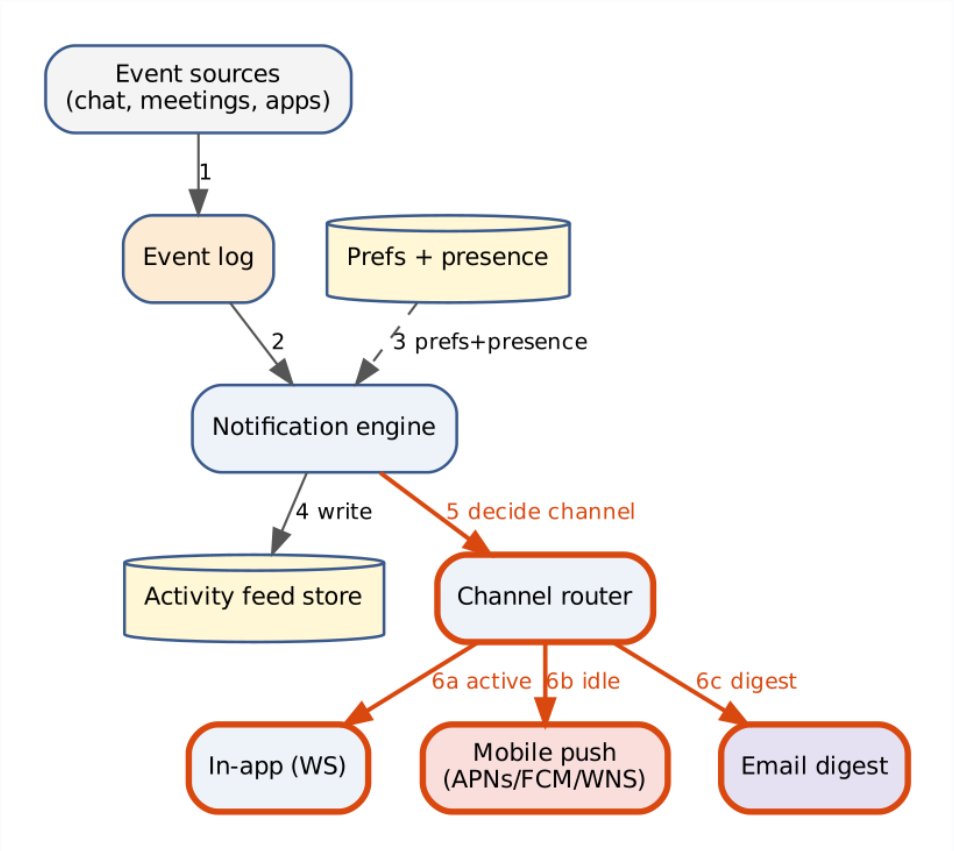
Step 2 · Decide and record (1–2 min)



> **You:** “The notification engine expands targets, applies preferences, tenant policy and quiet hours, dedupes and aggregates, and writes every notification to the user's activity feed — the source of truth for read state.”

Why this step now: Policy and the source of truth.

Step 3 · Deliver by presence (1 min)



> **You:** "A router picks the channel by presence: active on desktop gets an in-app toast, idle gets mobile push, low-urgency goes to an email digest."

Why this step now: What makes Teams notifications feel right.

Narrate one request end to end (numbers match the arrows)

@mention

1. Event published (1) and consumed (2).
2. Engine reads prefs and presence (3), writes the activity item (4).
3. Router picks channel (5): in-app if active (6a), push if idle (6b).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Event sources (chat, meetings, apps)	Emit typed events	—	Feature services
Event log	Durable event stream	Decouple producers	Event Hubs / Kafka
Notification engine	Expand targets, apply prefs/policy/ DND, dedupe, aggregate	All rules in one place	Stream workers

Prefs + presence	User settings, tenant policy, active device	Decides channel	Cosmos DB + in-memory presence
Activity feed store	Notifications + read state	Source of truth	Partitioned by user
Channel router	Send to the right channel	—	Service
In-app (WS)	Toasts to active clients	Lowest latency	Gateway fleet
Mobile push (APNs/ FCM/WNS)	Phone notifications	Best effort	Provider APIs
Email digest	Batched summaries	Low-urgency	Scheduler + email

Before you leave the high-level design, check you have said:

☐ Feed is source of truth; push best effort
☐ Presence-aware routing
☐ Large-channel fan-out with priority lanes

60-second summary to say once the diagram is complete:

> **You:** "Event producers — chat, meetings, apps — publish to a log. The notification engine consumes events, expands targets (a channel mention can mean thousands of users), checks each user's preferences, tenant policy, quiet hours and presence, and decides: activity feed only, in-app toast, mobile push, or email digest later."
>
> "Presence is what makes Teams notifications feel right: if you're active on desktop, we show an in-app toast and suppress mobile push; if you're idle for a few minutes, we push to mobile. Every notification also lands in the activity feed store, and read state syncs across devices so reading on desktop clears the badge on your phone."

8. Deep dives

Deep dive 1: fan-out for large channels. A message in a 10,000-member team channel isn't 10,000 pushes — most members get only a feed badge. `@channel` mentions to large teams: fan out via a queue in batches, rate-limited per tenant, with priority lanes so 1:1 mentions aren't stuck behind a large broadcast.

Deep dive 2: dedupe and aggregation. Key notifications by `(user, thread_id, type)`; collapse "5 new messages in thread X" within a short window; push providers (APNs/FCM/WNS) support collapse keys. Prevents notification storms.

Deep dive 3: delivery guarantees. At-least-once from log to engine; idempotency by event_id; push providers are best-effort, so the activity feed is the source of truth — on app open, the client syncs from the feed. Mentions retried with backoff; token invalidation handled when providers report unregistered devices.

9–11. Failures and trade-offs

- Push provider outage → queue with TTL; activity feed still correct.
- Presence store unavailable → default to sending both (safe) but with collapse keys.

Decision	Choice	Why
Source of truth	Activity feed, not push	Push is best-effort
Routing	Presence-aware	Avoid double-buzzing users
Large channel fan-out	Async batched with priority lanes	Protect 1:1 latency

12. Security, compliance, cost

- Enterprise: tenant admin policies (e.g., no message previews in mobile push for regulated tenants), data residency for activity stores (EU Data Boundary), retention and eDiscovery requirements.
- Cost: push provider calls and email are cheap per unit but huge in volume — aggregation is also a cost lever.

13. Manager lens

> **You:** "Notifications touch every feature team. I'd run it as a platform with a typed event contract, so feature teams publish events and never call push providers directly. The PM partner and I would own the notification policy together — volume of notifications is a product health metric, and over-notifying drives users to mute everything."

14. Follow-ups

1. **"How do you measure notification quality?"** — Open/click-through rates, mute and disable rates, complaints; experiment with aggregation windows.
2. **"Quiet hours across time zones?"** — Store user time zone; scheduler holds non-urgent notifications; urgent ones (configured by user/admin) bypass.
3. **"A tenant wants no content in push notifications."** — Tenant policy flag; push contains only "New message from X" or nothing, and the client fetches content after authentication.

15–17. Say this

"Push is best-effort; the activity feed is the source of truth." · "Presence-aware routing is what stops us buzzing every device at once." · "Over-notifying is a product bug — I'd track mute rate as a guardrail metric."

Q18. Real-time collaborative document editing (co-authoring)

> Microsoft (Likely — Word/Loop co-authoring; also Google Docs) · Practice for the pattern

1. The prompt

"Design real-time co-authoring: multiple users edit the same document and see each other's changes instantly."

2. Clarify

Ask: "How many concurrent editors per document?" · "Offline editing?" · "Rich text or plain text?" · "Version history and comments?" · "Latency expectation for remote edits?"

State: "Up to 100 concurrent editors on hot documents, typically 2–5; rich text; offline edits merged later; remote edits visible within ~200 ms; full history."

3. Core design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

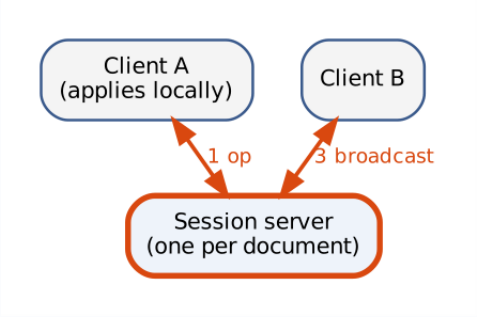
Step 1 · Local-first clients (30 s)



> **You:** “Each client applies edits locally the moment the user types.”

Why this step now: Latency starts with the client.

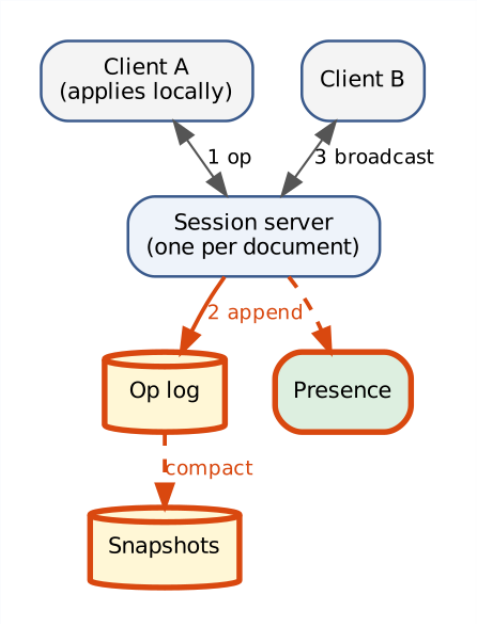
Step 2 · One sequencer per document (1–2 min)



> **You:** “All editors of a document connect to one session server that orders operations, transforms concurrent ones, and broadcasts the result.”

Why this step now: Turns a distributed problem into single-writer.

Step 3 · Durability and presence (1 min)



> **You:** “Ops are appended to a log before being acknowledged; snapshots every N ops bound load time. Cursors are broadcast but never stored.”

Why this step now: Recovery and history.

Narrate one request end to end (numbers match the arrows)

An edit

1. Client A applies locally and sends the op (1).
2. Server sequences, appends to the log (2), broadcasts the transformed op (3).
3. Client B transforms against pending local ops and applies.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Client A (applies locally)	Local edits, send ops	Typing never waits on network	Editor
Client B	Same	—	Editor
Session server (one per document)	Order ops (sequencer), transform/merge, broadcast	Single writer per doc	Stateful service with lease
Op log	Durable ops	History + recovery	Append-only store
Snapshots	Every N ops	Fast load	Object storage
Presence	Cursors, selections	Ephemeral	In-memory pub/sub

Before you leave the high-level design, check you have said:

☐ OT vs CRDT trade-off
☐ Session failover via lease + log replay
☐ Convergence testing

60-second summary to say once the diagram is complete:

> **You:** "Users must see their own keystrokes instantly, so clients apply edits locally first and send operations — 'insert "a" at position 42' — to a collaboration session server that owns that document. The server orders all operations and broadcasts them. The hard problem is that two users edit concurrently based on different versions, so positions shift. There are two standard solutions: Operational Transformation and CRDTs."

4. Deep dive: OT vs CRDT

	Operational Transformation (OT)	CRDT
How	Server is the central sequencer; transforms concurrent ops against each other (shift positions)	Each character has a unique, ordered ID; merges are commutative, no central transform needed
Pros	Compact ops; proven in Google Docs/Office-style systems	Works peer-to-peer and offline naturally; simpler server
Cons	Transform functions are subtle; needs a central server per doc	Metadata overhead per character; tombstones grow; garbage collection needed
Choice	Central session server with OT (or a sequenced CRDT) for online collaboration; CRDT merges are attractive for long offline edits	

Routing: all editors of a document connect to the same session server (consistent hashing on doc_id with a lease in a coordination service). If that server dies, another acquires the lease, loads the latest snapshot plus op log, and clients reconnect and resend unacknowledged ops.

Durability: append ops to a log before acknowledging; snapshot every N ops to bound load time; history = snapshots + ops.

Presence: cursors and selections are ephemeral — broadcast, never stored.

5. Estimates

- Typing generates ~5 ops/s per active editor; batch ops every ~50–100 ms per client. 1M concurrently edited documents × 3 editors × 5 ops/s = 15M ops/s globally → thousands of session servers; documents are independent, so it partitions cleanly.

6. Trade-offs, ops, manager lens

- Trade-off: one server per document creates a hot spot for 100+ editors — fine for typical docs; for huge docs, split by section or degrade (read-only for viewers, editing for a subset).
- SLIs: op round-trip latency, convergence errors (clients' document hashes diverge — detect via periodic checksums), session failover time.
- Manager lens: "Convergence bugs are data-loss bugs, so I'd invest in randomized testing — fuzz concurrent edit sequences and assert all replicas converge — before scaling the feature."

7. "Say this"

"Local first, so typing never waits on the network." · "One sequencer per document turns a distributed problem into a single-writer one." · "Convergence bugs are data-loss bugs — I'd fuzz them."

Chapter 5 — Databricks

Databricks at a glance

Loop	Hiring manager → panel presentation to directors and staff engineers → several technical/ architecture rounds: architecture interview (senior engineer), "systems" round mixing coding and design (staff IC), management interview (director), "technical fit" (defend your technical depth by walking through your current system)
Bar	Candidates report needing a "yes" from every round; system design reported as the hardest
How design is scored	Infrastructure depth: storage, metadata, consistency, distributed compute, multi-tenancy, cost. Familiarity with Spark, Delta Lake/lakehouse helps
Pitfalls	Box-level answers; not knowing what happens inside object storage or a distributed job; a systems-programming question that's too big for an hour — scope it down out loud (Part I, A10)

Databricks question bank

#	Question	Confidence	Source
Q19	Design an object-store system	Medium (2025 report)	Glassdoor Databricks EM review
Q20	Design a payment system like Visa/Mastercard	Medium	Glassdoor Databricks EM review
Q21	Design a scalable system to process streaming data (exactly-once into a lakehouse)	Medium	Databricks SWE guides (2026); Google variant reported
Q22	Design a multi-tenant job scheduler / compute control plane	Low / Likely	Domain pattern (control plane vs data plane)
—	Systems-programming problem (scope it; see A9 Example 5 log-structured KV store)	Medium	Glassdoor (Aug 2025)
—	Technical fit: walk through your current system in depth	Medium	Glassdoor Databricks EM review — use Q1

Q19. Design an object store (S3-like)

> Databricks (Medium) · also Google, Apple storage questions · Source: Glassdoor Databricks EM review (2025)

1. The prompt

"Design an object storage system." Expect deep follow-ups on metadata, durability, consistency, and listing — the parts that matter for a lakehouse.

2. Clarify

Ask:

- "Object size distribution — many small files, or large ones?" — small files stress metadata; large stress throughput.
- "Total capacity and growth?"
- "Consistency: strong read-after-write and list-after-write?" — lakehouse table formats depend on it (and on put-if-absent).
- "Durability and availability targets?"
- "Access patterns: write once, read many? Range reads (Parquet footers)?"
- "Multi-tenant? Single region or multi-region?"

State: "Exabyte scale long-term, start at 100 PB; objects from 1 KB to 5 TB with a long tail of small files; strong read-after-write and list consistency; conditional put (put-if-absent); 11 nines durability; 99.99% availability; single region with optional cross-region replication; range reads required."

3. Requirements

- **Must:** PUT/GET/DELETE object; multipart upload; LIST by prefix with pagination; range GET; conditional PUT (if-none-match); versioning; buckets with access policies.
- **Non-functional:** 11 nines durability; strong consistency for metadata ops; first-byte latency tens of ms; throughput scales with nodes.

4. Estimates

- 100 PB / average object 1 MB = **100 billion objects** → metadata at ~300 B each ≈ 30 TB — must be a distributed, sharded metadata store.
- Erasure coding 10+4 → 1.4× overhead → 140 PB raw; with 20 TB drives → ~7,000 drives; at 24 drives/node → ~300 storage nodes.
- Metadata ops: 1M ops/s at peak (lists and heads from analytics jobs) → hundreds of metadata partitions.

5. API

```
PUT    /{bucket}/{key}            headers: If-None-Match: *   (conditional create)  → 200
{etag, version}
GET    /{bucket}/{key}            Range: bytes=...                                   →
bytes
DELETE /{bucket}/{key}
GET    /{bucket}?list-type=2&prefix=...&continuation-token=...&delimiter=/          → keys
(sorted)
POST   /{bucket}/{key}?uploads  → upload_id;  PUT ...?partNumber=N&uploadId=...;  POST ...?
uploadId (complete)
```

6. Data model

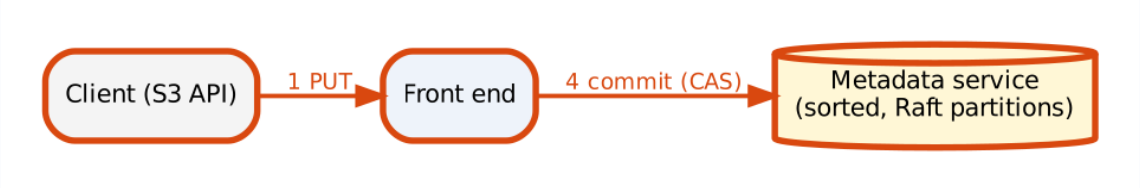
- **Metadata:** `(bucket, key, version) → {size, etag, created, storage_class, chunks: [{chunk_id, ec_group, offset, len}], state}`. Stored sorted by `(bucket, key)` so LIST by prefix is a range scan.
- **Placement:** `chunk_id → [storage nodes for each of 14 fragments]`.
- **Storage nodes:** append-only extent files on disk with per-block checksums; an index from chunk_id to (file, offset).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

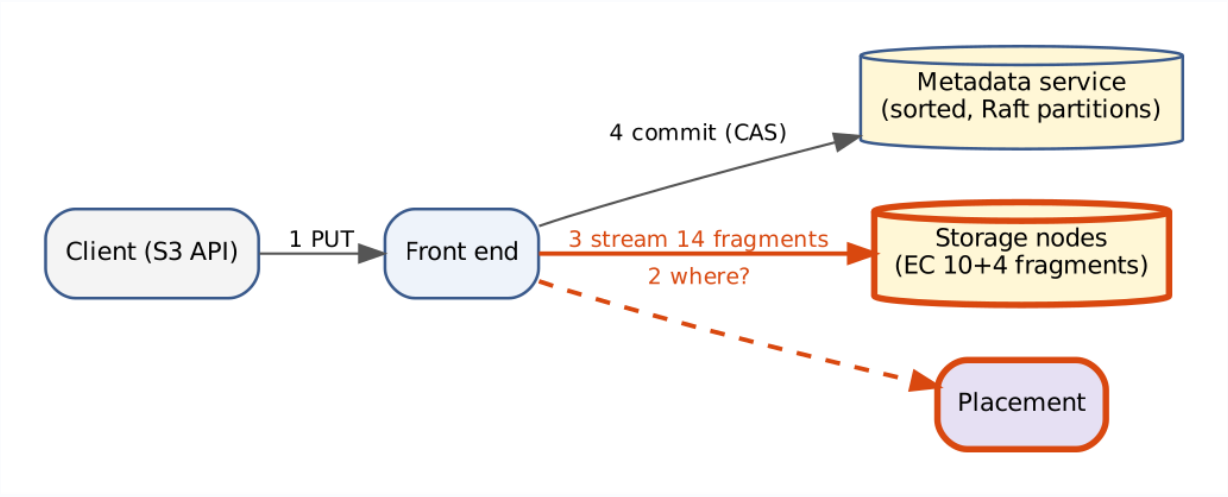
Step 1 · Metadata plane (1 min)



> **You:** "A stateless front end handles the S3-style API. Object metadata — bucket, key, version, and where the chunks are — lives in a sorted key-value store, range-partitioned, each partition a Raft group. That gives prefix listing as a range scan and conditional PUT as compare-and-set."

Why this step now: Split metadata from data immediately.

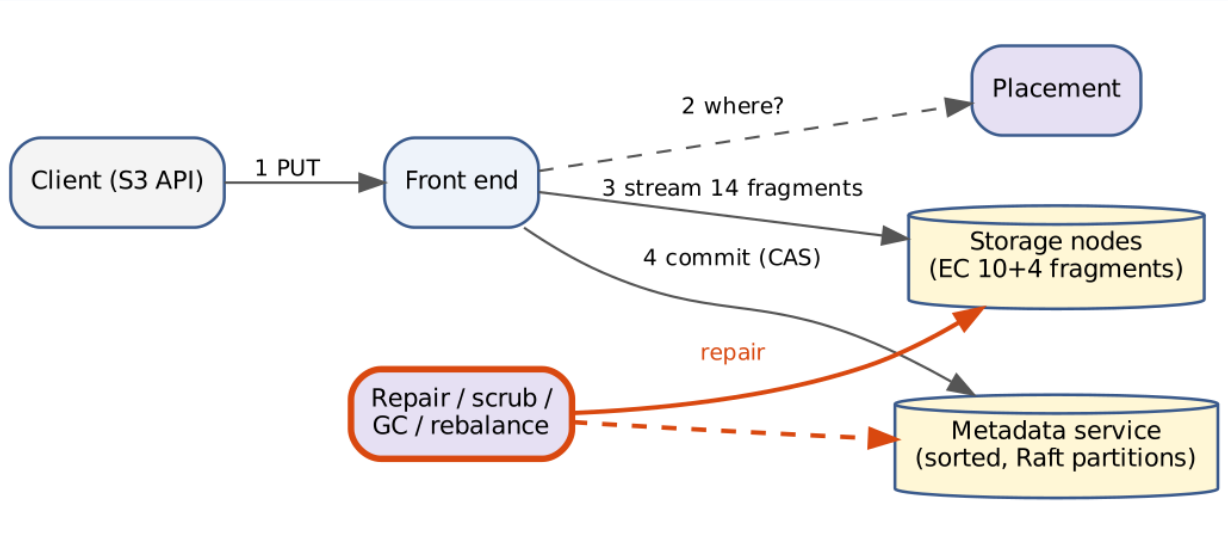
Step 2 · Data plane (1–2 min)



> **You:** "For a PUT, the front end chunks the object, erasure-codes each chunk into ten data and four parity fragments, asks placement for fourteen nodes in different failure domains, and streams fragments. Only after they're durable does it commit metadata — that commit is the moment the object becomes visible."

Why this step now: Commit point = linearization point.

Step 3 · Durability machinery (1 min)



> **You:** "Background workers re-protect fragments after node loss, scrub checksums to catch bit rot, garbage-collect orphan fragments from failed uploads, and rebalance."

Why this step now: Durability comes from repair speed.

Narrate one request end to end (numbers match the arrows)

PUT

1. Client PUT (1); front end gets placement (2).
2. Streams 14 fragments to storage nodes (3).

3. Commits metadata with CAS (4) — object visible now.

GET

1. Front end reads metadata from partition leader.
2. Reads 10 of 14 fragments (data fragments preferred), verifies checksums, returns bytes (range reads supported).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Client (S3 API)	PUT/GET/LIST	—	SDK
Front end	Auth, chunking, erasure coding, routing	Stateless; scales with traffic	Service fleet
Metadata service (sorted, Raft partitions)	bucket/key → version → chunk map	Strong consistency, listing, conditional PUT	Range-partitioned KV on Raft
Storage nodes (EC 10+4 fragments)	Store fragments in extents with checksums	Cheap durable bytes	JBOD nodes, extent files
Placement	Pick 14 nodes across failure domains	Durability needs spread	Placement service
Repair / scrub / GC / rebalance	Re-protect, detect bit rot, clean orphans	Durability is won here	Background workers

Before you leave the high-level design, check you have said:

☐ EC overhead math
☐ Commit point and read-after-write
☐ Put-if-absent for Delta
☐ Repair backlog as durability metric

60-second summary to say once the diagram is complete:

You: "I'll separate the metadata plane from the data plane, because they have opposite needs: metadata is small, must be strongly consistent and supports range listing; data is huge, write-once, and needs cheap durability."

"A PUT goes to a stateless front end. It splits the object into chunks — say 64 MB — erasure-codes each into 10 data and 4 parity fragments, and streams fragments to 14 storage nodes in different failure domains. Once enough fragments are durable, it commits a metadata record. The object becomes visible at the moment the metadata commit succeeds — that single atomic commit is what gives read-after-write consistency."

"The metadata service is a sorted key-value store, range-partitioned by bucket and key, each partition a Raft group. That makes LIST a range scan and lets conditional PUT be a compare-and-set inside one partition."

8. Deep dives

Deep dive 1: durability math and erasure coding.

- 3× replication tolerates 2 losses at 200% overhead; 10+4 Reed-Solomon tolerates any 4 losses at 40% overhead. Use EC for large/cold data, replication for small objects (EC on tiny objects is inefficient — pack small objects into larger extents first).
- Durability depends on **repair speed**: with ~7,000 drives and ~1–2% annual failure rate, you lose roughly 2 drives a week; if you re-protect fragments within hours, the chance of 5 overlapping failures in one group is vanishingly small. Spread fragments across racks/AZs.
- Scrubbing: read and verify checksums continuously to catch silent corruption (bit rot) before it combines with drive failures.

Deep dive 2: strong consistency and conditional writes (why Databricks cares).

- Read-after-write: GET reads metadata from the partition leader (or with a lease) → always sees the committed version.
- LIST consistency: listing reads the same sorted metadata; new keys appear immediately.
- Conditional PUT: `If-None-Match: *` → metadata partition does CAS: commit only if no current version. This is exactly the primitive Delta Lake's commit protocol needs (create `00011.json` only if absent).

```
def commit_put(bucket, key, new_meta, if_none_match=False, if_match_etag=None):
    part = metadata_partition_for(bucket, key)
    with part.raft_txn() as txn:                      # linearizable within partition
        cur = txn.get_latest(bucket, key)
        if if_none_match and cur is not None and not cur.deleted:
            raise PreconditionFailed
        if if_match_etag and (cur is None or cur.etag != if_match_etag):
            raise PreconditionFailed
        txn.put_version(bucket, key, new_meta)        # becomes visible on commit
    return new_meta.etag
```

Deep dive 3: hot prefixes and listing at scale.

- Keys like `logs/2026-10-07/...` concentrate writes in one key range → auto-split hot partitions by load, not only size; advise clients to spread prefixes (or hash-prefix internally).
- Listing billions of keys: paginate with continuation tokens (last key returned); delimiter-based "directory" listing computed by skipping ranges.

Deep dive 4: small files. Pack many small objects into large extent files on storage nodes (Haystack/f4-style) to avoid per-file overhead; metadata stores (extent, offset, length).

9. Scaling evolution

Stage	Design
Small	Metadata in a replicated SQL DB; 3× replication; simple placement

Stage	Design
Target	Raft-partitioned sorted metadata; EC 10+4; continuous scrubbing; auto-split partitions
10×	Storage classes (hot/cold/archive), tiering, multi-region replication, per-tenant cells

10. Failures

- Storage node dies → repair reconstructs missing fragments from 10 of the remaining 13 onto new nodes; prioritize groups with most losses.
- Metadata partition leader dies → Raft failover in seconds; requests retry.
- Partial PUT (client dies mid-upload) → uncommitted fragments are garbage-collected after a timeout; metadata never pointed to them.
- AZ loss → fragments spread so any one AZ loss leaves ≥ 10 fragments.

11. Trade-offs

Decision	Choice	Why
Replication vs EC	EC 10+4 for large, replication for small/hot	Cost vs latency/overhead
Metadata store	Sorted, range-partitioned, Raft	Strong consistency + prefix listing
Commit point	Metadata commit after data durable	Atomic visibility; no torn objects
Small objects	Packed into extents	Per-object overhead

12. Security, operations, cost

- Bucket policies, IAM, per-tenant encryption keys (envelope encryption), access logs, object lock for compliance.
- SLIs: durability (no lost objects — measured by scrub results), availability, first-byte latency, repair backlog; page on repair backlog growth.
- Cost: EC overhead, drive density, tiering cold data to cheaper media.

13. Manager lens

> **You:** "Teams: metadata service, storage nodes and durability (repair, scrub), front end and API, and a durability/SRE function. The metric I'd personally review weekly is repair backlog and time-to-re-protect, because that's where durability is actually won or lost. Any change to placement or EC parameters is a one-way door I'd sign off on."

14. Follow-ups

1. **"Why does a lakehouse need put-if-absent?"** — Delta commits create the next log file; two writers racing must not both succeed. Without it you need an external coordinator (e.g., a DynamoDB-based log store).

2. **"How do you make LIST fast for a prefix with 10 million keys?"** — Range scan with pagination; parallelize listing by splitting the key range; or avoid listing by using a table format's log (why Delta/Iceberg avoid list-heavy designs).
3. **"Range GET for Parquet footers?"** — Map byte range → chunk(s) → fragments; read only needed data fragments; cache footers.
4. **"How do you know you haven't lost data?"** — Scrubbing coverage (every byte verified every N days), audit of metadata ↔ fragment consistency, durability SLO reported.

15. Mock transcript

> **Interviewer:** Design an object store.
>
> **You:** Object size distribution, consistency needs, durability target? … I'll split metadata from data. Data: chunked, 10+4 erasure coded across failure domains. Metadata: sorted, range-partitioned Raft groups so listing is a range scan and conditional PUT is a compare-and-set. The object is visible when the metadata commit succeeds — that's the linearization point.
>
> **Interviewer:** What drives durability?
>
> **You:** Repair speed. With thousands of drives we lose a couple a week; durability comes from re-protecting within hours and scrubbing continuously, plus spreading fragments across racks and AZs.

16. Scoring guide

Rating	Signals
Strong Hire	Metadata/data split; EC math; commit point; strong consistency + conditional PUT tied to lakehouse needs; hot prefix handling; repair/scrub as durability
Hire	Chunking, replication, metadata DB, reasonable APIs
Lean No Hire	Files on a big NFS; no consistency or durability reasoning
No Hire	Can't explain where an object's bytes and metadata live

17. "Say this"

"The metadata commit is the linearization point." · "Durability is won by repair speed and scrubbing, not just replica count." · "Put-if-absent is the primitive that makes a transaction log on object storage possible."

Q20. Design a payment network like Visa/Mastercard

> Databricks (Medium) · related: Amazon Q5 (merchant side) · Source: Glassdoor Databricks EM review

1. The prompt

"Design a card payment network: merchants authorize card transactions in real time, and money settles between banks later."

2. Clarify

Ask: "Are we the network (routing between acquirers and issuers), or the issuer, or a processor?" · "Authorization latency target?" · "Peak TPS?" · "Do we need clearing and settlement, or only authorization?" · "Fraud scoring in the path?" · "Availability expectation?"

State: "We're the network. Authorization round trip under ~300 ms end to end, our hop under 50 ms; 60k TPS peak (holiday), 10× headroom target; clearing and settlement daily; real-time fraud scoring; availability effectively 99.999% — downtime means stores can't take cards."

3. Two very different flows

- **Authorization (real time):** merchant → acquirer → network → issuer → back. Decision only; money doesn't move.
- **Clearing and settlement (batch):** final amounts exchanged, fees computed, net positions between banks settled daily.

> **You:** "The most important insight is that these are two systems with opposite requirements. Authorization is low-latency, ultra-available, and must never be down. Clearing is batch, must be exactly correct, and can take hours. I'd design and staff them separately, joined by an immutable transaction log."

4. Estimates

- 60k TPS peak; ~1B transactions/day globally scale-of-magnitude (≈ 12k TPS avg).
- Txn log: 1B × 1 KB = 1 TB/day; retained for years.
- Latency budget for our hop: parse/validate 2 ms, route lookup 1 ms, fraud score 20–30 ms (parallel), network RTT to issuer dominant.

5. Messages (ISO 8583 / ISO 20022 style)

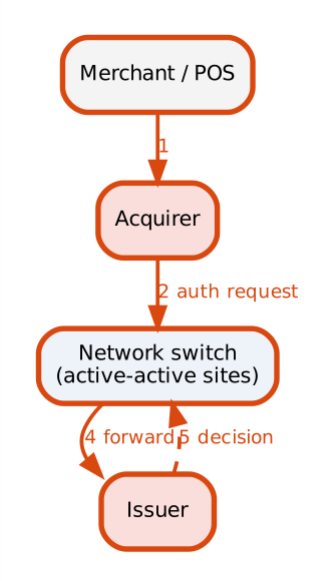
```
AuthRequest  {txn_id, pan_token, amount, currency, merchant_id, mcc, terminal, timestamp,
cryptogram}
AuthResponse {txn_id, approved: bool, auth_code, reason_code}
Reversal     {original_txn_id}
ClearingRecord {txn_id, final_amount, interchange_fee, ...}    (batch files)
```

6. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

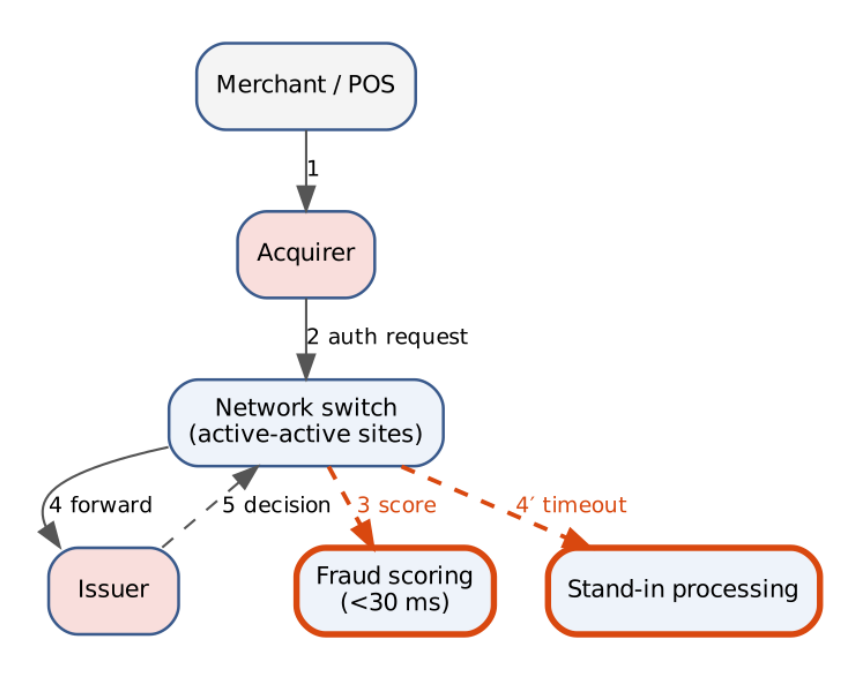
Step 1 · Authorization path (1 min)



> **You:** “A card swipe goes from the merchant to the acquirer to our switch, which validates the message, looks up the issuer by the card's BIN in an in-memory table, forwards it, and returns the decision. This path must essentially never be down.”

Why this step now: Real-time path first.

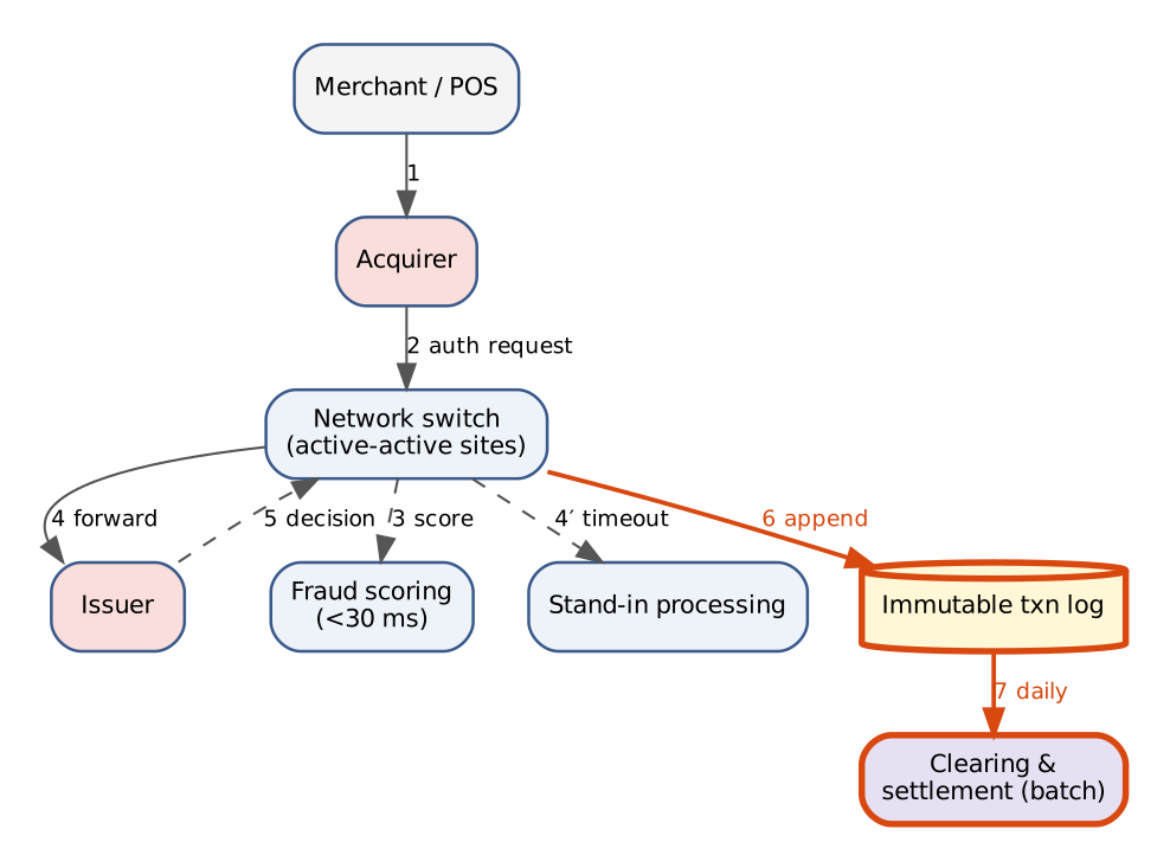
Step 2 · Fraud and stand-in (1 min)



> **You:** “Fraud scoring runs in parallel within the latency budget. If the issuer times out, stand-in processing decides using the issuer's rules so the cardholder can still pay.”

Why this step now: Availability features.

Step 3 · Log and settlement (1 min)



> **You:** “Every message is appended to an immutable replicated log. A daily batch computes net positions and fees and drives settlement — a separate system optimized for correctness.”

Why this step now: Two systems with opposite requirements.

Narrate one request end to end (numbers match the arrows)

Authorization

1. POS → acquirer (1) → switch (2).
2. Switch scores fraud in parallel (3) and forwards to issuer (4).
3. Issuer decides (5); on timeout stand-in decides (4′).
4. Message logged (6); response returned.

Settlement

1. Daily job reads the log (7), nets positions, computes fees, issues settlement instructions, reconciles.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Merchant / POS	Card present/not present	—	Terminals

Acquirer	Merchant's bank/ processor	—	Partner
Network switch (active-active sites)	Validate, route by BIN, respond	Must never be down	In-memory routing tables
Issuer	Approve/decline	Owns the cardholder	Partner
Fraud scoring (<30 ms)	Risk score in parallel	Stop fraud in the path	Model + feature store
Stand-in processing	Decide when issuer is down	Cardholders can still pay	Issuer-configured rules
Immutable txn log	Every message, replicated	Bridge to settlement + audit	Replicated log
Clearing & settlement (batch)	Net positions, fees, settlement	Correctness, not latency	Spark batch + ledger

Before you leave the high-level design, check you have said:

☐ Auth vs settlement split
☐ Active-active, no cross-site sync in auth path
☐ Stand-in processing
☐ PCI/HSM

60-second summary to say once the diagram is complete:

> **You:** "The authorization switch runs active-active in multiple data centers. It validates the message, looks up the issuer by the card's BIN range in an in-memory routing table, kicks off fraud scoring in parallel, forwards to the issuer over a persistent connection, and returns the issuer's answer. Every message is written to an immutable, replicated transaction log before the response is considered final."
>
> "If the issuer doesn't respond within its timeout, the network does **stand-in processing**: it approves or declines on the issuer's behalf using rules the issuer configured — limits, risk scores — so cardholders can still pay. Those stand-in decisions are reported to the issuer later."

7. Deep dives

Deep dive 1: availability at five nines. Multiple active sites; each authorization can be processed in any site; no cross-site synchronous dependency in the auth path; routing tables replicated and versioned; persistent connections to issuers from multiple sites. Deploys region by region with automated rollback; chaos testing.

Deep dive 2: idempotency and reversals. `txn_id` uniqueness; duplicate requests return the original response; a timeout on the merchant side triggers a reversal message so a "lost" approval doesn't hold funds.

Deep dive 3: clearing and settlement (correctness). End of day: aggregate cleared transactions per acquirer/issuer pair, compute interchange fees, produce net positions, send settlement instructions. Double-entry ledger, deterministic batch jobs, reconciliation files exchanged with banks, exceptions handling (chargebacks, disputes). This is a natural Spark/ lakehouse job — say it at Databricks.

Deep dive 4: fraud scoring in the path. Pre-computed features (cardholder spend profile, merchant risk) in a low-latency feature store; model inference < 20 ms; if scoring times out, decide by rules rather than block. Feedback loop from confirmed fraud and chargebacks.

8. Trade-offs

Decision	Choice	Why
Sync consistency across sites in auth path	No — independent sites, async log replication with per-site uniqueness	Latency and availability
Issuer unavailable	Stand-in processing	Cardholder can still pay
Settlement	Batch, deterministic, double-entry	Correctness over latency
Fraud timeout	Rules fallback	Never block on the model

9. Security

PCI DSS: tokenized PANs, HSMs for keys and cryptogram verification, network segmentation, strict access, audit. Say: "The card number should never exist in clear text outside an HSM boundary."

10. Manager lens

> **You:** "Authorization, clearing/settlement, fraud, and platform reliability are four teams with different on-call and change-management rules. Auth gets the strictest change process — staged, region by region, with instant rollback. I'd own the error budget policy: if auth burns its budget, feature work pauses."

11. Follow-ups

1. **"Network partition between your two sites?"** — Each continues independently; txn IDs include site prefix so no collisions; logs reconcile after healing.
2. **"How do you test changes without risking real payments?"** — Replay recorded traffic in a staging switch, shadow mode, canary by issuer.
3. **"Chargebacks?"** — Dispute workflow with state machine, evidence, deadlines; ledger adjustments in the next settlement cycle.

12. "Say this"

"Authorization and settlement are two systems: one optimizes for never being down, the other for never being wrong." · "Stand-in processing keeps the cardholder paying when the issuer is down." · "Clearing is a deterministic batch job — exactly the kind of workload a lakehouse is good at."

Q21. Design a streaming data pipeline with exactly-once results (lakehouse)

> Databricks (Medium) · Google: "logs and metrics pipeline processed in under a minute" (Medium) · Amazon logging analytics · Sources: Databricks SWE guides (2026); Aced Google SD guide (2026)

1. The prompt

"Design a system that ingests events from many sources and makes aggregated results available within a minute, with no double counting." Google's variant: several systems emit different log and metric types; collect, process in under a minute, serve to downstream consumers — and expect "how would you build the processing layer yourself?"

2. Clarify

Ask: "Event volume and size?" · "End-to-end freshness target?" · "Exactly-once results or approximate OK?" · "Late or out-of-order events — how late?" · "Downstream consumers: dashboards, alerts, ML features, ad hoc SQL?" · "Schema changes over time?"

State: "2M events/s at peak, ~500 B each (1 GB/s); results within 60 s; exact counts for billing-grade metrics; events up to 10 minutes late are common, up to 24 h rare; consumers are dashboards, alerting and analysts; schemas evolve."

3. Requirements

- **Must:** durable ingest; dedupe; windowed aggregations by event time; late data handling; exactly-once outputs; replay/backfill; queryable history.
- **Non-functional:** p95 freshness < 60 s; no data loss; cost-efficient storage; schema evolution without breaking consumers.

4. Estimates

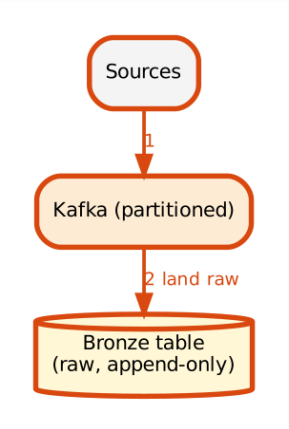
- 1 GB/s ≈ 86 TB/day raw; Parquet + compression ~5× → ~17 TB/day in bronze.
- Kafka: ~100–150 partitions at ~10 MB/s each.
- State: per-minute aggregates over 1M keys × 10 windows open ≈ 10M state entries — fits in a distributed state store (RocksDB-backed).

5. High-level design (medallion architecture)

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

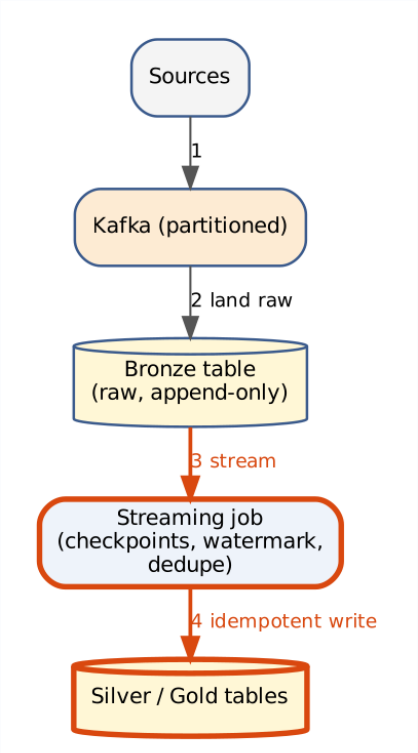
Step 1 · Durable raw landing (1 min)



> **You:** “Sources write to Kafka partitioned by entity. A job lands everything raw into a bronze Delta table — append-only — which is our replayable system of record.”

Why this step now: Replayable source first; it's the foundation of exactly-once.

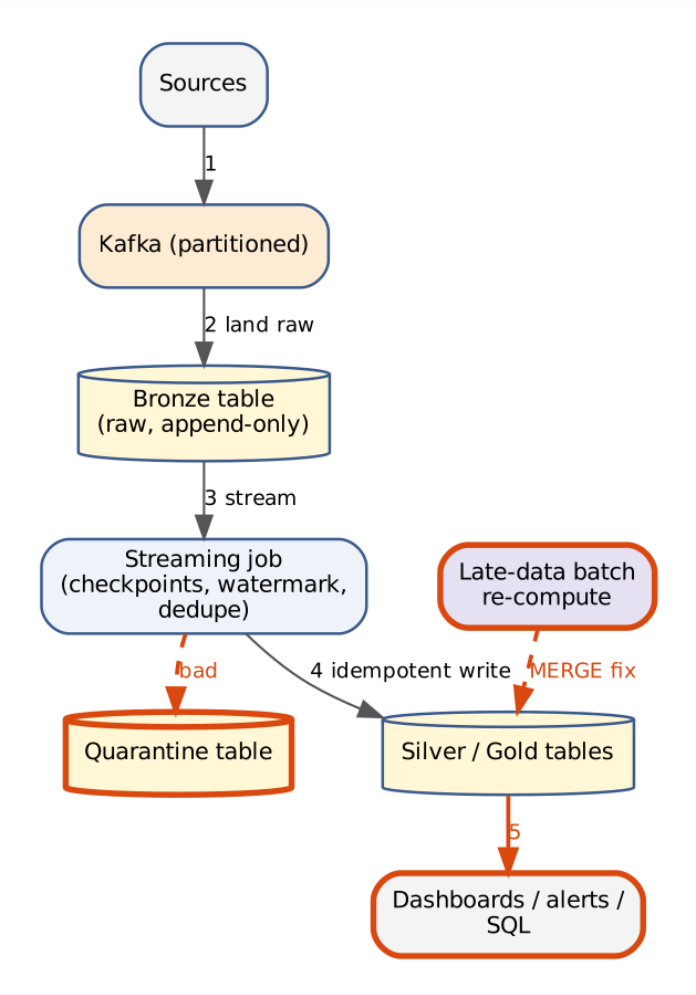
Step 2 · Processing to results (1–2 min)



> **You:** “A streaming job reads bronze, dedupes by event ID, aggregates per minute by event time with a ten-minute watermark, and writes to silver and gold tables. Offsets and state are checkpointed together; writes are transactional per batch.”

Why this step now: Exactly-once mechanics.

Step 3 · Edges: bad data, late data, consumers (1 min)



> **You:** “Unparseable records go to a quarantine table. Events later than the watermark land in silver and a periodic batch re-computes affected windows with MERGE. Dashboards and alerts read gold.”

Why this step now: Realistic edge cases.

Narrate one request end to end (numbers match the arrows)

An event

1. Produced to Kafka (1); landed raw in bronze (2).
2. Streamed (3): deduped, windowed, aggregated; written idempotently (4).
3. Gold read by dashboards/alerts (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Sources	Emit events with event_id, event_time	—	Apps/services

Kafka (partitioned)	Durable, replayable ingest	Replayable source	Kafka
Bronze table (raw, append-only)	Raw system of record	Replay button	Delta
Streaming job (checkpoints, watermark, dedupe)	Parse, validate, dedupe, aggregate by event time	Exactly-once processing	Structured Streaming / Flink
Silver / Gold tables	Clean data, per-minute aggregates	Serve consumers	Delta (MERGE, clustering)
Quarantine table	Bad records with reason	Never silently drop	Delta
Late-data batch re-compute	Fix windows for very late events	Correctness without huge state	Scheduled job
Dashboards / alerts / SQL	Consume results	—	BI, alerting

Before you leave the high-level design, check you have said:

☐ Replayable source + atomic checkpoint + idempotent sink
☐ Event time + watermark
☐ Data contracts

60-second summary to say once the diagram is complete:

> **You:** "Sources write to Kafka partitioned by a key that matters for ordering — for example, source and entity ID. A streaming job lands everything raw into a **bronze** Delta table — append-only, as the replayable system of record. A second streaming job reads bronze, parses and validates (bad records go to a quarantine table), dedupes by event_id, and writes clean **silver** data. Aggregation jobs compute per-minute metrics by event time into **gold** tables, which dashboards and alerts read."
>
> "Exactly-once comes from three pieces working together: Kafka offsets and operator state are checkpointed atomically, sinks are idempotent or transactional, and dedupe by event_id covers duplicates produced upstream."

6. Deep dives

Deep dive 1: exactly-once, precisely.

- Source: replayable (Kafka offsets).
- Processing: state + offsets checkpointed together (Structured Streaming / Flink checkpoints).
- Sink: Delta writes are transactional per micro-batch, and the batch ID is recorded so a retried batch is skipped (idempotent). For external sinks without transactions, use upserts keyed by (window, key).
- Upstream duplicates (producer retries): dedupe by event_id within a watermark-bounded window.

- *Say:* "Exactly-once is three guarantees stacked: replayable source, atomic checkpoint, idempotent sink. Drop any one and you're back to at-least-once."

Deep dive 2: event time, watermarks and late data.

- Window by event time, not arrival time. Watermark = max event time seen − 10 minutes: windows older than the watermark are finalized and their state dropped.
- Events later than the watermark (up to 24 h): written to silver anyway; a periodic batch job recomputes affected gold windows with MERGE — correctness restored without holding streaming state for 24 h.
- Trade-off: a longer watermark = more correct real-time numbers, more state, later final results.

Deep dive 3: "build the processing layer yourself" (Google). Partitioned consumers each own a set of Kafka partitions; each keeps an in-memory hash map `(window, key) → aggregate`, periodically flushed; on flush, write results + committed offsets in one transaction to the sink (or write results keyed idempotently, then commit offsets). On crash, restart from last committed offsets and rebuild state by replaying — or from a state snapshot plus replay. Rebalancing when consumers join/leave reassigns partitions (with state snapshots moving along).

```
# Simplified per-partition worker loop (at-least-once input + idempotent output =
effectively once)
state = load_snapshot(partition)          # {(window_start, key): count}, plus last_offset
consumer.seek(partition, state.last_offset + 1)
for batch in consumer.poll_batches(max_ms=1000):
    for ev in batch:
        if seen_recently(ev.event_id): continue          # dedupe within watermark window
        w = ev.event_time // 60 * 60
        state.counts[(w, ev.key)] += 1
        state.watermark = max(state.watermark, ev.event_time - LATENESS)
    closed = [k for k in state.counts if k[0] + 60 <= state.watermark]
    sink.upsert_many({k: state.counts.pop(k) for k in closed})   # idempotent by (window,
key)
    state.last_offset = batch.last_offset
    save_snapshot(partition, state)       # after sink write; replay is safe because
upserts are idempotent
```

Deep dive 4: small files and table maintenance. Micro-batches every few seconds create many small files → auto-compaction/OPTIMIZE, Z-order or clustering by common filters, VACUUM old versions per retention policy.

7. Failures

- Streaming job crash → restart from checkpoint; no loss, no double counting.
- Poison records → quarantine table with error reason; alert if rate spikes.
- Kafka retention shorter than an outage → bronze table becomes the replay source; set Kafka retention ≥ expected recovery time.
- Schema change upstream → schema evolution (additive) allowed automatically in bronze; breaking changes rejected to quarantine with alerts.

8. Trade-offs

Decision	Choice	Why
Lambda vs kappa	Kappa-ish: one streaming pipeline + batch correction for very late data	One codebase; correctness for stragglers
Micro-batch vs continuous	Micro-batch (seconds)	Simpler exactly-once; 60 s freshness is achievable
Watermark	10 minutes	Balances state size and correctness
Storage	Delta (bronze/silver/gold)	ACID, time travel, replay, SQL for analysts

9. Ops, cost, manager lens

- SLIs: freshness (event time → available in gold), records quarantined, consumer lag, checkpoint duration.
- Cost: autoscale streaming clusters on lag; compact files; tier old bronze to cheaper storage.
- Manager lens: "I'd treat data contracts like API contracts: producers own schemas, versioned, with CI checks. Most pipeline incidents are upstream schema surprises, so the contract is the reliability mechanism."

10. "Say this"

"Exactly-once is three guarantees stacked: replayable source, atomic checkpoint, idempotent sink." · "Bronze is my replay button." · "Most pipeline outages are schema surprises, so data contracts are my reliability mechanism."

Q22. Design a multi-tenant job scheduler / compute control plane

> Databricks (Low / Likely — core to the platform) · related: Amazon, Microsoft (Azure batch) · Practice for the pattern

1. The prompt

"Design the system that lets thousands of customers submit data jobs that run on clusters in their own cloud accounts."

2. Clarify

Ask: "Customer-account data plane or vendor-hosted (serverless)?" · "Job types — batch, streaming, interactive?" · "Scale: tenants, jobs/day, clusters?" · "Isolation and fairness requirements?" · "Start-up latency expectations?"

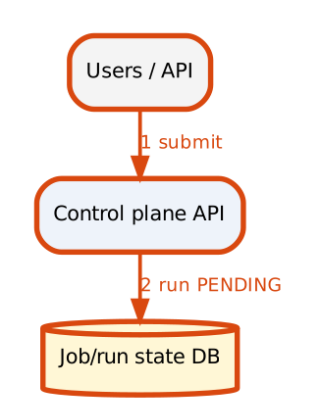
State: "10k tenants, 5M job runs/day, clusters launched in customer cloud accounts plus a serverless pool; batch and streaming; tenant isolation is non-negotiable; interactive start under 30 s using warm pools."

3. Design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

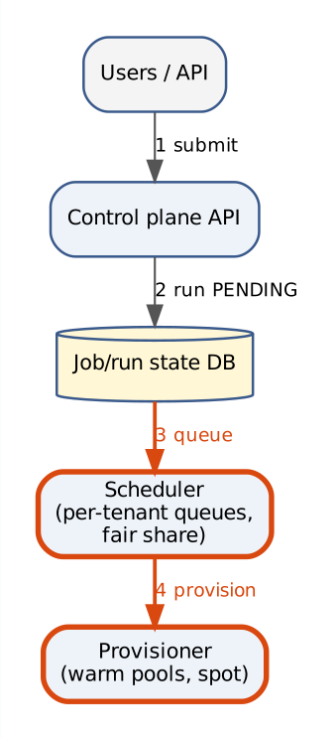
Step 1 · Control plane basics (30 s)



> **You:** “Users submit jobs to the control plane API, which records a run in a state store.”

Why this step now: Control vs data plane boundary first.

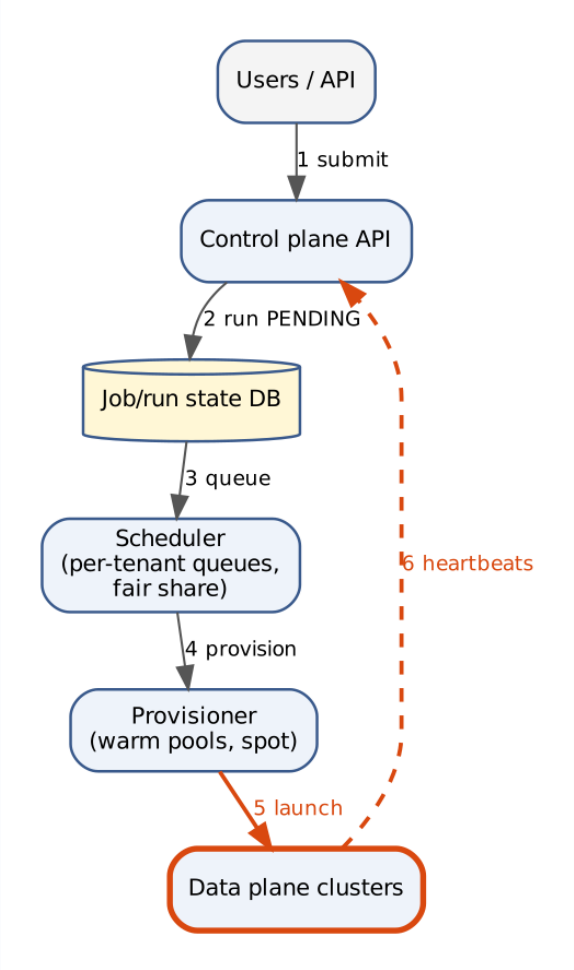
Step 2 · Scheduling and provisioning (1–2 min)



> **You:** “A scheduler with per-tenant queues and fair-share quotas decides when each run starts; a provisioner creates clusters in the customer's cloud account using warm pools and spot capacity.”

Why this step now: Fairness and cost.

Step 3 · Execution (30 s)



> **You:** “Clusters run the work and heartbeat status back; missed heartbeats mark runs lost and trigger retry policy.”

Why this step now: Closing the loop.

Narrate one request end to end (numbers match the arrows)

A job run

1. Submit (1); run created PENDING with unique (job, scheduled_time) (2).
2. Queued per tenant (3); provisioned (4); cluster launched (5).
3. Heartbeats update state (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Users / API	Define and trigger jobs	—	UI/API
Control plane API	Jobs, runs, clusters	Never touches customer data	Service

Job/run state DB	Run state machine	Exactly-once starts	Postgres / Spanner-like
Scheduler (per-tenant queues, fair share)	Decide when/where runs start	Fairness and quotas	Sharded by tenant with leases
Provisioner (warm pools, spot)	Create clusters in customer accounts	Start latency vs cost	Cloud APIs
Data plane clusters	Run customer code	Customer account	VMs + agent

Before you leave the high-level design, check you have said:

☐ Control plane never touches customer data
☐ Unique run key for exactly-once starts
☐ Jittered cron

60-second summary to say once the diagram is complete:

> **You:** "I'll separate the control plane — which we run, multi-tenant — from the data plane, where customer code and data live. The control plane stores job definitions and runs, decides when and where each run starts, provisions clusters, and tracks status. The data plane runs the workload and reports heartbeats. The control plane never touches customer data."

Key components:

- **Job state store:** run state machine `PENDING → QUEUED → PROVISIONING → RUNNING → SUCCEEDED|FAILED|CANCELLED`, with idempotent transitions and leases.
- **Scheduler:** per-tenant queues; quotas (max concurrent runs/cores); weighted fair share across tenants; priorities; cron triggers stored with next-fire-time index; at-least-once firing with dedupe by `(job_id, scheduled_time)`.
- **Provisioner:** calls cloud APIs with per-tenant credentials (scoped, short-lived); warm pools of pre-started VMs to cut start latency; spot instances with fallback to on-demand; handles cloud API rate limits and capacity errors with backoff and alternate instance types/zones.
- **Executor agent** in each cluster: pulls work, heartbeats, reports status; if heartbeats stop, the control plane marks the run lost and retries per policy.

4. Deep dives

Deep dive 1: exactly-once job starts. Scheduler instances are sharded by tenant (consistent hashing + leases). Creating a run is an insert with a unique key `(job_id, scheduled_time)` → duplicates from scheduler failover are rejected.

Deep dive 2: fairness and noisy neighbors. Dominant Resource Fairness or weighted fair queuing across tenants; hard quotas; per-tenant API rate limits; cell-based control plane (each cell serves a subset of tenants) so one tenant's storm can't affect all.

Deep dive 3: upgrades across thousands of workspaces. Control-plane releases rolled cell by cell; data-plane runtime versions pinned per job with gradual default upgrades.

5. Estimates

- 5M runs/day ≈ 58 starts/s avg, bursts at the top of the hour (cron) → 10–50× spikes → jitter cron fires and queue them.
- Heartbeats: 200k active clusters × 1 per 10 s = 20k/s.

6. Trade-offs and manager lens

Decision	Choice	Why
Control/data plane separation	Strict	Security, customer trust
Start latency vs cost	Warm pools sized by forecast	Interactive UX vs idle VM cost
Cron storms	Jitter + queue	Top-of-hour spikes

> **You:** "Cost is shared with customers here: idle warm pools cost us, slow starts cost them. I'd forecast warm pool size per region from historical demand and review cost per job start monthly. The isolation model is the decision I'd own — it's a one-way door for security."

7. "Say this"

"The control plane never touches customer data." · "Cron makes the top of the hour a thundering herd — jitter it." · "Cells limit the blast radius of any one tenant."

Chapter 6 — Google (secondary)

Google at a glance

Loop (EM)	Recruiter → technical screen → ~5 onsite rounds: People, Projects, System Design, Coding or Code Review, Googleyness/Leadership
How design is scored	You set scope from a one-line prompt; interviewers push past managed services and ask you to explain or build the component; trade-offs and scaling judgment; collaboration
Work surface	Plain Google Doc or basic whiteboard — practice drawing in plain text and simple shapes
Pitfalls	Naming products as black boxes; reproducing memorized architectures; promising to precompute everything without a memory answer

#	Question	Confidence	Source
Q23	Connection-degree (1st/2nd/3rd) system for a professional network	Medium (detailed recent report)	Aced Google SD guide (2026)
Q24	Count clicks/views on YouTube Shorts	Medium	Glassdoor Google EM review
Q25	Monitor ML model performance in real time	Medium	Glassdoor Google EM reviews
—	Logs & metrics pipeline in under a minute → **Q21**; video distribution → **Q27**; upgrade 5,000 servers → **Q3 patterns** (waves, health gates); URL shortener	Medium	Aced Google SD guide (2026)

Q23. Show 1st/2nd/3rd-degree connections instantly (LinkedIn-style)

> Google (Medium — detailed recent report; interviewer stayed on precomputation, caching, and asked the candidate to write the graph traversal instead of using a graph database) · Source: Aced Google SD guide (2026)

1. The prompt

"Given millions of users, when you open a profile the system should show instantly whether that person is a 1st-, 2nd- or 3rd-degree connection."

2. Clarify

Ask: "How many users and average connections? Max connections?" · "'Instantly' — what p99?" · "How fresh must degrees be after someone connects?" · "Do we need the path ('connected through Alice'), or just the degree?" · "Read volume — profile views per second?"

State: "900M users; average 400 connections, capped at 30k; p99 under 100 ms; new connections reflected within minutes for 2nd degree; degree only, path optional; 50k profile views/s."

Don't: "I'll use Neo4j." The reported interviewer explicitly set the graph database aside.

3. Estimates — why naive approaches fail

- 1st-degree list: 400 IDs × 8 B = 3.2 KB per user; all users ≈ 900M × 3.2 KB ≈ 2.9 TB → sharded store, cacheable.
- 2nd-degree set: ~400 × 400 = 160k (with overlap, ~100k) per user → 800 KB per user as raw IDs → **720 TB** if precomputed for everyone. Too big to store naively → compress or precompute only for active users.
- 3rd degree: ~400^3 = 64M — never materialize; answer by meeting in the middle.

4. API

```
GET /v1/degree?viewer=A&target=B   → {degree: 1|2|3|null, via?: [user_ids]}
```

5. Algorithm: bidirectional search with early exits

> **You:** "The trick is meeting in the middle. Degree 1: is B in A's friend list — a set lookup. Degree 2: do A's friends and B's friends intersect — a set intersection of two ~400-element sets, microseconds. Degree 3: does any friend of A have a friend in B's friend set — I expand the

> *smaller* side's friends one hop and check against the other side. That avoids ever building a 64-million-node frontier."

```
def degree(a, b, friends):          # friends(u) -> set of user ids, from a sharded cache
    if a == b: return 0
    fa, fb = friends(a), friends(b)
    if b in fa: return 1
    if fa & fb: return 2
    # Degree 3: expand the smaller side by one hop, test against the other side's set
    small, other = (fa, fb) if len(fa) <= len(fb) else (fb, fa)
    for u in small:                  # ~400 fetches; batch them in parallel (multi-get)
        if friends(u) & other:
            return 3
    return None                      # beyond 3rd degree
```

Cost: degree 3 worst case = ~400 friend-list fetches (batched multi-get across shards, ~a few ms each in parallel) + intersections — feasible at 100 ms p99 for most users. Hot path optimizations follow.

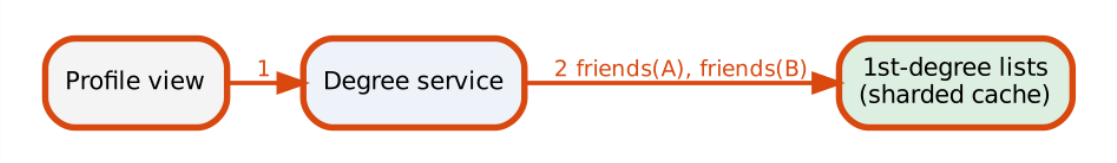
6. Precomputation and caching (the deep dive the interviewer stayed on)

- **Cache 1st-degree lists** in a distributed cache (2.9 TB fits across a cluster) — every query needs them.
- **Precompute 2nd-degree as compact sketches for active users:** a Bloom filter or roaring bitmap of 2nd-degree IDs per user (~100k IDs → ~120 KB as Bloom at 1% false positives, less as a compressed bitmap). Degree-3 check becomes: for each friend u of B, test `u ∈ second_degree(A)` — 400 in-memory membership tests, no fan-out. Verify positives against real lists to eliminate false positives.
- **Only for active users** (say 300M monthly): 300M × ~120 KB ≈ 36 TB — store in a sharded KV store, cache the hot subset.
- **Freshness:** connection events stream → recompute affected users' sketches asynchronously (a new edge A–C changes 2nd-degree sets of A, C and their friends — batch these updates).
- **Super-connectors** (30k connections): cap expansion, precompute for them specifically, or treat as special (their friend lists make everyone 2nd degree).

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

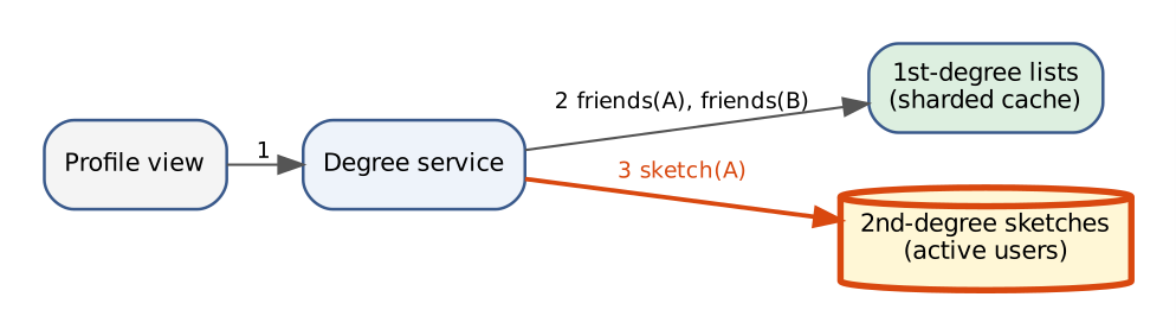
Step 1 · Degrees 1 and 2 on demand (1 min)



> **You:** “The degree service fetches both users' friend lists from a sharded cache. Degree 1 is a membership check, degree 2 a set intersection — microseconds.”

Why this step now: Show the cheap cases first.

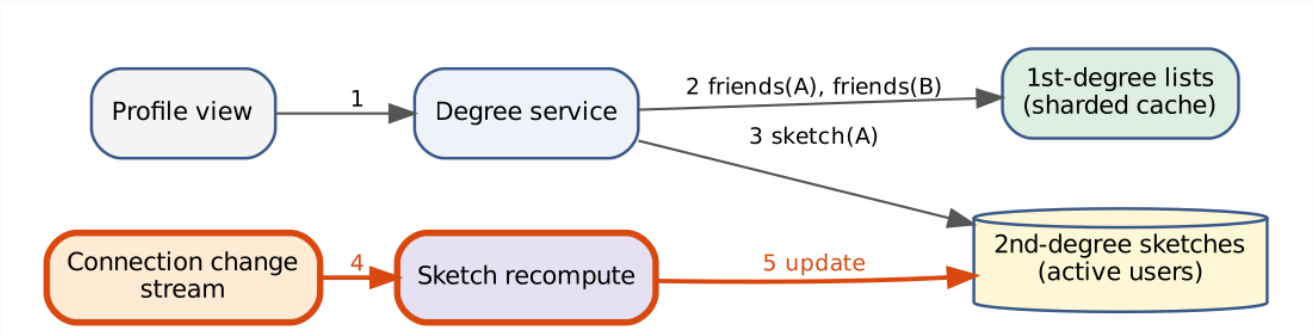
Step 2 · Degree 3 with precomputation (1–2 min)



> **You:** “For degree 3, I meet in the middle: test B's friends against a precomputed second-degree sketch of A — a Bloom filter or bitmap, stored only for active users — and verify positives.”

Why this step now: Address memory math explicitly.

Step 3 · Freshness (30 s)



> **You:** “Connection changes stream to workers that recompute affected sketches within minutes.”

Why this step now: Keep precompute correct.

Narrate one request end to end (numbers match the arrows)

degree(A,B)

1. Request (1); friend lists fetched (2) → degree 1/2.
2. Else check B's friends against A's sketch (3) → degree 3 (verify).
3. Edges change (4) → sketches refreshed (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech

Profile view	Ask degree(A,B)	—	App
Degree service	Compute 1st/2nd/3rd	—	Service
1st-degree lists (sharded cache)	friends(u)	Every check needs them	Sharded KV + cache
2nd-degree sketches (active users)	Bloom/bitmap of 2nd-degree IDs	Fast degree-3 checks, bounded memory	KV store
Connection change stream	New/removed edges	Keep sketches fresh	Kafka
Sketch recompute	Rebuild affected sketches	Freshness within minutes	Workers

Before you leave the high-level design, check you have said:

☐ Memory math (720 TB naive vs 36 TB sketches)
☐ Meet in the middle
☐ Super-connectors

60-second summary to say once the diagram is complete:

7. Answering "what about memory?"

The report specifically notes candidates get pushed on claiming they'd precompute everything. Answer with numbers: "Precomputing raw 2nd-degree sets for all 900M users is ~720 TB — no. Compressed sketches for 300M active users is ~36 TB — fine in a sharded store. 3rd degree is computed on demand by meeting in the middle, never stored."

8. Trade-offs

Decision	Choice	Why
Graph DB vs custom	Sharded adjacency lists + custom traversal	Predictable latency at this scale; we control the algorithm
Precompute 2nd vs compute	Precompute sketches for active users	Bounded memory, fast degree-3 checks
Exact vs approximate	Bloom + verify	Speed with correctness
Freshness	Async recompute within minutes	Connections are rare relative to views

9. Follow-ups

1. **"Show the path through mutual connections."** — Degree 2: return a few IDs from the intersection, ranked by relationship strength. Degree 3: record which friend u matched.
2. **"How do you shard friend lists?"** — By user_id hash; multi-get batches requests per shard.

3. **"What if both users are super-connectors?"** — Expand the smaller side; cap work and return "3rd+" if the budget is exceeded.

10. "Say this"

"Meet in the middle — never build a 64-million-node frontier." · "I'd precompute compact sketches for active users only, and here's the memory math." · "Expand the smaller side."

Q24. Count views/clicks on YouTube Shorts

> Google (Medium) · Meta (like counts), Amazon (ad clicks) · Source: Glassdoor Google EM review

1. Clarify and state

Ask: "What's the count used for — display to users, creator analytics, or monetization?" (accuracy differs) · "Volume?" · "Freshness?" · "Fraud/bot filtering?"

State: "50B views/day; display counts can lag a few seconds and be approximate; creator analytics accurate within minutes; monetization exact and fraud-filtered within hours."

> **You:** "The single most important clarification is that there are really three counts with different accuracy needs. I'll design one pipeline that produces all three, rather than three systems."

2. Estimates

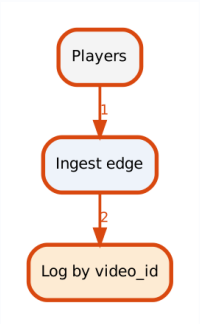
- 50B/day ≈ **580k events/s** average, ~1.5M/s peak.
- Naive: 1.5M increments/s on hot rows → a viral Short gets maybe 100k increments/s on one key → impossible on a single row.

3. Design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

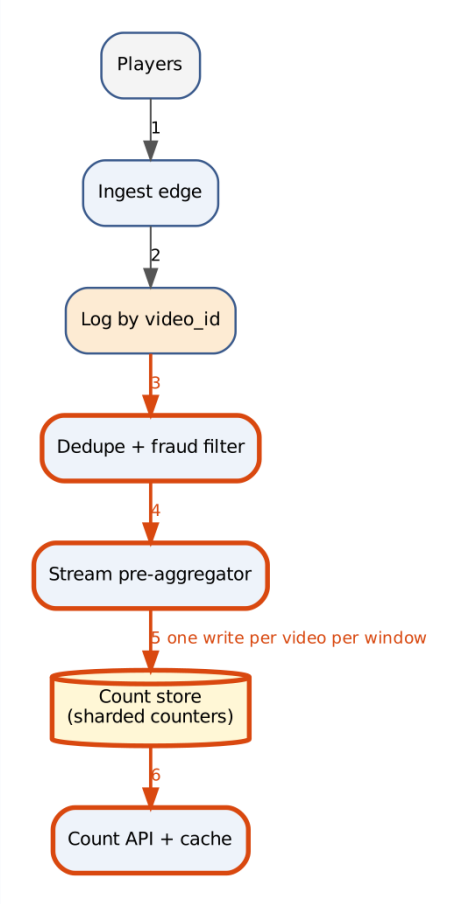
Step 1 · Durable ingest (30 s)



> **You:** “Players send view events with a view ID to an ingest edge that batches into a log partitioned by video.”

Why this step now: Never lose raw events.

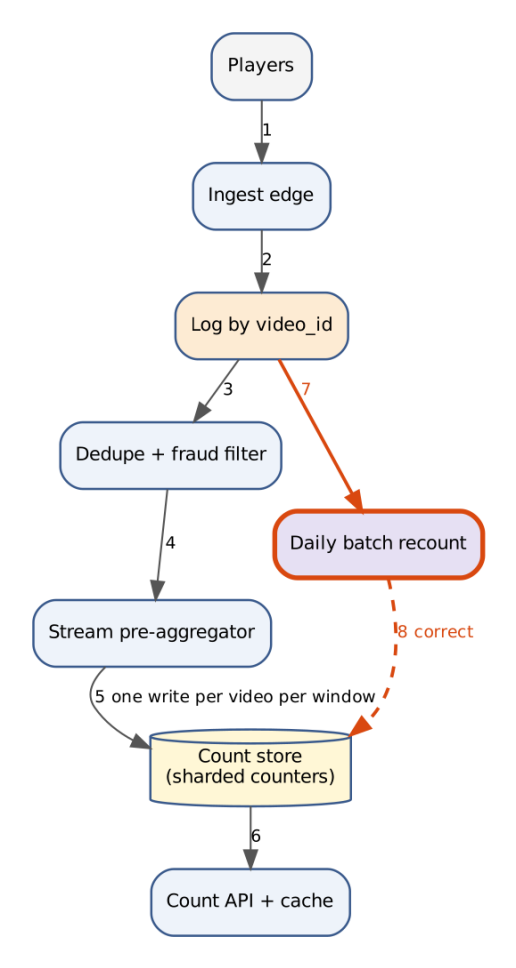
Step 2 · Real-time counts (1–2 min)



> **You:** "A stream job dedupes by view ID and filters bots, then pre-aggregates per video per few seconds and writes one increment per video per window into sharded counters. Display reads come from a cached count API."

Why this step now: This is where the hot-key problem is solved.

Step 3 · Exact counts (30 s)



> **You:** “A daily batch recomputes exact, fraud-filtered counts from the raw log and corrects totals; monetization reads only these.”

Why this step now: Three accuracy needs, one pipeline.

Narrate one request end to end (numbers match the arrows)

A view

1. Event sent (1) and logged (2).
2. Deduped (3), aggregated (4), written per window (5), served (6).
3. Batch recount from the log (7) corrects totals (8).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Players	View events with view_id	—	Client
Ingest edge	Validate, batch	—	Service
Log by video_id	Durable events	Replay + batch truth	Kafka

Dedupe + fraud filter	Drop repeats and bots	Counts must mean something	Stream job
Stream pre-aggregator	Sum per video per few seconds	Kills the hot-key problem	Flink / Dataflow
Count store (sharded counters)	Running totals	Serve display counts	Bigtable-like
Count API + cache	Display reads	—	Service + cache
Daily batch recount	Exact, fraud-filtered counts	Monetization truth	Batch job

Before you leave the high-level design, check you have said:

☐ Three counts, three accuracies
☐ Pre-aggregation math
☐ HyperLogLog for uniques

60-second summary to say once the diagram is complete:

> **You:** "Clients send view events with a view ID to an ingest edge, which batches into a log partitioned by video ID. A dedupe and fraud filter drops repeated view IDs and obvious bots. A stream aggregator sums views per video per few seconds in memory and writes *one* increment per video per flush instead of one per view — that's what kills the hot-key problem. Counts land in a sharded counter store and a cache serves display reads."
>
> "A daily batch job recomputes exact counts from the raw log, with full fraud signals, and corrects the stored totals. Monetization reads only the batch-verified numbers."

4. Deep dives

- **Pre-aggregation:** 1.5M events/s → with 5 s windows, if ~2M distinct videos get views in any 5-second window, writes drop to ~2M/5 s ≈ 400k updates/s across shards (and fewer with longer windows), and a viral video becomes 1 write per 5 s per aggregator instead of 100k/s.
- **Sharded counters for super-hot keys:** split the video's counter into N sub-counters; reads sum them (cache the sum).
- **Dedupe:** view_id generated client-side; dedupe within a time window using a probabilistic set (Bloom) per partition, exact in batch.
- **Approximate unique viewers:** HyperLogLog per video (12 KB each, ~1% error), mergeable across windows.
- **Lambda style by design:** stream for speed, batch for truth — say why it's justified here (money).

5. "Say this"

"Three counts, three accuracy needs, one pipeline." · "Pre-aggregate in the stream so a viral video is one write every few seconds, not a hundred thousand." · "Monetization reads only batch-verified counts."

Q25. Monitor ML model performance in real time

> Google (Medium) · relevant for Databricks (ML platform), Amazon, Meta · Source: Glassdoor Google EM reviews

1. Clarify

Ask: "What kind of model and decision — e.g., fraud, ranking, classification?" · "When do we get ground-truth labels — immediately, hours, weeks?" · "How many models and predictions/s?" · "What actions on detection — alert humans, auto-rollback?" · "Segments that matter (region, device, customer tier)?"

State: "Hundreds of models; the main one makes 100k predictions/s; labels arrive with delays from minutes (clicks) to weeks (fraud chargebacks); alert within 5 minutes of input/prediction drift; auto-rollback for severe regressions after a new model deploy."

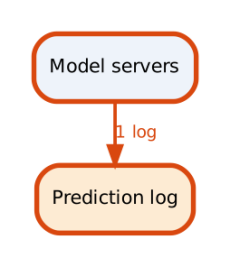
> **You:** "The core difficulty is that we usually can't measure accuracy in real time because labels are late. So I monitor three layers: system health right away, input and prediction distributions within minutes as proxies, and true accuracy when labels arrive."

2. Design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

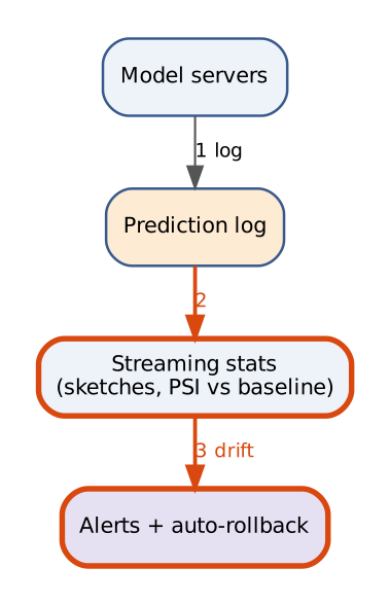
Step 1 · Log everything needed (30 s)



> **You:** “Model servers log features, predictions and a request ID to a prediction log.”

Why this step now: No monitoring without the right logs.

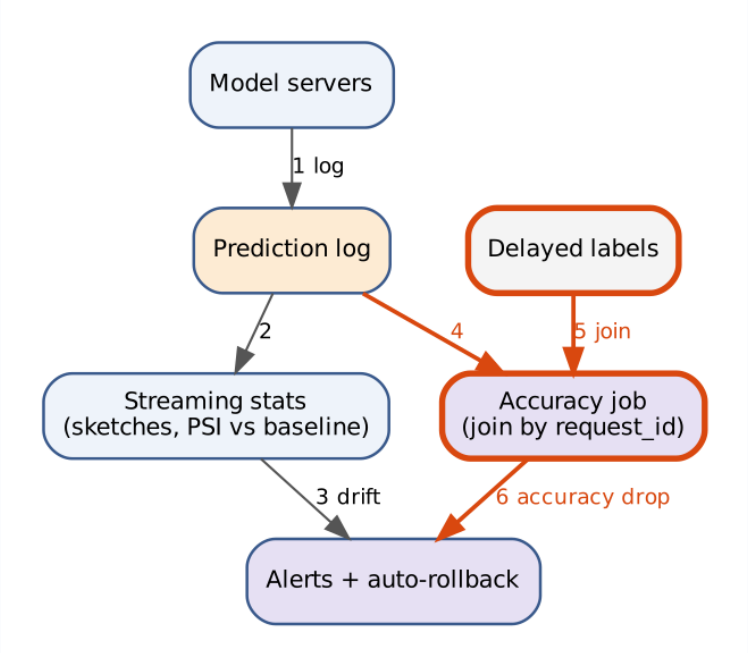
Step 2 · Real-time proxies (1–2 min)



> **You:** "A streaming job computes per-window distributions for key features and the prediction score, compares them with a baseline using PSI, per segment, and alerts — and can auto-roll back a canary model."

Why this step now: Answers 'real time' honestly.

Step 3 · True accuracy (1 min)



> **You:** "When labels arrive, a job joins them to predictions by request ID and computes accuracy per segment and model version."

Why this step now: Closes the loop with truth.

Narrate one request end to end (numbers match the arrows)

Monitoring loop

1. Predictions logged (1); drift computed (2) and alerted (3).
2. Labels joined later (4, 5); accuracy alerts (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Model servers	Predict; log features + predictions + request_id	Data source for monitoring	Serving stack
Prediction log	Stream of predictions	—	Kafka
Streaming stats (sketches, PSI vs baseline)	Drift per feature/segment per window	Labels are late; drift is the proxy	Stream job
Delayed labels	Outcomes arriving later	Ground truth	Event sources
Accuracy job (join by request_id)	Real performance per segment	Truth when available	Batch/stream join
Alerts + auto-rollback	Page owners; roll back canaries	Act on regressions	Alerting + deploy system

Before you leave the high-level design, check you have said:

☐ Three layers: health, drift, accuracy
☐ Segment-level
☐ Registered monitoring config per model

60-second summary to say once the diagram is complete:

Layer 1 — serving health (seconds): latency, error rate, timeouts, fallback rate per model version.

Layer 2 — distribution drift (minutes): stream processor computes per-window statistics of each important feature and of the prediction score: histograms or quantile sketches (t-digest), null rates, out-of-range values; compares to a training or trailing-week baseline with PSI (population stability index) or KL/JS divergence; per segment.

```
import math
def psi(expected_frac, actual_frac, eps=1e-6):
    # expected/actual: bucket fractions over the same bins (each sums to 1)
    return sum((a - e) * math.log((a + eps) / (e + eps))
               for e, a in zip(expected_frac, actual_frac))
# Rule of thumb: PSI < 0.1 stable, 0.1–0.25 moderate shift, > 0.25 significant shift
```

Layer 3 — true performance (hours–weeks): log `request_id → prediction`; join labels by request_id as they arrive; compute accuracy/precision/recall/calibration per segment per day; compare model versions.

Deployment safety: shadow mode (new model scores without acting), canary traffic split, automatic rollback when layer 1 or 2 regressions cross thresholds during the canary.

3. Estimates

- 100k predictions/s × ~2 KB feature log = 200 MB/s → sample features (e.g., 10%) for drift stats, log 100% of predictions with IDs for label joins.

4. Trade-offs and manager lens

Decision	Choice	Why
Wait for labels vs proxies	Proxies (drift) now, labels later	Labels are delayed
Alert thresholds	Per-model, tuned, segment-aware	Avoid alert fatigue
Auto-rollback	Only during canary, on clear signals	Humans decide on slow drift

> **You:** "The ownership question matters: the platform team owns the monitoring system; each model team owns its thresholds and on-call. I'd require every model to register a monitoring config before it can serve production traffic — that's the mechanism."

5. "Say this"

"Labels are late, so I monitor in three layers: health now, drift in minutes, accuracy when truth arrives." · "No model serves production without a registered monitoring config."

Chapter 7 — Netflix (secondary)

Netflix at a glance

Style	Bespoke, hard questions; deep on scale, resilience and operations; product-clone prompts ("Design Spotify") are unlikely
Behavioral weight	Nearly as heavy as design; anchored on the Culture Memo
Strong Hire signals	Failure thinking, graceful degradation, operational maturity, judgment under ambiguity

#	Question	Confidence	Source
Q26	Distributed rate limiter at millions of requests per second	Low–Medium (guides)	codemia Netflix guide (2026)
Q27	Global video distribution	Medium	Prepfully Netflix EM; interviewing.io; Aced Google list
—	Resilience to data-center failure; config management; recommendations; CDN caching	Low	designgurus / codemia guides

Q26. Distributed rate limiter at millions of requests per second

> Netflix (Low–Medium) · common everywhere (Amazon, Stripe-style APIs) · Source: codemia Netflix guide (2026)

1. Clarify

Ask: "What are we limiting — per user, per API key, per service, global?" · "Total RPS and number of keys?" · "How exact must limits be — is 5% over acceptable?" · "Latency budget for the check?" · "Fail open or fail closed if the limiter is down?" · "Multi-region — global limits or per region?"

State: "10M RPS across ~5,000 gateway nodes in 4 regions; limits per API key (100M keys) and per service; ~5% overshoot acceptable; < 1 ms added latency; fail open for most traffic, fail closed for expensive endpoints; limits are global per key."

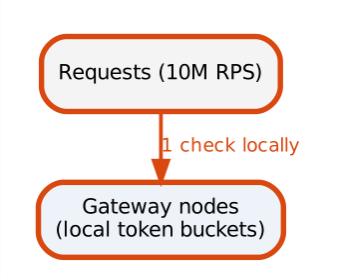
> **You:** "At ten million requests a second, a network call per request to a central counter — even Redis — adds latency and creates a hot spot. So the design is local decisions with global coordination: each node decides from local token buckets, and periodically syncs usage to a global service that hands out allocations."

2. Design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

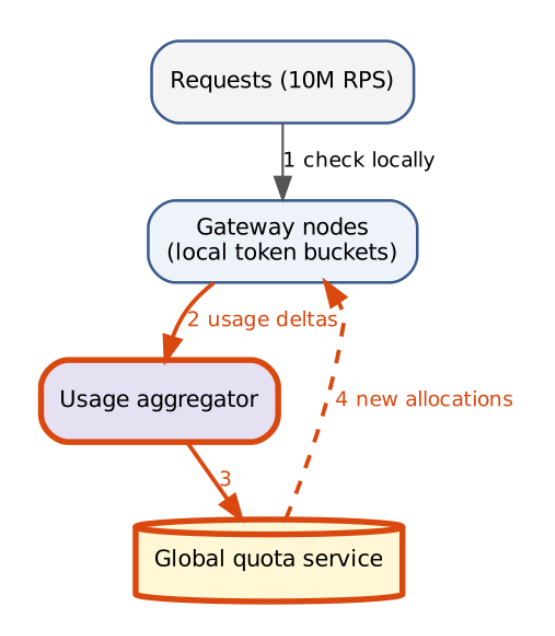
Step 1 · Local decisions (30 s)



> **You:** “Each gateway decides from local token buckets — no network call per request.”

Why this step now: At 10M RPS the check can't leave the box.

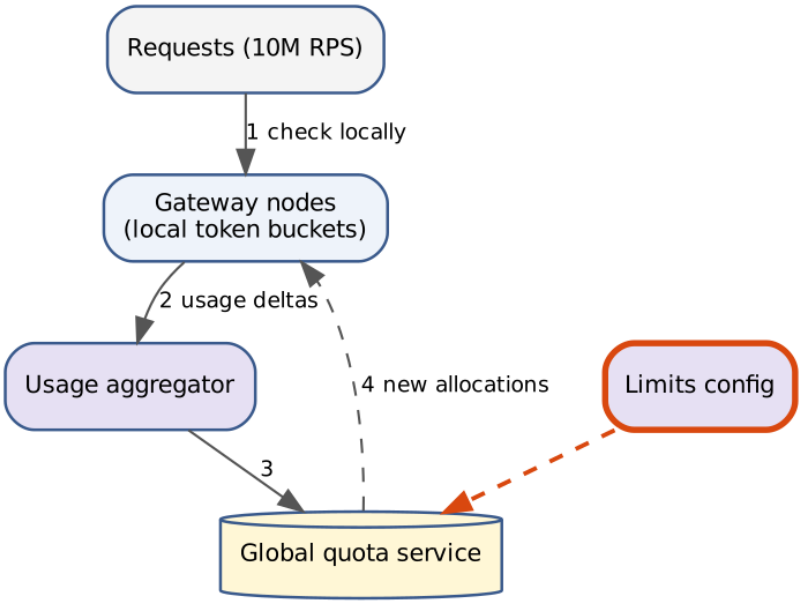
Step 2 · Global coordination (1–2 min)



> **You:** “Every few hundred milliseconds nodes report usage to an aggregator; a global quota service recomputes each key's allocation per node in proportion to demand and pushes it back.”

Why this step now: Accuracy without per-request coordination.

Step 3 · Configuration (30 s)



> **You:** “Limits come from a config service; low-volume keys are sticky-routed so one node owns them exactly.”

Why this step now: Operational completeness.

Narrate one request end to end (numbers match the arrows)

A request

1. Gateway checks local bucket (1).
2. Usage deltas reported (2, 3); allocations updated (4).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Requests (10M RPS)	—	—	—
Gateway nodes (local token buckets)	Decide locally, no network call	Latency and hot spots	In-process buckets (LRU-bounded)
Usage aggregator	Collect local usage every 100–500 ms	Global coordination	Sharded by key
Global quota service	Per-key allocations per node/region	Keep global limits approximately exact	Sharded store
Limits config	Who gets what	—	Config service

Before you leave the high-level design, check you have said:

☐ Overshoot bound stated
☐ Fail-open vs fail-closed per endpoint
☐ Sticky routing for small keys

60-second summary to say once the diagram is complete:

- **Local enforcement:** each gateway holds token buckets for keys it sees (LRU-bounded); decision in microseconds.
- **Allocation:** global limit L for a key is split across the nodes currently serving it, proportional to recent demand. Every 100–500 ms nodes report consumption; the quota service recomputes shares.
- **Low-volume keys** (most keys): sticky routing (consistent hashing of key → gateway) means one node owns the key's bucket exactly, no coordination.
- **Hot keys** spread across many nodes get demand-proportional shares.
- **Overshoot bound:** worst case ≈ (sync interval × number of nodes serving the key × local burst) — say the number and tie it to the 5% tolerance.

3. Estimates

- 10M RPS / 5,000 nodes = 2k RPS per node — local checks trivially cheap.
- Sync traffic: 5,000 nodes × (active keys per node, say 50k, batched) every 500 ms → compress to deltas; ~ hundreds of MB/s total to the aggregator tier, sharded by key.

4. Trade-offs

Decision	Choice	Why
Central counter vs local + sync	Local + sync	Latency and hot spots at 10M RPS
Exactness	~5% overshoot	Cost of exactness is a network call per request
Failure mode	Fail open by default, closed for expensive endpoints	Availability vs protection, per endpoint
Global vs regional limits	Regional allocations from a global budget	Avoid cross-region latency on the hot path

5. Failures

- Quota service down → nodes keep last allocations (decaying toward a conservative default).
- Node restarts → starts with a small conservative allocation until the next sync.
- Clock skew irrelevant locally (monotonic clocks); sync uses sequence numbers.

6. "Say this"

"At ten million RPS, the check can't leave the box." · "Local decisions, global coordination." · "I'd state the overshoot bound and agree it with the API owners."

Q27. Design global video distribution

> Netflix (Medium) · Google (video distribution, Medium) · Sources: Prepfully Netflix EM guide; interviewing.io; Aced Google SD guide (2026)

1. Clarify

Ask: "Catalog size and concurrent streams at peak?" · "Device types and network conditions?" · "Live or on-demand?" · "Startup time and rebuffer targets?" · "Do we control a CDN or use third parties?"

State: "On-demand; 20k titles; 15M concurrent streams at evening peak; average 5 Mbps; startup < 2 s; rebuffer ratio < 0.5%; we run our own CDN appliances embedded in ISPs plus third-party CDNs as backup."

2. Estimates

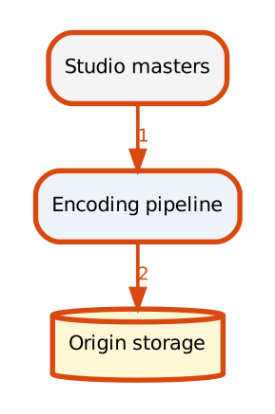
- 15M × 5 Mbps = **75 Tbps** at peak — impossible from a few data centers; must be served from the edge, inside ISP networks.
- Catalog: 20k titles × ~50 encodes (resolutions, bitrates, codecs, audio) × ~2 hours → each title ~100s of GB across ladders → a few PB total, popular subset fits in edge caches.

3. Design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

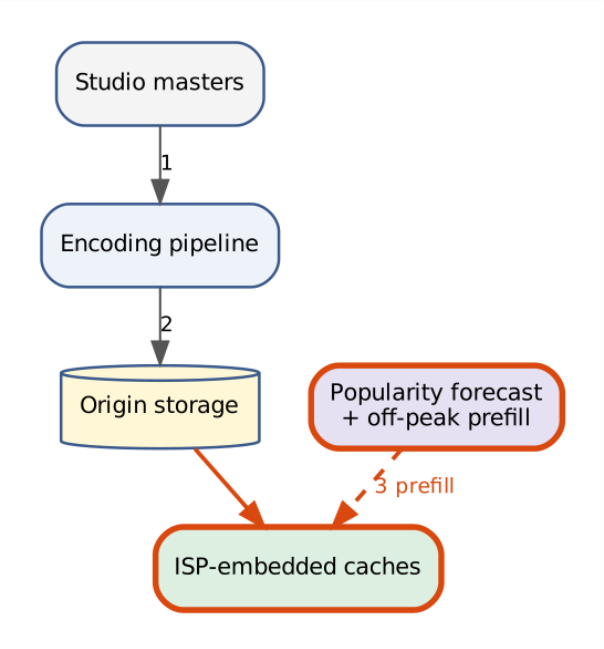
Step 1 · Prepare content (1 min)



> **You:** “Masters are chunked and encoded in parallel into a per-title bitrate ladder and stored at origin.”

Why this step now: Content first.

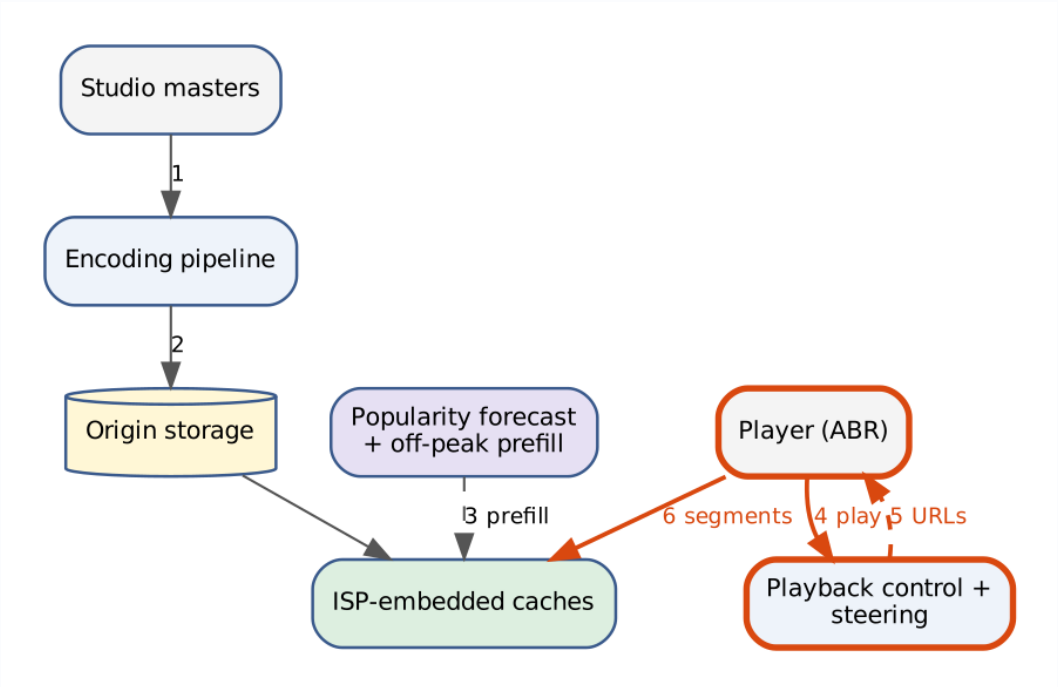
Step 2 · Place content near users (1–2 min)



> **You:** “Peak traffic is tens of terabits per second, so caches sit inside ISPs and are prefilled off-peak based on a popularity forecast.”

Why this step now: The math-driven key decision.

Step 3 · Play (1 min)



> **You:** "On play, the cloud control plane authorizes and steers the player to nearby healthy caches; the player streams segments with adaptive bitrate."

Why this step now: End-to-end experience.

Narrate one request end to end (numbers match the arrows)

Press play

1. Content encoded (1) and stored (2); prefilled to ISP caches (3).
2. Player requests play (4); gets ranked cache URLs (5).
3. Streams segments from the nearest cache (6), adapting bitrate.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Studio masters	Source content	—	—
Encoding pipeline	Chunked parallel encode, per-title ladders	Quality per bit	Batch workers
Origin storage	All encodes	—	Object storage
ISP-embedded caches	Serve segments near users	75 Tbps can't come from a few DCs	Appliances in ISPs
Popularity forecast + off-peak prefill	Fill caches overnight	Cheap bandwidth, high hit rate	Batch job
Player (ABR)	Adaptive bitrate playback	—	Client

Playback control + steering	Authorize, pick caches, manifest	Cloud control plane	Service

Before you leave the high-level design, check you have said:

☐ 75 Tbps math
☐ Prefill off-peak
☐ Control plane in cloud, bytes at edge
☐ QoE metrics

60-second summary to say once the diagram is complete:

> **You:** "Two halves: getting content ready and getting bytes to viewers. Content: studio masters are split into chunks and encoded in parallel into a bitrate ladder per title — per-title encoding tunes the ladder to the content's complexity. Outputs go to origin storage."
>
> "Delivery: because peak traffic is tens of terabits per second, we place cache appliances inside ISP networks and prefill them overnight with what we predict people will watch tomorrow. When a user hits play, the control plane — still in the cloud — authorizes, checks licensing, and a steering service returns a ranked list of nearby healthy caches holding the needed files. The player streams segments and adapts bitrate to its measured bandwidth and buffer."

4. Deep dives

- **Adaptive bitrate (ABR):** player picks each segment's bitrate from a buffer-based + throughput-based algorithm; start low for fast startup, ramp up; switch down before the buffer empties.
- **Proactive placement:** popularity forecast per region → prefill caches off-peak (cheap bandwidth); long tail served from regional/origin tiers on miss.
- **Steering:** choose caches by network proximity (BGP/ISP topology), health, load and content availability; return multiple URLs so the player can fail over mid-stream.
- **Resilience (Netflix's culture):** control plane multi-region active-active; regional evacuation drills; chaos testing; if steering fails, players fall back to third-party CDNs.

5. Metrics

Startup time, rebuffer ratio, average bitrate delivered, play failures, cache hit ratio per ISP, peak egress per cache. These are the product.

6. Trade-offs and manager lens

Decision	Choice	Why
Own CDN vs third-party	Own appliances in ISPs + third-party backup	Cost and quality at this scale

Decision	Choice	Why
Reactive vs proactive caching	Proactive prefill + reactive tail	Predictable demand; off-peak bandwidth is cheap
Encoding	Per-title ladders	Better quality per bit

> **You:** "Encoding, CDN/edge, client playback, and the streaming control plane are separate teams; the shared language is the quality-of-experience metrics. I'd tie every team's goals to rebuffer ratio and startup time, because those are what members feel."

7. "Say this"

"At 75 terabits a second, the only place to serve from is inside the ISP." · "Prefill off-peak, serve on-peak." · "The control plane is in the cloud; the bytes never are."

Part III — The classic canon (Q28–Q61)

These are the systems interviewers at every company draw from: TinyURL, WhatsApp, YouTube, Uber, Ticketmaster, Google Docs-style collaboration, ChatGPT and more. The list was cross-checked against Hello Interview's practice set so the common problems are all here.

How to use this part: start with the five pattern-teachers — **Q28 TinyURL, Q29 WhatsApp, Q30 YouTube, Q32 Uber, Q35 Ticketmaster**. Then work by pattern using the "Problems grouped by pattern" table in Part IV. Once you've done one problem in a group, the others go much faster.

The most-asked classics have every section, including mock transcripts. Less common ones are more compact but always include the staged high-level design walkthrough, deep dives, trade-offs and "say this" lines.

Part III.1 — Core consumer systems

Q28. Design TinyURL (URL shortener) — and Pastebin

> Classic · asked across Amazon, Google, Microsoft, Meta screens · Variants: Pastebin, bit.ly analytics, QR codes

1. The prompt

"Design a URL shortening service like TinyURL." Variants: custom aliases, expiration, click analytics; Pastebin (same design, the value is a text blob).

2. Clarify

Ask (and why):

- "How many new URLs per day and how many redirects?" — read:write ratio drives caching.
- "How short must the code be?" — sizes the keyspace.
- "Custom aliases? Expiration?" — uniqueness checks and cleanup.
- "Do we need click analytics?" — adds an async event pipeline.
- "Can a long URL map to multiple short URLs, or should duplicates return the same code?" — dedupe vs simplicity.
- "Any abuse concerns — phishing, malware links?" — safety checks on create.

Don't ask: "Should I use base62?" (decide it) · "Which database?"

State: "100M new URLs per day, 100:1 reads to writes, 7-character codes, optional custom aliases, default 5-year retention, basic click analytics, abuse checks on create."

3. Requirements

- **Must:** create short URL; redirect (HTTP 301/302) fast; custom alias; expiry.
- **Stretch:** analytics (clicks by day, country, referrer); user accounts; link editing.
- **Non-functional:** redirect p99 < 50 ms; 99.99% availability for redirects (broken links are very visible); codes unguessable enough to prevent enumeration if links are private.

4. Estimates

- Writes: 100M/day ≈ **1.2k/s**, peak ~3k/s.
- Reads: 100× → 10B/day ≈ **115k/s**, peak ~350k/s.
- Keyspace: base62 with 7 chars = 62^7 ≈ **3.5 trillion** codes; 100M/day × 365 × 5 = 182B over 5 years → ~5% of space used. Plenty.
- Storage: 182B × ~500 B ≈ **91 TB** over 5 years.

- Cache: 20% of daily-hot URLs: maybe 1B hot mappings × 500 B ≈ 500 GB → distributed cache.

5. API

```
POST /v1/urls      {long_url, custom_alias?, expires_at?}  Idempotency-Key  → {short_url, code}
GET  /{code}       → 301/302 Location: long_url   (404 if missing/expired)
GET  /v1/urls/{code}/stats?from=..&to=..            → {clicks_by_day, top_countries, top_referrers}
DELETE /v1/urls/{code}                              (owner only)
```

301 vs 302: 301 (permanent) lets browsers cache, so fewer requests reach us but we lose analytics for repeat clicks. 302 (temporary) keeps every click visible. **Choose 302 if analytics matter**, 301 if minimizing load matters. Say the trade-off out loud — interviewers ask.

6. Data model

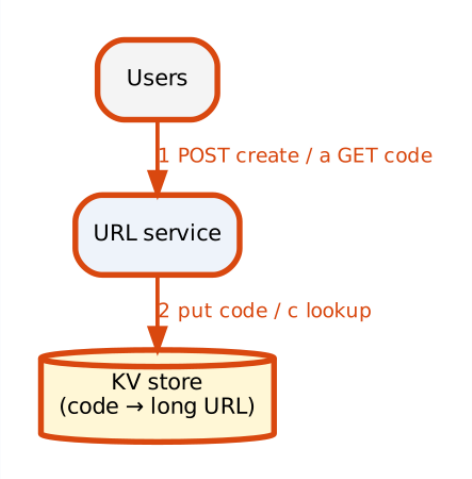
- `url(code PK, long_url, owner_id, created_at, expires_at)` in a key-value store (DynamoDB/ Cassandra) — single-key lookups, massive scale, no joins.
- Optional `long_url_hash → code` index if duplicates should return the same code.
- `click_events` → stream → aggregated `clicks(code, day, country, count)`.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

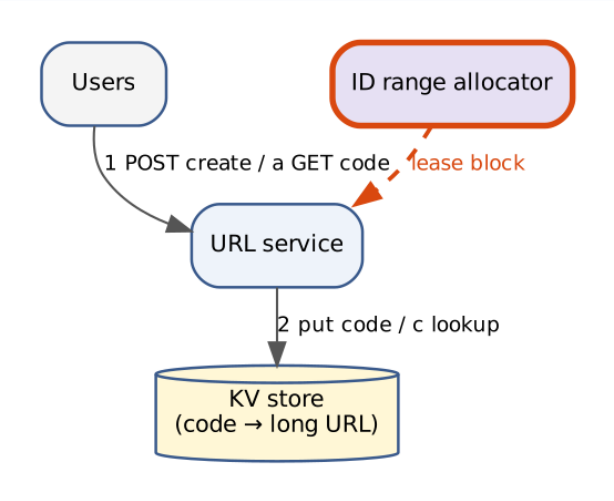
Step 1 · Simplest version (1 min)



> **You:** “A URL service creates codes and stores code-to-URL mappings in a key-value store; redirects look the code up and return a 302.”

Why this step now: Working system in one minute.

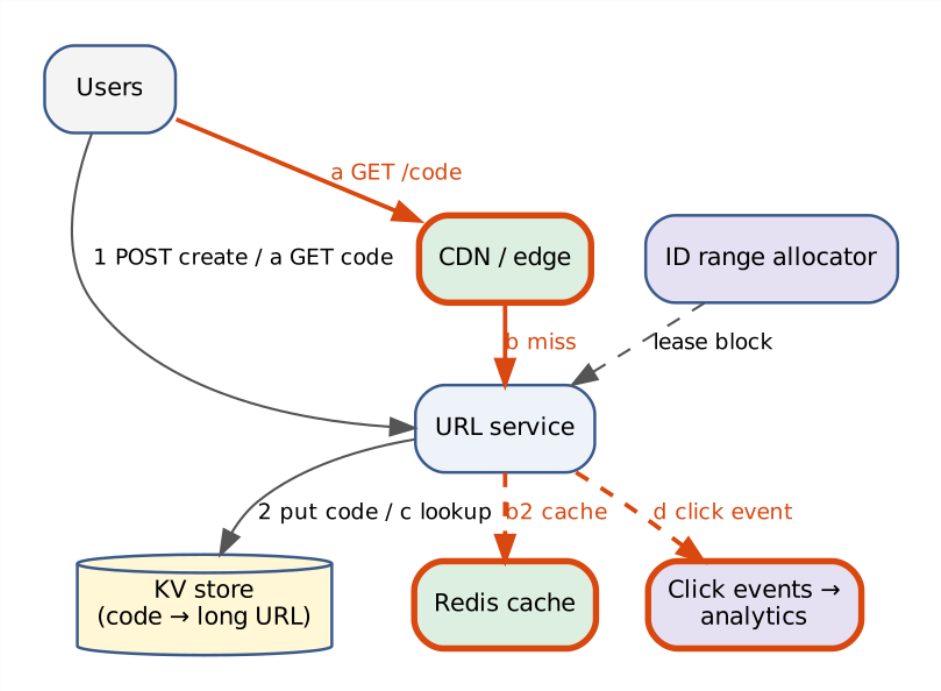
Step 2 · Collision-free codes (1 min)



> **You:** “Each server leases a block of a million IDs from a small allocator and base62-encodes them locally — no collisions, no coordination per write.”

Why this step now: Code generation is the classic follow-up; answer it structurally.

Step 3 · Fast reads and analytics (1–2 min)



> **You:** “Because reads are 100× writes, I add a CDN for hot links and Redis in front of the store; clicks are emitted asynchronously to an analytics pipeline.”

Why this step now: Optimize the dominant path last, with reasons.

Narrate one request end to end (numbers match the arrows)

Create

1. User posts long URL (1); server takes next ID from its leased block, base62-encodes, conditional put (2).

Redirect

1. Browser hits CDN (a); on miss, service (b) checks Redis, then KV (c).
2. 302 returned; click event emitted async (d).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Users	Create links, click links	—	Browser/app
URL service	Create codes, resolve redirects	Core logic	Stateless service
KV store (code → long URL)	Source of truth	Single-key lookups at scale	DynamoDB / Cassandra
ID range allocator	Lease blocks of 1M IDs to servers	Unique codes without per-write coordination	etcd / DB row
CDN / edge	Cache hot redirects	Viral links	CloudFront
Redis cache	Hot mappings	115k reads/s	Redis cluster
Click events → analytics	Async click counting	Keep redirects fast	Kafka + stream job

Before you leave the high-level design, check you have said:

☐ Keyspace math
☐ Leased ID blocks
☐ 301 vs 302
☐ Abuse checks

60-second summary to say once the diagram is complete:

> **You:** "Writes are rare and reads are 100× more, so I optimize the redirect path: CDN or edge cache first, then a distributed cache, then the key-value store. Code generation must never collide, so I'll use a counter-based ID encoded in base62 rather than hashing and retrying."

8. Deep dives

Deep dive 1: generating codes without collisions.

Approach	How	Pros	Cons
Hash + truncate	MD5/SHA of long URL, take 7 chars	Same URL → same code	Collisions → check-and-retry; slower writes
Random	7 random base62 chars, conditional insert	Unguessable	Collision retries grow as space fills (still rare at 5%)
Counter + base62	Global unique ID → base62	No collisions, simple	Sequential codes are guessable — scramble with a bijective shuffle if privacy matters

To avoid a single counter bottleneck, a **range allocator** (in ZooKeeper/etcd or a DB row) hands each app server a block of 1M IDs; servers allocate locally with no coordination. A crashed server wastes at most one block — fine with trillions available.

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
def base62(n: int) -> str:
    if n == 0: return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))

class RangeAllocator:
    """Each app server leases a block of IDs; no coordination per request."""
    def __init__(self, lease_block):           # lease_block() -> (start, end) from etcd/DB
        self.lease_block = lease_block; self.next = self.end = 0
    def next_id(self):
        if self.next >= self.end:
            self.next, self.end = self.lease_block()
        n = self.next; self.next += 1
        return n
```

Custom aliases: conditional insert (`attribute_not_exists(code)`); reject if taken. Reserve a disjoint namespace (e.g., aliases must contain a letter pattern the counter range never produces, or keep a separate check).

Deep dive 2: the redirect hot path. Edge/CDN caches popular codes (with short TTL so deletes/expiry take effect); app servers check Redis, then the KV store. Expiry checked on read; background sweeper deletes expired rows. Hot links (a viral tweet) are absorbed by the CDN.

Deep dive 3: analytics without slowing redirects. Redirect handler emits a click event asynchronously (fire-and-forget to a local buffer → Kafka); stream job aggregates by code/day/country. Never write analytics synchronously on the redirect path.

9. Scaling evolution

Stage	Design
Small	Single SQL DB with auto-increment ID → base62; one cache

Stage	Design
Target	KV store, range allocator, Redis + CDN, async analytics
10×	Multi-region read replicas of the KV store; regional ID ranges; edge redirects

10. Failures

- Cache cluster down → KV store absorbs reads (size it for that, or degrade with CDN stale-serve).
- Allocator down → servers keep using leased blocks; lease ahead before exhaustion.
- Region down → DNS fails over; KV replicated across regions (reads only need eventual consistency).

11. Trade-offs

Decision	Choice	Why
Code generation	Counter blocks + base62	No collisions, no coordination per write
Redirect type	302 if analytics, 301 otherwise	Analytics vs load
Store	KV	Single-key lookups at scale
Analytics	Async stream	Keep redirect p99 low

12. Security, operations, cost

- Abuse: check new URLs against malware/phishing lists, rate-limit creation per user/IP, allow takedowns. Preview pages for suspicious links.
- Enumeration: sequential codes reveal volume and allow scraping → scramble IDs if private links matter.
- SLIs: redirect latency and success, cache hit ratio, 404 rate.

13. Manager lens

> **You:** "It's a small system, so one team can own it. The decision I'd care about is abuse handling — a shortener is a phishing tool if we're careless, so trust and safety review is part of launch, not an afterthought."

14. Follow-ups

1. **"Same long URL twice — same code?"** — Optional: index by hash of long URL per user; trade storage for dedupe.
2. **"How do you handle 1M redirects/s for one link?"** — CDN caching at the edge; 301 for that link if analytics aren't needed.

3. **"Pastebin variant?"** — Store the text in object storage, metadata in KV; size limits; content-type; expiry; same code generation.
4. **"How do you shorten codes further?"** — 6 chars = 56B codes — enough for ~1.5 years at this rate; trade-off is length vs lifetime.

15. Mock transcript

> **Interviewer:** Design TinyURL.
>
> **You:** Volume, read ratio, code length, custom aliases, expiry, analytics? … 100M writes a day is ~1.2k/s; reads are 100×, ~115k/s. Seven base62 characters gives 3.5 trillion codes — we'd use about 5% in five years. I'll generate codes from leased ID blocks so there are no collisions and no per-request coordination.
>
> **Interviewer:** 301 or 302?
>
> **You:** 302 if we want click analytics, because 301 lets browsers cache and we'd stop seeing repeat clicks. For pure load reduction, 301.

16. Scoring

Strong Hire: keyspace math; collision-free generation with leased ranges; 301/302 trade-off; read path layering; async analytics; abuse. **No Hire:** hash with no collision plan; analytics written synchronously.

17. "Say this"

"Reads are 100× writes, so the redirect path gets all the optimization." · "Leased ID blocks give me uniqueness without coordination." · "301 saves load; 302 keeps analytics — I'd pick based on the product."

Q29. Design WhatsApp (1:1 and group chat) — also Messenger, Slack

> Classic · Meta (WhatsApp/Messenger), Microsoft (Teams chat), Amazon, Google

1. The prompt

"Design WhatsApp." Expect: 1:1 and group messaging, delivery and read receipts, offline delivery, online presence, media, multiple devices, end-to-end encryption.

2. Clarify

Ask: "DAU and messages per day?" · "Group size limit?" · "Do we store messages on the server after delivery?" (WhatsApp historically deletes after delivery) · "Multi-device support?" · "End-to-end encryption required?" · "Media types and sizes?"

State: "2B users, 1B DAU, 100B messages/day, groups up to 1,000 members, messages stored until delivered to all recipient devices then deleted from server (clients keep history), multi-device (up to 4), E2E encryption, media up to 100 MB via separate storage."

3. Requirements

- **Must:** send/receive 1:1 and group messages in real time; offline queueing; sent/delivered/read receipts; per-conversation ordering; media; presence/last seen.
- **Non-functional:** delivery p99 < 500 ms when both online; no message loss; at-least-once delivery with client-side dedupe; E2E encryption (server can't read content).

4. Estimates

- 100B messages/day ≈ **1.2M msgs/s**, peak ~3M/s.
- Concurrent connections: ~500M online at peak → at ~500k connections per gateway host → **~1,000+ hosts**.
- Storage (transient): messages held until delivery; average ~100 B encrypted; if 10% wait offline for a day → 10B × 100 B = 1 TB in the queue — small. Media dominates: say 5% messages with 200 KB average → 1 PB/day of media (stored in blob storage with TTL/client re-upload policies).

5. API (over a persistent connection)

```
connect(auth_token, device_id)                          → session on a gateway
send{client_msg_id, conv_id, ciphertext, media_ref?} → ack{server_msg_id, ts}
(sender's ✓)
deliver{server_msg_id, conv_id, from, ciphertext}    ← pushed to recipient devices
ack_delivered{server_msg_id}  → sender gets ✓✓
ack_read{conv_id, up_to_msg_id} → sender gets blue ✓✓
POST /v1/media (encrypted blob) → media_ref
```

6. Data model

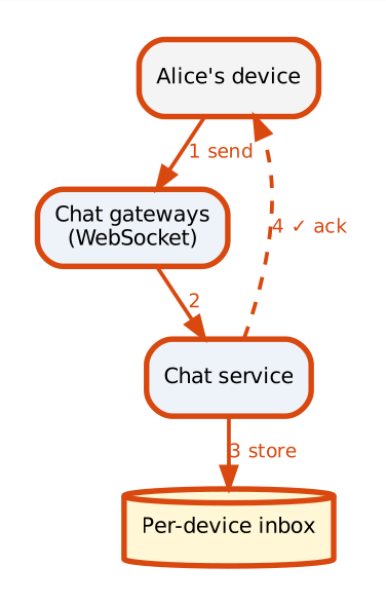
- `ConnectionRegistry: device_id → gateway_id` (in-memory, Redis) — where to route.
- `Inbox(device_id, server_msg_id, payload)` — undelivered messages per recipient device; wide-column store partitioned by device_id, ordered by message ID; deleted on delivery ack.
- `Conversation(conv_id, members[], type)`; `GroupMembers(group_id → user_ids)`.
- `Message IDs:` per-conversation monotonically increasing sequence or time-ordered IDs (ordering within a conversation).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

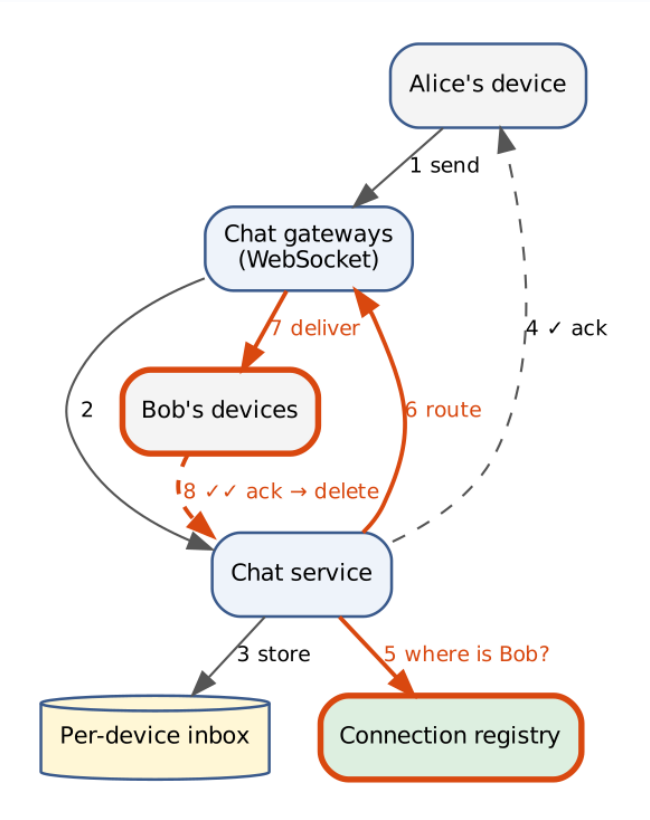
Step 1 · Send durably (1 min)



> **You:** “Alice's device holds a WebSocket to a gateway. The chat service assigns a server ID and per-conversation sequence, writes the message to each recipient device's inbox, then acks Alice — one tick.”

Why this step now: Durability before ack is the core guarantee.

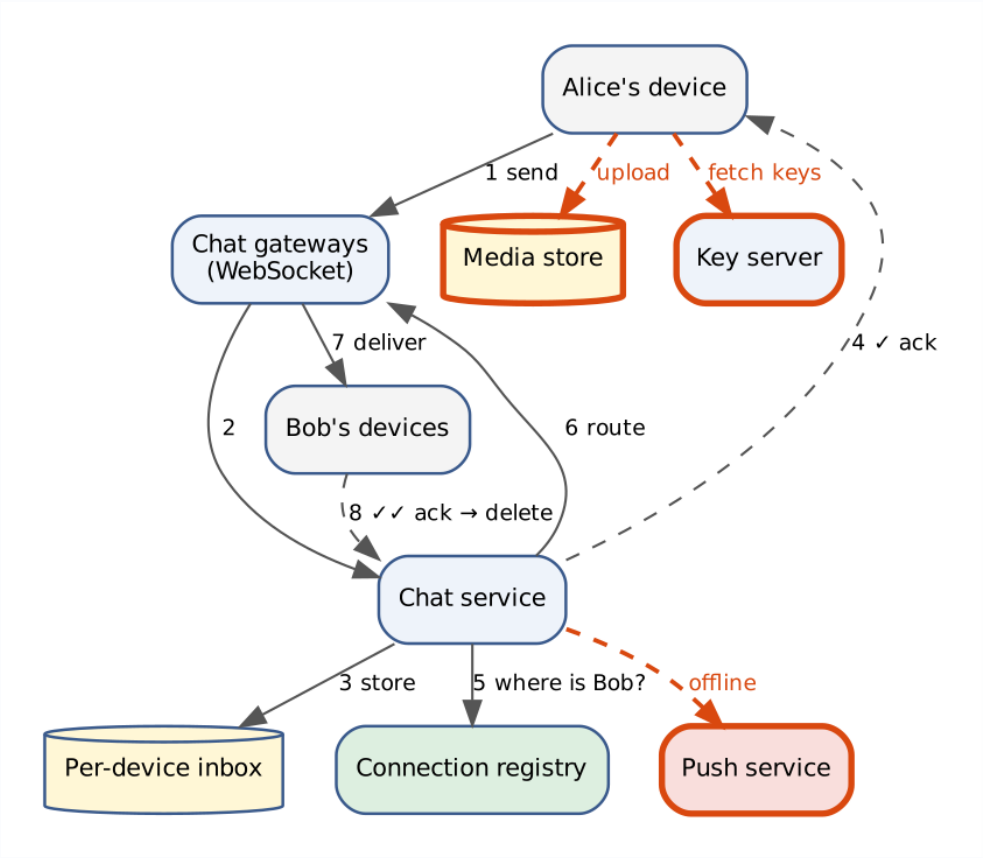
Step 2 · Deliver to online devices (1 min)



> **You:** “The service looks up Bob's gateway in the connection registry and pushes the message; Bob's ack deletes it from his inbox and Alice gets two ticks.”

Why this step now: Real-time path.

Step 3 · Offline, media, encryption (1–2 min)



> **You:** "If Bob is offline, the message waits in his inbox and we send a push notification. Media is uploaded encrypted to blob storage and referenced in the message. Clients fetch public keys from a key server to establish end-to-end encrypted sessions."

Why this step now: Completes the product.

Narrate one request end to end (numbers match the arrows)

Message delivery

1. Alice sends (1, 2); stored in Bob's device inboxes (3); Alice gets ✓ (4).
2. Registry lookup (5); routed (6) and delivered (7).
3. Bob acks (8): inbox entry deleted, Alice gets ✓✓.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Alice's device	Send encrypted messages	—	App
Chat gateways (WebSocket)	Hold connections	~500M concurrent	Gateway fleet
Chat service	Assign IDs/sequence, route	Core logic	Service

Per-device inbox	Undelivered messages	Durable before ack	Cassandra-like
Connection registry	device → gateway	Route to online recipients	Redis
Bob's devices	Receive, ack	—	App
Push service	Wake offline devices	Offline delivery	APNs/FCM
Media store	Encrypted attachments	Large files out of chat path	Blob storage
Key server	Public pre-key bundles	E2E sessions	Service

Before you leave the high-level design, check you have said:

☐ Connection math
☐ Durable before ack
☐ Per-conversation ordering
☐ E2E implications

60-second summary to say once the diagram is complete:

> **You:** "Clients keep a persistent connection to a chat gateway. When Alice sends, the gateway passes the message to a chat service that assigns a server ID, writes it to each recipient device's inbox, and acks Alice — that's one tick. If Bob's device is online, the chat service looks up his gateway in the connection registry and pushes the message; Bob's ack deletes it from his inbox and Alice gets two ticks. If he's offline, it waits in his inbox and we send a push notification."

8. Deep dives

Deep dive 1: delivery guarantees and ordering.

- **Durable before ack:** write to the recipient inbox before acking the sender → no loss.
- **At-least-once + dedupe:** retries can duplicate; clients dedupe by `client_msg_id`/ `server_msg_id`.
- **Ordering:** per conversation, not global. Route all messages of a conversation through one partition (hash conv_id) to assign sequence numbers; clients display by sequence.

Deep dive 2: group messages. For groups up to ~1,000, fan out on write: one inbox entry per member device (with E2E, per-device encryption is handled by the sender's client using sender keys — the server fans out one ciphertext per group with sender-key encryption). Large broadcast channels would use fan-out on read instead.

Deep dive 3: presence. Gateways know who is connected; heartbeats every ~30 s; presence changes published only to users who have the person's chat open or are contacts who recently interacted (subscriptions), not to all contacts — otherwise one user going online fans out to hundreds.

Deep dive 4: end-to-end encryption (Signal protocol). Each device has identity and pre-keys; senders fetch a recipient's public pre-key bundle from a key server, establish a session, and encrypt per device. The server routes ciphertext only. Multi-device = encrypt for each of the recipient's devices. Consequence for the design: no server-side search or content moderation of messages; spam detection uses metadata and user reports.

9. Scaling evolution

Stage	Design
Small	WebSocket server + one DB; poll for offline messages
Target	Gateway fleet + connection registry + per-device inbox + push
Global	Regional gateways; cross-region routing by recipient's home region; per-conversation sequencers

10. Failures

- Gateway crash → clients reconnect (jittered) to another gateway; registry updated; undelivered messages remain in inbox and are pushed on reconnect.
- Chat service crash after inbox write but before ack → sender retries; dedupe prevents duplicates.
- Push provider down → messages still wait in inbox; delivered on next app open.

11. Trade-offs

Decision	Choice	Why
Server storage	Delete after delivery	Privacy, cost; clients keep history
Group fan-out	On write (≤1k members)	Simple delivery; bounded size
Ordering	Per conversation	Global ordering is unnecessary and costly
Protocol	Persistent connection (WebSocket/MQTT-like)	Real-time, battery-efficient with heartbeats

12. Security, operations, cost

- E2E encryption; key change notifications; abuse via metadata, rate limits, reporting.
- SLIs: send→deliver latency, delivery success, connection count per gateway, reconnect rate, inbox backlog.

13. Manager lens

> **You:** "Teams: client, gateway/realtime infra, messaging core, media, and identity/keys. The decision I'd personally protect is E2E encryption — it constrains every other team (no server search, no content moderation), so it has to be an explicit product principle, not something we erode feature by feature."

14. Follow-ups

1. **"How do messages sync to a new device?"** — With server-side deletion, history comes from another device (encrypted backup or device-to-device transfer).
2. **"Read receipts in groups?"** — Per-member read pointers; aggregated "read by" lazily.
3. **"How do you send a 100 MB video?"** — Client encrypts, uploads to blob storage, sends a small message with the reference and key; recipients download and decrypt.
4. **"How do you avoid a celebrity's 'online' status flooding millions?"** — Presence subscriptions only for open chats; rate-limit presence updates.

15. Mock transcript

> **Interviewer:** Design WhatsApp.
>
> **You:** 100 billion messages a day is about 1.2 million a second, and ~500 million concurrent connections at peak — so the core is a large gateway fleet plus a connection registry. Delivery guarantee: write to the recipient's inbox before acking the sender; delete on recipient ack; dedupe on the client.
>
> **Interviewer:** Group of 500?
>
> **You:** Fan out on write — one inbox entry per member device — with sender-key encryption so the sender encrypts once per group.

16. Scoring

Strong Hire: connection registry + gateway math; durable inbox before ack; per-conversation ordering; group fan-out; presence scoped; E2E implications. **No Hire:** polling the DB per client; no offline story.

17. "Say this"

"Durable before ack, delete after delivery." · "Ordering is per conversation, never global." · "End-to-end encryption is a design constraint for every team, not a feature."

Q30. Design YouTube (upload, transcode, stream) — also TikTok, Spotify

> Classic · Google (video distribution reported), Netflix, Meta (Reels), Amazon (Prime Video)

1. The prompt

"Design YouTube: users upload videos, others watch them." Variants: live streaming, short-form feed (TikTok), music (Spotify — same delivery, smaller files, offline downloads).

2. Clarify

Ask: "Upload volume and average size?" · "Watch volume and concurrent viewers?" · "Which features: upload, playback, search, recommendations, comments, view counts?" (scope!) · "Global?" · "Live or on-demand?" · "Device mix and bandwidth?"

State: "Scope: upload, processing, playback, view counts; search and recommendations briefly. 500 hours of video uploaded per minute; 1B hours watched per day; global; on-demand only."

3. Requirements

- **Must:** resumable upload; transcode into multiple resolutions/codecs; adaptive streaming; metadata; view counts.
- **Non-functional:** playback start < 2 s; rebuffer < 1%; uploads never lost; processing within minutes; cost-efficient storage and egress.

4. Estimates

- Uploads: 500 h/min × 60 × 24 = 720k hours/day. Raw ~1 GB/h compressed source → 720 TB/day raw; transcoded ladders ~2× → ~1.5 PB/day total new storage.
- Watch: 1B hours/day at average 3 Mbps → 1B × 3600 s × 3 Mb ≈ 10.8 Eb/day ≈ **125 Tbps average** egress, peaks higher → CDN mandatory.
- Metadata: 1B videos × 1 KB = 1 TB — small.

5. API

```
POST /v1/uploads {title, size, content_type}  → {upload_id, part_urls[]}     (resumable multipart)
POST /v1/uploads/{id}:complete                → {video_id, status: processing}
GET  /v1/videos/{id}                          → metadata + manifest URL (HLS/DASH)
GET  <CDN>/videos/{id}/{rendition}/seg_{n}.m4s
POST /v1/videos/{id}/views                    (sampled view events)
```

6. Data model

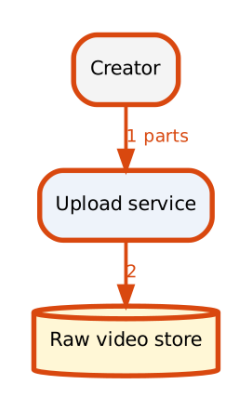
- `Video(video_id, owner, title, description, status, duration, renditions[], created_at)` — sharded SQL or wide-column.
- Object storage: `raw/{video_id}`, `encoded/{video_id}/{rendition}/segments`, manifests.
- `Channel`, `Subscription`, `Comment` stores; view counts in a counter system (see Q24).

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

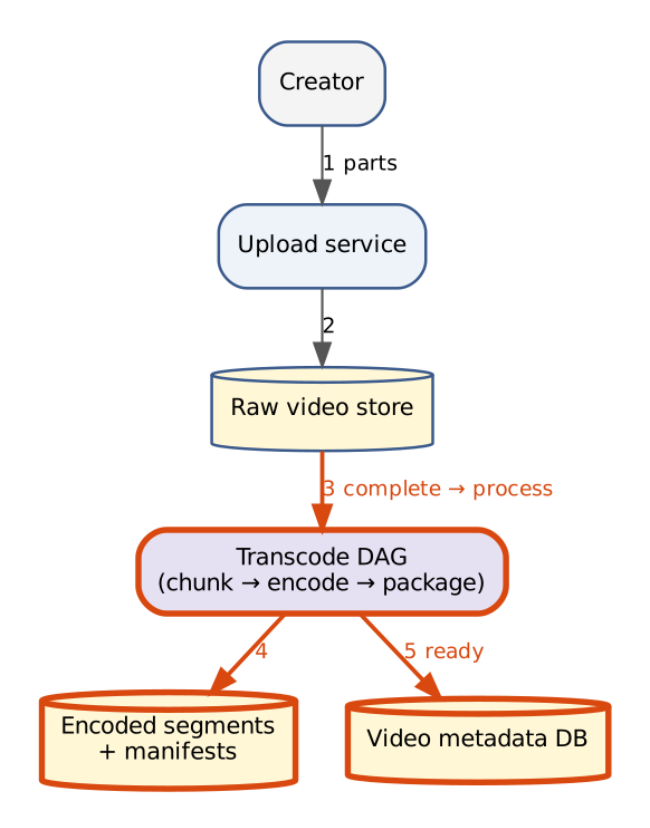
Step 1 · Upload (1 min)



> **You:** “Creators upload resumable parts directly to object storage through pre-signed URLs.”

Why this step now: Bytes bypass services.

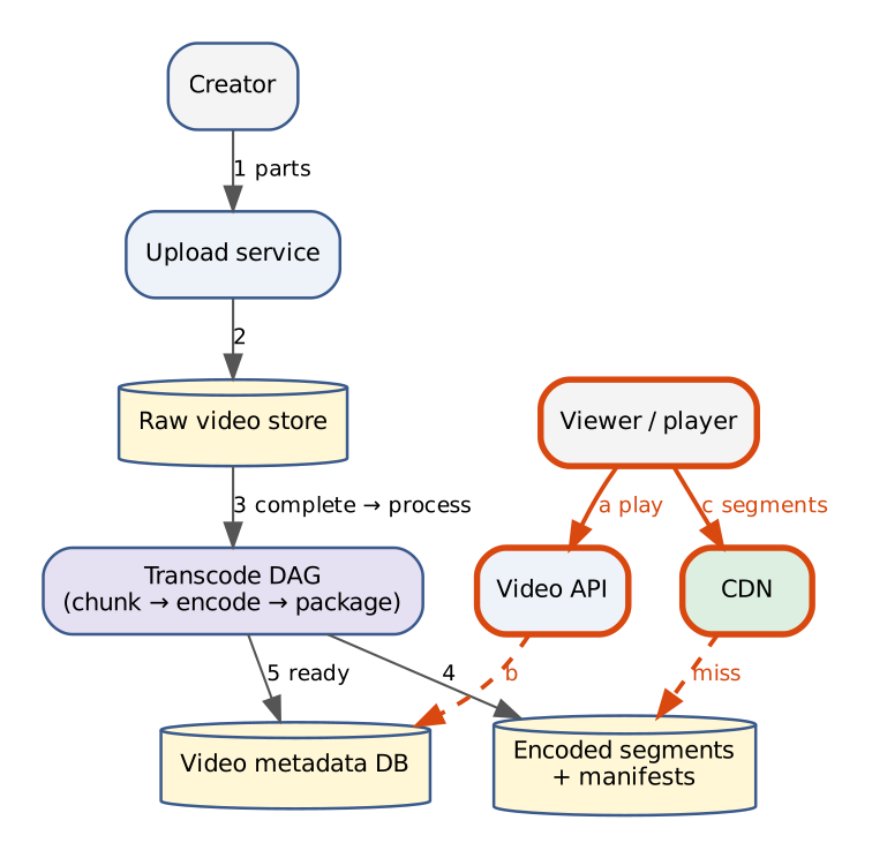
Step 2 · Process (2 min)



> **You:** “On completion, a DAG splits the video into chunks, transcodes them in parallel into a bitrate ladder, packages manifests, creates thumbnails, runs content checks, and marks the video ready.”

Why this step now: The key engineering of YouTube.

Step 3 · Watch (1–2 min)



> **You:** “Players fetch metadata and a manifest from the API, then stream segments from the CDN, adapting bitrate to bandwidth.”

Why this step now: Egress math justifies the CDN.

Narrate one request end to end (numbers match the arrows)

Upload to ready

1. Parts uploaded (1) to raw storage (2).
2. DAG processes (3), writes renditions (4), marks ready (5).

Playback

1. Player gets manifest (a, b).
2. Streams segments from CDN (c); misses go to origin.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Creator	Upload video	—	App
Upload service	Resumable multipart, presigned URLs	Large files	Service

Raw video store	Originals	Durable source	Object storage
Transcode DAG (chunk → encode → package)	Parallel transcoding, thumbnails, checks	Minutes not hours	Workflow + workers
Encoded segments + manifests	HLS/DASH renditions	Served by CDN	Object storage
Video metadata DB	Title, status, renditions	—	Sharded SQL
Viewer / player	ABR playback	—	Player
Video API	Metadata + manifest URL	—	Service
CDN	Segments at edge	125 Tbps egress	Multi-CDN

Before you leave the high-level design, check you have said:

☐ Egress math
☐ Chunked parallel transcoding
☐ ABR
☐ Storage tiering

60-second summary to say once the diagram is complete:

> **You:** "Two halves. Upload: clients upload resumable chunks straight to object storage using pre-signed URLs; on completion a processing pipeline splits the video into segments, transcodes them in parallel into a bitrate ladder, packages HLS/DASH manifests, and marks the video ready. Playback: the player fetches metadata and a manifest from our API, then pulls segments from the CDN, switching bitrate based on bandwidth."

8. Deep dives

Deep dive 1: the transcoding pipeline (DAG). Split source into ~2–10 s chunks at keyframes → parallel transcode tasks per chunk × rendition (240p…4K; H.264/VP9/AV1) → assemble/package → thumbnails, captions (speech-to-text), content checks (copyright fingerprinting, policy) → publish. Orchestrated as a DAG with retries per task; workers on spot instances; priority for popular creators. Parallel chunking turns a 1-hour video from hours to minutes.

Deep dive 2: adaptive bitrate streaming. Each rendition is split into segments; the manifest lists them. The player measures throughput and buffer and picks the next segment's bitrate. Start low for fast start, ramp up. CDN caches segments as static files.

Deep dive 3: storage and CDN cost. Popular videos (a small fraction) get most views → keep them on the CDN edge; long tail from origin/regional caches. Old, rarely watched videos → colder storage tiers; drop unpopular high renditions and re-transcode on demand if requested (cost lever).

9. Scaling evolution

Stage	Design
Small	Upload to S3; single transcode job per video; one CDN
Target	Chunked parallel DAG transcoding, ABR, CDN + regional caches
Huge	ISP-embedded caches (see Q27), per-title encoding, AV1, storage tiering

10. Failures

- Transcode worker dies → task retried (idempotent outputs keyed by chunk/rendition).
- Upload interrupted → resume missing parts.
- CDN PoP outage → player fails over to alternate URLs in manifest.

11. Trade-offs

Decision	Choice	Why
Upload path	Direct to object storage	Bytes bypass services
Transcode granularity	Chunk-level parallel	Speed, retry granularity
Renditions	Full ladder for popular, reduced for long tail	Cost
Protocol	HLS/DASH over HTTP	CDN-friendly, ABR

12. Security, operations, cost

- Content ID / copyright fingerprinting, policy classifiers, signed URLs for private videos, DRM for premium content.
- SLIs: upload success, processing time p95, start time, rebuffer ratio, play failures.
- Cost: egress and storage dominate — CDN hit ratio and codec efficiency are the biggest levers.

13. Manager lens

> **You:** "Upload/processing, playback/CDN, and the watch experience are separate teams; quality-of-experience metrics are the shared scorecard. Codec migrations (to AV1) are multi-year programs with a big cost payoff — I'd fund them explicitly."

14. Follow-ups

1. **"How do views get counted?"** — See Q24: event pipeline with dedupe and pre-aggregation; display counts approximate, monetization exact.
2. **"Live streaming?"** — Ingest via RTMP/SRT to regional ingest, real-time transcode, low-latency HLS with short segments; no long processing DAG.

3. **"How do you handle a video going viral right after upload?"** — CDN pulls from origin shield; pre-warm caches for popular creators' new uploads.
4. **"Spotify differences?"** — Small files, offline downloads with DRM, gapless playback, heavy personalization.

15–17. Mock, scoring, say this

> **You:** "125 terabits per second on average means the CDN is the product. Our services handle metadata and manifests; bytes come from the edge."

Strong Hire: egress math; resumable direct uploads; chunked parallel transcoding DAG; ABR; cost levers. **No Hire:** streaming whole files from app servers.

Say this: "Parallel chunk transcoding turns hours into minutes." · "The manifest is the contract between our backend and the player."

Q31. Design Twitter / X (post, home timeline, search, trends)

> Classic · Meta (feed), Amazon, Google, Microsoft · see Q15 for feed fan-out details

1. The prompt

"Design Twitter." Scope it: post tweets, follow, home timeline, user timeline, search, trending topics. Feed fan-out is covered deeply in Q15 — here focus on what Twitter adds: asymmetric follows, real-time search and trends.

2. Clarify and state

Ask: "Which features are in scope?" · "DAU, tweets/day, average and max followers?" · "Timeline ranked or reverse-chronological?" · "Search freshness?"

State: "300M DAU, 500M tweets/day, avg 200 followers, max 100M; home timeline mostly ranked; search must show tweets within seconds; trends per region updated every few minutes."

3. Estimates

- Tweets: 500M/day ≈ **6k/s**, peak ~20k/s.
- Timeline reads: 300M × 20 = 6B/day ≈ **70k/s**, peak ~200k/s.
- Fan-out on write: 6k × 200 = 1.2M timeline inserts/s (hybrid for celebrities, as in Q15).
- Search index: 500M tweets/day × ~300 B indexed ≈ 150 GB/day of index.

4. API

```
POST /v1/tweets {text, media_ids[], reply_to?}     → {tweet_id}
GET  /v1/timeline/home?cursor=..                   → tweets
GET  /v1/users/{id}/tweets?cursor=..
GET  /v1/search?q=..&cursor=..
GET  /v1/trends?region=..
```

5. Data model

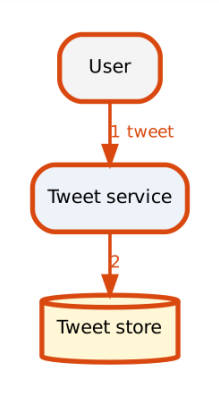
- `Tweet(tweet_id [Snowflake], author_id, text, created_at, reply_to, media)` sharded by tweet_id; user timeline index `author_id → [tweet_ids]`.
- `Follows(follower, followee)` both directions.
- Home timeline cache: `user_id → [tweet_ids]` (Redis lists, capped ~800).
- Search: inverted index segments by time.

6. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

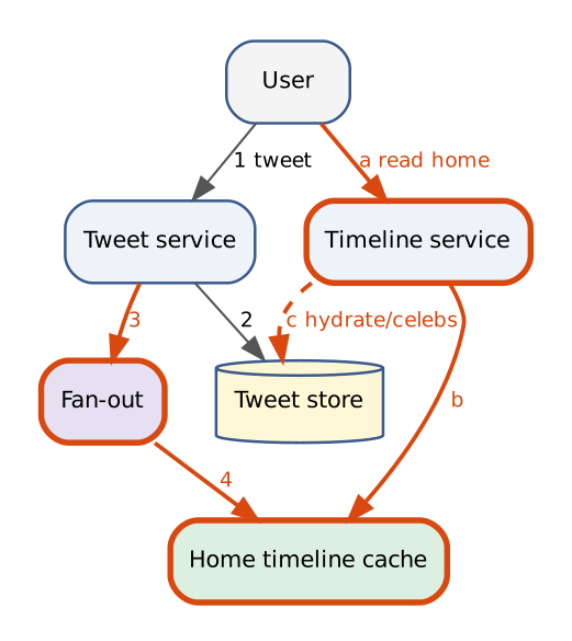
Step 1 · Post (30 s)



> **You:** “Tweets are written to a tweet store sharded by tweet ID, with per-user timelines.”

Why this step now: Simple write path.

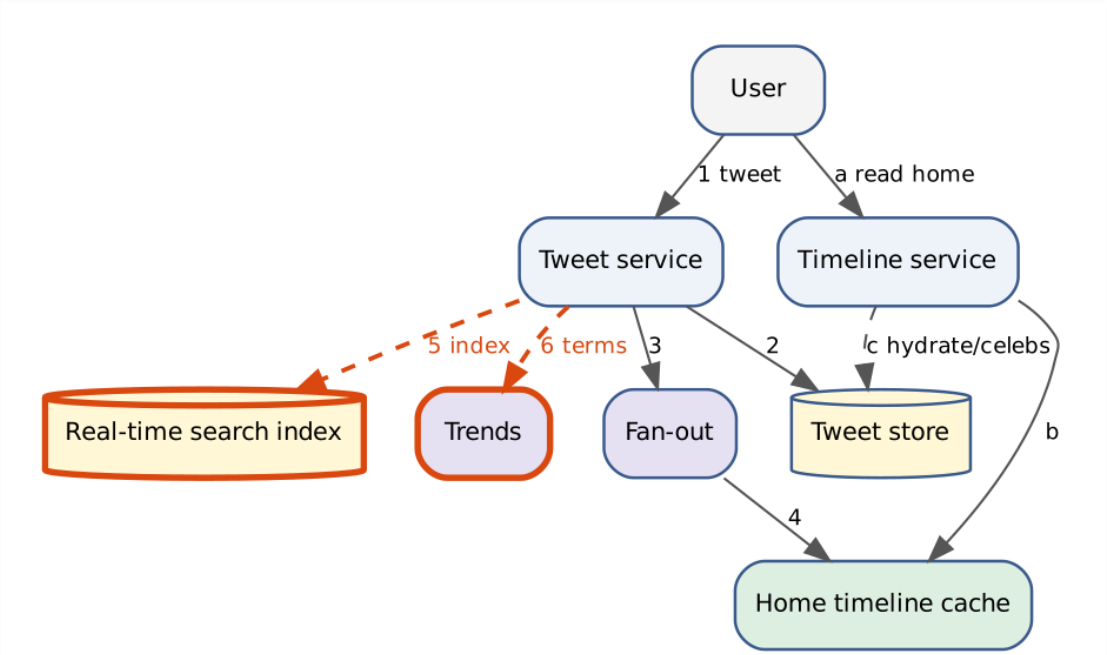
Step 2 · Home timeline (1–2 min)



> **You:** “Fan-out pushes tweet IDs to followers' home timelines in Redis except for celebrities, which are merged on read; the timeline service ranks and hydrates.”

Why this step now: Hybrid fan-out.

Step 3 · Search and trends (1 min)



> **You:** “The same tweet event feeds a real-time search indexer and a trends job that compares term velocity to a baseline per region.”

Why this step now: What Twitter adds beyond a feed.

Narrate one request end to end (numbers match the arrows)

Tweet

1. Posted (1, 2); fanned out (3, 4); indexed (5); terms counted (6).

Read

1. Home request (a); cached IDs (b); merged/hydrated (c).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User	Tweet, read	—	App
Tweet service	Write tweets	—	Service
Tweet store	Tweets + user timelines	Source of truth	Sharded store
Fan-out	Push IDs to followers	Cheap home timeline reads	Workers
Home timeline cache	Per-user tweet IDs	Fast reads	Redis
Timeline service	Merge + celebrity pull + rank	Hybrid	Service

Real-time search index	Time-partitioned inverted index	Search within seconds	Earlybird-like
Trends	Velocity vs baseline per region	Trending topics	Stream job + top-K

Before you leave the high-level design, check you have said:

☐ Hybrid fan-out
☐ Real-time index
☐ Trends = velocity

60-second summary to say once the diagram is complete:

> **You:** "Posting writes the tweet and enqueues fan-out. Fan-out workers push the tweet ID into followers' home timelines in Redis — except for accounts with huge followings, which are merged at read time. The same tweet event feeds a real-time search indexer and a trends counter. Reading the home timeline merges cached IDs with celebrity tweets, ranks, and hydrates."

7. Deep dives

Deep dive 1: real-time search (Earlybird-style). Indexers consume the tweet stream and append to in-memory, time-partitioned index segments; newest segment searchable within seconds; older segments flushed to disk/SSD. Query fans out across partitions, merges by relevance and recency. Archive search uses a separate, larger index.

Deep dive 2: trending topics. Count hashtags/terms per region in sliding windows (e.g., 5 min vs 24 h baseline); trend score = current rate vs expected rate (velocity, not raw volume — otherwise "the" and "good morning" always trend). Use count-min sketch + top-K heap per region (see Q39), spam filtering, human/policy review for sensitive trends.

Deep dive 3: timeline consistency. Your own tweet appears in your timeline immediately (client inserts it); followers see it in seconds; deletes filtered at hydration time.

8. Trade-offs, failures, ops

Decision	Choice	Why
Fan-out	Hybrid push/pull	Bounded write amplification
Search	Separate real-time index from archive	Freshness vs cost
Trends	Velocity vs baseline	Novelty, not popularity

Failures: fan-out lag → timelines a little stale (fine); search indexer lag → alert; Redis timeline loss → rebuild from follow graph + user timelines on read.

9. Manager lens, follow-ups, say this

> **You:** "Tweet ingestion, timelines, search, and trends each have their own team and SLOs; the tweet event stream is the shared contract."

Follow-ups: "Celebrity with 100M followers tweets?" → not fanned out; merged at read. "How do you rank?" → candidate retrieval + ML ranking (Q15). "Edit tweet?" → new version with edit history; caches invalidated by version.

Say this: "Trends measure acceleration, not volume." · "One event stream feeds timelines, search and trends."

Q32. Design Uber / Lyft (ride matching) — also DoorDash

> Classic · Amazon (logistics), Google (Maps), Meta, Microsoft · Netflix reported "backend for a ride-sharing service" (Prepfully)

1. The prompt

"Design the backend for a ride-hailing app: riders request rides, nearby drivers are matched, both see live location." Variant: DoorDash (three parties: customer, restaurant, courier; batching orders).

2. Clarify

Ask: "Scale: riders, drivers online, rides/day?" · "Driver location update frequency?" · "Matching goal — nearest driver, or optimize ETA/fairness?" · "Pricing (surge) in scope?" · "Payments in scope?" · "Global, many cities?"

State: "100M riders, 5M drivers, 1M drivers online at peak; 20M rides/day; drivers send location every 4 s; match on ETA; surge pricing briefly; payments via existing service; many cities."

3. Requirements

- **Must:** driver location ingestion; rider requests ride with pickup/destination; fare estimate; match to a driver who accepts; live tracking for both; trip lifecycle; completion and payment trigger.
- **Non-functional:** match within seconds; location freshness < 5 s; no double-assignment of a driver; high availability during peaks.

4. Estimates

- Location updates: 1M drivers / 4 s = **250k writes/s** — the hot path.
- Ride requests: 20M/day ≈ 230/s avg, ~1k/s peak.
- Location data is ephemeral: keep latest in memory; history to storage for analytics (250k × 100 B = 25 MB/s).

5. API

```
Driver:  WS  location{lat, lng, heading, ts, status}   (every 4 s)
         POST /v1/offers/{id}:accept | :decline
Rider:   POST /v1/rides {pickup, dropoff, product}  → {ride_id, fare_estimate}
         GET  /v1/rides/{id}  (+ WS for live driver location)
```

6. Data model

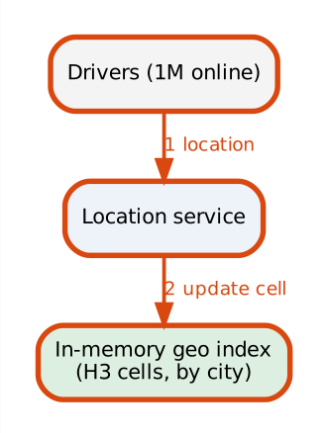
- **Geo index (in memory):** cell (H3/geohash) → set of available driver IDs with last location; sharded by city/cell.
- `Driver(id, status: offline|available|offered|on_trip, current_ride)` — status transitions must be atomic.
- `Ride(ride_id, rider, driver, state: requested→matched→arriving→in_progress→completed|cancelled, route, fare)` in a strongly consistent store.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

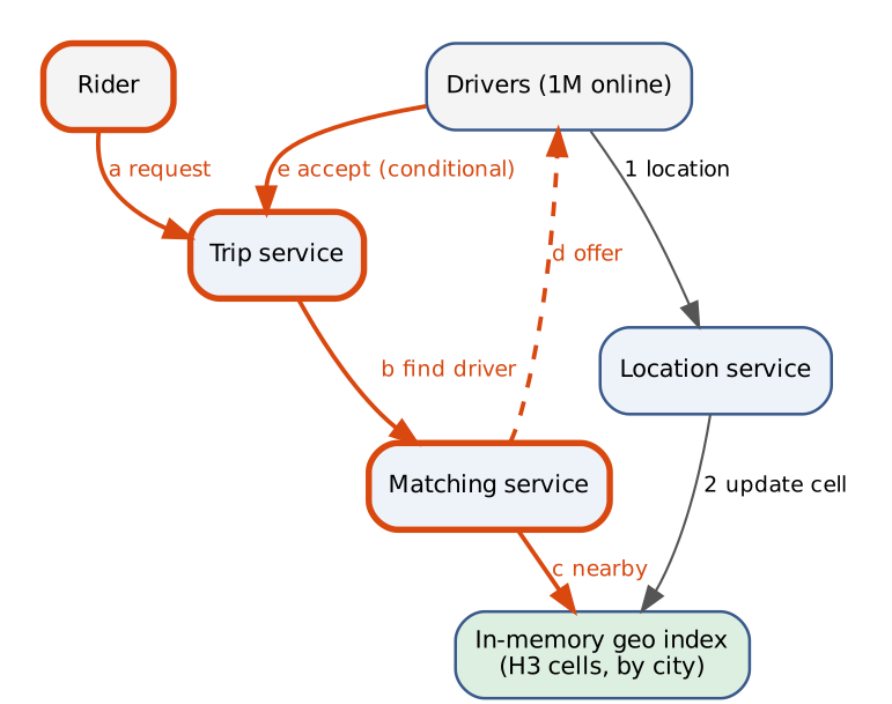
Step 1 · Driver supply (1 min)



> **You:** “Drivers stream locations every few seconds to a location service that updates an in-memory geo index sharded by city.”

Why this step now: Write math first.

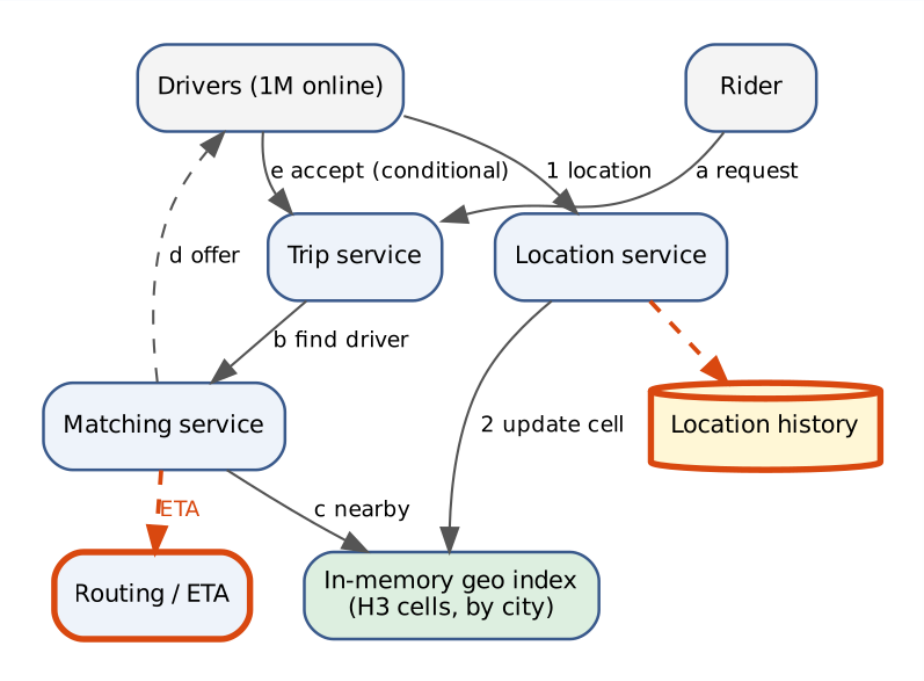
Step 2 · Matching (2 min)



> **You:** “A ride request creates a trip; matching finds available drivers in nearby cells, ranks by ETA, and offers to the best one with a timeout. Acceptance is a conditional update so no driver is double-booked.”

Why this step now: Core flow.

Step 3 · ETA and history (30 s)



> **You:** “Routing gives real ETAs; location history goes to storage asynchronously.”

Why this step now: Supporting systems.

Narrate one request end to end (numbers match the arrows)

Ride request

1. Drivers update locations (1, 2).
2. Rider requests (a); matching queries geo index (b, c).
3. Offer sent (d); driver accepts with conditional write (e).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Drivers (1M online)	Send location every 4 s	250k writes/s	Driver app
Location service	Ingest locations	Hot path	Stateless + WS
In-memory geo index (H3 cells, by city)	Latest driver positions	Only latest matters	Sharded in-memory
Rider	Request ride	—	Rider app
Trip service	Ride lifecycle	Source of truth	Service + SQL
Matching service	Candidates by ETA, offers	Core marketplace	Service
Routing / ETA	ETA per candidate	Better than straight-line distance	Maps service
Location history	Analytics, disputes	Async	Object storage/ TSDB

Before you leave the high-level design, check you have said:

☐ Location write math
☐ Atomic accept
☐ Shard by city

60-second summary to say once the diagram is complete:

> **You:** "Drivers stream locations over persistent connections to a location service that updates an in-memory geo index sharded by city. A rider's request goes to the trip service, which asks the matching service for candidates — available drivers in nearby cells ranked by ETA — and offers the ride to the best one with a short timeout. Accepting atomically sets the driver to busy and the trip to matched. Then both apps get live location via the same connections."

8. Deep dives

Deep dive 1: the geo index. H3 hexagon cells (~resolution 8–9 for city blocks). Each update: remove from old cell, add to new cell (in-memory map), keep last position. Query: cells within k rings of pickup → candidates. Shard by city (natural partition; cross-city trips are rare). Replicate in memory for availability; rebuild quickly because drivers resend every 4 s.

Deep dive 2: matching without double-booking. Offer one driver at a time (or a few in sequence) with a 10–15 s timeout. Acceptance is a conditional update: `UPDATE driver SET status='on_trip', ride=? WHERE id=? AND status='offered' AND offer_id=?`. If it fails, move to next candidate. Batch matching (every few seconds, solve assignment across many riders/drivers) improves global ETA at the cost of a little latency — mention as an optimization.

Deep dive 3: ETA and pricing. ETA from a routing service (Q34) with live traffic; surge multiplier per cell from supply/demand ratio over the last few minutes, smoothed to avoid oscillation.

9. Scaling evolution

Stage	Design
One city	Single service, Redis GEO, Postgres for trips
Many cities	Shard by city; in-memory geo index; event bus for trip events
Global	Regional deployments, cell-based architecture, dedicated matching per region

10. Failures

- Location service node dies → drivers reconnect; index repopulates in one update cycle (4 s).
- Matching crash mid-offer → offer has an ID and TTL; trip service retries; conditional update prevents double assignment.
- Region outage → trips in progress continue on apps (cached route); new requests fail over to another region where possible.

11. Trade-offs

Decision	Choice	Why
Location storage	In-memory index, history async	250k writes/s; only latest matters
Spatial index	H3 cells	Uniform neighbors, fast ring queries
Matching	Sequential offers + conditional accept	Simple, no double-booking
Partitioning	By city	Locality

12. Security, operations, cost

- Safety: trip sharing, anomaly detection (unexpected stops), identity verification.
- SLIs: match time, match rate, ETA accuracy, location freshness, cancellation rate.

13. Manager lens

> **You:** "Marketplace (matching/pricing), real-time location infra, trips, and maps are separate teams; matching and pricing changes ship behind city-level experiments because they move real money for drivers. I'd own the experimentation guardrails."

14. Follow-ups

1. **"DoorDash?"** — Three-sided: assign courier considering restaurant prep time; batch multiple orders per courier; ETA = prep + travel.
2. **"Rider and driver both see each other moving?"** — Location pushed over WebSocket to the trip's participants only, throttled.
3. **"Pool rides?"** — Matching considers route detours; more complex assignment.

15–17. Mock, scoring, say this

> **You:** "250k location writes a second, but only the latest position matters — so it lives in memory, sharded by city, and history goes to storage asynchronously."

Strong Hire: location write math; in-memory geo index by city; atomic accept; ETA-based matching. **No Hire:** writing every location to a SQL DB and querying with lat/lng ranges.

Say this: "Only the latest location matters, so it lives in memory." · "Acceptance is a conditional write — that's my no-double-booking guarantee."

Part III.2 — Location, booking, search and crawling

Q33. Design Yelp / a proximity service ("find restaurants near me") — also Nearby Friends

> Classic · Google, Amazon, Meta, Microsoft (Bing local)

1. The prompt

"Design a service that returns businesses near a location, filtered and ranked." Variant: Nearby Friends (moving points, privacy).

2. Clarify

Ask: "Number of businesses and search QPS?" · "Search radius options?" · "Filters (category, open now, rating) and ranking?" · "How often do businesses change?" (static vs moving points) · "Reviews and photos in scope?"

State: "200M businesses, 50k searches/s at peak, radius 0.5–20 km, filters and ranking by distance + rating; businesses change rarely (minutes of staleness fine); reviews in a separate service."

3. Requirements and estimates

- **Must:** nearby search with filters, pagination, business details; add/update business (eventually consistent).
- **NFR:** p99 < 200 ms; highly available; read-heavy.
- Data: 200M × 1 KB = 200 GB of business data; geo index (ID + coordinates + cell) ~200M × 30 B = 6 GB → **fits in memory on each search replica**. Say this — it's the key insight.

4. API

```
GET /v1/search?lat=..&lng=..&radius=2km&category=pizza&open_now=true&cursor=..  → {businesses[], next}
GET /v1/businesses/{id}
POST/PUT /v1/businesses (owners/admin)
```

5. Data model

- `Business(id, name, lat, lng, category, hours, rating, price)` in SQL (source of truth).

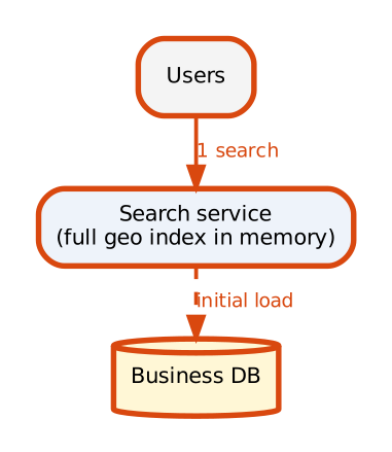
- Geo index: geohash/S2 cell → list of business IDs (+ precomputed attributes for filtering).

6. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

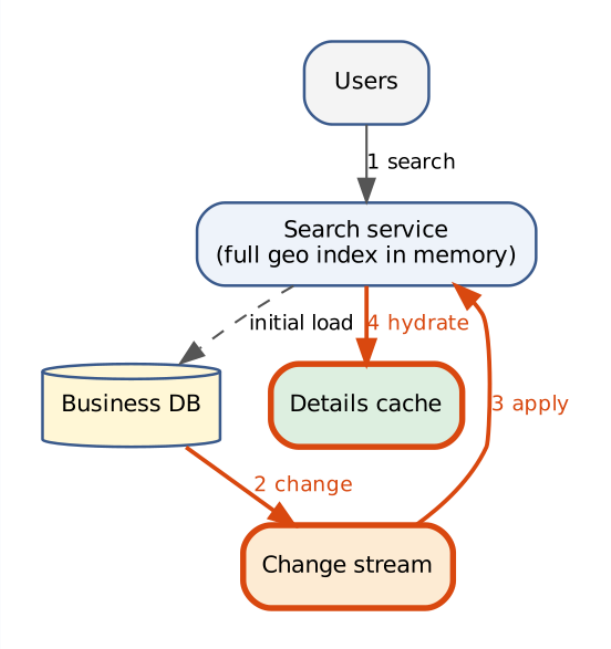
Step 1 · Search from memory (1 min)



> **You:** “Each search node holds the whole geo index in memory — 200M businesses is only a few gigabytes — and answers by covering the radius with cells.”

Why this step now: The key insight.

Step 2 · Keep it fresh and hydrate (1 min)



> **You:** “Business changes flow from the database through a change stream into each node's index; details come from a cache.”

Why this step now: Freshness without coupling.

Narrate one request end to end (numbers match the arrows)

Search

1. Query (1): cells covering circle → candidates → filter → rank → hydrate (4).

Update

1. DB change (2) applied to indexes (3) within minutes.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Users	Search nearby	—	App
Search service (full geo index in memory)	Cells covering radius → candidates → rank	6 GB index fits in RAM	Replicated service
Business DB	Source of truth	—	SQL
Change stream	Business updates	Refresh in-memory indexes	CDC → Kafka
Details cache	Business details	Hydration	Redis

Before you leave the high-level design, check you have said:

☐ Index fits in memory
☐ Spatial index choice
☐ Density adaptation

60-second summary to say once the diagram is complete:

> **You:** "Business data lives in a relational database. Because the geo index is only a few gigabytes, each search server keeps a full copy in memory, rebuilt or incrementally updated from a change stream every few minutes. A search computes the cells covering the circle, gathers candidates, filters, ranks by distance and quality, and hydrates details from a cache."

7. Deep dives

Deep dive 1: choosing the spatial index.

Index	How	Fit here
Geohash	Prefix strings; search cell + 8 neighbors	Simple; cell sizes fixed per precision

Index	How	Fit here
Quadtree	Split cells until ≤ N businesses	Adapts to density (Manhattan vs desert)
S2 / H3	Sphere-aware hierarchical cells; region covering	Accurate circles; production-grade
Postgres PostGIS / R-tree	DB spatial index	Fine at small scale

Pick: **S2 (or quadtree) in memory**; choose cell level by radius; expand rings until enough results.

Deep dive 2: ranking. Score = f(distance, rating, review count, open now, personalization). Retrieve ~500 candidates, compute score, return top 20; cursor holds the last score.

Deep dive 3: Nearby Friends (moving points). Locations update every 30 s; store latest in Redis keyed by user; pub/sub channel per user for friends who opted in; on update, publish to friends' channels; consumers compute distance. Privacy: opt-in, coarse location, expiry.

8. Trade-offs, failures, ops

Decision	Choice	Why
Index location	In memory on every search node	6 GB fits; no network hop
Freshness	Minutes	Businesses rarely move
Partition	Replicate, don't shard (until index outgrows RAM)	Simpler, faster

Failures: index rebuild failing → keep serving the old index; DB down → search still works (read-only).

9. "Say this"

"The whole geo index is a few gigabytes, so every search node holds it in memory." · "Quadtrees adapt to density; fixed grids don't."

Follow-ups: (1) "Index outgrows memory?" → shard by S2 cell ranges / region, route by query location. (2) "A new restaurant must appear within seconds?" → change stream applies incremental updates to in-memory indexes. (3) "Rank by 'best' not nearest?" → retrieve wider radius candidates, rank by learned score with distance as a feature.

Scoring: *Strong Hire* — notices the index fits in memory; picks and justifies a spatial index; separates search index from source of truth; handles density. *No Hire* — SQL `WHERE lat BETWEEN … AND lng BETWEEN …` on 200M rows per request.

Q34. Design Google Maps (routing and ETA)

> Classic · Google, Amazon (logistics routing), Uber-style questions

1. The prompt

"Design the routing and ETA part of a maps service." Clarify scope: map tiles rendering, search, routing, live traffic, navigation.

2. Clarify and state

Ask: "Scope: tiles, routing, traffic, turn-by-turn?" · "Route queries per second?" · "Freshness of traffic?" · "Modes: drive, walk, transit?"

State: "Focus on driving routes with live traffic ETA; map tiles briefly; 100k route requests/s at peak; traffic updated every minute from phone probes."

3. Estimates

- Road graph: ~1B nodes / 2B edges worldwide; with preprocessing (contraction hierarchies) a continent's graph fits in tens of GB of RAM per routing server.
- Traffic probes: 100M active navigating phones × 1 update/5 s = **20M location updates/s** → aggregated by road segment per minute.

4. API

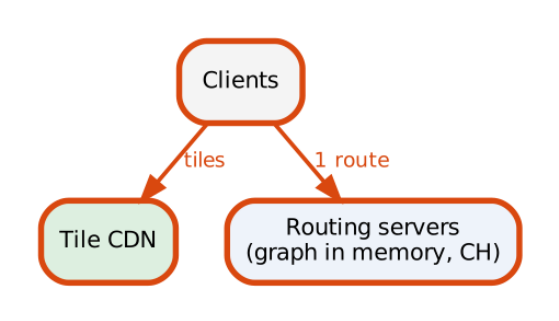
```
GET /v1/route?origin=..&dest=..&mode=drive&depart_at=now → {routes:[{polyline, eta, distance, steps}]}
GET /v1/tiles/{z}/{x}/{y}.pbf      (vector tiles via CDN)
POST /v1/probes (batched anonymized locations from navigating devices)
```

5. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

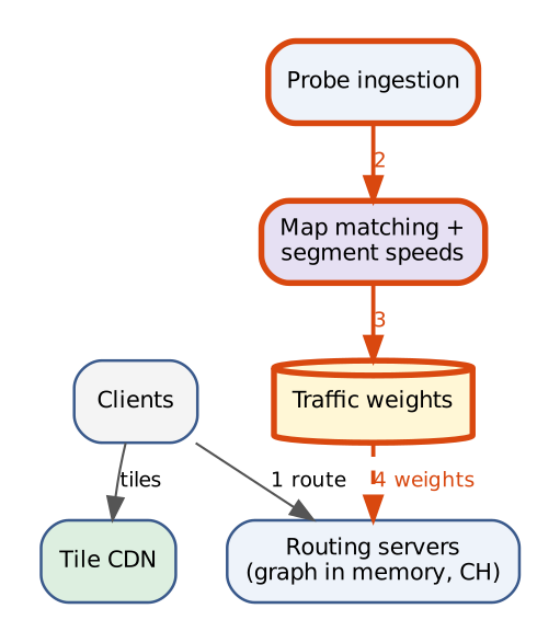
Step 1 · Tiles and routing (1 min)



> **You:** “Tiles are static vector files on a CDN. Routing servers hold the road graph in memory with contraction-hierarchy preprocessing so queries take milliseconds.”

Why this step now: Separate static from compute.

Step 2 · Live traffic (1–2 min)



> **You:** “Navigating phones send anonymized probes; a stream job map-matches them to road segments and computes current speeds every minute, which become routing weights.”

Why this step now: Freshness for ETA.

Narrate one request end to end (numbers match the arrows)

Route

1. Client requests route (1); server runs bidirectional hierarchy search with live weights (4).

Traffic

1. Probes ingested (2), matched and aggregated (3).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Clients	Request routes, tiles	—	App
Tile CDN	Vector tiles	Static, cacheable	CDN
Routing servers (graph in memory, CH)	Shortest paths in ms	Preprocessed graph	In-memory service
Probe ingestion	GPS from navigating phones	20M updates/s	Edge + Kafka
Map matching + segment speeds	Snap to roads, speed per segment per minute	Live traffic	Stream job
Traffic weights	Current + historical speeds	Edge weights for ETA	KV

Before you leave the high-level design, check you have said:

- ☐ Why plain Dijkstra is too slow
- ☐ Probe pipeline
- ☐ ETA blending

60-second summary to say once the diagram is complete:

> **You:** "Map tiles are static, pre-rendered vector tiles served from a CDN. Routing servers hold the road graph in memory, preprocessed so that a cross-country query explores thousands of nodes instead of millions. Live traffic comes from anonymized phone probes: a stream pipeline map-matches them to road segments and computes current speeds per segment every minute; routing uses those speeds as edge weights for ETA."

6. Deep dives

Deep dive 1: fast shortest paths. Plain Dijkstra on a continental graph is far too slow per request. **Contraction hierarchies:** preprocess by adding shortcut edges and ranking nodes; queries run bidirectional search upward in the hierarchy, exploring a tiny fraction of the graph → milliseconds. Live traffic changes weights → customizable variants (CCH / multi-level partitioning) let you re-weight quickly without full preprocessing.

Deep dive 2: ETA. Segment speeds from live probes, blended with historical speeds for that time of day; ML model corrects systematic errors (turns, traffic lights). Report ETA ranges; recompute during navigation and reroute if a better path appears.

Deep dive 3: map matching. Noisy GPS points → snap to the most likely road sequence with a hidden Markov model (Viterbi). Needed both for probes and for navigation.

7. Trade-offs and say this

Decision	Choice	Why
Tiles	Pre-rendered vector tiles on CDN	Static, cacheable
Routing	In-memory graph + hierarchy preprocessing	Millisecond queries
Traffic	Probe stream → segment speeds every minute	Fresh weights

Say this: "Preprocessing turns a million-node search into a few thousand." · "Tiles are static files; routing is in-memory compute; traffic is a streaming problem."

Follow-ups: (1) "A road closes?" → traffic/incident feed sets edge weight to infinity; customizable hierarchy re-weights in seconds; navigating users rerouted. (2) "Offline maps?" → download tiles + a regional routing graph to the device. (3) "How do you test ETA quality?" → compare predicted vs actual trip times by region and hour; track error percentiles.

Scoring: *Strong Hire* — separates tiles/routing/traffic; knows why Dijkstra alone is too slow and how preprocessing fixes it; probe pipeline for traffic. *No Hire* — "run Dijkstra on the database."

Q35. Design Ticketmaster (high-demand ticket booking) — also Airbnb / hotel booking

> Classic · Amazon, Microsoft, Meta, Google; Airbnb/Booking common at Amazon

1. The prompt

"Design a ticketing system where millions of fans try to buy seats for a popular concert when sales open." Variant: hotel/Airbnb booking (lower contention, date ranges).

2. Clarify

Ask: "Seated (specific seats) or general admission?" · "Peak demand — users at on-sale vs seats available?" · "How long can a seat be held during checkout?" · "Fairness requirements — queue, lottery, bot protection?" · "Payment in scope?"

State: "Reserved seating; 50k seats; 5M users arrive at on-sale; holds last 10 minutes; virtual waiting queue; bots are a major threat; payment via existing processor."

3. Requirements

- **Must:** browse events; show seat map availability; hold seats; checkout and pay; never sell a seat twice; release expired holds.
- **NFR:** correctness above all (no double-sell); survive 100× traffic spikes at on-sale; fair access.

4. Estimates

- 5M users in the first minutes vs 50k seats → **demand is 100× supply**; the system's real job is controlled admission.
- If all 5M hit the seat map at once: 5M requests in ~1 min ≈ 80k RPS on one event's data → hot key.

5. API

```
GET  /v1/events/{id}/seats        → availability map (cached, slightly stale OK)
POST /v1/events/{id}/holds {seat_ids[]} → {hold_id, expires_at}   (fails if any seat taken)
POST /v1/holds/{id}:checkout {payment_token} Idempotency-Key → {order_id}
Queue: GET /v1/events/{id}/queue/token  → {position, eta}  (waiting room)
```

6. Data model

- `Seat(event_id, seat_id, status: available|held|sold, hold_id, hold_expires_at, order_id)` — strongly consistent store, partitioned by event (and section for huge venues).

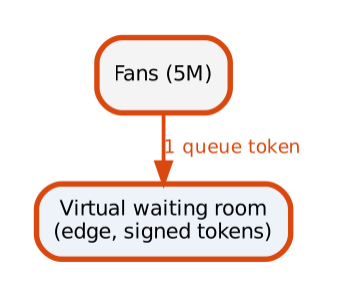
- `Hold(hold_id, user_id, seat_ids, expires_at)`, `Order(order_id, user, seats, payment_state)`.

7. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

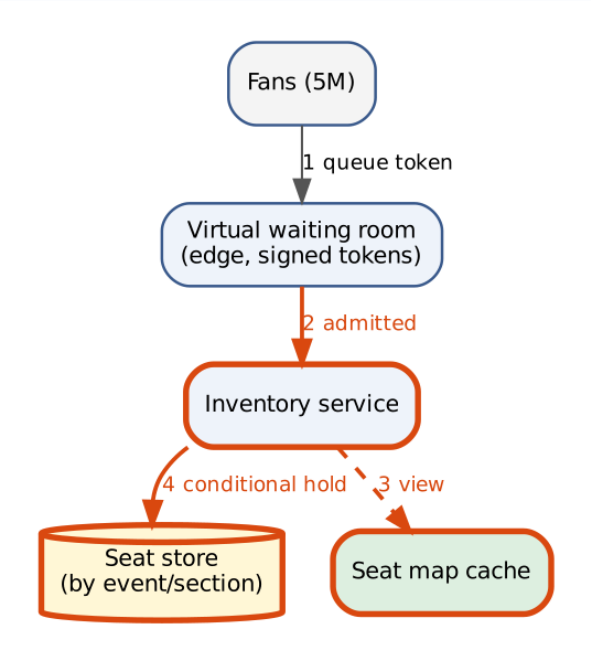
Step 1 · Admission control (1 min)



> **You:** "Because demand is 100× supply, everyone first gets a signed queue token at the edge, and we admit users at a rate inventory can handle."

Why this step now: Frame the real problem.

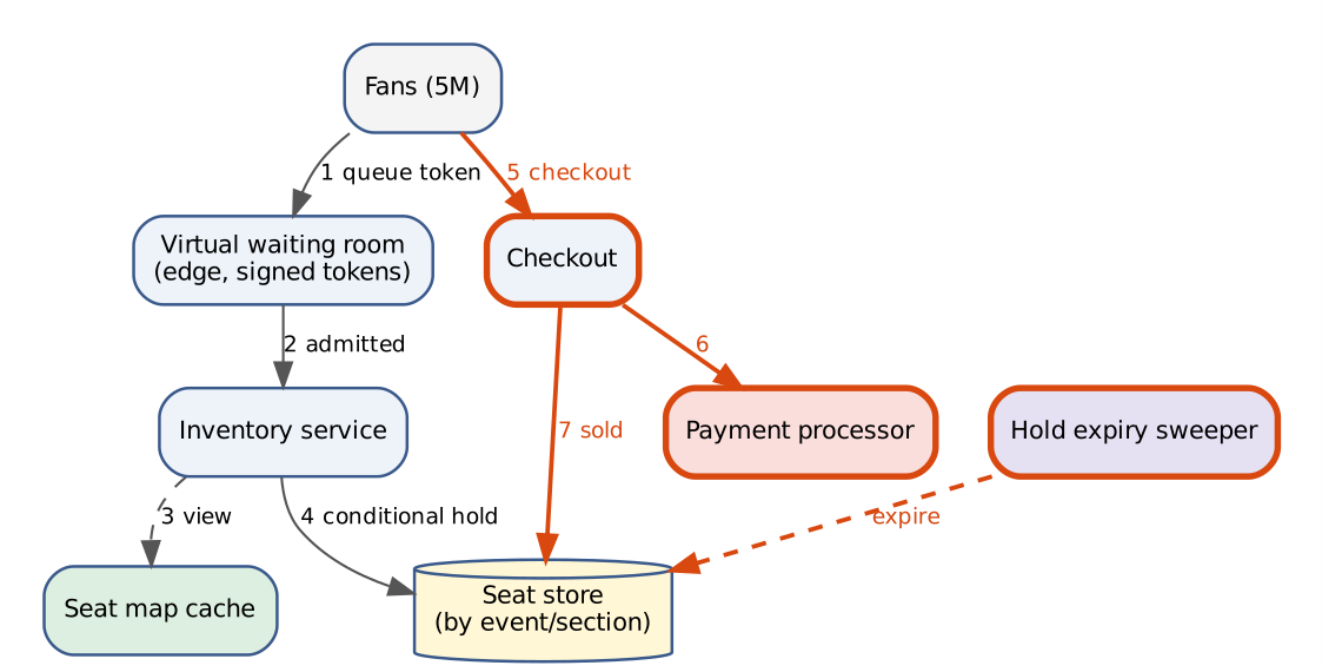
Step 2 · Holding seats (1–2 min)



> **You:** "Admitted users see a cached seat map; holding seats is a conditional update on the seat store that succeeds only if every seat is still available, with a 10-minute expiry."

Why this step now: Correctness core.

Step 3 · Checkout and expiry (1 min)



> **You:** "Checkout charges with an idempotency key and moves held seats to sold; a sweeper releases expired holds."

Why this step now: Completes the lifecycle.

Narrate one request end to end (numbers match the arrows)

Buy

1. Queue token (1); admitted (2); view seats (3); hold atomically (4).
2. Checkout (5), charge (6), mark sold (7).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Fans (5M)	Buy tickets	—	App
Virtual waiting room (edge, signed tokens)	Admit at a controlled rate	Demand ≫ supply	CDN workers
Inventory service	Hold seats atomically	No double-sell	Service
Seat store (by event/section)	Seat status, holds	Strong consistency	SQL / Redis+persist
Seat map cache	Availability view	Read load	Cache
Checkout	Pay, held → sold	Idempotent	Service

Payment processor	Charge	—	PSP
Hold expiry sweeper	Release expired holds	Seats return	Job

Before you leave the high-level design, check you have said:

☐ Demand vs supply framing
☐ Conditional hold
☐ Bots/fairness

60-second summary to say once the diagram is complete:

> **You:** "Because demand is a hundred times supply, the first component is a virtual waiting room: users get a queue token at the edge, and we admit them to the booking flow at a rate the inventory service can handle. Admitted users see a cached seat map, pick seats, and the inventory service atomically moves those seats from available to held with a ten-minute expiry. Checkout charges the card with an idempotency key and moves held to sold. Expired holds return to available."

8. Deep dives

Deep dive 1: never double-sell. Hold = conditional update for all requested seats in one transaction: `UPDATE seats SET status='held', hold_id=?, hold_expires_at=now()+10m WHERE event=? AND seat_id IN (...) AND (status='available' OR (status='held' AND hold_expires_at < now()))` and require rows affected = number of seats. Alternatives: Redis with Lua scripts for speed, then persist; or a single-writer per event section (actor) that serializes holds.

Deep dive 2: the waiting room. Edge (CDN/worker) issues signed queue tokens with random order for those who arrived before on-sale (lottery = fair vs bots), FIFO after. Admission rate controller watches inventory service latency and seats remaining; stops admission when sold out and tells everyone immediately.

Deep dive 3: bots and fairness. Verified fan pre-registration, CAPTCHA/proof-of-work at the queue, per-account limits, device fingerprinting, purchase limits per payment instrument.

Airbnb/hotel variant: contention is low; inventory = (listing, date) rows; booking = conditional update across a date range in one transaction; search is the hard part (geo + availability + price), served from a search index updated from the inventory stream.

9. Trade-offs and failures

Decision	Choice	Why
Admission	Virtual queue with rate control	Protects inventory from 100× demand
Consistency	Strong for holds	No double-selling
Seat map	Cached, seconds stale	Read load; holds are the source of truth

Decision	Choice	Why
Hold expiry	10 min TTL, lazy + sweeper	Seats return to sale

Failures: payment fails → hold stays until expiry, user can retry; inventory partition down → admission paused for that event.

10. Manager lens and say this

> **You:** "The on-sale is a planned event, so I'd run it like a launch: load test at 2× expected, game day with the waiting room, war room on the day, and a clear decision owner for pausing sales."

Say this: "When demand is 100× supply, the system's job is admission control." · "The hold is a conditional write; the seat map is just a cache."

Q36. Design search autocomplete (typeahead)

> Classic · Google, Amazon, Meta, Microsoft (Bing), LinkedIn

1. The prompt

"Design the system that suggests the top 10 completions as a user types in a search box."

2. Clarify and state

Ask: "Scale of queries and keystrokes?" · "How fresh must suggestions be — trending terms within minutes?" · "Personalized?" · "Languages?" · "Filter offensive suggestions?"

State: "10B searches/day; ~5 keystrokes each sent with debouncing → ~25B suggestion requests/day; top 10 by popularity; trending terms reflected within ~15 minutes; light personalization; filter policy-violating suggestions."

3. Estimates

- 25B/day ≈ **290k RPS**, peak ~1M RPS → must be served from memory, likely at the edge.
- p99 latency budget ~50 ms (users type fast).
- Distinct queries in the index: say 1B prefixes' top-10 lists × ~200 B ≈ 200 GB → shard by prefix range or cache only popular prefixes (most traffic hits short prefixes).

4. API

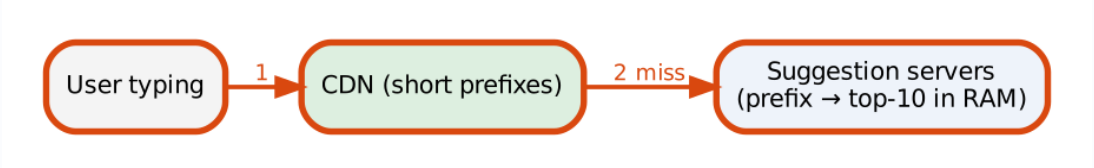
```
GET /v1/suggest?q=new%20yo&lang=en&loc=..  → {suggestions:["new york", "new york times", ...]}
```

5. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

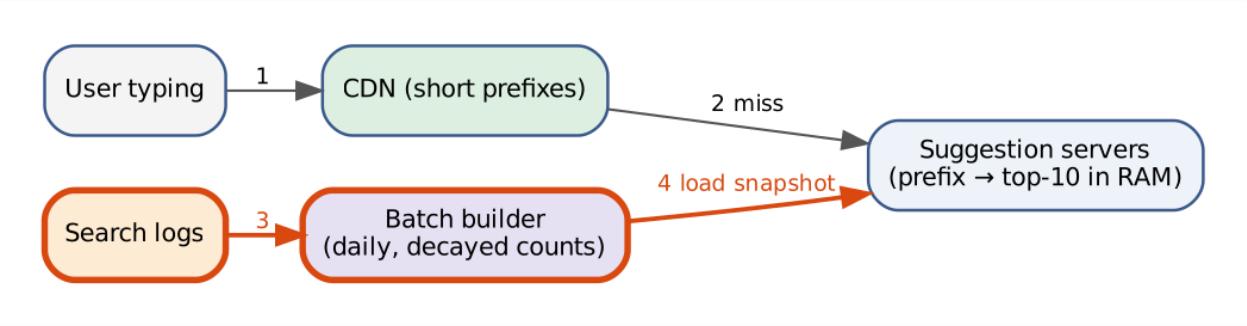
Step 1 · Serve from memory (1 min)



> **You:** “Suggestions come from servers holding precomputed top-10 lists for each prefix in memory; short popular prefixes are cached at the CDN; clients debounce.”

Why this step now: Read path first — it's the hard constraint.

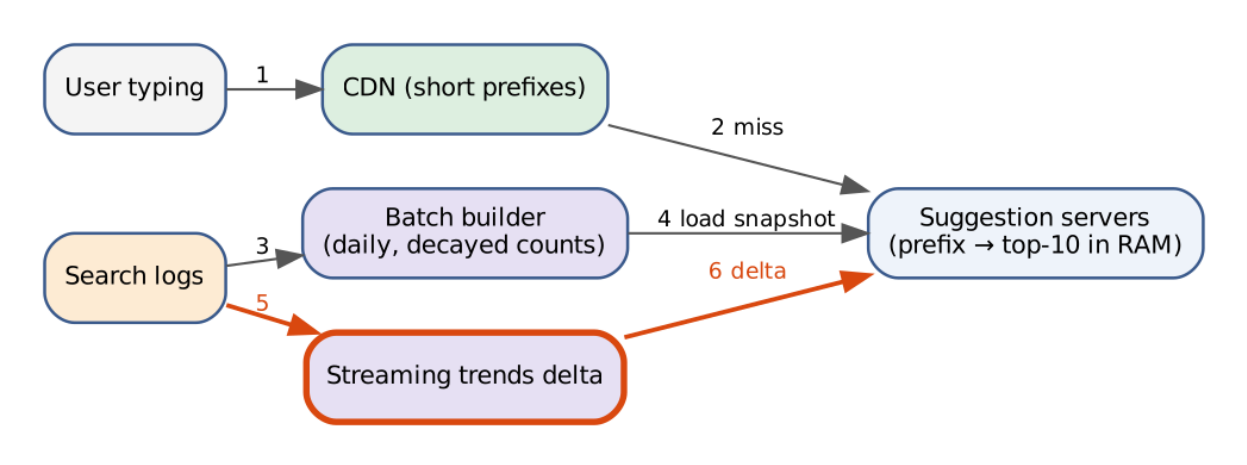
Step 2 · Build offline (1 min)



> **You:** “A daily batch counts queries with time decay and builds the prefix lists.”

Why this step now: Where the answers come from.

Step 3 · Freshness (30 s)



> **You:** “A streaming job detects trending queries in the last 15 minutes and pushes small deltas.”

Why this step now: Trends without full rebuilds.

Narrate one request end to end (numbers match the arrows)

Keystroke

1. Request (1) → CDN or server (2) → lookup → top-10.

Build

1. Logs (3) → batch (4); trends (5) → delta (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User typing	Debounced requests	—	Browser
CDN (short prefixes)	Popular prefixes	Absorb load	CDN
Suggestion servers (prefix → top-10 in RAM)	O(prefix) lookups	1M RPS	Replicated shards
Search logs	Query events	Signal	Kafka
Batch builder (daily, decayed counts)	Build prefix→top-K	Completeness	Spark job
Streaming trends delta	Last 15 min trending	Freshness	Stream job

Before you leave the high-level design, check you have said:

☐ QPS math
☐ Precomputed top-K
☐ Skew-aware sharding

60-second summary to say once the diagram is complete:

> **You:** "Two systems: an offline/near-line builder and an online server. Search logs flow into an aggregation pipeline that counts queries with time decay and builds, for every prefix, its precomputed top-10 list. Online servers hold those lists in memory — a trie or simply a prefix-to-list map — and answer in microseconds. Clients debounce keystrokes and cache results; popular prefixes are cached at the CDN."

6. Deep dives

Deep dive 1: data structure. A trie where each node stores its top-K completions precomputed (so lookup = walk the prefix, return the stored list; no traversal of the subtree at query time). At scale, flatten into a key-value map `prefix → top10` (simpler to shard and replicate).

```
import heapq
from collections import defaultdict

def build_prefix_topk(query_counts, k=10, max_prefix=20):
    """query_counts: {query: decayed_count}. Returns {prefix: [top-k queries]}."""
    buckets = defaultdict(list)                  # prefix -> min-heap of (count, query)
    for q, c in query_counts.items():
        for i in range(1, min(len(q), max_prefix) + 1):
            h = buckets[q[:i]]
            if len(h) < k: heapq.heappush(h, (c, q))
            elif c > h[0][0]: heapq.heapreplace(h, (c, q))
    return {p: [q for c, q in sorted(h, reverse=True)] for p, h in buckets.items()}
```

Deep dive 2: freshness. Full rebuild daily (batch over logs) + a streaming layer that counts trending queries in the last 15 minutes and merges them into results for affected prefixes (small delta index). Time-decayed counts so last week matters more than last year.

Deep dive 3: sharding. Shard by prefix (first 1–2 chars are skewed: "s" ≫ "x") → use a sharding map based on traffic, not alphabet. Replicate shards heavily (read-only data). Personalization: blend global list with the user's recent queries on the client or a small per-user store.

7. Trade-offs, ops and say this

Decision	Choice	Why
Compute at query time vs precompute	Precompute top-K per prefix	1M RPS needs O(prefix) lookups
Freshness	Daily batch + streaming delta	Trending without full rebuilds
Caching	Client debounce + CDN for short prefixes	Cuts load dramatically

Policy: blocklist filters applied at build time and again at serve time (fast kill switch).

Say this: "Store the answer at every prefix — query time is just a lookup." · "Batch for completeness, a streaming delta for trends."

Follow-ups: (1) "Offensive suggestion goes viral?" → serve-time blocklist with instant push; remove from builder. (2) "Multi-language?" → per-locale indexes; normalization (accents, scripts). (3) "Typos?" → fuzzy matching on short prefixes via edit-distance candidates, or spelling-corrected prefixes precomputed.

Scoring: *Strong Hire* — QPS math → precomputed top-K per prefix in memory; freshness via streaming delta; skew-aware sharding; client debounce. *No Hire* — `SELECT … WHERE query LIKE 'prefix%' ORDER BY count` per keystroke.

Q37. Design a web crawler

> Classic · Google, Microsoft (Bing), Amazon, OpenAI-style data teams

1. The prompt

"Design a crawler that downloads billions of web pages for a search index." Variants: crawl for LLM training data, price monitoring.

2. Clarify and state

Ask: "How many pages and how fast?" · "Recrawl frequency?" · "Content types (HTML only)?" · "Politeness constraints?" · "Store raw pages or just extracted text?" · "Dynamic JS rendering needed?"

State: "15B pages per month (refresh cycle), HTML focus, obey robots.txt and per-host rate limits, store raw compressed pages, JS rendering for a small subset."

3. Estimates

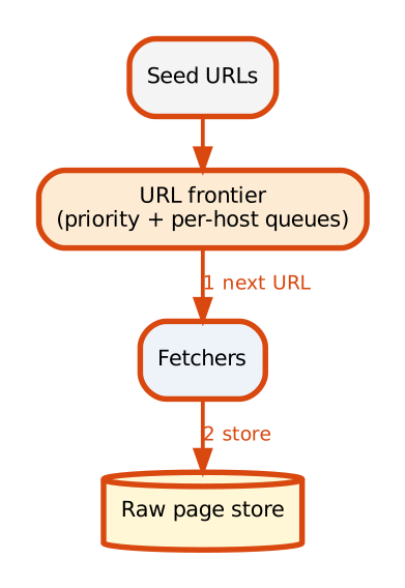
- 15B/month ≈ 500M/day ≈ **5.8k pages/s**. At ~100 KB/page → 580 MB/s download bandwidth; 50 TB/day raw, ~10 TB compressed.
- URL frontier: tens of billions of URLs → must be on disk, partitioned.

4. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

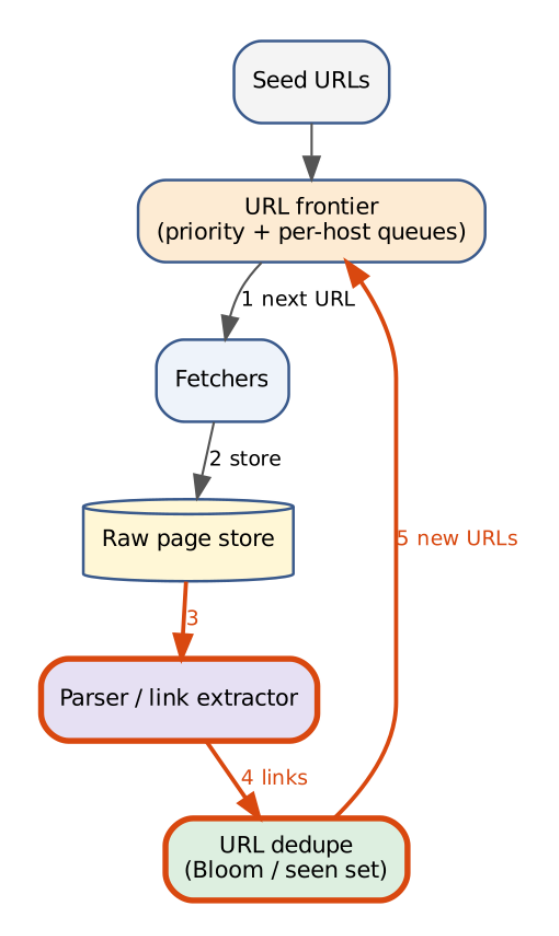
Step 1 · Fetch loop (1 min)



> **You:** “The frontier hands fetchers the next URL respecting per-host delays; fetchers check robots.txt, download, and store raw pages.”

Why this step now: Core loop.

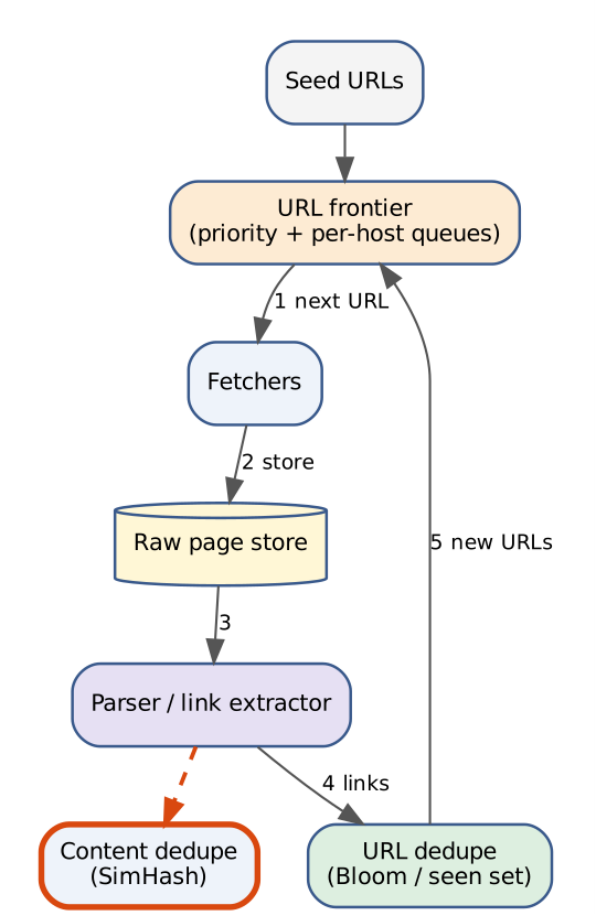
Step 2 · Discover links (1 min)



> **You:** “Parsers extract links, normalize them, check a Bloom-filter seen set, and add new URLs back to the frontier.”

Why this step now: Closing the loop.

Step 3 · Dedupe content (30 s)



> **You:** “SimHash detects near-duplicate pages before indexing.”

Why this step now: Quality.

Narrate one request end to end (numbers match the arrows)

Crawl cycle

1. Frontier gives URL (1); fetch and store (2).
2. Parse (3); dedupe links (4); enqueue new (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Seed URLs	Starting points	—	—
URL frontier (priority + per-host queues)	What to fetch next, politely	The heart of the crawler	Disk-backed queues
Fetchers	DNS (cached), robots.txt, download	—	Workers
Raw page store	Compressed pages	Index input	Object storage

Parser / link extractor	Text + links	—	Workers
URL dedupe (Bloom / seen set)	Skip seen URLs	Billions of URLs	Bloom filter
Content dedupe (SimHash)	Near-duplicate pages	Waste and spam	Service

Before you leave the high-level design, check you have said:

☐ Frontier: priority + politeness
☐ Partition by host
☐ Traps

60-second summary to say once the diagram is complete:

> **You:** "The heart of a crawler is the URL frontier — the queue of what to fetch next, organized so we're polite to each host and fetch important pages first. Fetchers take URLs from the frontier, resolve DNS through a cache, check robots.txt, download, and store raw pages. Parsers extract text and links; links are normalized, deduplicated against a seen-URL set, and added back to the frontier."

5. Deep dives

Deep dive 1: the frontier (priority + politeness). Two layers (Mercator design): front queues by priority (PageRank, freshness need); back queues one per host, each with a next-allowed-fetch time based on crawl delay. A heap of hosts by next-allowed time decides which host's queue to pull from. Partition the frontier by host hash across machines so each host is owned by one crawler node (politeness enforced locally).

Deep dive 2: deduplication. URL dedupe: normalize (lowercase host, remove fragments, sort params) and check a Bloom filter / distributed seen-set (billions of entries; Bloom with ~1% false positives costs ~1.2 GB per billion URLs). Content dedupe: hash for exact duplicates; SimHash for near-duplicates (mirrors, boilerplate).

Deep dive 3: robustness. Spider traps (infinite calendars, session IDs) → max depth per host, URL length limits, per-host page budgets. Timeouts, max page size. DNS caching (DNS is a surprising bottleneck). Recrawl scheduling by change frequency (news hourly, static pages monthly).

6. Trade-offs and say this

Decision	Choice	Why
Frontier	Priority front queues + per-host back queues	Importance + politeness
Partitioning	By host	Politeness and locality
Dedupe	Bloom for URLs, SimHash for content	Memory-efficient at billions

Say this: "The frontier is the crawler — everything else is plumbing." · "Partition by host so politeness is a local decision."

Follow-ups: (1) "How do you prioritize what to recrawl?" → estimate change rate per page from history; importance × change probability. (2) "JavaScript-heavy pages?" → separate rendering queue with headless browsers for a sampled subset; expensive, so prioritize. (3) "A crawler node dies?" → its host partitions reassigned; frontier is durable on disk, fetch state re-derived.

Scoring: *Strong Hire* — frontier with priority + politeness; partition by host; URL and content dedupe with memory math; traps and robots.txt. *No Hire* — a single BFS queue hammering hosts.

Part III.3 — Search, ranking and infrastructure building blocks

Q38. Design a web search engine (Google Search)

> Classic · Google, Microsoft (Bing), Amazon (product search), Apple (Spotlight/Siri search)

1. The prompt

"Design a search engine: given a query, return the 10 most relevant web pages in under 200 ms." Builds on the crawler (Q37). Variant: e-commerce product search (structured filters + ranking).

2. Clarify and state

Ask: "Index size?" · "QPS?" · "Freshness for news?" · "Ranking signals in scope?" · "Spelling correction, snippets?"

State: "50B pages indexed, 100k queries/s, p99 < 200 ms, fresh index for news within minutes, ranking by text relevance + link authority + learned model, snippets and spelling correction."

3. Estimates

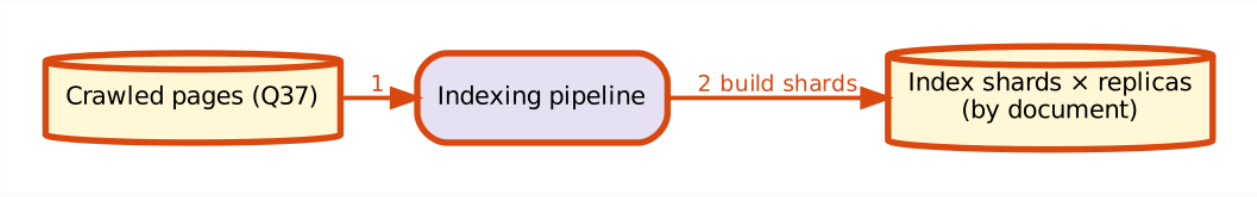
- Index size: 50B docs × ~10 KB indexed text ≈ 500 TB of text → inverted index maybe ~30–50% of that compressed → **hundreds of TB**, split across thousands of machines (document-sharded).
- Each query fans out to every shard → 100k QPS × N shards → replicas multiply capacity: each shard has R replicas; serving cluster = N × R machines.

4. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

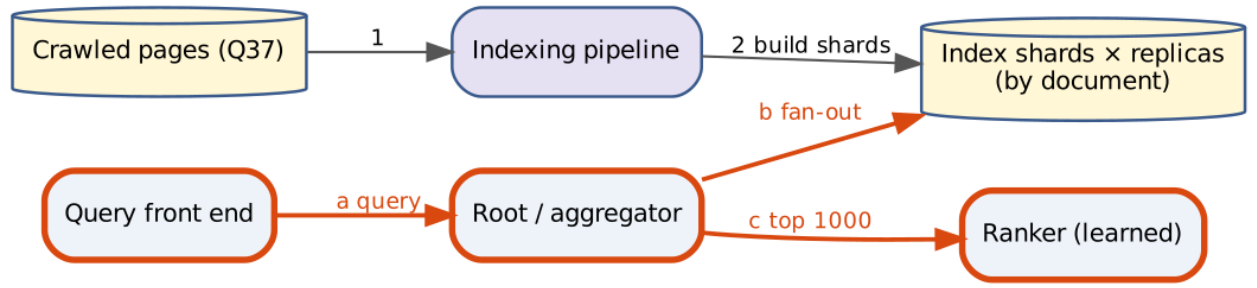
Step 1 · Offline indexing (1 min)



> **You:** "Crawled pages are processed into inverted-index shards partitioned by document, plus quality signals."

Why this step now: Build before serve.

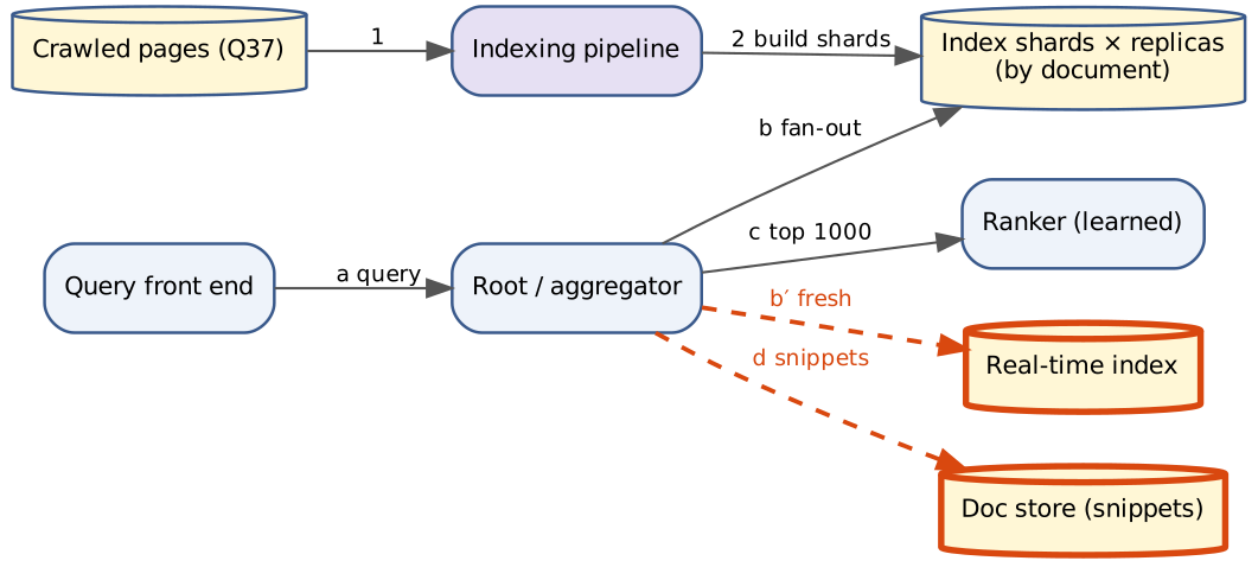
Step 2 · Query serving (2 min)



> **You:** "A query is rewritten, fanned out to one replica of every shard, each returns its local top results, the root merges, and a learned ranker reorders the best thousand."

Why this step now: Multi-stage ranking within 200 ms.

Step 3 · Freshness and snippets (30 s)



> **You:** “A small real-time index is queried in parallel for fresh content; snippets come from a document store.”

Why this step now: Completes the result page.

Narrate one request end to end (numbers match the arrows)

Search

1. Query (a) → fan-out to shards (b, b′) → merge → rank (c) → snippets (d).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Crawled pages (Q37)	Input	—	Object storage
Indexing pipeline	Parse, build inverted-index shards, signals	Offline heavy lifting	Batch + streaming
Index shards × replicas (by document)	Posting lists	Hundreds of TB	Serving clusters
Query front end	Spelling, rewrite	—	Service
Root / aggregator	Fan out, merge, hedge	Thousands of shards	Service
Ranker (learned)	Re-rank top ~1,000	Quality	ML service
Real-time index	Fresh news/pages	Minutes freshness	In-memory index
Doc store (snippets)	Snippets	Result display	KV

Before you leave the high-level design, check you have said:

☐ Document sharding
☐ Tiered ranking
☐ Hedged requests

60-second summary to say once the diagram is complete:

> **You:** "Offline, crawled pages are parsed and an indexing pipeline builds inverted-index shards — partitioned by document — plus signals like link authority. Online, a query goes to a front end that corrects spelling and expands the query, then a root server fans out to one replica of every index shard. Each shard returns its local top results; the root merges them, a ranking stage reorders the best few hundred with a learned model, and snippets are generated from document stores."

5. Deep dives

Deep dive 1: inverted index and serving. Term → posting list (doc IDs, positions, term frequency), compressed (delta + varint). Shards by document so each shard answers independently; posting lists sorted by a static quality score so a shard can stop early (top-k with early termination, WAND/BMW algorithms) rather than scanning every match.

Deep dive 2: tiered ranking. Stage 1 per shard: cheap BM25 + static quality → top ~1,000. Stage 2 at root: learned ranking model over ~1,000 candidates with richer features (query-document semantic match via embeddings, freshness, clicks). Stage 3: diversification, policy, ads blending.

Deep dive 3: freshness. Main index rebuilt in large batches; a small real-time index (news, recent pages) updated continuously and queried in parallel; results merged.

Tail latency: with thousands of shards, one slow shard slows every query → hedged requests (send to a second replica after p95 delay), partial results if a shard misses the deadline.

6. Trade-offs and say this

Decision	Choice	Why
Sharding	By document	Independent shards, simple merge
Ranking	Multi-stage (cheap → expensive)	Latency budget
Freshness	Batch main index + real-time tier	Cost vs freshness
Tail latency	Hedged requests + partial results	Fan-out amplifies slow shards

Follow-ups: "Product search differences?" → structured filters (facets), inventory/price freshness, conversion-optimized ranking. "How do you evaluate ranking?" → human-rated relevance sets (NDCG) + online A/B tests on click metrics.

Say this: "Cheap ranking everywhere, expensive ranking on few." · "With thousands of shards, the slowest shard is your latency — hedge requests."

Q39. Design Top-K / trending (heavy hitters) — "top 100 songs/videos/hashtags in the last hour"

> Classic · Google (YouTube), Amazon (best sellers), Meta/Twitter (trends), Spotify

1. The prompt

"Return the top K most-viewed items in the last minute, hour and day." Variant: trending (rate of change rather than raw count).

2. Clarify and state

Ask: "Event volume?" · "Exact or approximate?" · "Windows: sliding or tumbling?" · "Global or per region/category?" · "Latency of results?"

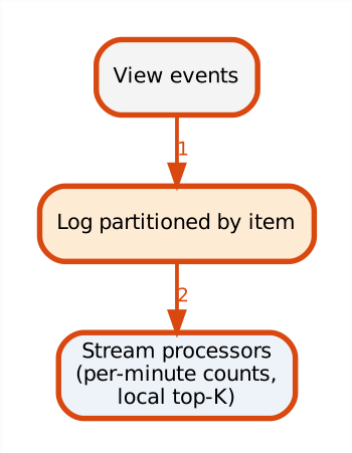
State: "1M events/s, K = 100, windows of 1 min / 1 h / 24 h, approximate fine for 1 min/1 h, exact for daily reports; per region; results within ~1 minute."

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

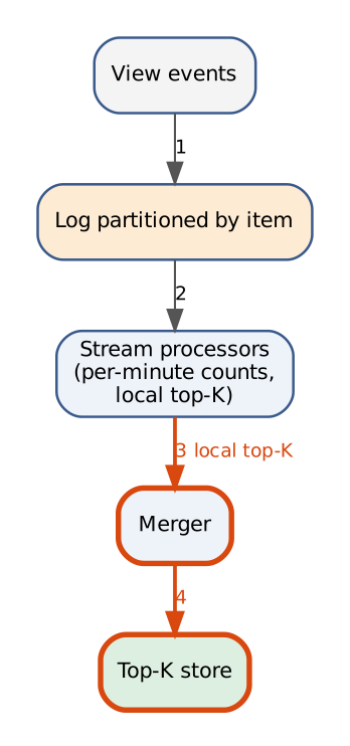
Step 1 · Partitioned counting (1 min)



> **You:** “Events are partitioned by item ID so each item's count lives in one processor, which keeps per-minute counts and a local top-K.”

Why this step now: Correctness comes from partitioning.

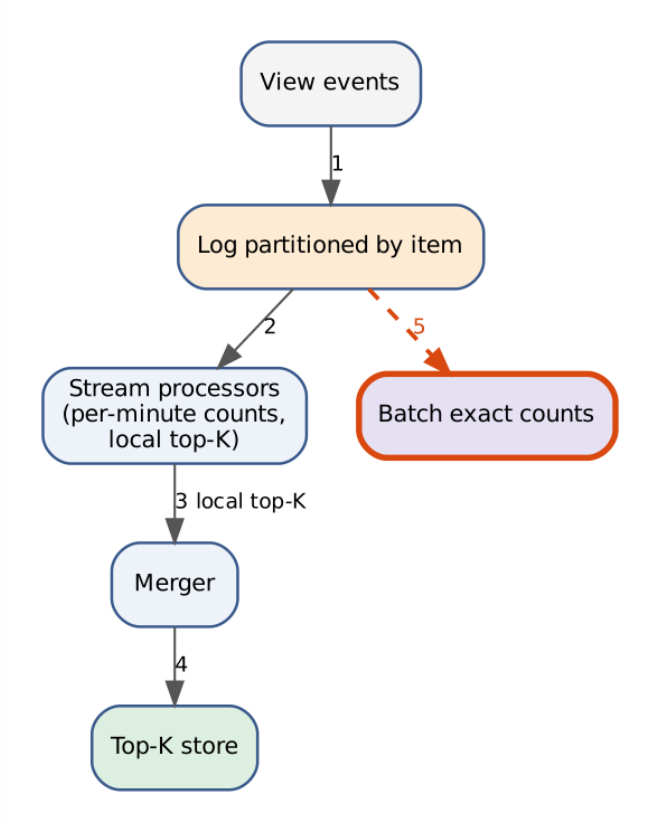
Step 2 · Global merge (1 min)



> **You:** “A merger combines local top-Ks into the global top-K per window; hour and day results come from merging minute aggregates.”

Why this step now: Serving results.

Step 3 · Exact batch (30 s)



> **You:** “A batch job computes exact daily counts for reports.”

Why this step now: Accuracy where needed.

Narrate one request end to end (numbers match the arrows)

Count

1. Events (1) partitioned (2); local top-K per minute (3); merged (4); batch truth (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
View events	1M/s	—	—
Log partitioned by item	Each item's events in one place	Mergeable top-K	Kafka
Stream processors (per-minute counts, local top-K)	Count + local heap	Fast path	Flink
Merger	Global top-K per window	Union of local top-Ks is exact	Service
Top-K store	Results per window/ region	Serve reads	Redis

Batch exact counts	Daily truth	Reports	Spark

Before you leave the high-level design, check you have said:

- ☐ Partition by item
- ☐ Count-min sketch when needed
- ☐ Trending vs top

60-second summary to say once the diagram is complete:

> **You:** "Two paths. A fast path: stream processors partitioned by item ID count events in one-minute windows, keep a local top-K per partition, and a merger combines partition top-Ks into a global top-K per window; hour and day results are built by merging minute aggregates. A slow path: a batch job computes exact daily counts from the raw log for reports."

4. Deep dives

Deep dive 1: why partition by item ID. Each item's count lives on exactly one partition → local top-K per partition is correct for its items → global top-K = top-K of the union of partition top-Ks (correct because every item's full count is in one place).

Deep dive 2: memory at scale — count-min sketch + heap. If distinct items per window are too many for exact counting, use a count-min sketch (fixed memory, overestimates only) to estimate counts and a min-heap of size K for candidates.

```
import hashlib, heapq
class CountMinTopK:
    def __init__(self, width=2**16, depth=4, k=100):
        self.w, self.d, self.k = width, depth, k
        self.t = [[0]*width for _ in range(depth)]
        self.heap, self.inheap = [], {}          # (est_count, item)
    def _idx(self, item, i):
        h = hashlib.blake2b(f"{i}:{item}".encode(), digest_size=8).digest()
        return int.from_bytes(h, "big") % self.w
    def add(self, item, n=1):
        est = min(self._incr(item, i, n) for i in range(self.d))
        if item in self.inheap:
            self.inheap[item] = est
            self.heap = [(self.inheap[x], x) for _, x in self.heap];
heapq.heapify(self.heap)
        elif len(self.heap) < self.k:
            heapq.heappush(self.heap, (est, item)); self.inheap[item] = est
        elif est > self.heap[0][0]:
            _, old = heapq.heapreplace(self.heap, (est, item))
            del self.inheap[old]; self.inheap[item] = est
    def _incr(self, item, i, n):
        j = self._idx(item, i); self.t[i][j] += n; return self.t[i][j]
    def top(self):
        return sorted(self.heap, reverse=True)
```

(The heap re-build on update is O(K) — fine for K = 100; production code uses an indexed heap.)

Deep dive 3: trending ≠ top. Trending score = (count in last window) / (expected count from baseline, e.g., same hour last week), with a minimum volume threshold; otherwise perennial items always win.

5. Trade-offs and say this

Decision	Choice	Why
Exactness	Approximate real time + exact batch	Speed vs accuracy
Partitioning	By item	Makes local top-K mergeable
Long windows	Merge minute aggregates	No need to keep raw events in state

Follow-ups: "Top-K per country?" → key = (country, item); per-country heaps. "Skewed item (one viral video) overloads a partition?" → pre-aggregate at producers or split hot keys with salting and merge.

Say this: "Partition by item and the per-partition top-Ks merge correctly." · "Trending is surprise relative to a baseline, not raw volume."

Q40. Design a distributed message queue (Kafka)

> Classic · Amazon (SQS/Kinesis), Google ("build the queue yourself"), Microsoft (Event Hubs), Databricks, LinkedIn

1. The prompt

"Design a distributed, durable, high-throughput message queue / log like Kafka." Expect: partitions, replication, consumer groups, delivery semantics, retention.

2. Clarify and state

Ask: "Queue semantics (delete on consume) or log (retain and replay)?" · "Throughput and message size?" · "Ordering requirements?" · "Delivery guarantees?" · "Retention?"

State: "Log semantics with replay; 10 GB/s writes across topics; messages ~1 KB; ordering per partition key; at-least-once by default with an exactly-once option; retention 7 days."

3. Estimates

- 10 GB/s ≈ 10M msgs/s. At ~100 MB/s per broker disk/network budget → ~100 brokers for writes, ×3 replication on disk → 30 GB/s of disk writes total → ~300 brokers, or fewer with bigger nodes.
- Storage: 10 GB/s × 86,400 × 7 × 3 replicas ≈ **18 PB** — tiered storage (old segments to object storage) is attractive.

4. API

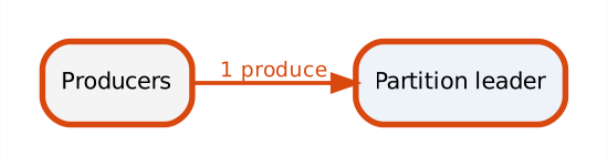
```
produce(topic, key, value, acks=all) → (partition, offset)
fetch(topic, partition, offset, max_bytes) → records
commit(group, topic, partition, offset)
Consumer group membership: join/heartbeat/rebalance
```

5. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

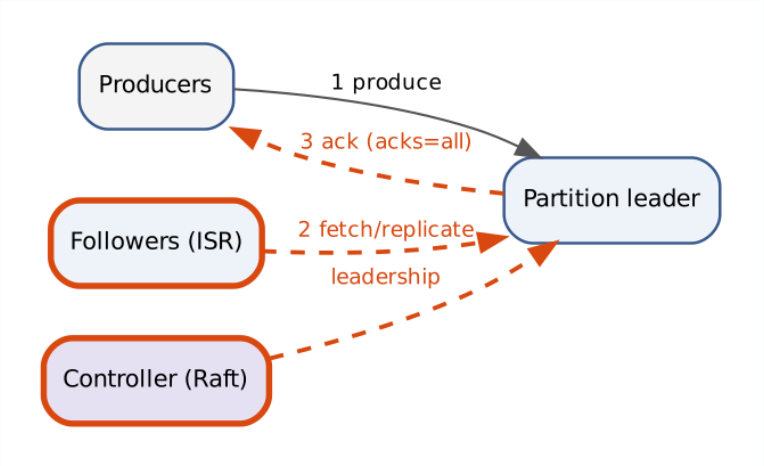
Step 1 · Append (1 min)



> **You:** “Producers hash the key to a partition and append to its leader, which writes sequentially to segment files.”

Why this step now: Simplest log.

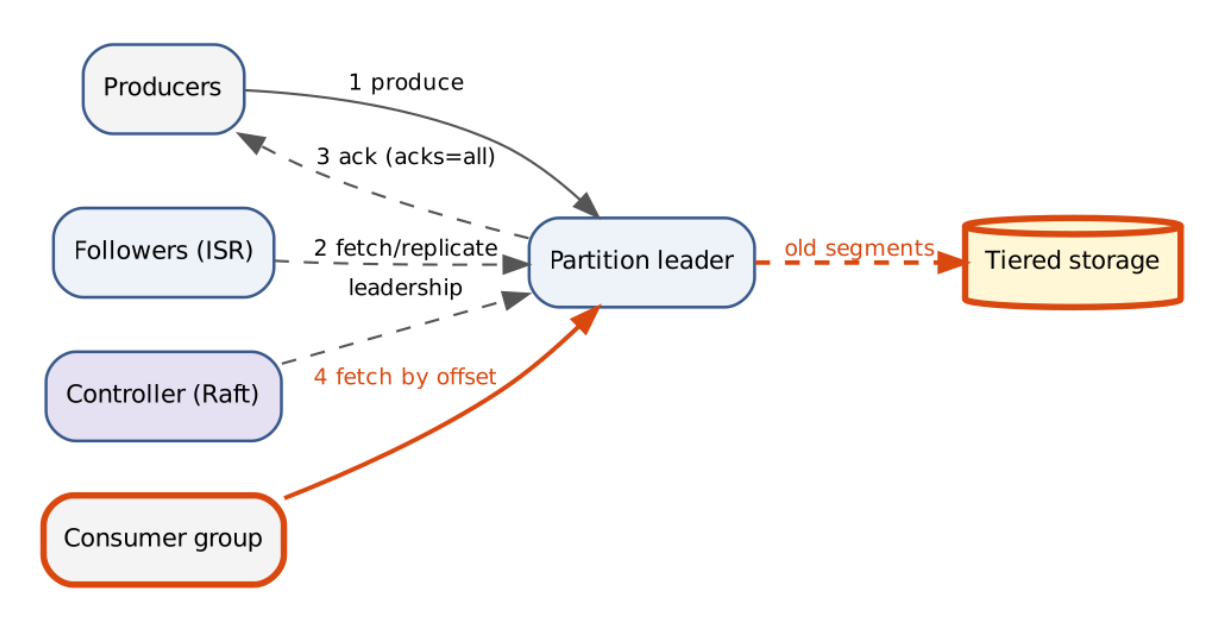
Step 2 · Replication (1–2 min)



> **You:** “Followers fetch from the leader; a record is committed once all in-sync replicas have it; a Raft controller manages leadership.”

Why this step now: Durability.

Step 3 · Consumption and retention (1 min)



> **You:** "Consumer groups split partitions among members and commit offsets; old segments move to object storage."

Why this step now: Readers and cost.

Narrate one request end to end (numbers match the arrows)

Message

1. Produce (1); replicate (2); ack after ISR (3); consume by offset (4).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Producers	Write by key	—	Clients
Partition leader	Append to segment log	Ordering per partition	Broker
Followers (ISR)	Replicate	Durability	Brokers
Controller (Raft)	Metadata, leader election	No split brain	Quorum
Consumer group	Read by offset, commit	Parallelism = partitions	Clients
Tiered storage	Old segments	PB-scale cost	Object storage

Before you leave the high-level design, check you have said:

☐ Partition = unit of order/parallelism
☐ ISR + acks=all
☐ Delivery semantics

60-second summary to say once the diagram is complete:

> **You:** "A topic is split into partitions; each partition is an append-only log stored as segment files, replicated to three brokers with one leader. Producers hash the key to a partition and write to its leader; the leader appends and followers fetch from it. Once all in-sync replicas have the record, it's committed. Consumers in a group each own some partitions, read sequentially by offset, and commit offsets. A controller — a Raft quorum — manages metadata: which broker leads which partition."

6. Deep dives

Deep dive 1: storage design (why it's fast). Sequential appends to segment files; reads are sequential too; OS page cache + zero-copy (sendfile) to serve consumers; sparse offset index per segment (offset → file position); retention by deleting whole old segments; log compaction for keyed topics keeps the latest value per key.

Deep dive 2: replication and durability. Leader + followers; in-sync replica set (ISR) = followers caught up within a lag bound. `acks=all` + `min.insync.replicas=2` → a write survives one broker loss. High-watermark = last offset replicated to all ISR; consumers read only up to it. Leader failure → controller elects a new leader from the ISR (unclean election off to avoid data loss).

Deep dive 3: delivery semantics. At-least-once: commit offsets after processing. Idempotent producers (producer ID + sequence numbers) remove duplicate writes on retry. Exactly-once processing: transactions that atomically write outputs and commit input offsets.

Deep dive 4: consumer groups. Partitions are the unit of parallelism: max consumers per group = partitions. Rebalancing reassigns partitions on join/leave (cooperative rebalancing avoids stop-the-world).

7. Trade-offs and say this

Decision	Choice	Why
Ordering	Per partition only	Global order kills throughput
Durability vs latency	acks=all, ISR ≥ 2	No loss on single failure
Storage	Local segments + tiered to object storage	Cost at PB scale
Metadata	Raft controller quorum	No external ZooKeeper dependency

Follow-ups: "Queue vs log?" → SQS deletes on consume, competing consumers, per-message visibility timeout; Kafka retains and lets many groups replay. "Hot partition?" → better keys, more partitions, or salting with ordering trade-off.

Say this: "Partitions are the unit of order, parallelism and replication." · "Sequential I/O plus the page cache is why a log is fast."

Q41. Design a distributed unique ID generator

> Classic · Twitter Snowflake pattern; asked at Amazon, Google, Meta, Microsoft, Uber

1. The prompt

"Generate unique 64-bit IDs at 1M+ IDs/s across many machines, roughly time-ordered."

2. Clarify and state

Ask: "Size limit (64-bit)?" · "Must IDs be sortable by time?" · "Strictly increasing or roughly?" · "Throughput?" · "Can IDs be guessable?"

State: "64-bit numeric, roughly time-sortable, unique across ~1,000 machines, 1M+ IDs/s total, no central bottleneck."

3. Options

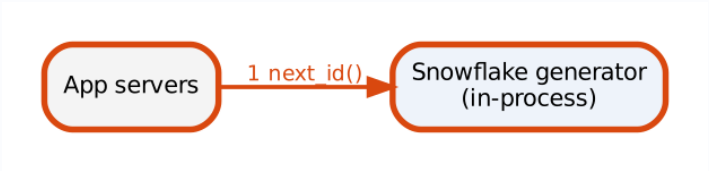
Option	Pros	Cons
DB auto-increment	Simple, strictly ordered	Bottleneck, single point of failure
Multi-master DB with step (server k uses k, k+N, …)	Simple scale-out	Hard to add servers; not time-ordered
UUIDv4 (128-bit)	No coordination	128 bits, not sortable, poor index locality
UUIDv7 / ULID	Sortable, no coordination	128 bits
Range/block allocation	Fast, compact	Not time-ordered; gaps on crash
Snowflake	64-bit, time-ordered, no per-ID coordination	Needs unique worker IDs, clock discipline

4. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

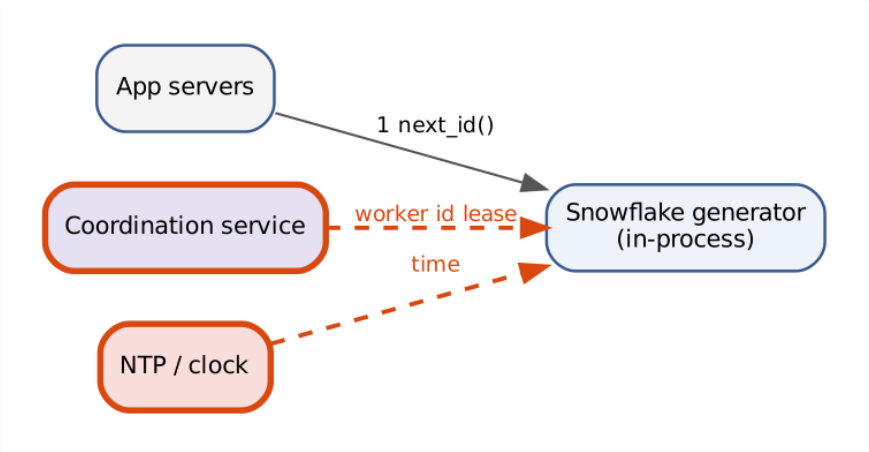
Step 1 · Local generation (30 s)



> **You:** “Each app server generates IDs in-process: 41 bits of milliseconds, 10 bits of worker ID, 12 bits of sequence.”

Why this step now: No coordination per ID.

Step 2 · Uniqueness guarantees (1 min)



> **You:** “Worker IDs are leased from a coordination service; clocks are NTP-disciplined, and the generator refuses to issue if the clock moves backwards.”

Why this step now: Where duplicates could come from.

Narrate one request end to end (numbers match the arrows)

ID

1. App calls next_id (1): timestamp, worker, sequence combined.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
App servers	Need IDs	—	Services
Snowflake generator (in-process)	time | worker | seq	No network per ID	Library
Coordination service	Lease worker IDs	Unique worker IDs	etcd / ZooKeeper
NTP / clock	Time	Clock discipline	NTP

Before you leave the high-level design, check you have said:

☐ Bit layout math

☐ Worker ID leasing
☐ Clock going backwards

60-second summary to say once the diagram is complete:

> **You:** "Snowflake layout: 1 sign bit, 41 bits of milliseconds since a custom epoch — that's about 69 years — 10 bits of worker ID for 1,024 workers, and 12 bits of sequence, so 4,096 IDs per millisecond per worker. Worker IDs are leased from a coordination service at startup. Generating an ID is purely local."

```
import threading, time
class Snowflake:
    EPOCH = 1_700_000_000_000          # custom epoch (ms)
    def __init__(self, worker_id: int):
        assert 0 <= worker_id < 1024
        self.worker, self.seq, self.last = worker_id, 0, -1
        self.lock = threading.Lock()
    def _now(self): return int(time.time() * 1000)
    def next_id(self) -> int:
        with self.lock:
            ts = self._now()
            if ts < self.last:                      # clock moved backwards
                raise RuntimeError("clock moved backwards; refuse to issue")
            if ts == self.last:
                self.seq = (self.seq + 1) & 0xFFF
                if self.seq == 0:                   # 4096 used this ms: wait for next ms
                    while ts <= self.last: ts = self._now()
            else:
                self.seq = 0
            self.last = ts
            return ((ts - self.EPOCH) << 22) | (self.worker << 12) | self.seq
```

5. Deep dives and say this

- **Clock skew:** NTP can move clocks backwards → refuse to issue (or wait) until time catches up; alert. Monotonic clocks can't be used directly because they aren't comparable across machines.
- **Worker ID assignment:** lease from ZooKeeper/etcd with TTL; a restarted node can't reuse an ID still leased (prevents duplicates).
- **Ordering:** IDs are only roughly ordered across machines (clock differences), strictly within one worker.

Follow-ups: "Need more workers?" → rebalance bits (e.g., 12 worker bits, 10 sequence bits). "Hide volume from competitors?" → don't expose raw IDs publicly, or encrypt them.

Say this: "41 bits of time, 10 of worker, 12 of sequence — 4,096 IDs per millisecond per worker, no coordination per ID." · "If the clock goes backwards, refuse to issue rather than risk a duplicate."

Q42. Design a real-time gaming leaderboard

> Classic · Amazon (gaming/Twitch), Microsoft (Xbox), Meta, Apple (Game Center)

1. The prompt

"Design a leaderboard showing global top 100 and each player's rank, updated in real time." Variants: weekly/monthly boards, friends leaderboard, per-region.

2. Clarify and state

Ask: "Players and score update rate?" · "Global rank for every player or just top N?" · "Ties?" · "Time-bounded boards (weekly)?" · "Anti-cheat?"

State: "50M monthly active players, 5M daily; 2k score updates/s avg, 10k peak; show top 100 and any player's exact rank; ties broken by earlier time; monthly boards; server-validated scores."

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

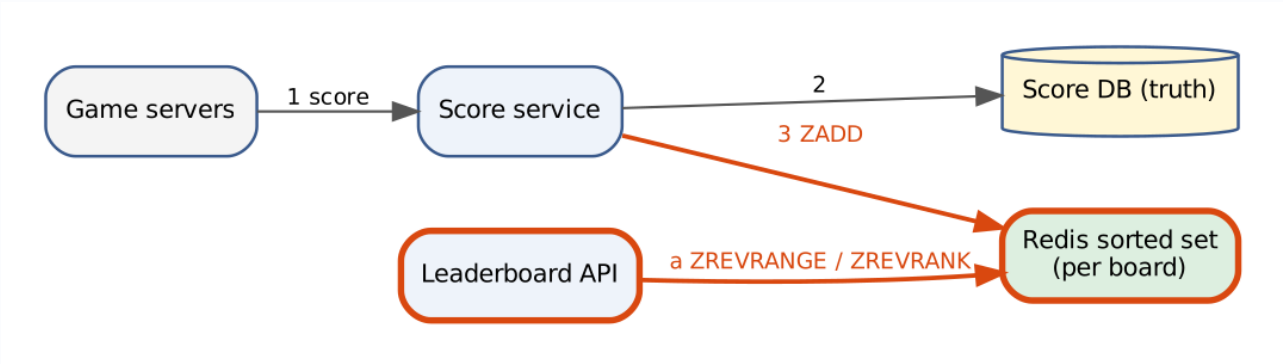
Step 1 · Durable scores (30 s)



> **You:** “Game servers report validated scores to a score service that persists them.”

Why this step now: Truth first.

Step 2 · Ranking (1 min)



> **You:** "The score service updates a Redis sorted set per leaderboard; the API reads top 100 and a player's rank in logarithmic time."

Why this step now: The data structure is the answer.

Narrate one request end to end (numbers match the arrows)

Update

1. Score (1) persisted (2) and added to ZSET (3).

Read

1. Top-N / rank from ZSET (a).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Game servers	Validated scores	Server-authoritative	—
Score service	Validate, write	—	Service
Score DB (truth)	Score history	Rebuild source	SQL
Redis sorted set (per board)	Rank + top-N in log time	Fast ranking	Redis
Leaderboard API	Top 100 (cached 1 s), my rank	—	Service

Before you leave the high-level design, check you have said:

☐ Why sorted set
☐ Ties
☐ Scaling beyond one node

60-second summary to say once the diagram is complete:

> **You:** "Game servers report validated scores to a score service, which writes the score history durably and updates a Redis sorted set per leaderboard. Sorted sets give top-N and a player's

> rank in logarithmic time. Reads for top 100 are cached for a second; rank lookups go straight to the sorted set."

4. Deep dives

Deep dive 1: why a sorted set. Redis ZSET = skip list + hash: `ZINCRBY` / `ZADD` O(log n), `ZREVRANK` O(log n), `ZREVRANGE 0 99` O(log n + 100). 50M members × ~100 B ≈ 5 GB → fits in one Redis instance (replicated). SQL `ORDER BY score` with `COUNT(*) WHERE score >` for rank is O(n) per query — too slow at this scale.

Deep dive 2: ties. Encode composite score = score × 10^{10} + (MAX_TS − timestamp) so higher score wins and earlier achievement breaks ties, within float precision limits (or keep score in ZSET and resolve ties in a secondary step).

Deep dive 3: scaling beyond one node. If players grow 100× (5B entries): shard by score range (each shard holds a score band; rank = sum of counts in higher shards + local rank) — requires rebalancing as distributions shift; or approximate ranks for the long tail ("top 12%") using histograms, exact for top N.

Durability: ZSET is a cache of the truth; scores persisted in a database; rebuild leaderboard from DB on failure; monthly boards = new key per month, old ones archived.

5. Trade-offs and say this

Decision	Choice	Why
Ranking structure	Redis sorted set	O(log n) rank and top-N
Truth	Durable DB; ZSET rebuildable	Cache can fail
Huge scale	Score-range sharding or approximate tail ranks	Exact rank for everyone is expensive

Follow-ups: "Friends leaderboard?" → fetch friends' scores (few hundred) and sort on read. "Cheaters?" → server-authoritative scores, anomaly detection, remove and recompute.

Say this: "A sorted set gives me rank and top-N in log time." · "The sorted set is a cache — the database is the truth."

Part III.4 — Finance, real-time media, AI and messaging

Q43. Design a stock exchange (order matching) — also Robinhood (brokerage)

> Classic · Amazon, Google, Meta, fintech; Robinhood variant on Hello Interview

1. The prompt

"Design a stock exchange that matches buy and sell orders." Variant (Robinhood): a brokerage app — users place orders that the broker routes to exchanges, with real-time quotes and portfolio views.

2. Clarify and state

Ask: "Exchange or brokerage?" · "Order types — limit, market, cancel?" · "Throughput and latency targets?" · "Number of symbols?" · "Market data distribution?" · "Fairness guarantees?"

State: "Exchange: limit, market and cancel orders; 10k symbols; peak 1M orders/s across all symbols; matching latency in microseconds to low milliseconds; strict price-time priority; market data to many subscribers; full audit trail."

3. Requirements

- **Must:** accept, validate and risk-check orders; match by price-time priority; publish trades and order book updates; cancel; daily settlement handoff.
- **Non-functional:** deterministic, fair ordering; no lost or duplicated orders; very low and predictable latency; recover exactly after crashes.

4. Estimates

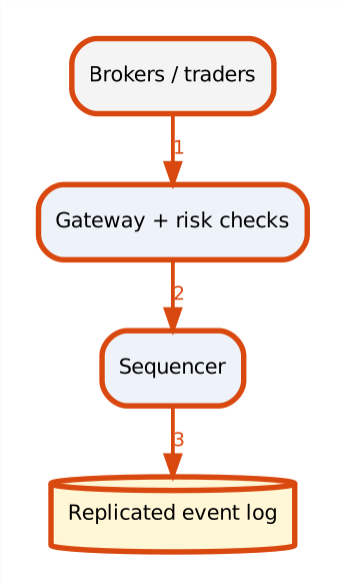
- 1M orders/s overall, but per symbol far less; the hottest symbol might see ~50k orders/s → single-threaded matching per symbol can handle this in memory.
- Event log: 1M × 200 B = 200 MB/s; 17 TB/day.

5. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

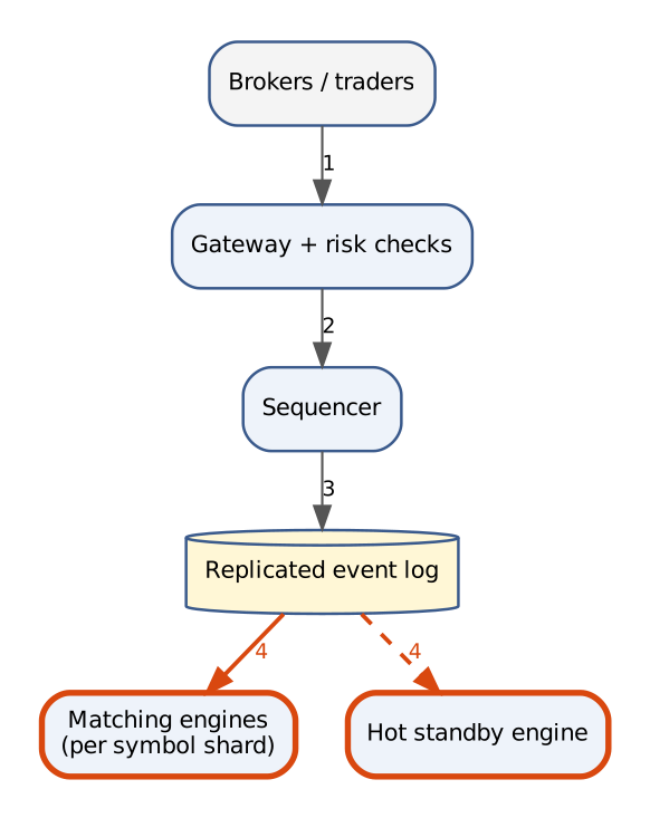
Step 1 · Sequence everything (1 min)



> **You:** “Orders pass risk checks and a sequencer stamps each with a global sequence number and writes it to a replicated log.”

Why this step now: Determinism first.

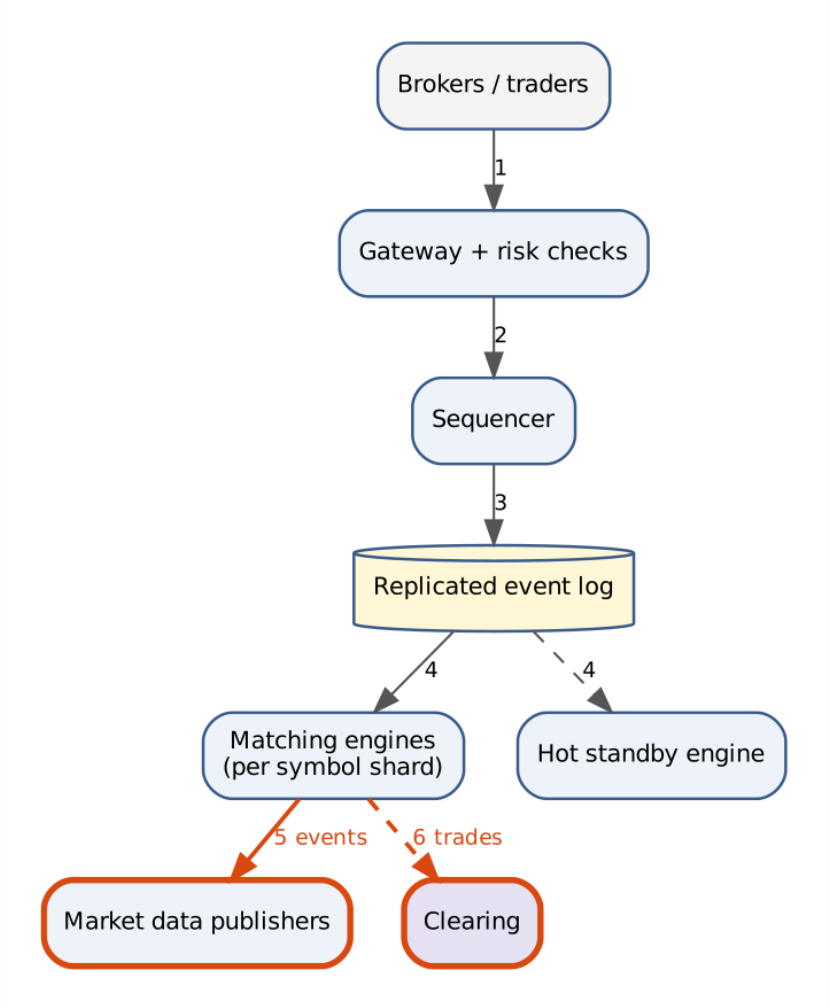
Step 2 · Match (1–2 min)



> **You:** “Matching engines per symbol consume the sequenced log and match against in-memory order books by price-time priority; a hot standby consumes the same log.”

Why this step now: Core engine.

Step 3 · Publish (30 s)



> **You:** “Trades and book changes go to market data feeds and clearing.”

Why this step now: Outputs.

Narrate one request end to end (numbers match the arrows)

Order

1. Order in (1), risk-checked (2), sequenced and logged (3).
2. Engine matches (4); events published (5) and cleared (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Brokers / traders	Orders	—	FIX/API
Gateway + risk checks	Auth, pre-trade risk	—	Service
Sequencer	Global sequence number	Determinism, fairness	Single-writer

Replicated event log	Sequenced orders	Recovery by replay	Log
Matching engines (per symbol shard)	In-memory order books	Microseconds	Single-threaded
Hot standby engine	Same input, same state	Instant failover	Replica
Market data publishers	Trades, book updates	Fan-out	Multicast/WS
Clearing	Settlement	Post-trade	Batch

Before you leave the high-level design, check you have said:

☐ Sequencer + log determinism
☐ In-memory book
☐ Replay recovery

60-second summary to say once the diagram is complete:

> **You:** "Orders arrive through a gateway that authenticates brokers and runs pre-trade risk checks. A **sequencer** stamps every inbound order with a global sequence number and writes it to a replicated event log — that's what makes the system deterministic. Matching engines, one per symbol shard, consume the sequenced stream and match against an in-memory order book. Every trade and book change is itself an event; market data publishers fan them out, and a clearing system consumes trades for settlement."

6. Deep dives

Deep dive 1: the order book. Per symbol: bids sorted by price descending, asks ascending; each price level holds a FIFO queue of orders (price-time priority). Data structure: sorted map of price → linked list, plus hash map order_id → node for O(1) cancels. A new buy limit order matches against the best asks while price crosses, then rests the remainder.

```
from collections import deque
import bisect

class Book:
    def __init__(self):
        self.bids, self.asks = {}, {}          # price -> deque of [order_id, qty]
        self.bid_px, self.ask_px = [], []      # sorted price lists (bids ascending; best =
last)
    def _level(self, side, px):
        book, pxs = (self.bids, self.bid_px) if side == "B" else (self.asks, self.ask_px)
        if px not in book:
            book[px] = deque(); bisect.insort(pxs, px)
        return book[px]
    def limit(self, oid, side, px, qty):
        trades = []
        opp, opp_px = (self.asks, self.ask_px) if side == "B" else (self.bids, self.bid_px)
        while qty and opp_px:
            best = opp_px[0] if side == "B" else opp_px[-1]
            if (side == "B" and best > px) or (side == "S" and best < px): break
            q = opp[best]
            while qty and q:
                o = q[0]; fill = min(qty, o[1])
                trades.append((oid, o[0], best, fill))
                qty -= fill; o[1] -= fill
                if o[1] == 0: q.popleft()
            if not q:
                del opp[best]; opp_px.remove(best)
        if qty: self._level(side, px).append([oid, qty])
        return trades
```

Deep dive 2: determinism and recovery. Because matching is a pure function of the sequenced input, a hot standby engine consuming the same log produces identical state; failover is instant. Recovery = latest snapshot + replay from the log. This is the classic event-sourcing / state-machine-replication design.

Deep dive 3: latency. Single-threaded engine per symbol shard (no locks), pinned cores, preallocated memory, kernel-bypass networking at real exchanges. For an interview, the key point is: no database or network round trip on the matching path; durability comes from the sequenced log written before matching.

Robinhood variant: user orders → brokerage order service (balance/risk checks, idempotency) → smart order router to exchanges/market makers via FIX; fills update positions and cash in a ledger; real-time quotes fanned out over WebSockets with per-symbol subscriptions; market-open spikes require queueing and autoscaling.

7. Trade-offs and say this

Decision	Choice	Why
Ordering	Central sequencer + replicated log	Fairness and determinism
Matching	In-memory, single-threaded per symbol	Microsecond latency, no locks

Decision	Choice	Why
Recovery	Snapshot + replay; hot standby	Exact state reconstruction

Follow-ups: "Sequencer is a bottleneck?" → partition sequencing by symbol group. "Market data to 1M subscribers?" → multicast within exchange; tiered distributors/CDN-like fan-out externally.

Say this: "Sequence first, then match — the log makes the system deterministic." · "The order book lives in memory; durability lives in the log."

Q44. Design Zoom (video conferencing)

> Classic · Microsoft (Teams), Google (Meet), Apple (FaceTime), Amazon (Chime)

1. The prompt

"Design a video conferencing system supporting meetings of 2 to 1,000 participants." Variants: webinars (1 → 10k), recording, screen share.

2. Clarify and state

Ask: "Meeting sizes?" · "Latency target?" · "Recording?" · "E2E encryption?" · "Global?"

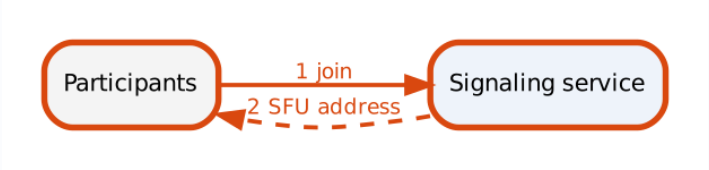
State: "Meetings up to 300 interactive participants; mouth-to-ear latency < 200 ms; recording optional; global; encryption in transit, E2E optional for small meetings."

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

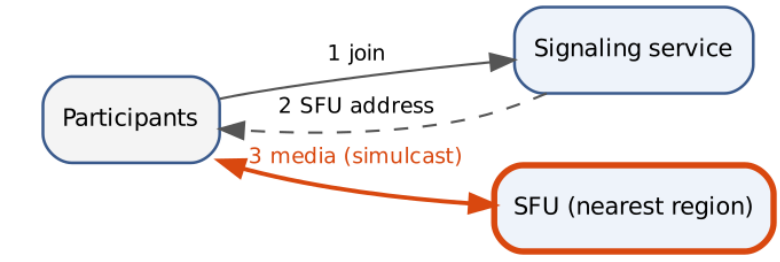
Step 1 · Signaling (30 s)



> **You:** “Clients join through a signaling service that authenticates and assigns the nearest media server.”

Why this step now: Control vs media split.

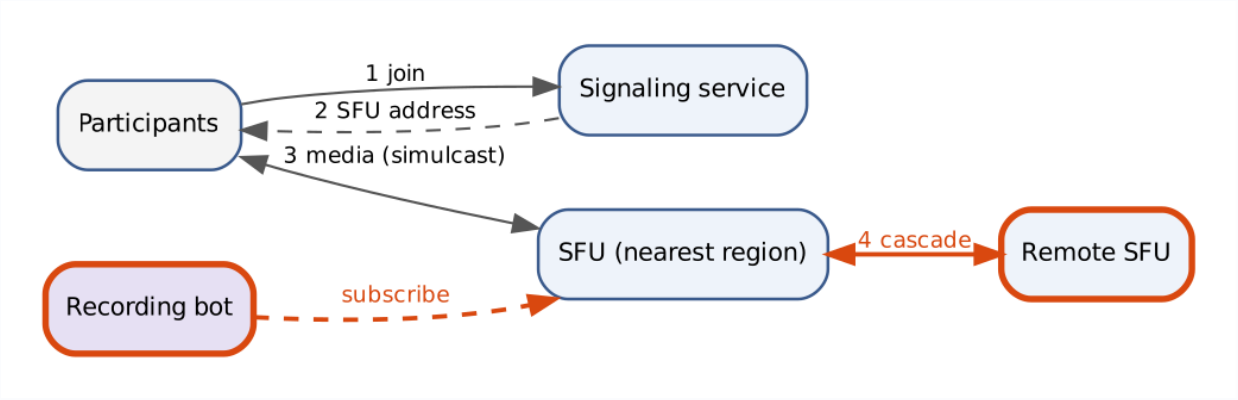
Step 2 · SFU media (1–2 min)



> **You:** “Each client sends simulcast streams once to the SFU, which forwards the right quality to each receiver.”

Why this step now: Why it scales.

Step 3 · Global and recording (1 min)



> **You:** “SFUs cascade across regions; a recording bot joins as a participant.”

Why this step now: Extensions.

Narrate one request end to end (numbers match the arrows)

Join call

1. Join (1), get SFU (2), exchange media (3); cross-region via cascade (4).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Participants	Send/receive media	—	WebRTC clients
Signaling service	Join, auth, assign SFU	Setup only	Service + WS
SFU (nearest region)	Forward streams without decoding	Scales to hundreds	Media servers
Remote SFU	Other region	Global meetings	Cascaded
Recording bot	Join, compose, store	Async recording	Service

Before you leave the high-level design, check you have said:

☐ Mesh vs MCU vs SFU
☐ Simulcast
☐ Audio priority

60-second summary to say once the diagram is complete:

> **You:** "Signaling and media are separate. Clients join through a signaling service that authenticates, finds the meeting, and assigns the nearest media server — an SFU, a selective

> forwarding unit. Each client sends its audio and video once to the SFU, which forwards streams to the others without decoding them. Clients send multiple video qualities (simulcast) and the SFU forwards the right one per receiver based on their bandwidth and layout."

4. Deep dives

Deep dive 1: mesh vs MCU vs SFU.

Topology	How	Fits
Mesh (P2P)	Everyone sends to everyone	2–4 people; upload explodes after that
MCU	Server decodes, mixes, re-encodes one stream	Low client cost; very expensive server CPU, adds latency
SFU	Server forwards selected streams	Scales to hundreds; low latency; standard choice

Deep dive 2: network adaptation. UDP/RTP (WebRTC) for media; jitter buffers; forward error correction; simulcast or SVC layers; bandwidth estimation; only forward video for active speakers and visible tiles; audio prioritized over video.

Deep dive 3: large and global meetings. Cascade SFUs across regions: participants connect to the nearest SFU; SFUs exchange streams over the backbone. Webinars: few senders, many receivers → SFU tree or switch to HLS-style streaming for very large audiences (higher latency is acceptable).

Recording: a recording bot joins as a participant, receives streams, composes and stores them; process asynchronously.

5. Estimates, trade-offs, say this

- 720p video ~1.5 Mbps; a 10-person meeting sends 10 uplinks and ~90 downlink streams of mixed quality through the SFU → bandwidth, not CPU, is the SFU's main limit.

Say this: "Signaling sets up the call; media never touches the signaling path." · "SFUs forward without decoding — that's why they scale." · "Audio first: people forgive frozen video, not broken audio."

Q45. Design ChatGPT (an LLM chat service)

> Classic (newer) · Hello Interview "ChatGPT"; relevant at Microsoft (Copilot), Google, Meta, Amazon, Apple, Databricks

1. The prompt

"Design a ChatGPT-like service: users chat with an LLM, responses stream token by token, conversations persist."

2. Clarify and state

Ask: "Users and messages/day?" · "Models — one or several sizes?" · "Streaming responses?" · "Conversation history length?" · "Tools/RAG/file uploads?" · "Free vs paid tiers and rate limits?" · "Safety requirements?"

State: "50M DAU, 1B messages/day, several model sizes, streaming, history persisted, file uploads with retrieval, free and paid tiers with quotas, safety filtering."

3. Estimates

- 1B messages/day ≈ **12k requests/s** average, peaks ~3× → ~35k/s.
- Tokens: ~1,500 input (with history) + 500 output per request → 35k × 500 = 17.5M output tokens/s at peak. If a GPU serves ~2,000 output tokens/s with batching for a mid-size model → **~9,000 GPUs** at peak for output alone (plus prefill). GPUs are the cost, so capacity, batching and routing dominate the design.

4. API

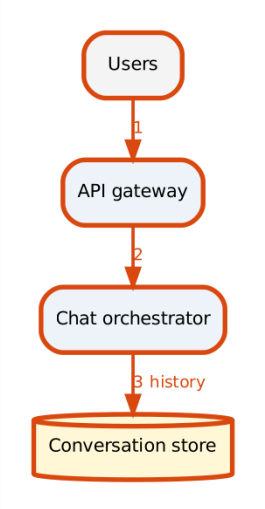
```
POST /v1/conversations/{id}/messages {content, attachments?, model}  → SSE stream of tokens
GET  /v1/conversations?cursor=..
GET  /v1/conversations/{id}
```

5. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

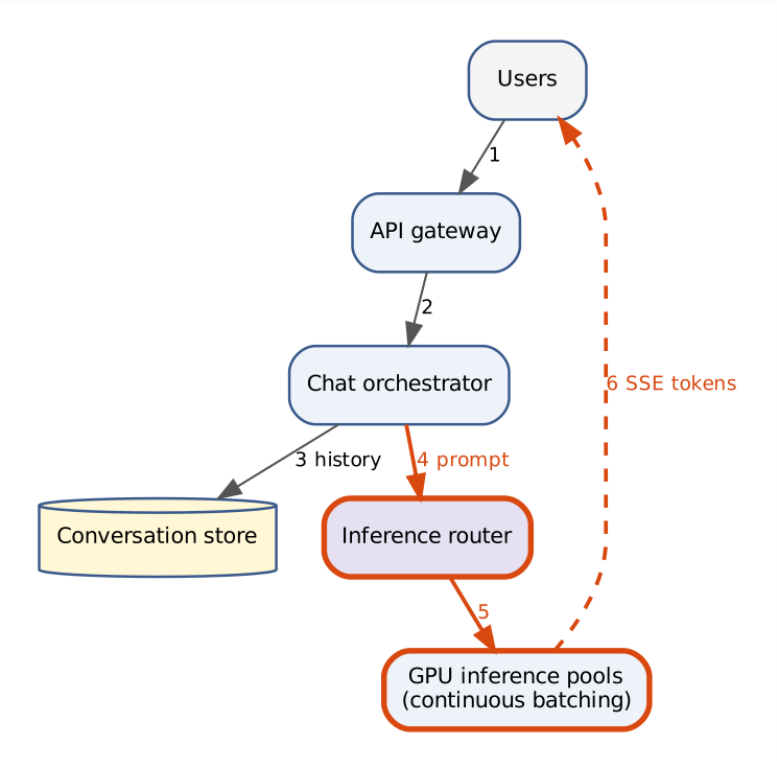
Step 1 · Request path (1 min)



> **You:** “Messages pass a gateway enforcing quotas; an orchestrator loads history and assembles the prompt; turns are persisted.”

Why this step now: Product flow.

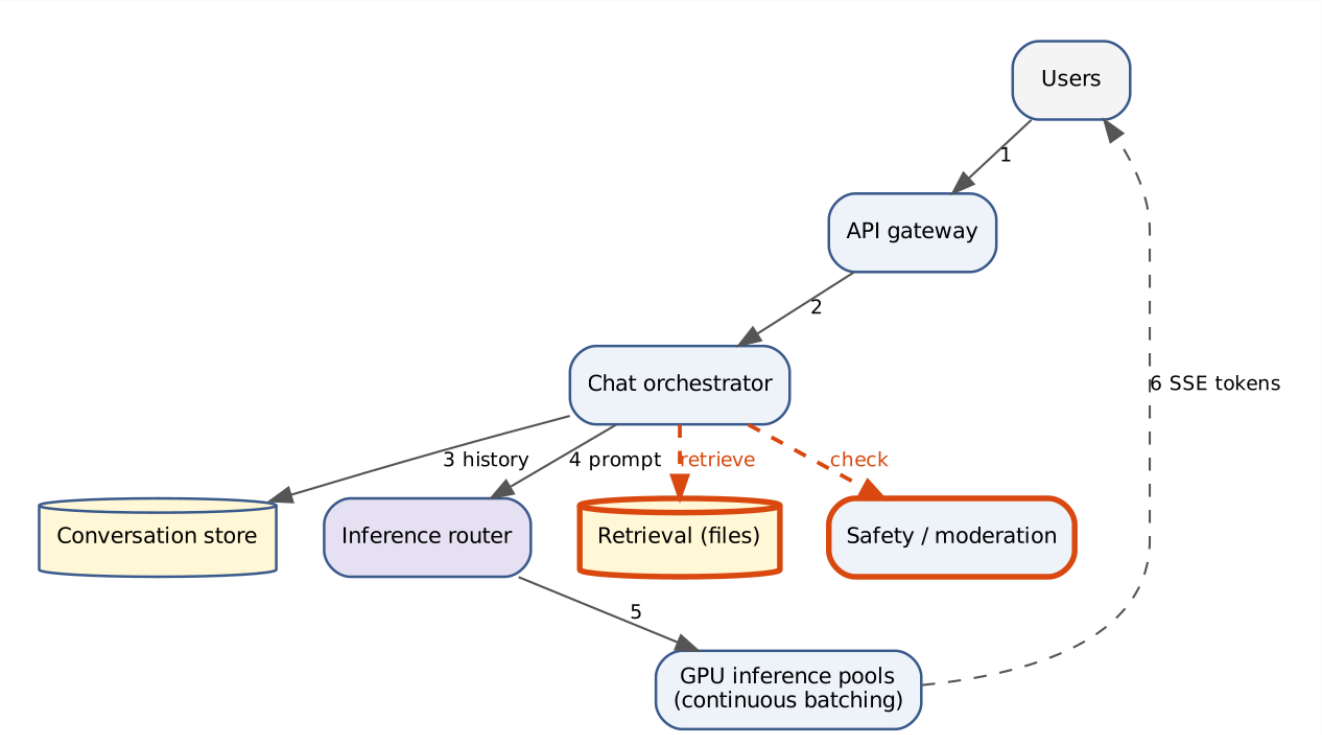
Step 2 · Inference (1–2 min)



> **You:** “An inference router chooses a model pool and replica with capacity; GPU servers with continuous batching stream tokens back over server-sent events.”

Why this step now: GPU math drives this.

Step 3 · Retrieval and safety (1 min)



> **You:** “Uploaded files are retrieved from a vector index; input and streaming output are checked by safety classifiers.”

Why this step now: Completeness.

Narrate one request end to end (numbers match the arrows)

Message

1. Request (1, 2); history (3); prompt to router (4) → GPU (5); tokens streamed (6); turn saved.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Users	Chat, stream tokens	—	Web/app
API gateway	Auth, tier quotas	GPU protection	Gateway
Chat orchestrator	History, safety, prompt assembly	Core logic	Service
Conversation store	Messages	History	Wide-column
Inference router	Pick model pool/replica	Capacity + cost	Service
GPU inference pools (continuous batching)	Generate tokens	The scarce resource	vLLM-like

Retrieval (files)	Vector index per user	Grounding	Vector DB
Safety / moderation	Input + streaming output checks	Policy	Classifiers

Before you leave the high-level design, check you have said:

☐ GPU capacity math
☐ Continuous batching
☐ Admission control / priority

60-second summary to say once the diagram is complete:

> **You:** "A user message goes through an API gateway that authenticates and enforces per-tier quotas. The chat orchestrator loads recent conversation history, runs input safety checks, optionally retrieves context from uploaded files, assembles the prompt, and sends it to an inference router. The router picks a model pool and GPU replica with capacity; the inference server streams tokens back, which we relay to the client over server-sent events while running streaming output moderation. When done, we persist the turn and update usage."

6. Deep dives

Deep dive 1: inference serving efficiency. Continuous batching (add new sequences each decoding step), paged KV-cache, prefix caching for shared system prompts and conversation history, quantization, speculative decoding. Separate prefill-heavy and decode-heavy work. Route small/cheap model for simple requests.

Deep dive 2: capacity and fairness. Per-tier quotas and token-bucket limits; priority queues (paid > free); admission control when GPUs are saturated (queue with ETA, or degrade to smaller model); autoscaling is slow (GPU nodes take minutes) → keep headroom and forecast peaks.

Deep dive 3: long conversations. Context window limits → summarize older turns, retrieve relevant earlier turns, cap history tokens; store full history in the database regardless.

Deep dive 4: safety. Input and output classifiers, policy-based refusal, abuse detection, audit logs; for tools, permission checks and sandboxing.

7. Data model, trade-offs, say this

- `Conversation(id, user_id, title, created_at)`; `Message(conv_id, seq, role, content, model, tokens)` in a wide-column store partitioned by conversation; attachments in object storage + vector index per user.

Decision	Choice	Why
Streaming	SSE	Simple, one-way, works through proxies

Decision	Choice	Why
Batching	Continuous	GPU utilization
Overload	Priority queues + model fallback	GPUs can't autoscale instantly

Say this: "The GPU is the scarce resource, so the design is about batching, routing and admission control." · "Stream tokens to users while moderation runs on the stream."

Q46. Design a recommendation system (Netflix / Amazon / YouTube recommendations)

> Classic · Netflix (reported in guides), Amazon, Meta, Google, Apple (App Store/Music)

1. The prompt

"Design the system that recommends items (videos/products) on a user's home page."

2. Clarify and state

Ask: "Users and items scale?" · "What is the objective — clicks, watch time, purchases, long-term satisfaction?" · "Freshness — how quickly should recent behavior change recommendations?" · "Cold start for new users/items?" · "Latency?"

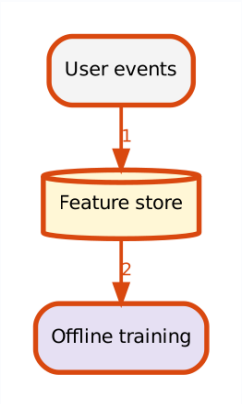
State: "200M users, 10M items, objective = long-term engagement with guardrails, recent behavior reflected within minutes, p99 < 150 ms for the home page."

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

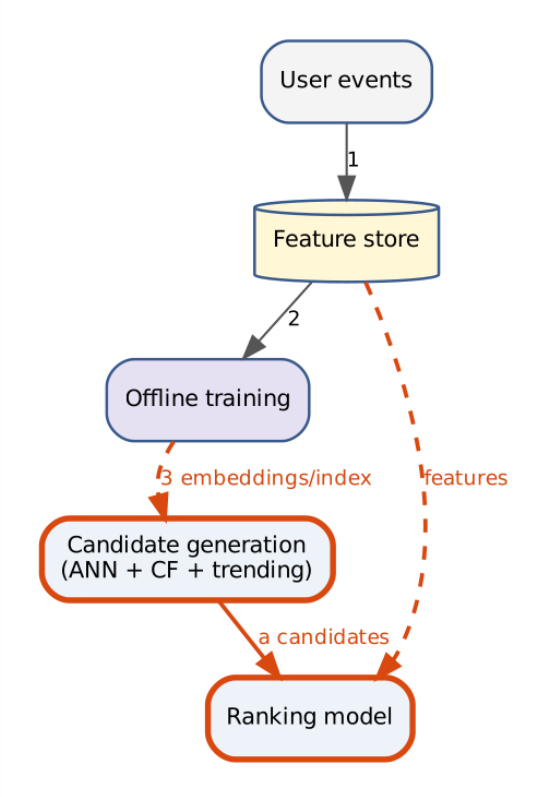
Step 1 · Data and training (1 min)



> **You:** “User events feed a feature store; offline training produces models and item embeddings.”

Why this step now: Foundation.

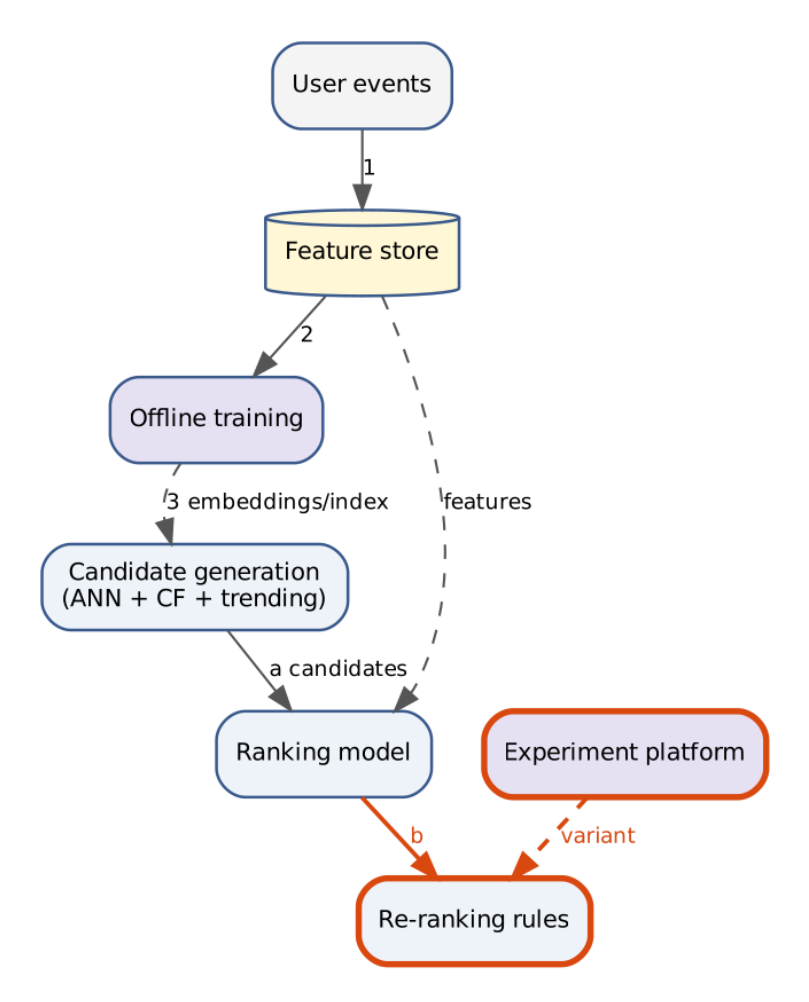
Step 2 · Serving funnel (1–2 min)



> **You:** “Candidate generation narrows millions to about a thousand with ANN, collaborative filtering and trending; the ranker scores them with rich features.”

Why this step now: Funnel.

Step 3 · Polish and measure (30 s)



> **You:** “Re-ranking applies diversity and filters; an experiment platform decides which model wins.”

Why this step now: Quality and evaluation.

Narrate one request end to end (numbers match the arrows)

Home page

1. Candidates (a) → rank → re-rank (b) → render; events logged (1).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User events	Views, clicks	Signal	Apps
Feature store	User/item features online+offline	Train/serve consistency	Feature store
Offline training	Models, embeddings	Learn	Spark/GPU

Candidate generation (ANN + CF + trending)	10M → ~1,000	Cheap recall	ANN index
Ranking model	Score candidates	Precision	Model server
Re-ranking rules	Diversity, filters	Product quality	Service
Experiment platform	A/B tests	Decide winners	Platform

Before you leave the high-level design, check you have said:

☐ Funnel
☐ Feature store consistency
☐ Online experiments

60-second summary to say once the diagram is complete:

> **You:** "Recommendations are a funnel. Candidate generation narrows 10 million items to about a thousand using several cheap sources — embedding similarity to the user, items similar to recent views, trending, follows. A ranking model scores those candidates with rich features from a feature store and predicts the objective. A re-ranking step applies business rules: diversity, freshness, already-watched filters. Offline, pipelines train models and build embedding indexes; online, events update user features in near real time."

4. Deep dives

Deep dive 1: candidate generation. Two-tower model: user tower and item tower produce embeddings; approximate nearest neighbor (ANN) index over item embeddings returns top candidates for a user embedding in milliseconds. Plus collaborative filtering ("people who watched X watched Y"), content-based for cold-start items, and popularity.

Deep dive 2: ranking and features. Gradient-boosted trees or deep models with features: user history, item stats, context (device, time), cross features. Feature store guarantees the same feature definitions in training (point-in-time correct) and serving (low latency).

Deep dive 3: evaluation. Offline metrics (recall@k, NDCG) are proxies; online A/B tests decide; guardrail metrics (diversity, complaints, long-term retention) prevent optimizing clickbait.

Cold start: new users → popularity + onboarding questions + contextual signals; new items → content embeddings and exploration traffic (bandits).

5. Trade-offs and say this

Decision	Choice	Why
Architecture	Multi-stage funnel	Latency vs model complexity

Decision	Choice	Why
Freshness	Streaming user features + daily retrain	Recent intent matters
Objective	Long-term engagement with guardrails	Avoid short-term clickbait

Say this: "Cheap retrieval over millions, expensive ranking over hundreds." · "Offline metrics pick candidates for A/B tests; A/B tests pick winners."

Q47. Design Gmail (email service)

> Classic · Google, Microsoft (Outlook), Apple (iCloud Mail), Amazon (SES)

1. The prompt

"Design an email service: send, receive, store, search, with spam filtering." Focus on receive path, storage and search.

2. Clarify and state

Ask: "Users and emails/day?" · "Storage per user?" · "Search requirements?" · "Attachments size?" · "Interoperability (SMTP with other providers)?"

State: "1.5B users, 100B emails/day received (half spam), 15 GB per user, full-text search, attachments up to 25 MB, standard SMTP interop."

3. Estimates

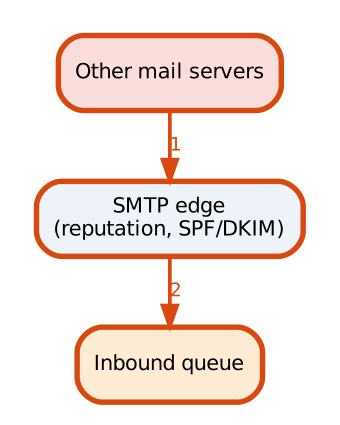
- 100B/day ≈ **1.2M emails/s** inbound (mostly spam rejected early).
- Storage: 1.5B × 15 GB quota (actual average maybe 2 GB) = ~3 EB actual → deduplicate attachments, compress, tier cold mail.

4. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

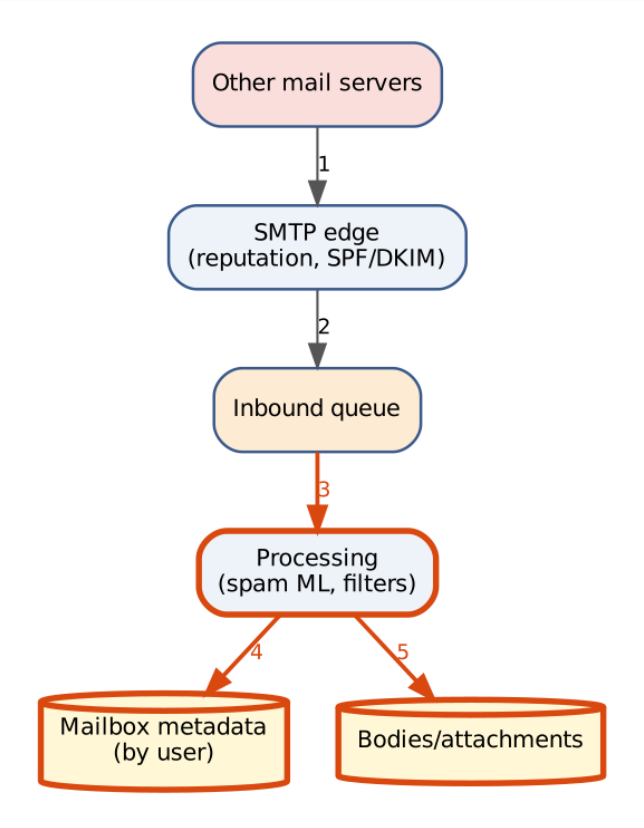
Step 1 · Inbound (1 min)



> **You:** “Mail arrives over SMTP at edge servers that check reputation and authentication and reject obvious spam, then queue the rest.”

Why this step now: Volume control.

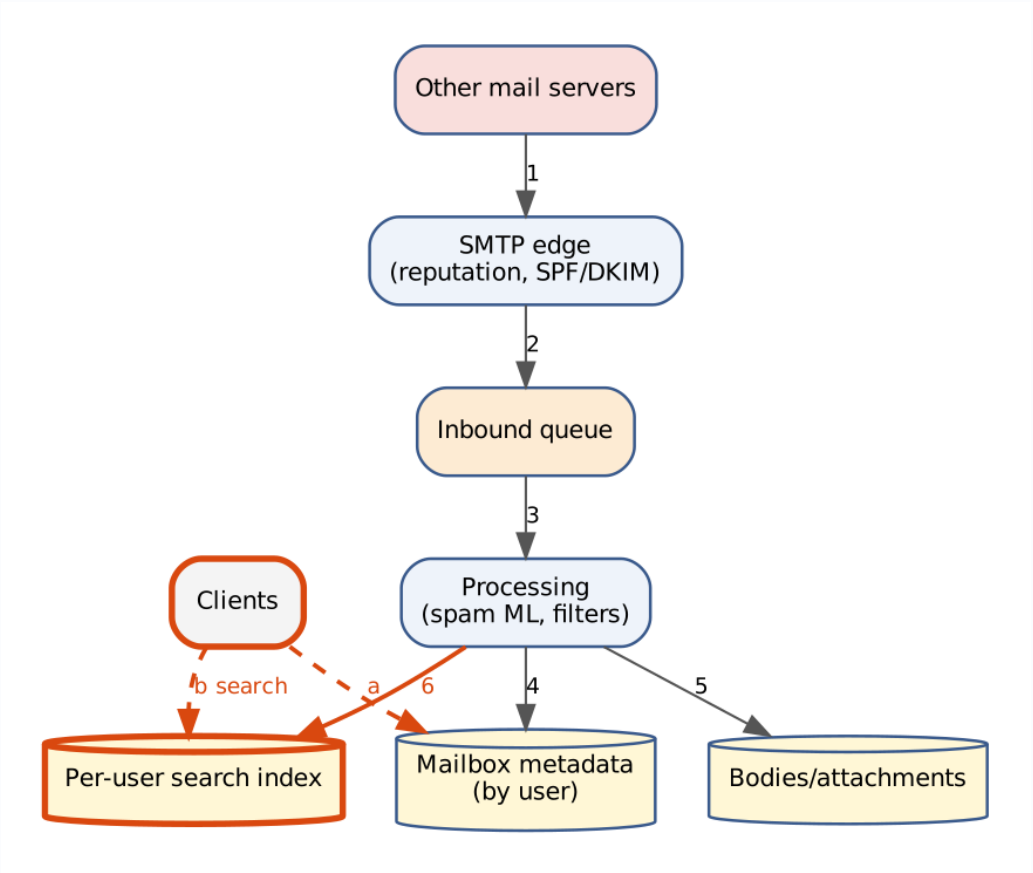
Step 2 · Process and store (1 min)



> **You:** “Workers classify spam, apply filters, store bodies in blob storage and metadata in a mailbox store partitioned by user.”

Why this step now: Core storage.

Step 3 · Search and clients (30 s)



> **You:** “A per-user index is updated for search; clients read mailboxes and search only their own index.”

Why this step now: Read path.

Narrate one request end to end (numbers match the arrows)

Receive

1. SMTP in (1), queued (2), processed (3), stored (4, 5), indexed (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Other mail servers	SMTP inbound	Interop	SMTP
SMTP edge (reputation, SPF/DKIM)	Reject spam early	Half of mail is spam	MTA fleet
Inbound queue	Buffer	Spikes	Queue
Processing (spam ML, filters)	Classify, label	—	Workers
Mailbox metadata (by user)	Labels, threads	—	Wide-column

Bodies/attachments	Content-addressed	Dedup	Blob storage
Per-user search index	Full text	Owner-only search	Index shards by user
Clients	Read, search	—	Web/app

Before you leave the high-level design, check you have said:

☐ Reject early
☐ Per-user index
☐ Metadata vs blob split

60-second summary to say once the diagram is complete:

> **You:** "Inbound mail arrives over SMTP at edge servers that check sender reputation and authentication (SPF, DKIM, DMARC) and reject obvious spam early. Accepted mail goes to a queue; processing workers run spam and malware classification, apply user filters, store the message body and attachments, write metadata to the mailbox store, update the per-user search index, and notify the user's clients. Sending goes the other way through outbound SMTP with retries and reputation management."

5. Deep dives

Deep dive 1: mailbox storage. Metadata (`user_id, thread_id, message_id, labels, read, date`) in a wide-column store partitioned by user (Bigtable-style), bodies and attachments in blob storage (content-addressed for dedup of common attachments). Labels instead of folders → a message can be in many views; label indexes per user.

Deep dive 2: per-user search. Each user's index is small and only they search it → per-user inverted index sharded by user, updated on arrival; queries never fan out across users.

Deep dive 3: spam. Layered: connection-level reputation, authentication checks, content ML models, user feedback ("report spam") as training signal, rate limits on outbound to protect sender reputation.

6. Trade-offs and say this

Decision	Choice	Why
Search index	Per user	Only owner searches; small, local
Storage	Metadata in wide-column, bodies in blob	Different access patterns
Spam	Reject early at SMTP edge	Most inbound volume is spam

Say this: "Half the inbound volume is spam, so the cheapest place to stop it is the SMTP edge." · "Search is per user, so it never fans out across users."

Part III.5 — Marketplaces, social and real-time apps

Q48. Design a local delivery service (Gopuff / Instacart-style)

> Classic · Hello Interview (Easy) · Amazon (Fresh/Now), Uber Eats, DoorDash

1. Prompt and clarify

"Design a service where customers order items from nearby micro-warehouses and get delivery in under 30 minutes."

Ask: "How many warehouses and items?" · "How is 'available near me' defined — which warehouses can reach me in time?" · "Order volume?" · "Inventory accuracy needs?" · "Do we design driver dispatch too?"

State: "500 warehouses, ~10k SKUs each, 1M orders/day, availability = items in stock at any warehouse that can deliver to you within 30 min, no overselling, dispatch reuses the Uber pattern (Q32)."

2. Requirements, estimates, API

- **Must:** browse available items for my address; place order (reserve inventory); track delivery.
- 1M orders/day ≈ 12/s avg, ~100/s peak; availability queries ~10k/s at peak (browsing) → cacheable.

```
GET  /v1/availability?lat=..&lng=..&q=..        → items with quantity (from reachable warehouses)
POST /v1/orders {items[], address} Idempotency-Key → {order_id, eta}
GET  /v1/orders/{id}
```

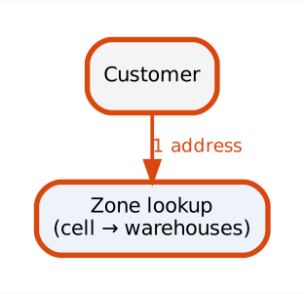
Core entities: Warehouse(id, location, delivery polygon/travel-time zone), Inventory(warehouse_id, sku, qty, reserved), Order, OrderItem, Delivery.

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

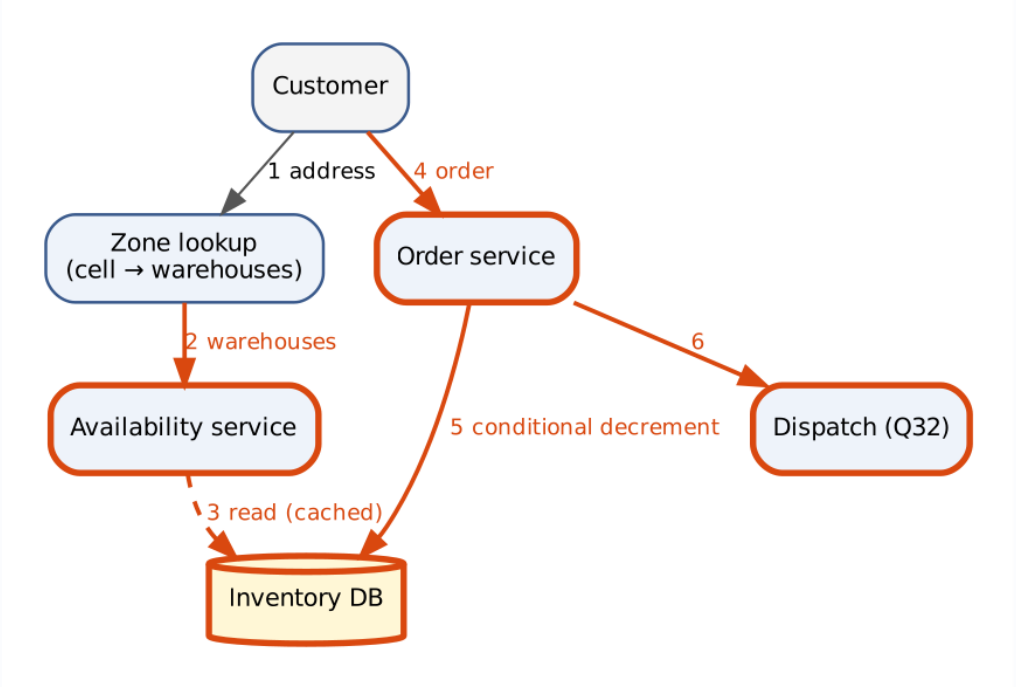
Step 1 · Which warehouses can serve me (1 min)



> **You:** “Each warehouse's 30-minute delivery zone is precomputed as geo cells, so an address maps to nearby warehouses with a lookup.”

Why this step now: The tricky requirement first.

Step 2 · Availability and ordering (1–2 min)



> **You:** “Availability reads cached inventory for those warehouses; placing an order reserves stock with a conditional decrement in one transaction, then dispatch takes over.”

Why this step now: Correctness on the write path.

Narrate one request end to end (numbers match the arrows)

Order

1. Address → warehouses (1, 2); browse (3); order reserves stock (4, 5); dispatch (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Customer	Browse, order	—	App
Zone lookup (cell → warehouses)	Reachable warehouses	Precomputed isochrones	KV
Availability service	Items in stock nearby	Read-heavy	Service + cache
Inventory DB	Qty per warehouse/ SKU	No overselling	SQL
Order service	Reserve + create order	—	Service
Dispatch (Q32)	Assign courier	—	Service

Before you leave the high-level design, check you have said:

☐ Precomputed delivery zones
☐ Conditional inventory decrement
☐ Browse vs commit

60-second summary to say once the diagram is complete:

> **You:** "First, which warehouses can serve this address? I precompute each warehouse's 30-minute delivery zone from travel times and store it as a set of geo cells, so a lookup is: address → cell → nearby warehouses. Availability then queries inventory for those warehouses, cached for a few seconds. Placing an order reserves inventory with a conditional decrement in one warehouse — or splits across warehouses — then hands off to dispatch."

4. Deep dives

- **Inventory correctness:** `UPDATE inventory SET qty = qty - n WHERE warehouse=? AND sku=? AND qty >= n` inside a transaction for all items; if any fails, roll back and show what's unavailable. Inventory is per warehouse, so contention is local.
- **Availability reads at scale:** browsing tolerates seconds of staleness → read replicas or a cache keyed by (warehouse set, sku); order placement checks the source of truth.
- **Reachability:** travel-time isochrones precomputed per warehouse (refreshed for traffic at a few time-of-day buckets) and rasterized to H3 cells.

5. Trade-offs and say this

Decision	Choice	Why
"Near me"	Precomputed delivery zones as cells	O(1) lookup per address
Inventory	Strong consistency per warehouse	No overselling
Browsing	Cached, seconds stale	Read volume

Say this: "Precompute which warehouses can reach each cell; at request time it's a lookup." · "Browse from cache, commit against the source of truth."

Q49. Design Instagram (photo upload, feed, stories)

> Classic · Meta (reported: "Instagram design") · see Q15 for feed fan-out

1. Prompt and clarify

"Design Instagram: post photos/videos, follow users, view a feed, view stories that expire after 24 hours."

State: "500M DAU, 100M posts/day, follow graph with celebrity skew, ranked feed, stories expire in 24 h, media heavy."

2. Estimates

- Uploads: 100M/day ≈ 1.2k/s; at ~2 MB per photo after client compression → 200 TB/day of originals plus resized variants (~3×).
- Feed reads: 500M × 10 = 5B/day ≈ 58k/s, peak ~170k/s.
- Stories: ~500M story items/day, viewed many times, expire → TTL storage.

3. API and entities

```
POST /v1/media:upload → presigned URL;  POST /v1/posts {media_ids[], caption}
GET  /v1/feed?cursor=..;   POST /v1/stories {media_id};   GET /v1/stories/tray
```

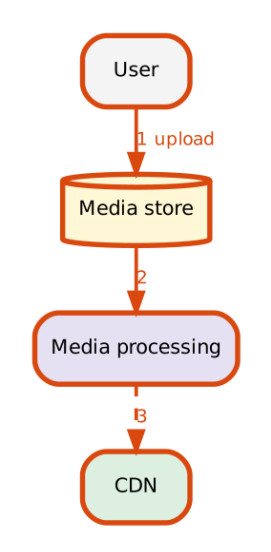
Core entities: User, Follow, Post, Media (variants), Story (expires_at), FeedItem, StoryView.

4. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

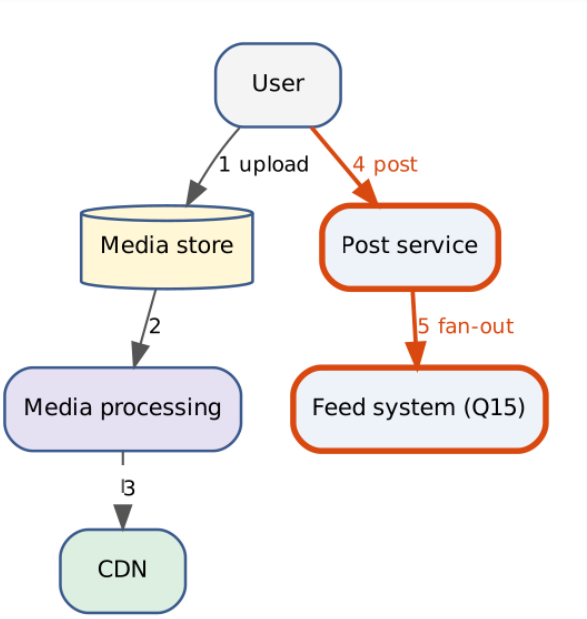
Step 1 · Media upload (1 min)



> **You:** “Clients compress and upload media directly to object storage; a pipeline creates resized variants served by a CDN.”

Why this step now: Bytes bypass services.

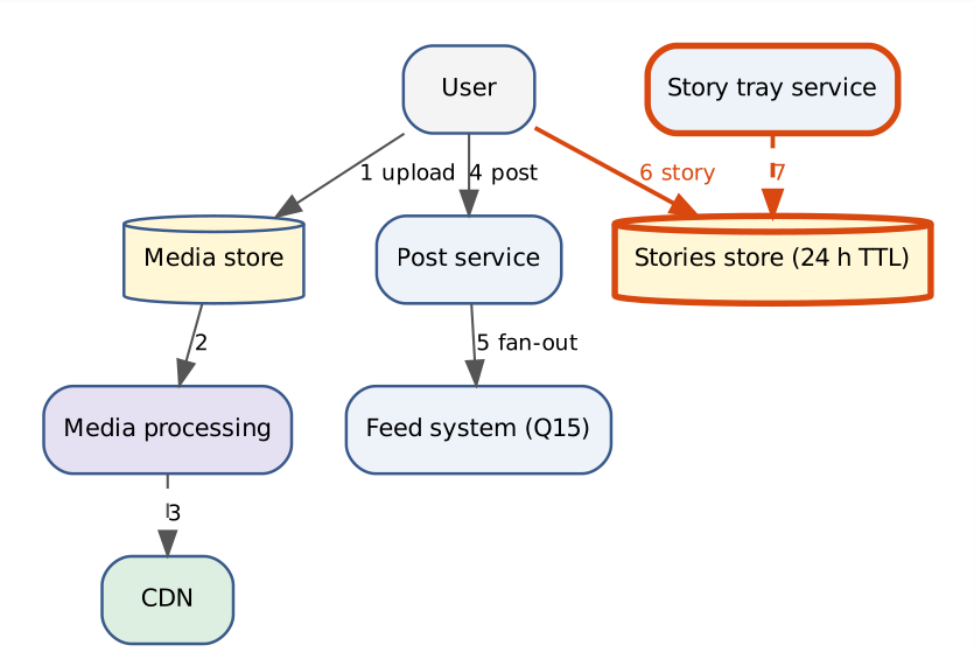
Step 2 · Posts and feed (1 min)



> **You:** “Post metadata is written and fanned out to followers' feeds — push for normal accounts, pull for celebrities.”

Why this step now: Reuse Q15.

Step 3 · Stories (1 min)



> **You:** “Stories live in a TTL store; the tray is built from followees with active stories, ranked by closeness.”

Why this step now: Separate product with expiry.

Narrate one request end to end (numbers match the arrows)

Post

1. Upload (1), process (2, 3), post (4), fan-out (5).

Story

1. Story stored with TTL (6); tray built (7).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User	Post, view	—	App
Media store	Originals + variants	—	Object storage
Media processing	Resize, thumbnails	—	Workers
CDN	Serve media	Immutable, cache forever	CDN
Post service	Post metadata	—	Service
Feed system (Q15)	Fan-out + ranking	—	Service
Stories store (24 h TTL)	Story items	Expiry	Cassandra TTL

Story tray service	Ranked tray	—	Service

Before you leave the high-level design, check you have said:

☐ Direct media upload
☐ Hybrid fan-out
☐ Stories as TTL data

60-second summary to say once the diagram is complete:

> **You:** "Media goes straight from the client to object storage via pre-signed URLs; a processing pipeline creates resized variants and thumbnails, served through a CDN. Creating a post writes metadata and triggers feed fan-out — push for normal accounts, pull for celebrities, as in the News Feed design. Stories are separate: a per-user list of story items with a 24-hour TTL, and the story tray is built from the stories of people you follow, ordered by affinity."

5. Deep dives

- **Media pipeline:** client-side compression and resizing first (saves bandwidth); server generates 3–4 sizes; CDN with long TTL (media is immutable — new edit = new URL).
- **Stories:** write-heavy and short-lived → store in a TTL-capable store (Cassandra TTL / Redis with expiry); tray = for each followee with active stories (precomputed "has active story" set) → ranked by closeness; viewed state per viewer.
- **Feed:** hybrid fan-out (Q15), ranking by engagement predictions.

6. Trade-offs and say this

Say this: "Media is immutable, so the CDN can cache it forever." · "Stories are a separate product with a TTL, not feed items that expire."

Q50. Design Tinder (swipe and match)

> Classic · Hello Interview (Medium) · Meta (Dating), Bumble

1. Prompt and clarify

"Design a dating app: users see nearby profiles matching their preferences, swipe left/right, and get notified on a mutual like."

Ask: "DAU, swipes per user?" · "Preference filters (age, distance, gender)?" · "Should a user ever see the same profile twice?" · "How fast must a match notify?"

State: "20M DAU, 100 swipes/user/day = 2B swipes/day ≈ 23k/s avg, ~70k/s peak; filters by distance and preferences; never re-show swiped profiles; match notification within seconds."

2. API and entities

```
GET  /v1/feed?count=20           → profiles (precomputed stack)
POST /v1/swipes {target_id, decision: like|pass}  → {match: bool}
```

Core entities: Profile(id, location, prefs), Swipe(swiper, target, decision, ts), Match(user_a, user_b).

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

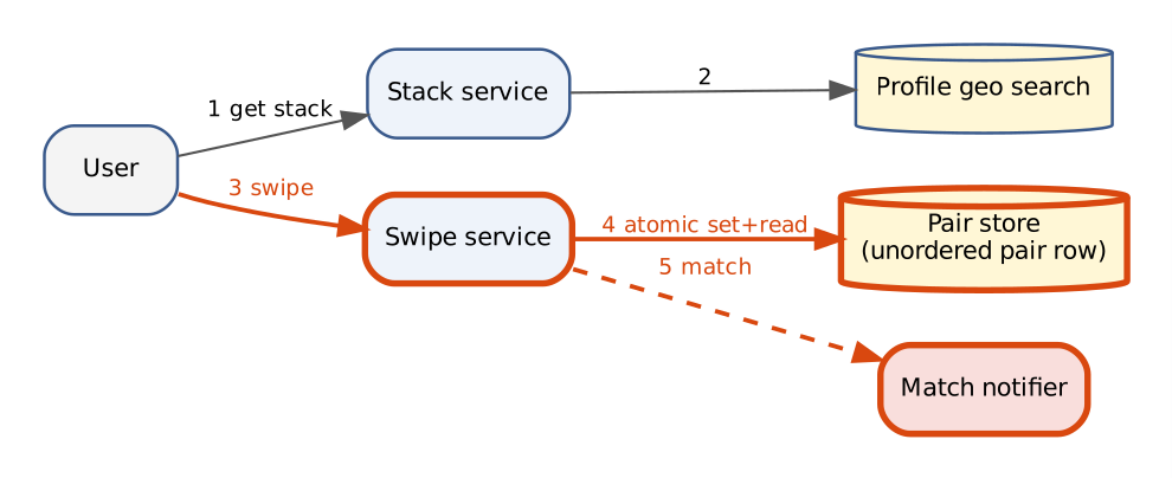
Step 1 · Profile stack (1 min)



> **You:** “A stack service builds each user's queue of candidates from a geo-indexed profile search filtered by mutual preferences, excluding already-swiped profiles.”

Why this step now: Read path.

Step 2 · Swipes and matches (1–2 min)



> **You:** "Swipes are written per pair in one row; a like atomically sets my side and reads the other side, so a mutual like creates exactly one match, pushed to both users."

Why this step now: The race condition is the deep dive.

Narrate one request end to end (numbers match the arrows)

Swipe

1. Stack fetched (1, 2); swipe (3) atomic on pair row (4); match → notify (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User	Swipe	—	App
Stack service	Candidate queue	Precomputed	Service + cache
Profile geo search	Nearby, filtered	—	Search index
Swipe service	Record swipes	—	Service
Pair store (unordered pair row)	Both decisions in one row	Atomic match check	KV with conditional write
Match notifier	Push both users	—	Push

Before you leave the high-level design, check you have said:

☐ Pair-row atomic match
☐ Bloom for swiped
☐ Precomputed stacks

60-second summary to say once the diagram is complete:

> **You:** "A recommendation service builds each user's stack of candidates from a geo-indexed profile search filtered by mutual preferences, removes already-swiped profiles, ranks them, and caches the next batch. A swipe is written to a swipe store partitioned by user; for a like, we check whether the target already liked us — if yes, it's a match, written atomically and pushed to both users."

4. Deep dives

- **Match consistency (the key race):** A and B like each other at the same moment → both checks may miss. Fix: store swipes keyed by the unordered pair `(min(a,b), max(a,b))` in one row with two columns; an atomic update sets my column and reads the other in the same operation (conditional write / Redis Lua script) → exactly one match created.
- **Never show twice:** per-user swiped set — Bloom filter for fast exclusion during stack generation (false positives just skip a profile, acceptable); authoritative swipe table for correctness.
- **Stack generation:** geo search (Q33) with filters, precomputed in the background when the stack runs low.

5. Say this

"Store both sides of a pair in one row so a mutual like is a single atomic operation." · "A Bloom filter of swiped profiles is fine here — a false positive only hides one profile."

Q51. Design LeetCode (online judge with contests)

> Classic · Hello Interview (Medium) · Amazon, Microsoft, Google (code execution)

1. Prompt and clarify

"Users read problems, submit code in many languages, see results; contests with a live leaderboard for up to 100k participants."

State: "Problems mostly read-only; 10k submissions/min normally, 100k participants in contests submitting ~5 times each over 90 minutes (~5.5k submissions/min average, bursty at the start and end); results within seconds; strong isolation for untrusted code."

2. API and entities

```
GET  /v1/problems/{id};  POST /v1/submissions {problem_id, language, code} → {submission_id}
GET  /v1/submissions/{id} (poll or WS);   GET /v1/contests/{id}/leaderboard?page=..
```

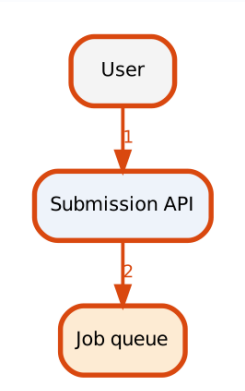
Core entities: Problem, TestCase (hidden), Submission(status, runtime, memory), Contest, ContestScore.

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

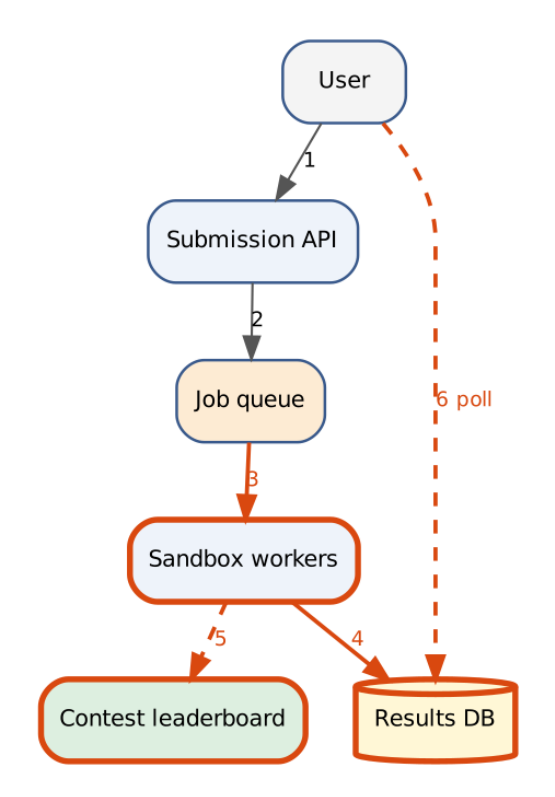
Step 1 · Submit and queue (1 min)



> **You:** “Users submit code; submissions go to a queue so contest bursts don't overwhelm execution.”

Why this step now: Absorb bursts.

Step 2 · Execute safely (1–2 min)



> **You:** “Workers run code in isolated sandboxes — no network, strict limits — against hidden tests and write results; contest scores update a sorted-set leaderboard.”

Why this step now: Security and fairness.

Narrate one request end to end (numbers match the arrows)

Submission

1. Submit (1), queue (2), run in sandbox (3), result (4), leaderboard (5), client polls (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
User	Submit, poll	—	Browser
Submission API	Validate, enqueue	—	Service
Job queue	Pending runs	Bursts	SQS/Kafka
Sandbox workers	Run code vs tests	Isolation	gVisor/Firecracker
Results DB	Verdicts	—	SQL
Contest leaderboard	Ranks	—	Redis ZSET

Before you leave the high-level design, check you have said:

☐ Sandbox isolation
☐ Queue for bursts
☐ Pre-warmed images

60-second summary to say once the diagram is complete:

> **You:** "Submissions are queued; workers pull jobs and run code in isolated sandboxes — containers with seccomp, no network, CPU/memory/time limits, or microVMs — against hidden test cases, then write results. The client polls or gets a push. During contests, results also update a leaderboard in a Redis sorted set."

4. Deep dives

- **Security of running untrusted code:** per-submission sandbox (gVisor/Firecracker microVM), read-only filesystem, no network, resource limits, kill on timeout; never run on hosts holding secrets.
- **Contest spikes:** queue absorbs bursts; pre-scale workers before contest start; pre-warmed language images; priority for contest submissions.
- **Leaderboard:** score = problems solved × large constant − penalty time; ZSET for rank (Q42); cache top pages for 1–2 s.

5. Say this

"The queue absorbs the contest burst; sandboxes isolate untrusted code." · "Never run user code on a box that can reach anything."

Q52. Design Strava (activity tracking)

> Classic · Hello Interview (Medium) · Apple (Fitness), Google (Fit), Garmin, Amazon

1. Prompt and clarify

"Users record runs/rides with GPS, upload activities, see stats and a feed of friends' activities; optional live tracking and segment leaderboards."

State: "50M MAU, 5M activities/day, GPS point every second while recording (stored on device, uploaded at end), live tracking for opt-in users, segment leaderboards."

2. Estimates and entities

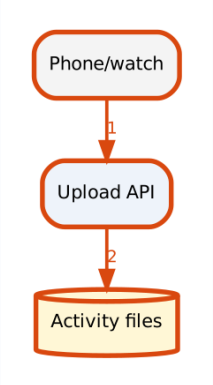
- 5M activities/day × ~3,600 points × 30 B ≈ 540 GB/day of GPS data → compress, store as files in object storage, summary in DB.
- **Core entities:** Athlete, Activity(summary stats, polyline), ActivityStream (raw points file), Segment, SegmentEffort, Follow.

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

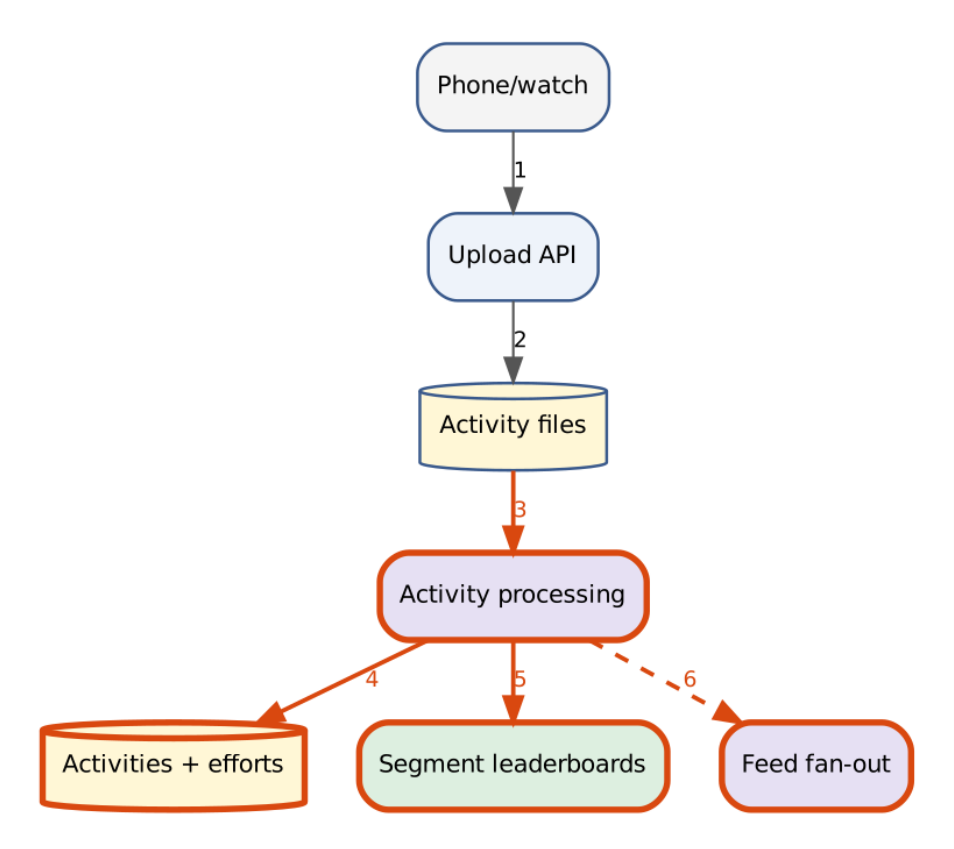
Step 1 · Record offline, upload (1 min)



> **You:** "The device records GPS locally and uploads the activity file at the end, idempotently."

Why this step now: Unreliable connectivity.

Step 2 · Process and share (1–2 min)



> **You:** “Processing computes stats, matches segments, writes summaries and efforts, and fans out to followers' feeds.”

Why this step now: Core value.

Narrate one request end to end (numbers match the arrows)

Activity

1. Upload (1, 2); process (3); store (4); efforts (5); feed (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Phone/watch	Record GPS	Offline-first	App
Upload API	Idempotent upload	—	Service
Activity files	Raw streams	—	Object storage
Activity processing	Stats, segment matching	—	Workers
Activities + efforts	Summaries	—	SQL
Segment leaderboards	Top efforts	—	Redis ZSET

Feed fan-out	Friends' feeds	—	Workers

Before you leave the high-level design, check you have said:

☐ Offline-first
☐ Files vs rows
☐ Segment matching

60-second summary to say once the diagram is complete:

> **You:** "Recording happens on the device, which buffers points locally — connectivity during a run is unreliable. On finish, the app uploads the activity file; processing computes distance, pace, elevation, matches the route against known segments, writes summary and efforts, and fans out to followers' feeds. Live tracking, if enabled, sends a point every few seconds to a location service that watchers subscribe to."

4. Deep dives

- **Offline-first recording:** device is the source of truth until upload; idempotent upload by client-generated activity ID.
- **Segment matching:** index segments by geo cells; for each activity, find candidate segments whose start/end cells are on the route; verify with point-to-polyline distance; compute effort time.
- **Segment leaderboards:** ZSET per segment (Q42) for top efforts; per-athlete best effort only.

5. Say this

"Record on the device, upload when done — the network can't be trusted mid-run." · "Raw GPS streams are files; summaries are rows."

Q53. Design an online auction (eBay-style bidding)

> Classic · Hello Interview (Medium) · Amazon, eBay

1. Prompt and clarify

"Users list items with an end time; others place bids; the highest valid bid wins; everyone sees the current price in real time."

State: "10M active auctions, 100k bids/min overall, popular auctions get hundreds of bids per second near the end; bids must be strictly ordered and never lost; live price updates within a second."

2. API and entities

```
POST /v1/auctions {item, start_price, ends_at}
POST /v1/auctions/{id}/bids {amount} Idempotency-Key → {accepted, current_price}
GET  /v1/auctions/{id} (+ WS/SSE for live updates)
```

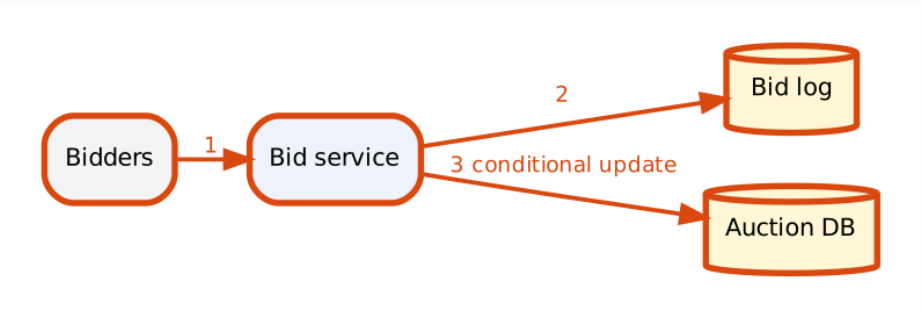
Core entities: Auction(id, seller, current_price, current_winner, ends_at, status), Bid(auction_id, bid_id, bidder, amount, ts, accepted).

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

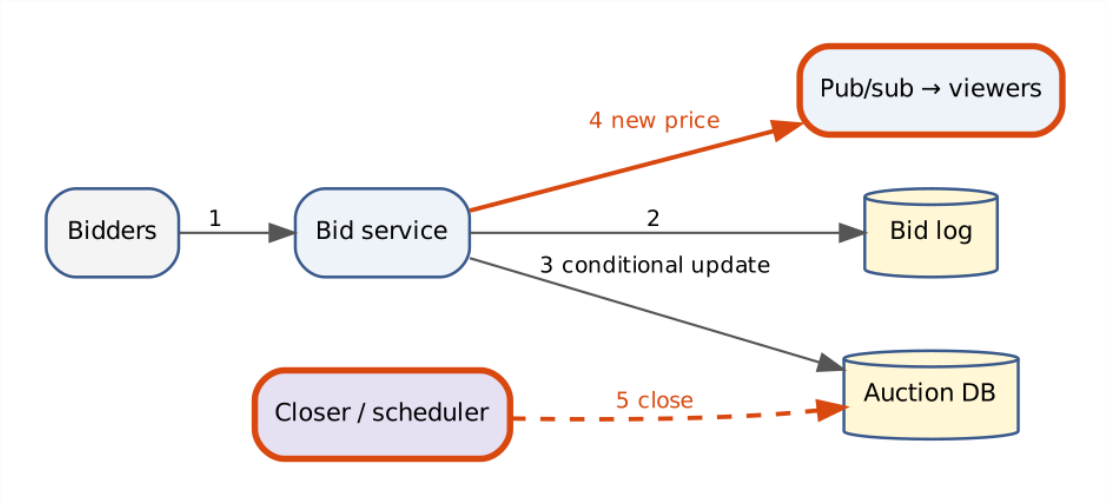
Step 1 · Place bids (1 min)



> **You:** “Bids hit a bid service that logs every attempt and does a conditional update on the auction row — accept only if higher by the increment and still open.”

Why this step now: Consistency per auction.

Step 2 · Live updates and closing (1 min)



> **You:** "Accepted bids are published to viewers over WebSockets; a scheduler closes auctions at end time and triggers payment."

Why this step now: Real-time and lifecycle.

Narrate one request end to end (numbers match the arrows)

Bid

1. Bid (1) logged (2), conditional update (3), broadcast (4); auction closed by scheduler (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Bidders	Bid	—	App
Bid service	Validate, conditional update	—	Service
Bid log	Every attempt	Audit	Kafka/append-only
Auction DB	Current price/winner	Truth	SQL
Pub/sub → viewers	Live price	—	WS gateways
Closer / scheduler	End auctions	—	Scheduler

Before you leave the high-level design, check you have said:

☐ Conditional update per auction
☐ Server clock
☐ Anti-sniping

60-second summary to say once the diagram is complete:

> **You:** "Bids go to a bid service that does a conditional update on the auction row: accept only if the amount beats the current price by the minimum increment and the auction is still open. Accepted bids are recorded and published; viewers of that auction get the new price over a WebSocket. A scheduler closes auctions at their end time and triggers payment and notifications."

4. Deep dives

- **Consistency under contention:** `UPDATE auctions SET price=?, winner=? WHERE id=? AND price + min_increment <= ? AND now() < ends_at` — atomic per auction; or serialize bids per auction through a partitioned queue (one consumer per auction partition) for very hot auctions.
- **Durability:** write every bid attempt to a durable log first (audit and dispute resolution), then process.
- **Ending fairly:** server time is authoritative; "sniping" protection option — extend end time if a bid arrives in the final N seconds.
- **Proxy bidding:** store each bidder's max; the system auto-increments on their behalf.

5. Say this

"Each auction is its own consistency boundary — a conditional update per auction." · "The server's clock decides when an auction ends, never the client's."

Q54. Design a news aggregator (Google News)

> Classic · Hello Interview (Medium) · Google, Apple (News), Microsoft (Start/MSN)

1. Prompt and clarify

"Collect articles from thousands of publishers, group them into stories, and show users a personalized, fresh feed."

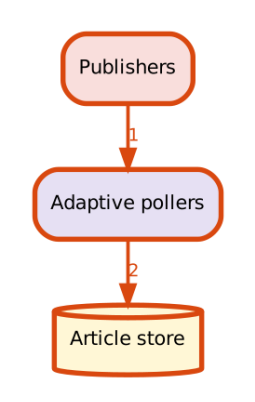
State: "50k publishers, 2M new articles/day, freshness within minutes for breaking news, 100M DAU, personalized feed with top stories."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

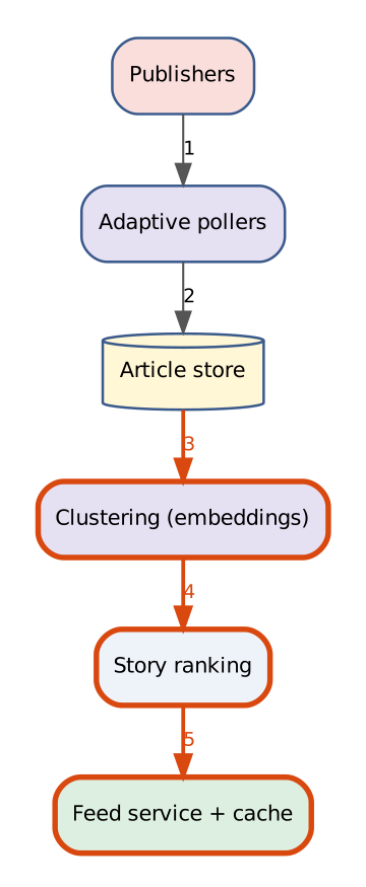
Step 1 · Ingest (1 min)



> **You:** “Feeds and sitemaps are polled at each publisher's publishing rate; articles fetched and parsed.”

Why this step now: Freshness vs cost.

Step 2 · Cluster and rank (1–2 min)



> **You:** “Articles are embedded, deduplicated and clustered into stories; stories are ranked by freshness, authority and size; feeds are cached per region and personalized.”

Why this step now: The product value.

Narrate one request end to end (numbers match the arrows)

Article to feed

1. Polled (1), stored (2), clustered (3), ranked (4), served (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Publishers	RSS/sitemaps	—	—
Adaptive pollers	Fetch new articles	—	Workers
Article store	Text + entities	—	DB
Clustering (embeddings)	Group into stories	—	ML job
Story ranking	Score stories	—	Service
Feed service + cache	Regional top + personalization	—	Service + cache

Before you leave the high-level design, check you have said:

☐ Adaptive polling
☐ Cluster into stories
☐ Cache per region

60-second summary to say once the diagram is complete:

> **You:** "Ingestion pulls RSS feeds and sitemaps on a schedule tuned per publisher (news sites every few minutes) plus publisher push APIs, then fetches and parses articles. A processing pipeline extracts text and entities, deduplicates near-identical articles, and clusters articles about the same event into stories using embeddings. Ranking combines freshness, source authority, story size and user interests. The feed service serves cached top stories per region with per-user personalization on top."

3. Deep dives

- **Clustering:** embed each article; assign to an existing story if similarity to the cluster centroid exceeds a threshold within a time window, else create a new story; periodically re-cluster.
- **Freshness vs crawl cost:** adaptive polling per source based on publishing rate.
- **Serving:** most users see the same top stories per region → cache heavily; personalize by re-ranking a cached candidate set.

Say this: "Cluster articles into stories, then rank stories, not articles." · "Poll each source at the rate it actually publishes."

Q55. Design a price tracking service (CamelCamelCamel)

> Classic · Hello Interview (Medium) · Amazon-adjacent; crawling + alerting pattern

1. Prompt and clarify

"Track prices of products on e-commerce sites, show price history charts, and alert users when a price drops below their target."

State: "500M tracked products, 10M users with 50M alerts, prices refreshed hourly for popular items and daily for the long tail, alerts within an hour of a drop; respect site rate limits."

2. Estimates

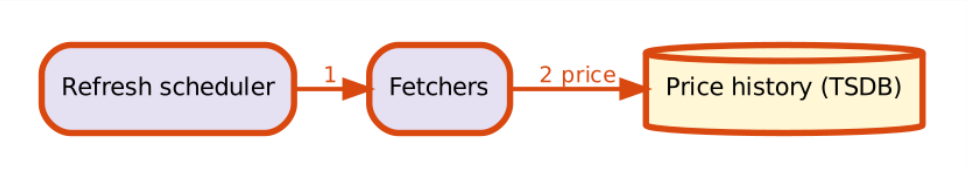
- Refresh: say 20M popular hourly + 480M daily → 20M/3600 + 480M/86400 ≈ 5.6k + 5.6k ≈ **11k fetches/s**.
- Price history: 500M × ~2 points/day × 16 B ≈ 16 GB/day → time-series store.

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

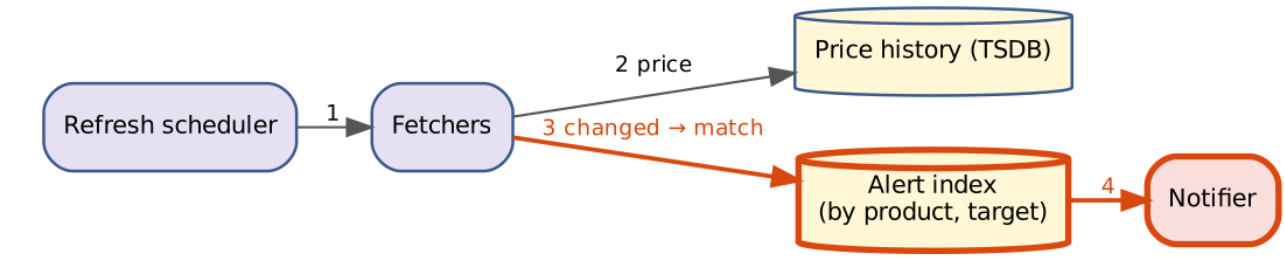
Step 1 · Fetch prices (1 min)



> **You:** “A scheduler prioritizes products by watchers and volatility; fetchers respect site rate limits and write every price to a time-series store.”

Why this step now: Scale is in the refresh.

Step 2 · Alert (1 min)



> **You:** “A price change looks up alerts indexed by product with targets at or above the new price and notifies those users once.”

Why this step now: Turn data into value.

Narrate one request end to end (numbers match the arrows)

Price drop

1. Scheduled fetch (1), recorded (2), alerts matched (3), notified once (4).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Refresh scheduler	Priority by watchers × volatility	—	Service
Fetchers	Polite fetch / partner APIs	—	Workers
Price history (TSDB)	Charts	—	TSDB
Alert index (by product, target)	Who to notify	—	KV/SQL
Notifier	Email/push	—	Service

Before you leave the high-level design, check you have said:

☐ Prioritized refresh
☐ Alert index
☐ Politeness

60-second summary to say once the diagram is complete:

> **You:** "A scheduler prioritizes which products to refresh — popular and alert-watched ones more often. Fetchers retrieve prices politely (per-site rate limits, or partner APIs where available), and every observed price goes into a time-series store. A price-change event triggers alert evaluation: an index of alerts by product finds users whose target is now met, and a notifier emails or pushes them once."

4. Deep dives

- **Prioritized refresh:** priority = watchers × price volatility × time since last check (crawler frontier from Q37).
- **Alert matching:** alerts indexed by product_id with targets sorted → on a new price, range-query alerts with target ≥ price; mark fired to avoid repeats.
- **Charts:** downsample history (daily min/max) for long ranges.

Say this: "Refresh priority follows who's watching and how volatile the price is." · "Alerts are indexed by product so a price change finds its watchers in one lookup."

Q56. Design Facebook post search

> Classic · Hello Interview (Hard) · Meta, LinkedIn, Twitter

1. Prompt and clarify

"Users search posts by keyword; results sortable by recency or by likes; new posts and like counts update quickly."

State: "1B posts/day, 10B likes/day, 100k searches/s, results by recency or like count, new posts searchable within a minute, like counts can lag a few minutes, no privacy filtering for this version (state it explicitly)."

2. Estimates

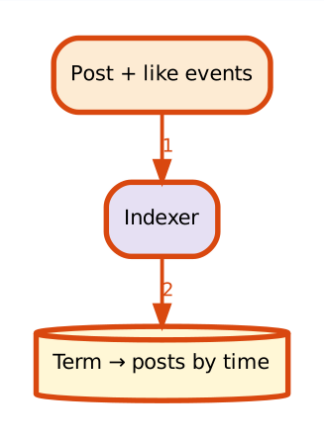
- Writes: posts 1B/day ≈ 12k/s; likes 10B/day ≈ **115k/s** — likes are the hard part because each changes a sort key.

3. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

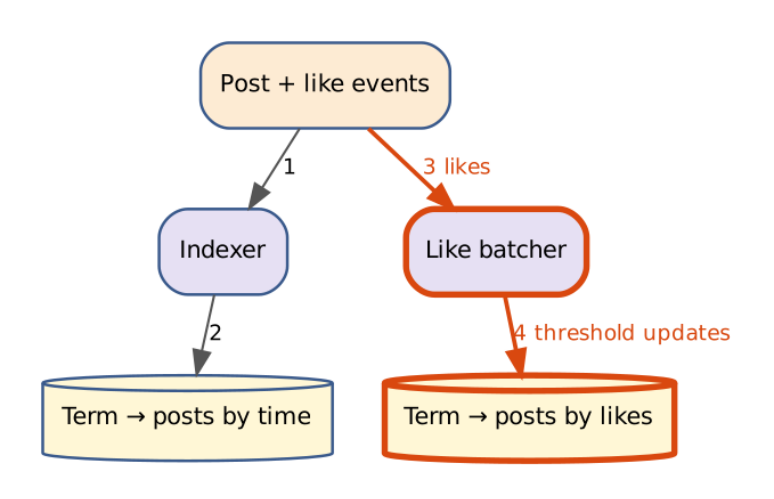
Step 1 · Index posts (1 min)



> **You:** “Post events are tokenized and appended to per-term posting lists ordered by time.”

Why this step now: Recency search.

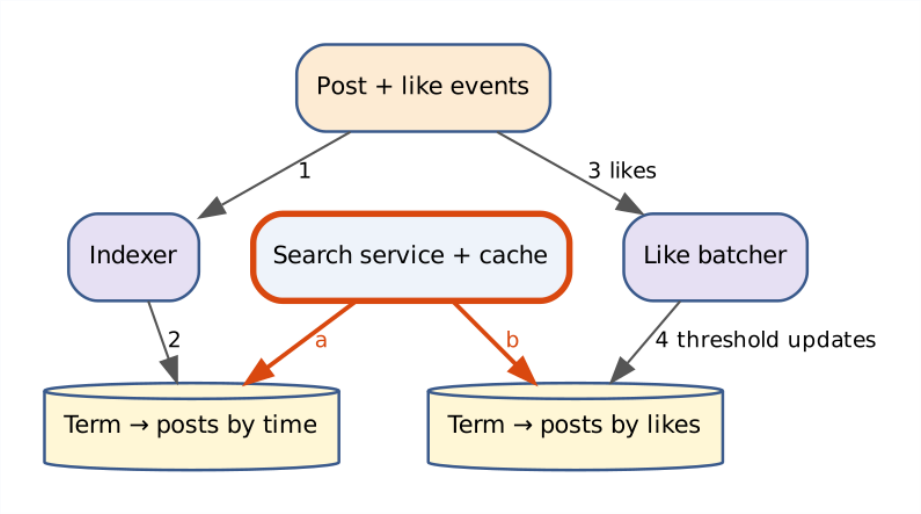
Step 2 · Index likes cheaply (1 min)



> **You:** “Likes are batched per post and the like-ordered lists are updated only when counts cross thresholds.”

Why this step now: 10× more likes than posts.

Step 3 · Query (30 s)



> **You:** “Search reads and intersects posting lists for the query terms and returns the top results; popular queries cached.”

Why this step now: Read path.

Narrate one request end to end (numbers match the arrows)

Write

1. Posts indexed (1, 2); likes batched (3) into popularity lists (4).

Query

1. Read lists (a, b), intersect, return.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Post + like events	Writes	—	Kafka
Indexer	Tokenize	—	Workers
Term → posts by time	Recency lists	—	KV lists
Like batcher	Aggregate likes	Tame writes	Stream job
Term → posts by likes	Popularity sorted sets	—	Redis ZSET
Search service + cache	Intersect, rank	—	Service

Before you leave the high-level design, check you have said:

☐ Two orderings
☐ Batch likes
☐ Intersection

60-second summary to say once the diagram is complete:

> **You:** "Posts and likes flow as events into ingestion. Indexers tokenize posts and append post IDs to per-term posting lists — one ordered by time, one by like count. Likes are batched: we aggregate like counts per post over a few minutes and only update the like-sorted index for posts whose counts crossed meaningful thresholds. Search reads the posting list for each term, intersects, and returns the top results; popular queries are cached."

4. Deep dives

- **Two orderings:** recency list = append-only (new posts at the head); like-ordered list = sorted set per term updated in batches.
- **Taming like writes:** batch likes per post per few minutes; update the like index only on logarithmic thresholds (1, 2, 4, 8 … likes), since ranking only needs approximate magnitude; exact counts fetched at display time.
- **Multi-term queries:** intersect posting lists; cap list lengths for very common terms (keep top N by likes, recent N by time).

Say this: "Likes are 10× posts, so I batch them and only reorder on threshold crossings." · "Two posting lists per term: one by time, one by popularity."

Q57. Design online chess (real-time multiplayer game)

> Classic · Hello Interview (Hard) · Microsoft (Xbox), Amazon (Luna/Twitch), Meta

1. Prompt and clarify

"Players find opponents of similar rating, play live games with clocks, and spectators can watch."

State: "10M DAU, 1M concurrent games at peak, moves must be validated server-side, clocks authoritative on the server, reconnect within the game time, matchmaking by rating within seconds."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

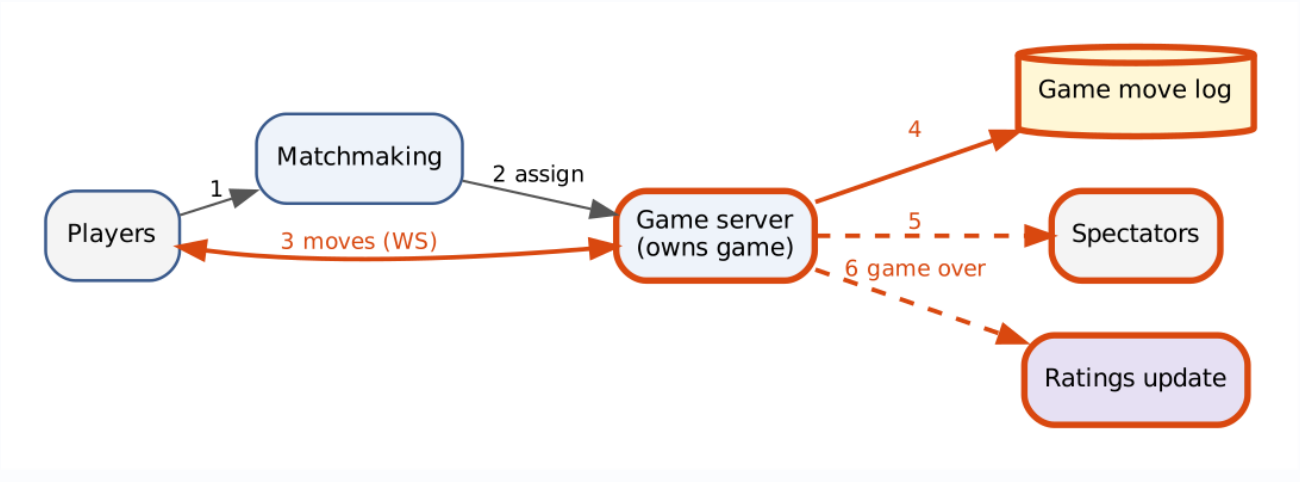
Step 1 · Matchmaking (1 min)



> **You:** “Players enter rating-bucketed queues; matchmaking pairs close ratings, widening over time, and assigns a game server.”

Why this step now: Get two players together.

Step 2 · Play (1–2 min)



> **You:** “Both connect to the game server that owns the game; it validates moves, updates clocks, logs moves and broadcasts to players and spectators.”

Why this step now: Single owner per game.

Narrate one request end to end (numbers match the arrows)

Game

1. Queue (1), assigned (2), moves (3) logged (4), broadcast (5), ratings (6).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Players	Queue, play	—	App
Matchmaking	Pair by rating	—	Service + queues
Game server (owns game)	Validate, clocks	—	Stateful service
Game move log	Replay/recovery	—	DB
Spectators	Watch	—	App
Ratings update	Elo/Glicko	—	Job

Before you leave the high-level design, check you have said:

☐ Single owner per game
☐ Server clocks
☐ Reconnect by replay

60-second summary to say once the diagram is complete:

> **You:** "Matchmaking keeps rating-bucketed queues and pairs players with close ratings, widening the window the longer they wait. A game is assigned to a game server that owns its state for its lifetime; both players connect over WebSockets. Each move goes to the game server, which validates it, updates the board and clocks, persists the move to a game log, and broadcasts to both players and spectators."

3. Deep dives

- **Single owner per game:** route game_id to one server (consistent hashing + lease) → no distributed locking; state = move list (event log) so a replacement server rebuilds by replay.
- **Clocks:** server timestamps receipt of each move and deducts from the mover's clock; clients show local countdowns but the server decides flag-falls; compensate for latency with a small lag allowance.
- **Reconnect:** client reconnects, fetches move list since last known ply.
- **Ratings:** Elo/Glicko updated after game end asynchronously.

Say this: "One server owns each game, so there's nothing to lock." · "The server's clock is the only clock that matters."

Q58. Design a flash sale (limited inventory, huge spike)

> Classic · Hello Interview (Hard) · Amazon (Lightning Deals, Prime Day), Alibaba (Singles' Day)

1. Prompt and clarify

"Sell 10,000 units of a product at a big discount starting at 12:00; 10M users try to buy at once."

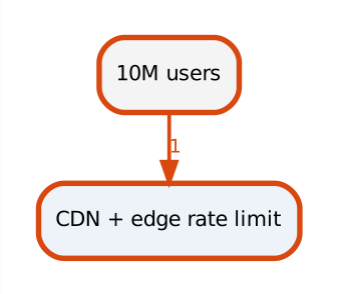
State: "10k units, 10M users within the first minute, no overselling, fair-ish allocation, unpaid reservations released after 5 minutes, the rest of the site must stay healthy."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

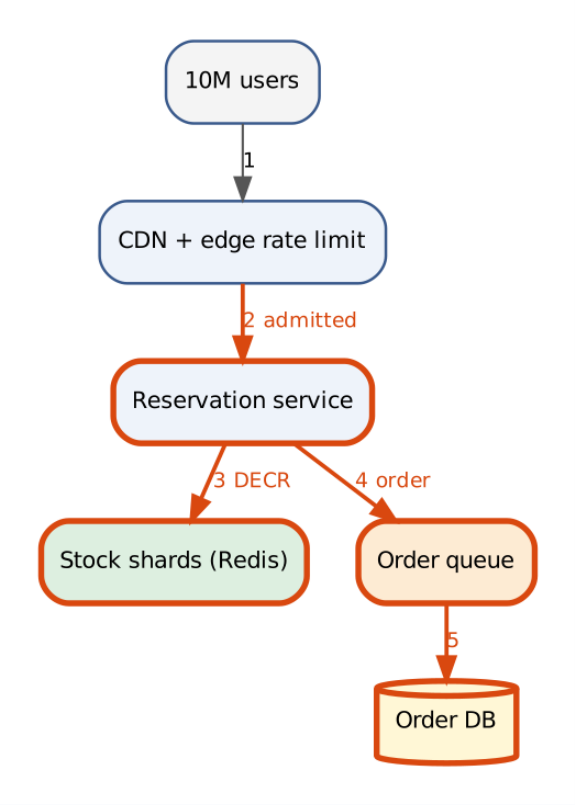
Step 1 · Edge absorbs the crowd (1 min)



> **You:** “Pages and countdown come from the CDN; at sale time, an edge layer rate-limits and short-circuits once sold out.”

Why this step now: Admission control.

Step 2 · Atomic stock, async orders (1–2 min)



> **You:** “A reservation service atomically decrements sharded stock counters in Redis; winners get a token, and orders are written asynchronously through a queue.”

Why this step now: Never oversell, never melt the DB.

Narrate one request end to end (numbers match the arrows)

Purchase

1. Edge (1) admits (2); stock decremented (3); order queued (4) and written (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
10M users	Buy	—	Browsers
CDN + edge rate limit	Static pages, sold-out flag	Shield backends	CDN workers
Reservation service	Atomic decrement	—	Service
Stock shards (Redis)	In-memory stock	Throughput	Redis Lua
Order queue	Async order writes	Protect DB	Kafka
Order DB	Orders	—	SQL

Before you leave the high-level design, check you have said:

☐ Sold-out at edge
☐ Sharded stock
☐ Separate cell

60-second summary to say once the diagram is complete:

> **You:** "Like Ticketmaster, demand is a thousand times supply, so the design is admission control. Static pages and the countdown are served from the CDN. At 12:00, purchase attempts hit an edge layer that rate-limits per user and passes requests to a reservation service which atomically decrements an in-memory stock counter — Redis with a Lua script. Winners get a reservation token and go to checkout; everyone else is told 'sold out' immediately at the edge once stock hits zero. Orders are written asynchronously through a queue to the order database."

3. Deep dives

- **Atomic stock in memory:** Redis `DECR` (or Lua check-and-decrement) handles hundreds of thousands of ops/s; shard the stock across N keys (e.g., 10 keys × 1,000 units) to spread load, try a random shard, fall back to others.
- **Sold-out fast path:** once a shard is empty, a flag at the edge short-circuits requests without touching backends.
- **Isolation:** run the flash sale on a separate cell/cluster so the spike can't take down normal shopping.
- **Reservation expiry:** unpaid reservations returned to stock after 5 minutes (TTL + sweeper).

Say this: "Decrement stock in memory atomically; write orders asynchronously." · "Tell the 99.9% 'sold out' at the edge — they should never reach the database."

Q59. Design a donations website (GoFundMe-style, viral spikes)

> Classic · Hello Interview (Hard) · payments + spike handling

1. Prompt and clarify

"Users create fundraisers; donors give money; totals update; a campaign can go viral (e.g., a disaster appeal) with huge spikes."

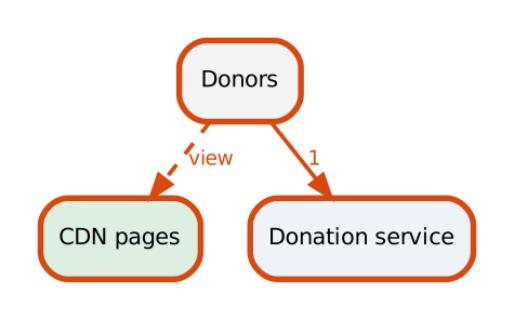
State: "Normal 50 donations/s, viral peaks 20k/s on one campaign; never lose or double-charge a donation; totals can lag a few seconds; payments via a processor."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

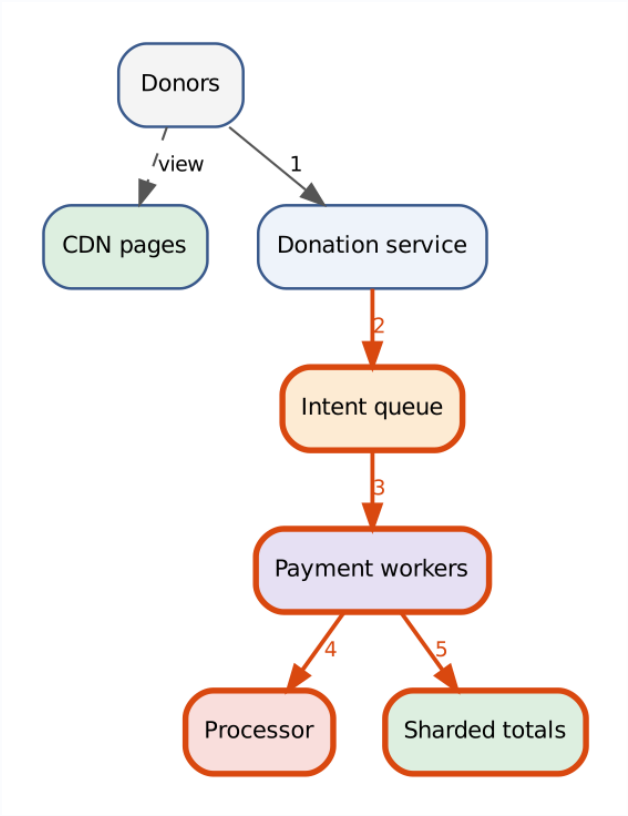
Step 1 · Record intent (1 min)



> **You:** “Campaign pages come from the CDN; a donation records an intent durably with an idempotency key.”

Why this step now: Never lose a donation.

Step 2 · Charge at a safe rate (1 min)



> **You:** “Payment workers drain the intent queue at the processor's allowed rate and update a sharded campaign total.”

Why this step now: Viral spikes.

Narrate one request end to end (numbers match the arrows)

Donation

1. Intent recorded (1, 2); charged (3, 4); total updated (5).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Donors	Donate	—	Browser
CDN pages	Campaign pages	Spikes	CDN
Donation service	Record intent	—	Service
Intent queue	Buffer	Processor limits	Kafka/SQS
Payment workers	Charge idempotently	—	Workers
Processor	Charge	—	PSP
Sharded totals	Campaign total	Hot key	Redis

Before you leave the high-level design, check you have said:

☐ Intent before charge
☐ Queue vs processor limits
☐ Sharded counter

60-second summary to say once the diagram is complete:

> **You:** "Campaign pages are static and cached at the CDN, with the total fetched from a cached endpoint. A donation goes to a donation service that records the intent with an idempotency key, then hands off to the payment processor. To absorb viral spikes, intents go through a durable queue and payment workers process them at the rate the processor allows; the donor sees 'processing' and then confirmation. Successful payments update the campaign total through a sharded counter."

3. Deep dives

- **Never lose a donation:** write intent durably before charging; idempotency keys to the processor; state machine (CREATED → CHARGED → CONFIRMED / FAILED); reconciliation with processor reports (Q5).
- **Hot campaign total:** sharded counters (N sub-counters summed on read, cached 1–2 s).
- **Spike handling:** queue decouples the web tier from the processor's rate limits; autoscale web tier; CDN for pages.

Say this: "Record the intent durably, then charge — never the other way round." · "A viral campaign is a hot key; shard its counter."

Q60. Design GitHub Actions (CI/CD job runner)

> Classic · Hello Interview (Hard) · Microsoft (GitHub, Azure DevOps), Amazon (CodeBuild), Databricks (job platform)

1. Prompt and clarify

"When code is pushed, run workflows defined in the repo: jobs with steps and dependencies, on hosted runners, streaming logs to users."

State: "10M workflow runs/day, jobs as DAGs, hosted Linux/Windows/macOS runners plus self-hosted, run isolation per job, logs streamed live, artifacts and caches stored."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

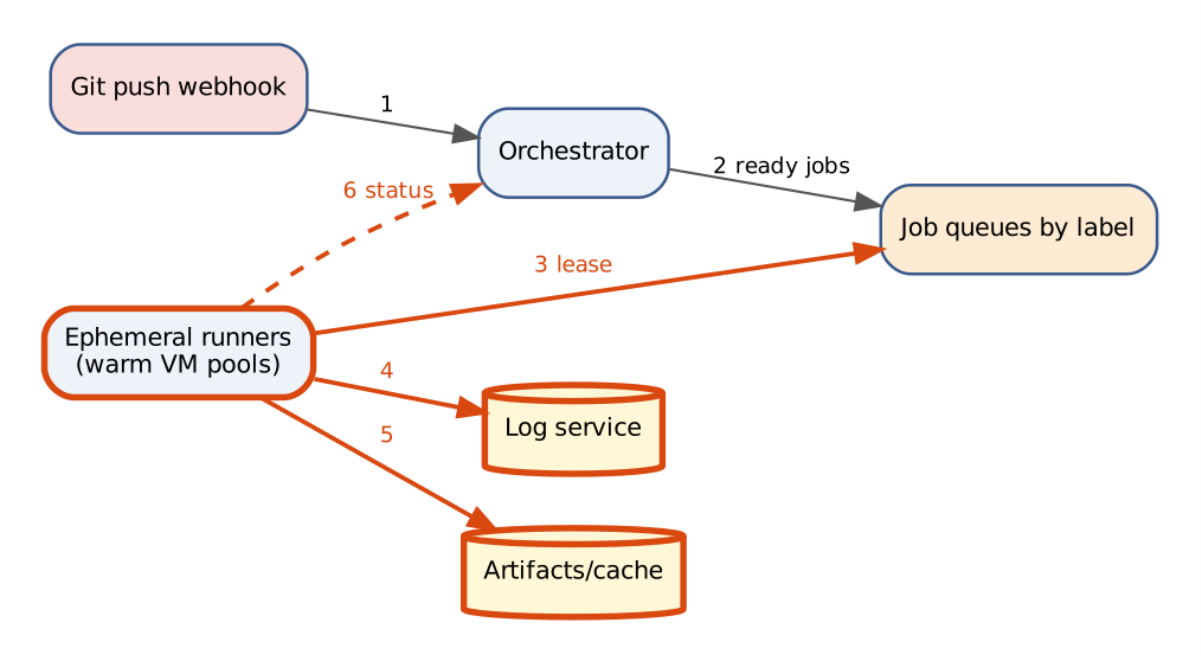
Step 1 · Trigger and plan (1 min)



> **You:** “A push webhook creates a run; the orchestrator parses the workflow into a DAG and enqueues jobs whose dependencies are met.”

Why this step now: Workflow as DAG.

Step 2 · Run and report (1–2 min)



> **You:** “Ephemeral runners from warm pools long-poll queues by label, run steps in isolation, stream logs, upload artifacts and report status; completion releases dependents.”

Why this step now: Isolation and scheduling.

Narrate one request end to end (numbers match the arrows)

Run

1. Webhook (1); ready jobs enqueued (2); runner leases (3), logs (4), artifacts (5), reports (6) → dependents released.

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Git push webhook	Trigger	—	Webhook
Orchestrator	DAG, job states	—	Service + DB
Job queues by label	os/size	—	Queue
Ephemeral runners (warm VM pools)	Execute jobs	Isolation	VMs
Log service	Live + archived logs	—	Stream + object storage
Artifacts/cache	Outputs	—	Object storage

Before you leave the high-level design, check you have said:

☐ DAG release
☐ One VM per job
☐ Fair scheduling

60-second summary to say once the diagram is complete:

> **You:** "A push webhook creates a workflow run: the orchestrator parses the workflow file into a DAG of jobs and enqueues the jobs whose dependencies are met. Runners — fresh VMs from warm pools — long-poll the job queue for jobs matching their labels, execute steps in isolation, stream logs to a log service, upload artifacts, and report status. When a job completes, the orchestrator enqueues its dependents."

3. Deep dives

- **Isolation:** one ephemeral VM per job for hosted runners (destroyed after); secrets injected at runtime, masked in logs; untrusted forks get restricted tokens.
- **Scheduling:** queues per runner label (os/size); per-org concurrency limits for fairness; warm pools sized by forecast (most pushes happen in business hours).
- **Exactly-once job execution:** jobs leased to a runner with heartbeats; on lease expiry, mark failed or retry (steps aren't generally idempotent, so default to fail + manual rerun).
- **Logs:** runners stream chunks to a log service; live viewers subscribe; final logs stored in object storage.

Say this: "A workflow is a DAG; the orchestrator just releases jobs whose dependencies are done." · "One fresh VM per job — isolation is the security model."

Q61. Design an ad click aggregator

> Classic · Hello Interview (Hard) · Google, Meta, Amazon Ads, Microsoft Ads · see Q24 for the counting core

1. Prompt and clarify

"Users click ads; we redirect them to the advertiser and let advertisers query click counts by ad and time window with low latency; counts must be accurate for billing."

State: "10k clicks/s average, 50k peak; advertisers query per-minute aggregates over arbitrary ranges in under a second; deduplicate fraud/duplicate clicks; billing-grade accuracy within hours."

2. High-level design

High-level design walkthrough — build it in this order

Draw the design in stages, left to right along the path of a request. **Orange** boxes and arrows are what you add in that step. Say the script while you draw — never draw silently for more than ~20 seconds.

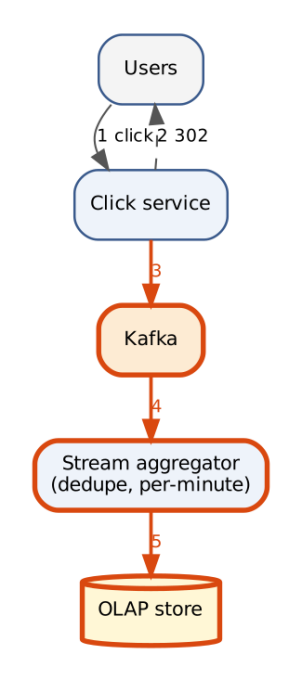
Step 1 · Redirect first (30 s)



> **You:** “The click service records the click with its ID and redirects the user immediately.”

Why this step now: Never block the user.

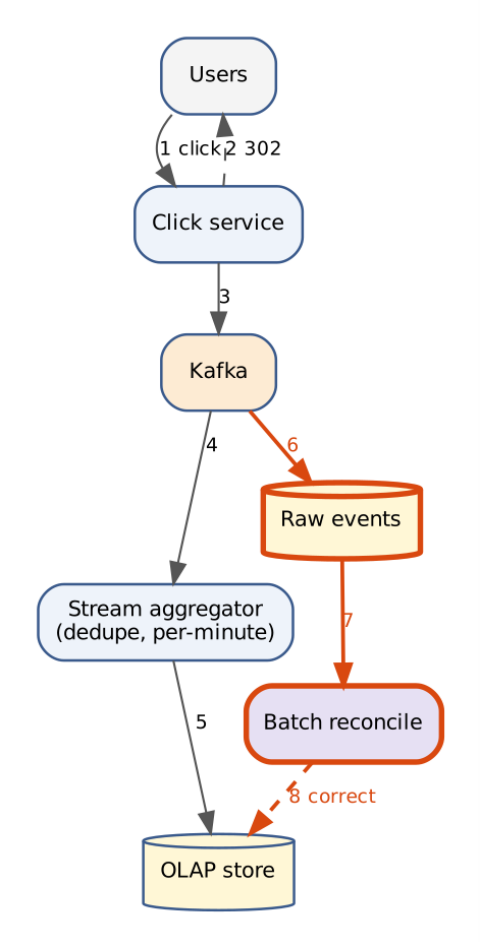
Step 2 · Real-time aggregates (1–2 min)



> **You:** “Click events flow through Kafka to a stream job that dedupes and aggregates per ad per minute into an OLAP store for advertiser queries.”

Why this step now: Fresh numbers.

Step 3 · Billing truth (30 s)



> **You:** “Raw events land in object storage; a batch job recomputes exact counts and reconciles.”

Why this step now: Money needs exactness.

Narrate one request end to end (numbers match the arrows)

Click

1. Click (1) → redirect (2); event (3) aggregated (4, 5); raw (6) reconciled (7, 8).

Component table — what each box is for

Component	Responsibility	Why it exists	Typical tech
Users	Click ad	—	Browser
Click service	Log + redirect	Fast	Service
Kafka	Click stream	—	Kafka
Stream aggregator (dedupe, per-minute)	Counts	—	Flink
OLAP store	Advertiser queries	Fast range sums	Pinot/Druid

Raw events	Replay	—	Object storage
Batch reconcile	Exact counts	Billing	Spark

Before you leave the high-level design, check you have said:

☐ Redirect first
☐ Dedupe by click ID
☐ Stream + batch

60-second summary to say once the diagram is complete:

> **You:** "The click service records the click event with a click ID and redirects the user immediately — the redirect must never wait on analytics. Events go to Kafka; a stream processor dedupes by click ID, filters fraud, and aggregates counts per ad per minute into an OLAP store that advertisers query. In parallel, raw events land in object storage, and a periodic batch job recomputes exact counts and reconciles the real-time numbers — that's the billing source of truth."

3. Deep dives

- **Dedupe:** click ID generated server-side when the ad is served (impression ID); duplicates within a window dropped (stateful dedupe keyed by click ID).
- **Query store:** pre-aggregated minute buckets in an OLAP database (Druid/Pinot/ClickHouse) partitioned by ad and time → fast range sums.
- **Hot ads:** partition stream by ad_id with salting for huge advertisers; merge partial counts.
- **Reconciliation:** batch (Lambda-style) recomputation from raw logs corrects real-time counts — justified because money is involved.

Say this: "Redirect first, count later." · "Stream for freshness, batch for the bill."

Part IV — Index and practice plan

Top 20: the minimum set to master

If time is short, master these. They cover the most-reported questions at your primary targets and the patterns that transfer to almost everything else.

Priority	Problem	Why
1	Q28 TinyURL	Read-heavy design, ID generation, caching layers
2	Q29 WhatsApp	Real-time delivery, connection registry, ordering
3	Q30 YouTube	Upload pipeline, transcoding, CDN, ABR
4	Q32 Uber	Geo indexing, matching, contention
5	Q35 Ticketmaster	Admission control, holds, strong consistency
6	Q1 Your system at 10×	Amazon and Databricks ask it; your strongest story
7	Q9 Legacy migration	Apple's most detailed prompt; your cloud modernization
8	Q2 Distributed logging	Amazon + Google; ingest, tiering, quotas
9	Q14 Live comments	Meta's most-reported prompt; fan-out math
10	Q10 Storage + EU regionalization	Apple, Databricks; consistency, residency
11	Q19 Object store	Databricks; storage internals
12	Q21 Streaming pipeline	Databricks + Google; exactly-once
13	Q3 OTA updates	Amazon; rollout control loops, security
14	Q5 Payments	Idempotency, ledgers (transfers to Q20, Q59)
15	Q15 News Feed	Meta; hybrid fan-out (transfers to Q31, Q49)
16	Q4 Dropbox	Amazon 2026; chunking, sync, conflicts
17	Q17 Notifications	Microsoft; channels, presence, preferences
18	Q11 Distributed cache	Apple; stampedes, invalidation
19	Q45 ChatGPT	AI systems now come up everywhere
20	Q39 Top-K	Counting and streaming pattern (transfers to Q24, Q61)

Problems grouped by pattern

Practice one problem per group first; the others in the group then go much faster.

Pattern	Problems
Read-heavy + caching	Q28 TinyURL, Q11 cache, Q33 Yelp, Q36 typeahead
Fan-out and feeds	Q15 feed, Q31 Twitter, Q49 Instagram, Q14 live comments, Q17 notifications
Real-time messaging and sessions	Q29 WhatsApp, Q44 Zoom, Q57 chess, Q18 Docs
Media and large files	Q30 YouTube, Q27 video distribution, Q4 Dropbox, Q16 device sync, Q3 OTA
Geo and location	Q32 Uber, Q33 Yelp, Q34 Maps, Q48 local delivery, Q52 Strava, Q50 Tinder
Contention and booking	Q35 Ticketmaster, Q58 flash sale, Q53 auction, Q6 lockers, Q7 promotions
Money and correctness	Q5 payments, Q20 card network, Q59 donations, Q43 exchange
Counting and analytics	Q24 click counter, Q39 Top-K, Q61 ad clicks, Q2 logging, Q13 observability, Q25 ML monitoring
Search and crawling	Q38 search engine, Q37 crawler, Q56 post search, Q54 news, Q55 price tracking, Q8 Alexa
Infrastructure building blocks	Q40 Kafka, Q41 IDs, Q26 rate limiter, Q12 load balancer, Q10 KV store, Q19 object store
Data and compute platforms	Q21 streaming, Q22 job scheduler, Q60 GitHub Actions, Q51 LeetCode
Migration and evolution	Q9 migration, Q1 reverse design
AI and ML systems	Q45 ChatGPT, Q46 recommendations, Q25 ML monitoring, Q8 Alexa

Master index: all 61 problems

#	Problem	Part	Key pattern
Q1	Reverse design: your system at 10×	II	Finding the real bottleneck
Q2	Distributed logging	II	Ingest log, tiering, quotas
Q3	Firmware / OTA updates	II	Rollout control loop, CDN
Q4	Dropbox / Google Drive	II	Chunking, metadata vs bytes
Q5	Parking payment system	II	Idempotency, ledger
Q6	Amazon Locker	II	Capacity vs physical slot
Q7	Promotions / cash back	II	Fast path vs money path
Q8	Alexa voice search	II	Latency budget, ranking
Q9	Legacy storage migration	II	Dual-write, backfill, cutover
Q10	Storage system / KV store + EU	II	Raft partitions, residency

#	Problem	Part	Key pattern
Q11	Distributed / web cache	II	Consistent hashing, stampedes
Q12	Load balancer	II	L4/L7, health, P2C
Q13	Observability + cost	II	Cardinality, forecasting
Q14	FB Live Comments	II	Two-level fan-out
Q15	News Feed / Instagram feed	II	Hybrid fan-out
Q16	Hardware-software device sync	II	Offline-first, battery
Q17	Notification system (Teams)	II	Prefs, presence, channels
Q18	Google Docs / co-editing	II	OT vs CRDT
Q19	Object store (S3)	II	Erasure coding, commit point
Q20	Card payment network	II	Auth vs settlement
Q21	Streaming pipeline, exactly-once	II	Checkpoints, watermarks
Q22	Job scheduler	II	Control vs data plane
Q23	Connection degree	II	Meet in the middle
Q24	View/click counter	II	Pre-aggregation
Q25	ML model monitoring	II	Drift as a proxy
Q26	Rate limiter	II	Local + global coordination
Q27	Global video distribution	II	Edge caching, prefill
Q28	TinyURL / Bitly / Pastebin	III	Keyspace, ID blocks
Q29	WhatsApp / Messenger	III	Inbox, connection registry
Q30	YouTube / TikTok / Spotify	III	Transcode DAG, ABR
Q31	Twitter / X	III	Fan-out, search, trends
Q32	Uber / DoorDash	III	Geo index, atomic accept
Q33	Yelp / proximity / Nearby Friends	III	Spatial index in memory
Q34	Google Maps	III	Contraction hierarchies
Q35	Ticketmaster / Airbnb	III	Admission control, holds
Q36	Typeahead / autocomplete	III	Precomputed top-K
Q37	Web crawler	III	Frontier, politeness
Q38	Search engine	III	Inverted index, tiered rank
Q39	Top-K / YouTube Top K / trending	III	Partitioned heavy hitters
Q40	Distributed message queue (Kafka)	III	Partitions, ISR

#	Problem	Part	Key pattern
Q41	Unique ID generator	III	Snowflake bits
Q42	Game leaderboard	III	Sorted sets
Q43	Stock exchange / Robinhood	III	Sequencer, order book
Q44	Zoom / video conferencing	III	SFU, simulcast
Q45	ChatGPT	III	GPU batching, admission
Q46	Recommendation system	III	Retrieval → ranking funnel
Q47	Gmail	III	Per-user index, spam edge
Q48	Local delivery (Gopuff)	III	Precomputed delivery zones
Q49	Instagram	III	Media pipeline, stories TTL
Q50	Tinder	III	Atomic pair match
Q51	LeetCode / online judge	III	Sandboxes, queues
Q52	Strava	III	Offline-first, segments
Q53	Online auction	III	Conditional bid per auction
Q54	News aggregator	III	Clustering into stories
Q55	Price tracking	III	Prioritized refresh, alert index
Q56	FB post search	III	Two orderings, batched likes
Q57	Online chess	III	Single owner per game
Q58	Flash sale	III	Edge sold-out, atomic stock
Q59	Donations website	III	Intent before charge
Q60	GitHub Actions / CI	III	DAG release, ephemeral runners
Q61	Ad click aggregator	III	Stream + batch reconcile

Variants map: "they asked X — practice Y"

If asked…	Practice	Change to emphasize
Pastebin	Q28	Content in object storage
Facebook Messenger, Slack, Discord	Q29	Channels/servers, search, larger groups
Spotify, Netflix playback, TikTok	Q30 / Q27	Offline downloads, recommendations
Instagram feed	Q15 / Q49	Media-heavy, celebrity skew
DoorDash, Uber Eats	Q32 / Q48	Three-sided matching, prep time
Airbnb, hotel booking	Q35	Date-range inventory, search

If asked…	Practice	Change to emphasize
Google Drive, OneDrive	Q4	Sharing, permissions
Ad click aggregator, metrics	Q61 / Q24 / Q13	Exactness for billing
Distributed cron, Airflow	Q22 / Q60	DAGs, exactly-once starts
Payment system, digital wallet	Q5 / Q20 / Q59	Ledger, idempotency
Notification system	Q17	Channels and preferences
Key-value store, DynamoDB	Q10	Partitioning, replication
Twitch / live streaming	Q30 follow-ups / Q14	Low-latency ingest, live chat

Company matrix for Part II (Q1–Q27)

● = reported for this company · ○ = strong pattern match / likely · blank = less relevant

#	Problem	Amazon	Apple	Meta	Microsoft	Databricks	Google	Netflix
Q1	Reverse design: your system at 10×	●	○	○	○	●	○	○
Q2	Distributed logging	●	○	○	○	○	●	○
Q3	Firmware / OTA updates	●	○	○			○	
Q4	Dropbox file sync	●	○	○	○		○	
Q5	Parking payment system	●	○		○	○		
Q6	Locker pickup	●						
Q7	Promotions / cash back	●						
Q8	Alexa voice search	●	○				○	
Q9	Legacy storage migration	○	●	○	○	○	○	○
Q10	Storage system + EU regionalization	○	●		○	○	○	
Q11	Web / distributed cache	○	●	○	○		○	○
Q12	Load balancer from scratch	○	●				○	○
Q13	Observability + cost forecasting	○	●		○	○		○
Q14	Live comments at millions of viewers	○		●	○		○	○
Q15	News Feed / Instagram	○		●			○	

#	Problem	Amazon	Apple	Meta	Microsoft	Databricks	Google	Netflix
Q16	Hardware-software (device sync)	○	○	●				
Q17	Teams notifications	○	○	○	●		○	
Q18	Collaborative editing		○		○		○	
Q19	Object store	○	○		○	●	○	
Q20	Card payment network	○				●		
Q21	Streaming pipeline, exactly-once	○		○	○	●	●	○
Q22	Multi-tenant job scheduler	○			○	○		
Q23	Connection degree			○	○		●	
Q24	View/click counting	○		○			●	○
Q25	ML model monitoring	○	○	○	○	○	●	○
Q26	Rate limiter at 10M RPS	○	○	○	○		○	●
Q27	Global video distribution	○	○				●	●

Practice tracker

Copy this table into a spreadsheet. A problem is "done" when your third cold attempt hits every checkpoint in 45 minutes.

#	Attempt 1 date / score	Attempt 2 (day 7)	Attempt 3 (day 30)	Gaps to fix
Q28				
Q29				
Q30				
Q32				
Q35				
Q1				
Q9				
…				

Self-scoring rubric (1–4 each): requirements and assumptions · estimation · high-level design by minute 20 · depth of deep dives · trade-offs stated · failures and operations · manager lens woven in · communication and pace. A Strong Hire attempt averages 3.5+ with no 1s.

Sources

Reported questions and loop details (Parts I–II) come from 2025–2026 candidate reports and guides; treat them as signals, since formats vary by team. Classic problems (Part III) were cross-checked against Hello Interview's practice list.

- Aced/Exponent — Amazon SDM Interview Guide (2026); Apple EM Interview Guide (2026) and Apple EM interview experience; Google System Design Interview (2026); Amazon System Design Interview (2026)
- IGotAnOffer — Amazon SDM interview (Feb 2026 update); Amazon System Design Interview (Apr 2026 update)
- Glassdoor — Meta EM reviews (real-time comments, News Feed, Instagram, hardware-software); Amazon senior SWE (Dropbox clone, Jul 2026) and SDM reviews; Databricks EM reviews (object store, payment system, systems programming, panel presentation, Feb 2026); Google EM reviews (YouTube Shorts clicks, ML model monitoring)
- Dataford — Google and Databricks EM interview experiences
- CleverPrep — Microsoft SWE guide (Teams notification system); Blind and IGotAnOffer — Microsoft EM loop and "As Appropriate" round
- Prepfully, interviewing.io, codemia — Netflix interview guides
- Hello Interview — System design practice list and delivery framework (hellointerview.com)
- Technical background: DeCandia et al., "Dynamo" (2007); Ongaro & Ousterhout, "Raft" (2014); Armbrust et al., "Delta Lake" (VLDB 2020); Nishtala et al., "Scaling Memcache at Facebook" (NSDI 2013); Eisenbud et al., "Maglev" (NSDI 2016); Geisberger et al., "Contraction Hierarchies" (2008); Amazon Builders' Library; Kleppmann, *Designing Data-Intensive Applications*